

Introduction to .NET

What is .NET?

Ans: It is a product of Microsoft launched in the year 2002, which can be used for building various kinds of Applications like: Web, Mobile, Desktop, Micro services, Cloud, Machine Learning, Game Development and IoT (Internet of Things).

How to develop all the above applications by using .NET?

Ans: To develop the above applications, .NET provides with a set of Programming Languages, Technologies & Servers using which we can build any kind of Application.

What are the Programming Languages, .NET provides to us?

Ans: In .NET there are 30+ programming languages available for a developer to build applications and programmers have a chance of choosing any 1 language from the list.

Features of .NET: there are 2 important features in .NET, those are:

1. Language Independent
2. Platform Independent

1. Language Independent: .NET is a collection of programming Languages i.e.; it provides us multiple languages for building our applications and developers can choose any 1 language from the list to build their applications. At the time of launching .NET in 2002, Microsoft has given **30+ Languages** like C#, VB.NET, Fortran.NET, Python.NET (Iron Python), Cobol.NET, VCPP.NET, Pascal.NET, J#.NET, etc. Most of these languages are extension to some existing languages, like:

C, CPP	=>	C#
Cobol	=>	Cobol.NET
Pascal	=>	Pascal.NET
Fortran	=>	Fortran.NET
Visual Basic	=>	VB.NET
Visual CPP	=>	VCPP.NET
Python	=>	Python.NET (Iron Python)
Java	=>	J#.NET
	=>	F#
	=>	ML.NET

Note: As of today, we don't have all these 30+ languages in usage, what we have is only **5 languages** in usage like **C#, VB.NET, F#, Iron Python** and **ML.NET**, and the most popular of all these languages is "**C#**". Because .NET is a collection of languages, programmers always have a choice to choose a language based on his previous experience or interest to build their applications, for example:

Task: Write a program for printing from 1 to 100 by using a for loop.

C# Source Code => Compiled by using C# Compiler => CIL Code

```
static void Main()
{
    for (inti = 1; i<= 100; i++)
```

```

{
    Console.WriteLine(i);
}
}

```

VB Source Code => Compiled by using VB Compiler => CIL Code

```

Shared Sub Main()
    For i As Integer = 1 To 100 Step 1
        Console.WriteLine(i)
    Next i
End Sub

```

F# Source Code => Compiled by using F# Compiler => CIL Code

```

let main() =
    For i = 1 To 100 Do
        printfn "%i" i
    main()

```

The output code that is generated after compilation of a program that is implemented by using a .NET Language is called CIL (Common Intermediate Language) Code or MSIL (Microsoft Intermediate Language).

COBOL, Pascal, FORTRAN, and C Languages are Procedural Programming Languages and the drawback in this approach is they don't provide security and re-usability of code. To overcome the drawbacks of Procedural Programming Language's in early 80's we are provided with a new approach known as **Object Oriented Programming** which provides security and re-usability.

All Object-Oriented Programming Languages have an important feature that is **"Code Re-usability"** i.e., the code we write in 1 program can be consumed from another program, for example:

C++ Source Code => Compiled by using C++ Compiler => Generates Object Code => Which can be consumed from another C++ Program.

Java Source Code => Compiled by using Java Compiler => Generates Byte Code => Which can be consumed from another Java Program.

C# Source Code => Compiled by using C# Compiler => Generates CIL Code => Which can be consumed from any .NET Language Program.

F# Source Code => Compiled by using F# Compiler => Generates CIL Code => Which can be consumed from any .NET Language Program.

VB Source Code => Compiled by using VB Compiler => Generates CIL Code => Which can be consumed from any .NET Language Program.

Note: Re-usability in CPP and Java Languages is only with-in that language whereas the same re-usability in .NET Languages is across all languages of .NET, and this is what we call as Language Independent.

If any 2 languages want to communicate or interoperate with each other they need to cross 2 hurdles:

1. There should not be any mismatch in compiled code.
2. There should not be any mismatch in data types.

Lang1 (int is 2 bytes) => Object Code

Lang2 (int is 4 bytes) => Object Code

Note: In .NET Languages we will not face compiled code mismatch because all languages are generating CIL or MSIL Code only after the compilation. They don't face data type mis-match problem also because all languages of .NET adopt a rule known as "Uniform Data Type Structure" i.e., similar types will always be same in size irrespective of their names.

2. Platform Independent: it is an approach of executing an application that is developed on 1 platform, in other platforms.

What is a Platform?

Ans: A platform is an environment under which an application executes, and it is a combination of 2 things, those are Micro-Processor and Operating System.



Note: up to 1995, application that are developed by using programming languages that are present in the market (E.g., C, CPP, VB, Cobol, Pascal, Fortran, VCPP) are all platform dependent i.e., if we develop any application by using any of these languages on 1 platform, we can't run them on other platforms. For example, we can't install MS Office on Linux or Mac OS, so it is a platform dependent application.

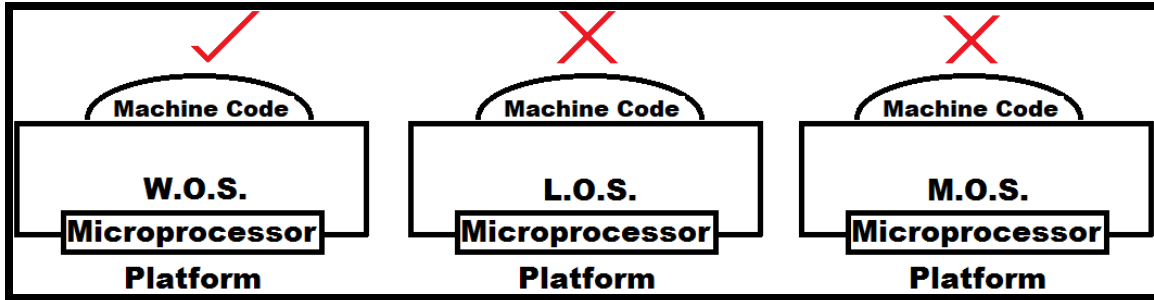
Why older programming languages are platform dependent?

Ans: Applications that are developed by using programming languages that are present in the market before 1995 are all platform dependent, because in all these languages when we compile the Source Code, they will generate Machine Code based on the O.S. where they are compiled, so Machine Code that is generated for 1 O.S is not understandable to other OS's.

Application developed by using C++ language on Windows OS:

Source Code => Compiled by C++ Compiler => Machine Code

Machine Code means operating system understandable code and this code has an advantage and dis-advantage. Advantage is to run the Machine Code we don't require to install CPP Software on client machines whereas dis-advantage is the above Machine Code runs only on Windows but not on any other OS.



Note: any application which directly sits on the top of OS is always a platform dependent application.

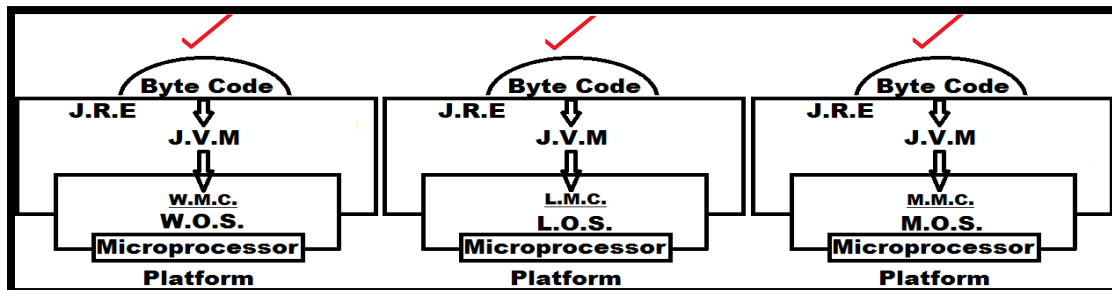
What is Platform Independent?

Ans: Applications that are developed by using Java and .NET Languages are Platform Independent i.e., these applications once developed on a Platform can run on any other Platform (i.e., write once and run anywhere).

Application developed by using Java language on Windows OS:

Source Code => Compiled by Java Compiler => Byte Code

Byte Code is not OS understandable, so OS is not at all responsible to execute this code. We can run this Byte Code, we need to install a software provided by Java known as JRE (Java Runtime Environment) and if this software is installed on the Client's Computer we can run the Byte Code where ever we want, because inside of the JRE there is a component called as JVM (Java Virtual Machine) and that JVM contains a compiler called "JIT (Just In Time) Compiler" which will convert Byte Code into Machine Code based on the OS where it was executing.



Note: JRE software is platform dependent i.e., we are provided with this JRE separately for each OS and this makes the Byte Code platform independent.

Windows Machine installed with Windows JRE:

Byte Code => JVM => Converts into Windows Machine Code

Linux Machine installed with Linux JRE:

Byte Code => JVM => Converts into Linux Machine Code

Mac Machine installed with Mac JRE:

Byte Code => JVM => Converts into Mac Machine Code

Solaris Machine installed with Solaris JRE:

Byte Code => JVM => Converts into Solaris Machine Code

We can download JRE from the below sites:

<https://www.java.com/en/download/manual.jsp>

<https://www.oracle.com/in/java/technologies/javase-jre8-downloads.html>

.NET: Microsoft launched .NET in the year 2002.

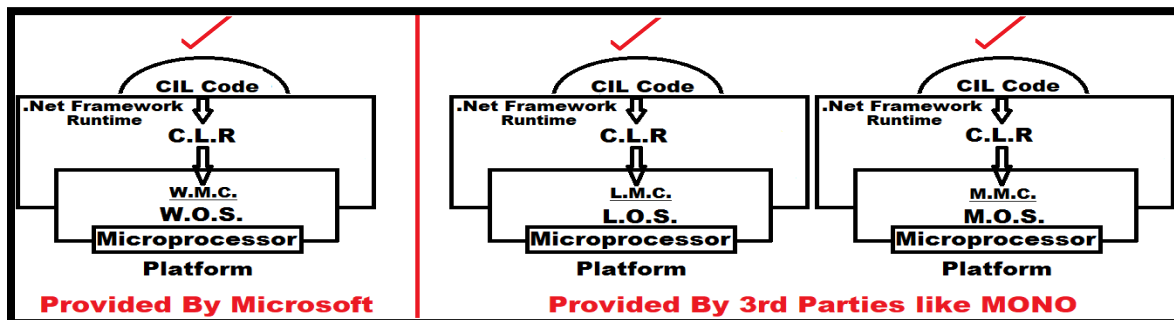
Application developed by using .NET languages on Windows OS:

Source Code => Compiled by a Language Compiler => CIL Code

Note: as said earlier, .NET is a collection of programming languages so with whatever .NET Language we develop the application and compile the source code by using an appropriate language compiler, the outcome will be “CIL” (Common Intermediate Language) code only.

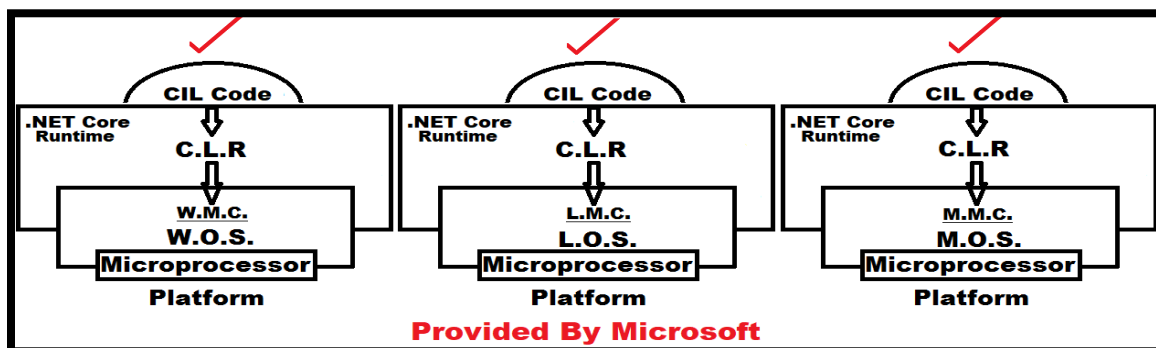
We will install CIL Code on Client Machines and to run that code we need to install software known as “.NET Runtime” and inside of this Runtime there will be a component called CLR (Common Language Runtime) which will convert CIL Code into Native Machine Code.

In the year 2002 when Microsoft launched .NET in the market, they provided their first Runtime for Windows O.S. only but not for any other O.S.’s, but they made the specifications to develop the Runtime as open, so 3rd party companies came forward and developed the Runtime’s for other O.S.’s also and the name of that runtime is “.NET Framework”. The first version of .NET Framework is 1.0 and the last version is 4.8.

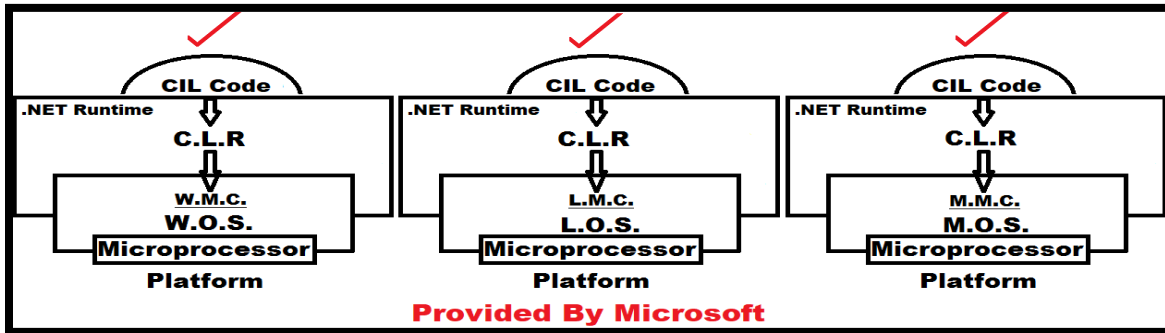


Note: with .NET Framework Runtime there is a criticism on .NET that it is not fully Platform Independent because Microsoft has given it only for Windows.

In the year 2016 Microsoft launched a new Runtime into the market with the name “.NET Core” and this runtime is provided for Windows, Linux, and Mac machines also. The first version of .NET Core is 1.0 and the last version is 3.1.



On November 10, 2020, Microsoft launched a new **Runtime** into the market by combining **.NET Framework** & **.NET Core** as **1 .NET** which starts from version **5.0** and the latest is **8.0** launched on November 2023.

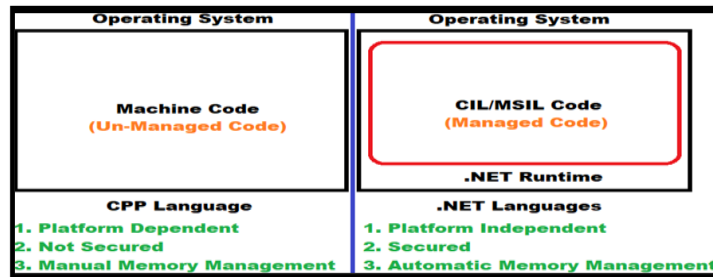


Note: the new .NET is nothing but .NET Core only but with-out again calling .NET Core and .NET Framework they made the name simple as just “.NET”.

What is a .NET Runtime?

Ans: It's software which must be installed on Client's Machine if at all we want to run .NET Application's on that Machine which sits on top of the O.S. and executes the CIL Code by masking the functionalities of an OS.

In case of platform dependent languages like Cobol, C, CPP, Visual Basic, etc. Compiled Code i.e., Machine Code runs directly under OS., whereas in case of .NET Languages, CIL Code will run under the .NET Runtime.



Note: Application's that directly run under the O.S. are known as Un-Managed App's whereas App's that run under .NET Runtime are known as Managed App's.

Applications that run under these runtime's are provided with the following features:

- Platform Independent or Portable
- Secured
- Automatic Memory Management

The development of .NET started with the development of this Runtime in late 90's originally under the name **“NGWS (Next Generation Windows Systems)”** and to develop this software first they prepared a specification known as **“CLI Specifications”**, where CLI stands from Common Language Infrastructure. This CLI Specification describes 4 aspects in it, those are:

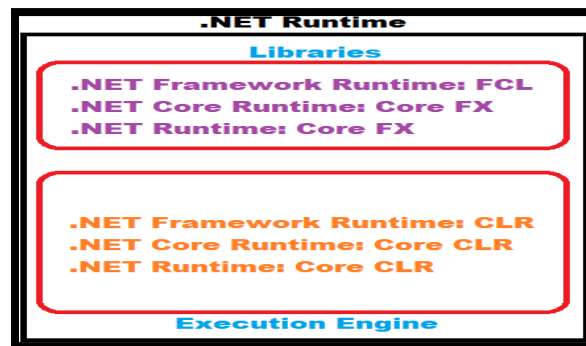
1. **CLS (Common Language Specification):** it's a set of base rules all Languages of .NET must follow to interoperate with each other, most importantly after compilation of source code all those languages need to generate the same type of output code known as CIL Code, so that when any 2 languages want to interoperate with each other, then compiled code mismatch will not come into picture.

2. **CTS (Common Type System):** According to this all languages of .NET should follow a standard regarding the Data Types i.e., “Uniform Data Type Structure” which means similar types must always be same in size irrespective of their names.

Note: Because of these CLS and CTS only, all language of .NET can interoperate or communicate with each other.

3. **Metadata:** Information about program structure is language-independent, so that it can be referenced between languages and tools, making it easy to work with code written in a language the developer are not aware.
4. **VES (Virtual Execution System):** this is nothing but CLR or Common Language Runtime.

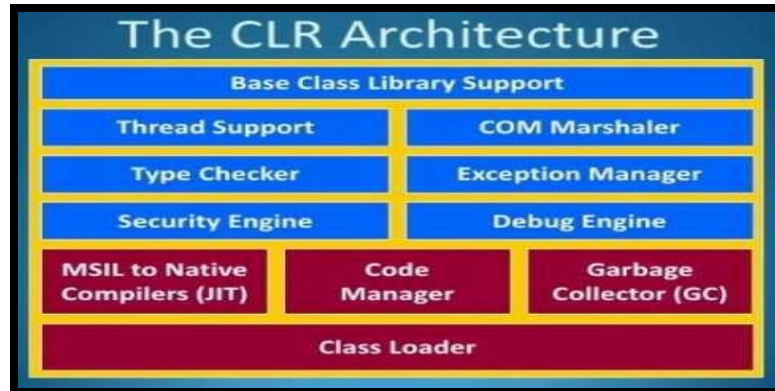
The runtime software internally contains 2 main components in it, those are the “Libraries” and an “Execution Engine” as following:



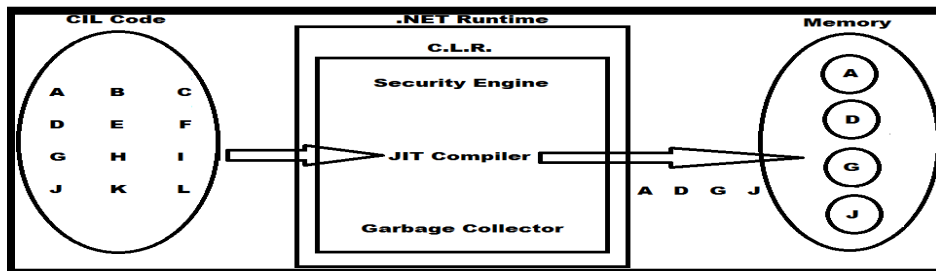
Libraries: A library is a set of re-usable functionalities, and every programming language has built-in libraries to it like **Header Files** in C & CPP Languages and **Packages** in Java Language same as that .NET Languages are also provided with built-in libraries and we call them “FCL (Framework Class Libraries)” in .NET Framework and “CORE FX” in .NET Core and .NET.

Execution Engine: as discussed earlier, .NET Applications will not run under the OS, but they will be running under the Runtime and in this Runtime, we have an Execution Engine responsible for the execution of Applications and we call this as “CLR (Common Language Runtime)” in .NET Framework and “CORE CLR” in .NET Core and .NET.

CLR and **Core CLR** are known as execution engine of .NET Runtime, where all .NET Application run under the supervision of this CLR and it internally it contains various components in it to manage various actions, like:



1. **Security Engine:** this is responsible for the security of our applications, i.e., it will take care that applications don't directly interact with the OS, as well as OS don't directly interact the application.
2. **JIT Compiler:** this is the compiler which is responsible for converting CIL Code into Machine Code based on the platform where we are executing the application adopting a process known as "Conversion gradually during the program's execution".



3. **Garbage Collector:** it is responsible for "Automatic Memory Management" where "Memory Management" is a process of allocation and de-allocation of memory that is required for a program to execute, and this is of 2 types:
 - ② Manual or Explicit
 - ② Automatic or Implicit

Manual or Explicit means, in this case programmers are responsible for allocation and de-allocation of the memory explicitly. Automatic or Implicit means, here programmers are not at all responsible for allocation and de-allocation of the memory and on behalf of the programmers Garbage Collector will take the responsibility for memory management.

What is Application Software?

Ans: Application software is commonly defined as any program or number of programs designed for end-users. In that sense, any end user program can be called an "application." People often use the term "application software" to talk about bundles or groups of individual software applications, using a different term, "application program" to refer to individual applications. Examples of application software include items like Notepad, WordPad, Microsoft Word, Microsoft Excel, or any of the Web Browsers used to navigate the Internet, etc.

Another way to understand application software is, in a very basic sense, every program that you use on your computer is a piece of application software. The operating system, on the other hand, is system software. Historically, the application was generally born as computers evolved into systems where you could run a particular codebase on a given operating system. Even social media platforms have come to resemble applications, especially on our mobile phone devices, where individual applications are given the nickname "apps." So, while the term

“application software” can be used broadly, it’s an important term in describing the rise of sophisticated computing environments.

How to develop Application software?

Ans: There are two basic camps of software development: Applications Development and Systems Development. Applications Development is focused on creating programs that meet the users' needs. These can range from mobile phone apps, video games, enterprise-level accounting software. Systems Development is focused on creating and maintaining operating systems and to do this we need to familiar with some Programming Language. Thousands of different programming languages have been created, and more are being created every year. Many programming languages are written in an imperative form (i.e., as a sequence of operations to perform) while other languages use the declarative form (i.e., the desired result is specified, not how to achieve it).

What is a Programming Language?

Ans: A programming language is a formal language comprising a set of instructions that produce various kinds of output. Programming languages are used in computer programming to implement algorithms. Most programming languages consist of instructions for computers. Since the early 1800s, programs have been used to direct the behavior of machines such as Jacquard looms, music boxes and player pianos.

“A computer programming language is a language used to write computer programs, which involves a computer performing some kind of computation or algorithm and possibly control external devices such as printers, disk drives, robots, and so on.”

Anyone can come up with ideas, but a developer will be able to turn those ideas into something concrete. Even if you only want to work on the design aspects of software, you should have some familiarity with coding and be able to create basic prototypes. There are a huge variety of programming languages that we can learn. Very early computers, such as Colossus is thus regarded as the world's first programmable, electronic, digital computer, although it was programmed by switches and plugs and not by a stored program.

Slightly later, programs could be written in machine language, where the programmer writes each instruction in a numeric form the hardware can execute directly. For example, the instruction to add the value in two memory location might consist of 3 numbers: an “opcode” that selects the “add” operation, and two memory locations. The programs, in decimal or binary form, were read in from punched cards, paper tape, and magnetic tape or toggled in on switches on the front panel of the computer. Machine languages were later termed first-generation programming languages (1GL).

The next step was development of so-called second-generation programming languages (2GL) or assembly languages, which were still closely tied to the instruction set architecture of the specific computer. These served to make the program much more human-readable and relieved the programmer of tedious and error-prone address calculations.

The first high-level programming languages, or third-generation programming languages (3GL), were written in the 1950s. John Mauchly's Short Code, proposed in 1949, was one of the first high-level languages ever developed for an electronic computer. Unlike machine code, Short Code statements represented mathematical expressions in understandable form. However, the program had to be translated into machine code every time it ran, making the process much slower than running the equivalent machine code.

At the University of Manchester, Alick Glennie developed Autocode in the early 1950s. As a programming language, it used a compiler to automatically convert the language into machine code. The first code and compiler were developed in 1952 for the Mark 1 computer at the University of Manchester and is the first compiled high-level programming language.

In 1954, FORTRAN was invented at IBM by John Backus. It was the first widely used high-level general purpose programming language to have a functional implementation, as opposed to just a design on paper. It is still a popular language for high-performance computing and is used for programs that benchmark and rank the world's fastest supercomputers.

Another early programming language was devised by Grace Hopper in the US, called FLOW-MATIC. It was developed for the UNIVAC I at Remington Rand during the period from 1955 until 1959. Hopper found that business data processing customers were uncomfortable with mathematical notation, and in early 1955, she and her team wrote a specification for an English programming language and implemented a prototype. The FLOW-MATIC compiler became publicly available in early 1958 and was substantially complete in 1959. FLOW-MATIC was a major influence in the design of COBOL.

COBOL an acronym for “common business-oriented language” is a compiled English-like computer programming language designed for business use. It is imperative, procedural and, since 2002, object-oriented. COBOL is primarily used in business, finance, and administrative systems for companies and governments. COBOL is still widely used in applications deployed on mainframe computers, such as large-scale batch and transaction processing jobs. But due to its declining popularity and the retirement of experienced COBOL programmers, programs are being migrated to new platforms, rewritten in modern languages. Most programming in COBOL is now purely to maintain existing applications.

Pascal is an imperative and procedural programming language, designed by Niklaus Wirth as a small, efficient language intended to encourage good programming practices using structured programming and data structuring. It is named in honor of the French mathematician, philosopher, and physicist Blaise Pascal. Pascal enabled defining complex data types and building dynamic and recursive data structures such as lists, trees, and graphs. Pascal has strong typing on all objects, which means that one type of data cannot be converted or interpreted as another without explicit conversions.

C is a general-purpose, imperative procedural computer programming language supporting structured programming, lexical variable scope, and recursion, with a static type system. By design, C provides constructs that map efficiently to typical machine instructions. It has found lasting use in applications previously coded in assembly language. Such applications include operating systems, various application software for computers that range from super computers to PLCs and embedded systems. A successor to the programming language B, C was originally developed at Bell Labs by Dennis Ritchie between 1972 and 1973 to construct utilities running on UNIX. It was applied to re-implementing the kernel of the UNIX operating system. During the 1980s, C gradually gained popularity. It has become one of the most widely used programming languages, with C compilers from various vendors available for most existing computer architectures and operating systems. C has been standardized by the ANSI since 1989 (ANSI C) and by the International Organization for Standardization (ISO).

C++ is a general-purpose programming language developed by Danish computer scientist Bjarne Stroustrup at Bell Labs since 1979 as an extension of the C programming language, or "C with Classes" as he wanted an efficient and flexible language like C that also provided high-level features for program organization. The

language has expanded significantly over time, and modern C++ now has object-oriented, generic, and functional features in addition to facilities for low-level memory manipulation. It is almost always implemented as a compiled language, and many vendors provide C++ compilers, including the Free Software Foundation, LLVM, Microsoft, Intel, Oracle, and IBM, so it is available on many platforms. C++ has also been found useful in many contexts, with key strengths being software infrastructure and resource-constrained applications, including desktop applications, video games, servers (e.g., e-commerce, Web search, or SQL Servers), and performance-critical applications (e.g., telephone switches or space probes).

Objective-C is a general-purpose, object-oriented programming language that adds Smalltalk-style messaging to the C programming language. It was the main programming language supported by Apple for macOS, iOS, and their respective application programming interfaces (APIs). The language was originally developed in the early 1980s. It was later selected as the main language used by NeXT for its NeXTSTEP operating system, from which macOS and iOS are derived. Objective-C source code 'implementation' program files usually have .m filename extensions, while Objective-C 'header/interface' files have .h extensions, the same as C header files. Objective-C++ files are denoted with a .mm file extension.

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed, and garbage collected. It supports multiple programming paradigms, including structured (particularly, procedural), object-oriented, and functional programming. Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0 released in 2000 and Python 3.0, released in 2008, was a major revision of the language that is not completely backward compatible, i.e., Python 2 code does not run unmodified on Python 3. The Python 2 language was officially discontinued in 2020 (first planned for 2015) and now only Python 3.5.x and later are supported.

Java is a general-purpose programming language that is class-based and object-oriented, and designed to have as few implementation dependencies as possible. It is intended to let application developers write once, run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need of recompilation. Java applications are typically compiled to byte code that can run on any Java virtual machine (JVM) regardless of the underlying computer architecture. The syntax of Java is like C and C++. Java was originally developed by James Gosling at Sun Microsystems (which has since been acquired by Oracle) and released in 1995 as a core component of Sun Microsystems' Java platform.

C# (pronounced see sharp, like the musical note C#, but written with the number sign) is a general-purpose, multi-paradigm programming language encompassing strong typing, lexically scoped, imperative, declarative, functional, generic, object-oriented (class-based), and component-oriented programming disciplines. It was developed around 2000 by Microsoft as part of its .NET initiative and later approved as an international standard by ECMA in 2002 and ISO in 2003. C# was designed by Anders Hejlsberg, and its development team is currently led by "Mads Torgersen". The most recent version is 9.0, which was released on November 2020 alongside Visual Studio 2019.

What is .NET?

Ans: .NET is a free, cross-platform, open-source developer platform for building many different types of applications like Desktop, Web, Mobile, Games and IOT by using multiple languages, editors, and libraries.

What is a Platform?

Ans: It is the environment in which a piece of software is executed. A platform can also be called as the stage on which computer programs can run. Platform can refer to the type of processor (CPU) on which a given operating system runs, the type of operating system on a computer or the combination of the type of hardware and the type of operating system running on it. An example of a common platform is Microsoft Windows running on x86 architecture. Other well-known desktop computer platforms include Linux/Unix and macOS

What is Cross-platform?

Ans: In computing, cross-platform software (also multi-platform software or platform-independent software) is computer software that is implemented to run on multiple platforms. For example, a cross-platform application may run on Microsoft Windows, Linux, and macOS. Cross-platform programs may run on as many as all existing platforms, or on few platforms.

What is meant by developing applications using multiple languages?

Ans: .NET languages are programming languages that are used to produce libraries and programs that conform to the Common Language Infrastructure (CLI) specifications. Most of the CLI languages compile entirely to the Common Intermediate Language (CIL), an intermediate language that can be executed using the Common Language Runtime, implemented by .NET Framework, .NET Core, and Mono. As the program is being executed, the CIL code is just-in-time compiled to the machine code appropriate for the architecture on which the program is running. While there are currently over 30+ languages in .NET, but only a small number of them are widely used and supported by Microsoft. List of .NET languages include C#, F#, Visual Basic, C++, Iron Python, etc. and the most popular and widely used language as a developer choice is C#. Visit the following link to view the list of .NET Languages: https://microsoft.fandom.com/wiki/Microsoft_.NET_Languages

What is CLI (Common Language Infrastructure)?

Ans: The Common Language Infrastructure (CLI) is an open specification (technical standard) developed by Microsoft and standardized by ISO (International Organization for Standardization) and ECMA (European Computer Manufacturers Association) that describes about executable code and a runtime environment that allows multiple high-level languages to be used on different computer platforms without being rewritten for specific architectures. This implies it is platform independent. The .NET Framework, .NET Core and Mono are implementations of the CLI.

CLI specification describes the following four aspects:

1. The Common Language Specification (CLS):
2. The Common Type System (CTS):
3. The Metadata:
4. The Virtual Execution System (VES):

What is .NET Framework and .NET Core?

Ans: .NET is a developer platform made up of tools, programming languages, and libraries for building many different types of applications. There are various implementations of .NET, and each implementation allows .NET code to execute in different places - Linux, macOS, Windows, iOS, Android, and many more. Various implementations of the .NET include:

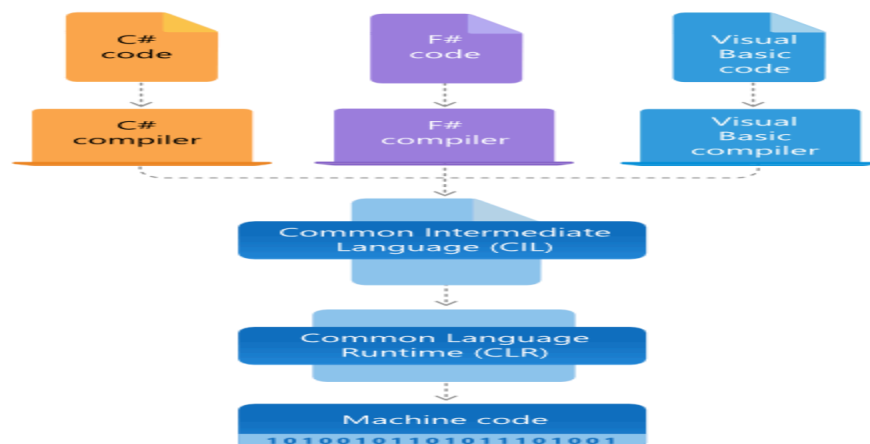
1. **.NET Framework:** it is the original implementation of .NET, and it supports running websites, services, desktop apps, and more on Windows.

2. **.NET Core:** it is a cross-platform implementation for running websites, services, and console apps on Windows, Linux, and macOS.
3. **Xamarin/Mono:** it is a .NET implementation for running apps on all the major mobile operating systems, including iOS and Android.

Architecture of .NET Framework: The two major components of .NET Framework are the .NET Framework Class Library and the Common Language Runtime.

1. The Class Library provides a set of APIs and types for common functionality. It provides types for strings, dates, numbers, etc. The Class Library includes APIs for reading and writing files, connecting to databases, drawing, and more.
2. The Common Language Runtime (CLR) is the heart of .NET Framework and the execution engine that handles running applications. It provides services like thread management, garbage collection, type-safety, exception handling, and more.

Architecture of .NET Framework CLR: .NET applications can be written in any .NET Language like C#, F#, or Visual Basic. Source Code we write by using some .NET Language is compiled into a language-agnostic Common Intermediate Language (CIL) and the compiled code is stored as assemblies (files with a ".dll" or ".exe" extension). When we run the applications, CLR takes the assemblies and uses a just-in-time compiler (JIT) to turn it into machine code that can execute on the specific architecture of the computer it is running on.



.NET Framework FAQ's

What is .NET Framework used for?

Ans: .NET Framework is used to create and run software applications. .NET apps can run on many operating systems, using different implementations of .NET. .NET Framework is used for running .NET apps on Windows.

Who uses .NET Framework?

Ans: Software developers and the users of their applications both use .NET Framework:

- Users need to install .NET Framework to run application built with the .NET Framework. In most cases, .NET Framework is already installed with Windows. If needed, you can download .NET Framework.
- Software developers use .NET Framework to build many different types of applications - websites, services, desktop apps, and more with Visual Studio. Visual Studio is an integrated development

environment (IDE) that provides development productivity tools and debugging capabilities. See the .NET customer showcase for examples of what people is building with .NET.

Why do I need .NET Framework?

Ans: You need .NET Framework installed to run applications on Windows that were created using .NET Framework. It is already included in many versions of Windows. You only need to download and install .NET Framework if prompted to do so.

How does .NET Framework work?

Ans: .NET Framework applications can be written in many languages like C#, F#, or Visual Basic and compiled to Common Intermediate Language (CIL). The Common Language Runtime (CLR) runs .NET applications on a given machine, converting the CIL to machine code. See Architecture of .NET Framework for more info.

What are the main components/features of .NET Framework?

Ans: The two major components of .NET Framework are the Common Language Runtime (CLR) and the .NET Framework Class Library. The CLR is the execution engine that handles running applications. The Class Library provides a set of APIs and types for common functionality.

How many versions do we have for .NET Framework?

Ans: There are multiple versions of .NET Framework, but each new version adds new features but retains features from previous versions. List of .NET Framework Versions:

.NET Framework 1.0	.NET Framework 1.1	.NET Framework 2.0	.NET Framework 3.0
.NET Framework 3.5	.NET Framework 4	.NET Framework 4.5	.NET Framework 4.5.1
.NET Framework 4.5.2	.NET Framework 4.6	.NET Framework 4.6.1	.NET Framework 4.6.2
.NET Framework 4.7	.NET Framework 4.7.1	.NET Framework 4.7.2	.NET Framework 4.8

Can you have multiple .NET Frameworks installed?

Ans: Some versions of .NET Framework are installed side-by-side, while others will upgrade an existing version (known as an in-place update). In-place updates occur when two .NET Framework versions share the same CLR version. For example, installing .NET Framework 4.8 on a machine with .NET Framework 4.7.2 and 3.5 installed will perform an in-place update of the 4.7.2 installation and leave 3.5 installed separately.

.NET Framework Version	CLR Version
.NET Framework 4.x	4.0
.NET Framework 2.x and 3.x	2.0
.NET Framework 1.1	1.1
.NET Framework 1.0	1.0

How much does .NET Framework cost?

Ans: .NET Framework is free, like the rest of the .NET platform. There are no fees or licensing costs, including for commercial use.

Which version of .NET Framework should I use?

Ans: In most cases, you should use the latest stable release and currently, that's .NET Framework 4.8. Applications that were created with any 4.x version of .NET Framework will run on .NET Framework 4.8. To run an application that was created for an earlier version (for example, .NET Framework 3.5), you should install that version.

What is the support policy for .NET Framework?

Ans: .NET Framework 4.8 is the latest version of .NET Framework and will continue to be distributed with future releases of Windows. If it is installed on a supported version of Windows, .NET Framework 4.8 will continue to also be supported.

Can customers continue using the .NET Framework and get support?

Ans: Yes. Many products both within and outside Microsoft rely on .NET Framework. The .NET Framework is a component of Windows and receives the same support as Windows version which it ships with or on which it is installed. .NET Framework 4.8 is the latest version of .NET Framework and will continue to be distributed with future releases of Windows. If it is installed on a supported version of Windows, .NET Framework 4.8 will continue to also be supported.

Architecture of .NET Core: The two main components of .NET Core are CoreCLR and CoreFX, respectively, which are comparable to the Common Language Runtime (CLR) and the Framework Class Library (FCL) of the .NET Framework's Common Language Infrastructure (CLI) implementation.

1. **CoreFX** is the foundational class libraries for .NET Core. It includes types for collections, file systems, console, JSON, XML, and many others.
2. **CoreCLR** is the .NET execution engine in .NET Core, performing functions such as garbage collection and compilation to machine code. As a CLI implementation of Virtual Execution System (VES), CoreCLR is a complete runtime and virtual machine for managed execution of .NET programs and includes a just-in-time compiler called RyuJIT.

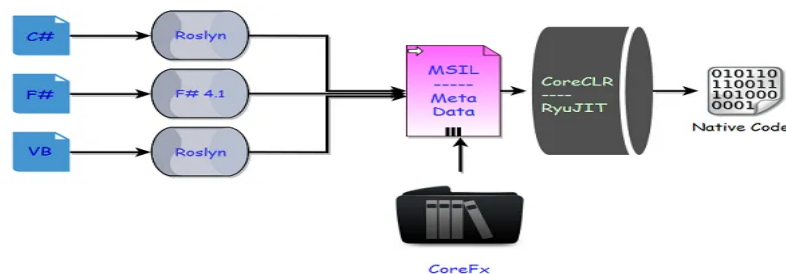
Note: .NET Core releases have a single product version, that is, there is no separate CLR version.

What is CoreFX?

Ans: CoreFX, also referred to as the Unified Base Class Library, consists of the basic and fundamental classes that form the core of the .NET Core platform. These set of libraries comprise the System.* (and to a limited extent Microsoft.*) namespaces. Majority of the .NET Core APIs are also available in the .NET Framework, so you can think of CoreFX as an extension of the .NET Framework Class Library.

What is CoreCLR?

Ans: CoreCLR is the .NET execution engine in .NET Core which is a complete runtime and virtual machine for managed execution of .NET programs and includes a just-in-time compiler called RyuJIT, performing functions such as garbage collection and compilation to machine code. CoreCLR is built from the same code base of the Framework CLR.



What is Roslyn?

Ans: Roslyn is the codename-that-stuck for the open-source compiler for C# and Visual Basic.NET. It is an open source, cross-platform, public language engine for C# and VB. The conversations about Roslyn were already ongoing when “Mads Torgersen” joined Microsoft in 2005 - just before .NET 2.0 would ship. That conversation was about rewriting C# in C# which is a normal practice for programming languages. But there was a more practical and important motivation: the creators of C# were not programming in C# themselves; they were coding in C++.

.NET CORE FAQ's

What is .NET Core?

Ans: The .NET Core platform is a new .NET stack that is optimized for open-source development. .NET Core has two major components. It includes a runtime that is built from the same codebase as the .NET Framework CLR. The .NET Core runtime includes the same GC and JIT (RyuJIT) but doesn't include features like Application Domains or Code Access Security. .NET Core also includes the base class libraries. These libraries are the same code as the .NET Framework class libraries but have been factored to enable to ship as smaller set of libraries. .NET Core refers to several technologies including ASP.NET Core, Entity Framework Core, and more.

What are the characteristics of .NET Core?

Ans: .NET Core has the following characteristics:

- **Cross Platform:** Runs on Windows, macOS, and Linux operating systems.
- **Open Source:** The .NET Core framework is open source, using MIT and Apache 2 licenses. .NET Core is a .NET Foundation project.
- **Modern:** It implements modern paradigms like asynchronous programming, no-copy patterns using struts', and resource governance for containers.
- **Performance:** Delivers high performance with features like hardware intrinsic, tiered compilation, and Span<T>.
- **Consistent Across Environments:** Runs your code with the same behavior on multiple operating systems and architectures, including x64, x86, and ARM.
- **Command-line Tools:** Includes easy-to-use command-line tools that can be used for local development and for continuous integration.
- **Flexible Deployment:** You can include .NET Core in your app or install it side-by-side (user-wide or system-wide installations). Can be used with Docker containers.

What is the composition of .NET Core?

Ans: NET Core is composed of the following parts:

- The .NET Core runtime, which provides a type system, assembly loading, a garbage collector, native interop, and other basic services. .NET Core framework libraries provide primitive data types, app composition types, and fundamental utilities.
- The ASP.NET Core runtime, which provides a framework for building modern, cloud-based, internet-connected apps, such as web apps, IOT apps, and mobile backend.
- The .NET Core SDK and language compilers (Roslyn and F#) that enable the .NET Core developer experience.
- The dotnet command, which is used to launch .NET Core apps and CLI commands. It selects and hosts the runtime, provides an assembly loading policy, and launches apps and tools.

What is .NET Core SDK?

Ans: The .NET Core SDK (Software Development Kit) includes everything you need to build and run .NET Core applications using command line tools or any editor like Visual Studio. It also contains a set of libraries and tools

that allow developers to create .NET Core applications and libraries. It contains the following components that are used to build and run applications:

1. The .NET Core CLI.
2. .NET Core libraries and runtime.
3. The dotnet driver.

What is .NET Core Runtime?

Ans: This includes everything you need to run a .NET Core Application. The runtime is also included in the SDK. When an app author publishes an app, they can include the runtime with their app. If they don't include the runtime, it's up to the user to install the runtime. There are three different runtimes you can install on Windows:

- ASP.NET Core runtime: Runs ASP.NET Core apps. Includes the .NET Core runtime.
- Desktop runtime: Runs .NET Core WPF and .NET Core Windows Forms desktop apps for Windows. Includes the .NET Core runtime.
- .NET Core runtime: This runtime is the simplest runtime and doesn't include any other runtime. It's highly recommended that you install both ASP.NET Core runtime and Desktop runtime for the best compatibility with .NET Core apps.

What's the difference between SDK and Runtime in .NET Core?

Ans: The SDK is all the stuff that is needed for developing a .NET Core application easier, such as the CLI and a compiler. The runtime is the “virtual machine” that hosts/runs the application and abstracts all the interaction with the base operating system.

What is the difference between .NET Core and .NET Framework?

Ans: .NET Core and .NET Framework share many of the same components and you can share code across the two. Some key differences include:

- .NET Core is cross-platform and runs on Linux, macOS, and Windows. .NET Framework only runs on Windows.
- .NET Core is open-source and accepts contributions from the community. The .NET Framework source code is available but does not take direct contributions.
- The majority of .NET innovation happens in .NET Core.
- .NET Framework is included in Windows and automatically updated machine-wide by Windows Update. .NET Core is shipped independently.

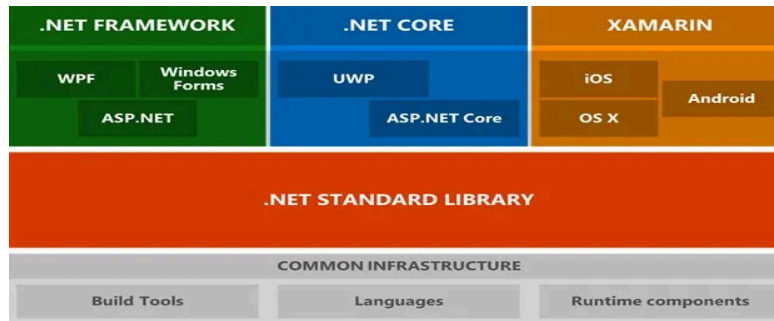
What is the difference between .NET Core and Mono?

Ans: To be simple, Mono is third party implementation of .Net Framework for Linux/Android/iOS and .Net Core is Microsoft's own implementation for same.

What's the difference between .NET Core, .NET Framework, and Xamarin?

Ans: difference between .NET Core, .NET Framework and Xamarin are:

- .NET Framework is the “traditional” flavor of .NET that's distributed with Windows. Use this when you are building a desktop Windows or UWP app or working with older ASP.NET 4.8.
- .NET Core is cross-platform .NET that runs on Windows, Mac, and Linux. Use this when you want to build console or web apps that can run on any platform, including inside Docker containers.
- Xamarin is used for building mobile apps that can run on iOS, Android, or Windows Phone devices.



What is the support policy to .NET Core?

Ans: .NET Core is supported by Microsoft on Windows, macOS, and Linux. It's updated for security and quality regularly (the second Tuesday of each month). .NET Core binary distributions from Microsoft are built and tested on Microsoft-maintained servers in Azure and follow Microsoft engineering and security practices.

Red Hat supports .NET Core on Red Hat Enterprise Linux (RHEL). Red Hat builds .NET Core from source and makes it available in the Red Hat Software Collections. Red Hat and Microsoft collaborate to ensure that .NET Core works well on RHEL (Red Hat Enterprise Linux).

Tizen (developed by Samsung) supports .NET Core on Tizen platforms.

How much does .NET Core cost?

Ans: .NET Core is an open-source and cross-platform version of .NET that is maintained by Microsoft and the .NET community on GitHub. All aspects of .NET Core are open source including class libraries, runtime, compilers, languages, ASP.NET Core web framework, Windows desktop frameworks, and Entity Framework Core data access library. There are no licensing costs, including for commercial use.

What is GitHub?

Ans: GitHub is a code hosting platform for collaboration and version control. It is a repository (usually abbreviated to “repo”) is a location where all the files for a particular project are stored which lets you (and others) work together on projects. Each project has its own repo, and you can access it with a unique URL. Git is an open-source version control system that was started by “Linus Torvalds” - the same person who created Linux. Git is similar to other version control systems—Subversion, CVS, and Mercurial to name a few.

What is the release schedule for .NET Core?

Ans: .NET Core 2.1 and .NET Core 3.1 are the current LTS releases made available on August 2018 and December 2019, respectively. After .NET Core 3.1, the product will be renamed to .NET and LTS releases will be made available every other year in November. So, the next LTS release will be .NET 6, which will ship in November 2021. This will help customers plan upgrades more effectively.

How many versions do we have for .NET Core?

Ans: This table tracks release dates and end of support dates for .NET Core versions.

Version	Original Release Date	Support Level	End of Support
.NET Core 3.1	December 3, 2019	LTS	December 3, 2022
.NET Core 3.0	September 23, 2019	EOL	March 3, 2020
.NET Core 2.2	December 4, 2018	EOL	December 23, 2019

.NET Core 2.1	May 30, 2018	LTS	August 21, 2021
.NET Core 2.0	August 14, 2017	EOL	October 1, 2018
.NET Core 1.1	November 16, 2016	EOL	June 27, 2019
.NET Core 1.0	June 27, 2016	EOL	June 27, 2019

EOL (end of life) releases have reached end of life, meaning it is no longer supported and recommended moving to a supported version.

LTS (long-term support) releases have an extended support period. Use this if you need to stay supported on the same version of .NET Core for longer.

.NET 5 (.NET Core vNext)

.NET 5 is the next step forward with .NET Core. This new project and direction are a game-changer for .NET. With .NET 5, your code and project files will look and feel the same no matter which type of app you're building. You'll have access to the same runtime, API, and language capabilities with each app. The project aims to improve .NET in a few keyways:

- Produce a single .NET runtime and framework that can be used everywhere and that has uniform runtime behaviors and developer experiences.
- Expand the capabilities of .NET by taking the best of .NET Core, .NET Framework, Xamarin and Mono.
- Build that product out of a single code-base that developers (Microsoft and the community) can work on and expand together and that improves all scenarios.

Microsoft skipped the version 4 because it would confuse users that are familiar with the .NET Framework, which has been using the 4.x series for a long time. Additionally, they wanted to clearly communicate that .NET 5 is the future for the .NET platform. They are also taking the opportunity to simplify naming. They thought that if there is only one .NET going forward, they don't need a clarifying term like "Core". The shorter name is a simplification and communicates that .NET 5 has uniform capabilities and behaviors. Feel free to continue to use the ".NET Core" name if you prefer it.

Runtime experiences:

Mono is the original cross-platform implementation of .NET. It started out as an open-source alternative to .NET Framework and transitioned to targeting mobile devices as iOS and Android devices became popular. Mono is the runtime used as part of Xamarin.

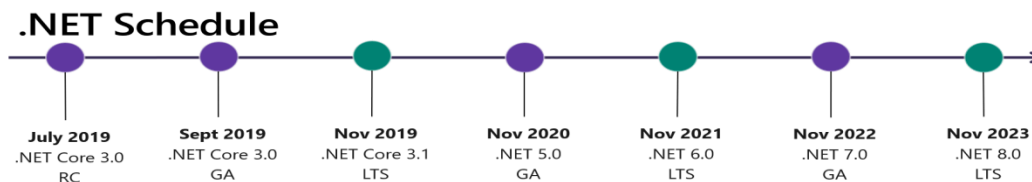
Core CLR is the runtime used as part of .NET Core. It has been primarily targeted at supporting cloud applications, including the largest services at Microsoft, and now is also being used for Windows desktop, IoT and machine learning applications.

Taken together, the .NET Core and Mono runtimes have a lot of similarities (they are both .NET runtimes after all) but also valuable unique capabilities. It makes sense to make it possible to pick the runtime experience you want. They are in the process of making Core CLR and Mono drop-in replacements for one another and will make it as simple as a build switch to choose between the different runtime options.

.NET – A unified platform



.NET Schedule: .NET 5 is shipped in November 2020, and then they intend to ship a major version of .NET once a year, every November:



- .NET Core 3.0 release in September
- .NET Core 3.1 = Long Term Support (LTS)
- .NET 5.0 release in November 2020
- Major releases every year, LTS for even numbered releases
- Predictable schedule, minor releases if needed

The .NET 5 Project is an important and exciting new direction for .NET. You will see .NET become simpler but also have broader and more expansive capability and utility. All new development and feature capabilities will be part of .NET 5, including new C# versions. We see a bright future ahead in which you can use the same .NET APIs and languages to target a broad range of application types, operating systems, and chip architectures. It will be easy to make changes to your build configuration to build your applications differently, in Visual Studio, Visual Studio for Mac, Visual Studio Code, and Azure DevOps or at the command line.

.NET 5.0 is the next major release of .NET Core following 3.1. They named this new release .NET 5.0 instead of .NET Core 4.0 for two reasons:

1. They skipped version numbers 4.x to avoid confusion with .NET Framework 4.x.
2. They dropped “Core” from the name to emphasize that this is the main implementation of .NET going forward. .NET 5.0 supports more types of apps and more platforms than .NET Core or .NET Framework.

Note: ASP.NET Core 5.0 is based on .NET 5.0 but retains the name “Core” to avoid confusing with ASP.NET MVC 5. Likewise, Entity Framework Core 5.0 retains the name “Core” to avoid confusing it with Entity Framework 5 and 6.

The .NET 5 projects is an important and exciting new direction for .NET. You will see .NET become simpler but also have broader and more expansive capability and utility. All new development and feature capabilities will be part of .NET 5, including new C# versions.

We see a bright future ahead in which you can use the same .NET APIs and languages to target a broad range of application types, operating systems, and chip architectures. It will be easy to make changes to your build configuration to build your applications differently, in Visual Studio, Visual Studio for Mac, Visual Studio Code, and Azure DevOps or at the command line.

The current and latest version of .NET is 8.0 that was launched in November, 2023 with lots of exciting features for building Platform Independent applications targeting Windows, Linux, and Mac.

C# Programming Language

C# (pronounced see sharp, like the musical note $\sharp$, but written with the number sign) is a general-purpose, programming language encompassing strong typing, lexically scoped, imperative, declarative, functional, generic, object-oriented (class-based), and component-oriented programming disciplines. It was developed around 2000 by Microsoft as part of its .NET initiative and later approved as an international standard by ECMA (European Computers Manufacturing Association) in 2002 and ISO (International Organization for Standardization) in 2003.

The name “C Sharp” was inspired by the musical notation where sharp indicates that the written note should be made a semitone i.e., higher in pitch. This is like the language name of C++, where “++” indicates that a variable should be incremented by 1 after being evaluated. The sharp symbol also resembles a ligature of 4 “+” symbols (in a two-by-two grid), further implying that the language is an increment of C++. Due to technical limitations of display and the fact that the sharp symbol is not present on most keyboard layouts, the number sign “#” was chosen to approximate the sharp symbol in the written name of the programming language.

C# was designed by Anders Hejlsberg, and its development team is currently led by Mads Torgersen. C# has Procedural; Object Oriented syntax based on C++ and includes influences from several programming languages, most importantly Delphi and Java with a particular emphasis on simplification. The first version of C# is 1.0 and the most recent stable version is 13.0, which was released in November 2024.

History: During the development of the .NET, the libraries were originally written using a managed code compiler system called “Simple Managed C” (SMC). In January 1999, Anders Hejlsberg formed a team to build a new language at the time called “COOL”, which stood for “C-like Object Oriented Language”.

Microsoft had considered keeping the name “COOL” as the final name of the language but chose not to do so for trademark reasons. By the time .NET project was publicly announced at the July 2000 in Professional Developers Conference, the language COOL had been renamed “C#”, and the libraries and ASP.NET runtime had been ported to “C#”.

Anders Hejlsberg is C#'s principal designer and lead architect at Microsoft and was previously involved with the design of Turbo Pascal, Borland Delphi, Visual J++ and Type Script languages also. In interviews and technical papers, he has stated that flaws in most major programming languages like C++, Java, Delphi, and Smalltalk drove the design of the C# language.

Design Goals: The ECMA standard lists these design goals for C#.

- The language is intended to be a simple, modern, general-purpose, and object-oriented language.
- The language, and implementations thereof, should provide support for software engineering principles such as strong type checking, array bounds checking, detection of attempts to use uninitialized variables, and automatic garbage collection.
- Support for internationalization is very important.
- The language is intended for use in developing software components suitable for deployment in distributed environments.
- Portability is very important for programmers, especially those already familiar with C and C++.
- C# is intended to be suitable for writing applications for both hosted and embedded systems, ranging from the very large that use sophisticated operating systems, down to the very small having dedicated functions.

Versions of the language: 1.0, 1.2, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0

New features in C# 2.0:

- Generics
- Partial types
- Anonymous methods
- Iterators
- Nullable value types
- Getter/setter separate accessibility
- Static classes
- Delegate inference
- Null coalescing operator

New features in C# 3.0:

- Implicitly typed local variables
- Object initializers
- Collection initializers
- Auto-Implemented properties
- Anonymous types
- Extension methods
- Query expressions
- Lambda expressions
- Expression trees
- Partial methods

New features in C# 4.0:

- Dynamic binding
- Named and optional arguments
- Generic covariant and contravariant
- Embedded interop types

New features in C# 5.0:

- Asynchronous methods
- Caller info attributes
- Compiler API

New features in C# 6.0:

- Static imports
- Exception filters
- Auto-property initializers
- Default values for getter-only properties
- Expression bodied members
- Null propagator
- String interpolation

- nameof operator
- Index initializers
- Await in catch/finally blocks

New features in C# 7.0:

- Out variables
- Tuples and deconstruction
- Pattern matching
- Local functions
- Expanded expression bodied members
- Ref locals and returns
- Discards
- Binary Literals and Digit Separators
- Throw expressions

New features in C# 7.1:

- Async main method
- Default literal expressions
- Inferred tuple element names
- Pattern matching on generic type parameters

New features in C# 7.2:

- Techniques for writing safe efficient code
- Non-trailing named arguments
- Leading underscores in numeric literals
- private protected access modifier
- Conditional ref expressions

New features in C# 7.3:

- Accessing fixed fields without pinning

- Reassigning ref local variables
- Using initializers on stackalloc arrays
- Using fixed statements with any type that supports a pattern
- Using additional generic constraints

New features in C# 8.0:

- Readonly members
- Default interface methods
- Pattern matching enhancements:
 - Switch expressions
 - Property patterns
 - Tuple patterns
 - Positional patterns
- Using declarations
- Static local functions
- Disposable ref structs
- Nullable reference types
- Asynchronous streams and asynchronous disposable
- Indices and ranges
- Null-coalescing assignment
- Unmanaged constructed types
- Enhancement of interpolated verbatim strings

New features in C# 9.0 (Supported on .NET 5 only):

- Records
- Init only setters
- Top-level statements
- Pattern matching enhancements
- Native sized integers

- Function pointers
- Suppress emitting localsinit flag
- Target-typed new expressions
- static anonymous functions
- Target-typed conditional expressions
- Covariant return types
- Extension GetEnumerator support for foreach loops
- Lambda discard parameters
- Attributes on local functions
- Module initializers
- New features for partial methods

New features in C# 10 (Supported on .NET 6 only):

- Record structs
- Improvements of structure types
- Interpolated string handlers
- global using directives
- File-scoped namespace declaration
- Extended property patterns
- Improvements on lambda expressions
- Allow const interpolated strings
- Record types can seal ToString()
- Improved definite assignment
- Allow both assignment and declaration in the same deconstruction
- Allow AsyncMethodBuilder attribute on methods
- CallerArgumentExpression attribute
- Enhanced #line pragma

New features in C# 11 (Supported on .NET 7 only):

- Raw string literals
- Generic math support
- Generic attributes
- UTF-8 string literals
- Newlines in string interpolation expressions

- List patterns
- File-local types
- Required members
- Auto-default structs
- Pattern match `Span<char>` on a constant string
- Extended `nameof` scope
- Numeric `IntPtr`
- `ref` fields and scoped `ref`
- Improved method group conversion to delegate
- Warning wave 7

New features in C# 12 (Supported on .NET 8 only):

- Primary constructors
- Collection expressions
- `ref readonly` parameters
- Default lambda parameters
- Alias any type
- Inline arrays
- Experimental attribute
- Interceptors

New features in C# 13 (Supported on .NET 9 only):

- Params Collections
- New lock type and semantics.
- New escape sequence - `\e`.
- Implicit indexer access in object initializers
- Enable `ref locals` and `unsafe contexts` in iterators and `async methods`
- Enable `ref struct` types to implement interfaces.
- Partial properties and indexers are now allowed in partial types.

.NET Framework, .NET Core and .NET 5 support for C# language versions:

Target Runtime	Version	C# Language Version
.NET	9.x	C# 13
.NET	8.x	C# 12
.NET	7.x	C# 11
.NET	6.x	C# 10
.NET	5.x	C# 9.0
.NET Core	3.x	C# 8.0
.NET Core	2.x	C# 7.3
.NET Framework	all	C# 7.3

Writing a program by using different Programming Approaches

To write a program we generally follow 2 different approaches in the industry:

1. Procedural Programming Approach
2. Object Oriented Programming Approach

Procedural Programming Approach: This is a very traditional approach followed by the industry to develop applications till 70's. E.g.: COBOL, Pascal, FORTRAN, C, etc.

In this approach a program is a collection of **members** like **variables** and **functions**, and the **members** that are defined inside the program should be explicitly called for execution and we do that calling from “**main**” function because it is the **entry point** of any **program** that is developed by using any **programming language**.

C Program

-Collection of Members (Variables & Functions)

void main() <= Entry Point

{

-Call the members from here for execution

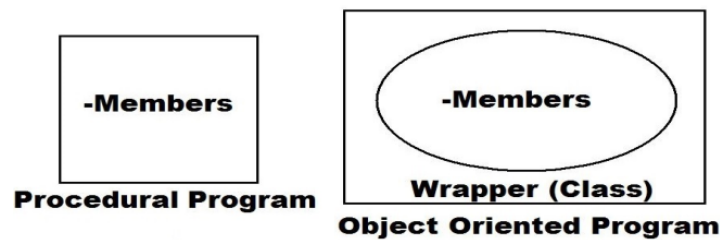
}

Note: the drawbacks of **procedural programming** languages are they don't provide **security** and **re-usability**.

Object Oriented Programming Approach: This came into existence in late 70's to overcome the drawbacks of **Procedural Programming Language's** by providing **Security** and **Re-usability**.

E.g.: C++, Python, Java, C#, etc.

In an **Object-Oriented Programming** approach also, a program is a Collection of **Members** like **Variables** and **Functions** only, but the main difference between **Object Oriented Languages** and **Procedural Languages** is, here to protect the **members** of a program we put them under a **container** or **wrapper** known as a **"class"**.



What is a class?

Ans: it is a **user-defined type** very much like **structures** we have learnt in **C** language, i.e., by using these we can define **new types**, whereas the difference between the two are, structure in **"C"** language can contain only **variables** in it but **class** of **Object-Oriented** languages can contain both **variables** and **functions** also.

Syntax to define Class and Structure:

struct <Name>	class <Name>
{	{
-Variables	-Variables
};	-Functions
	};

Example:

struct Student	class Employee
{	{
int Id;	int Id;
char Name[25];	string Name, Job;
float Marks, Fees;	float Salary;
};	-Can be defined with functions also
	};

In the above case **int**, **float** and **char** are pre-defined **structures** whereas **string** is a pre-defined **class** which we are calling them as **types**, same as that **Student** and **Employee** are also **types** (**user-defined**). The other difference between **int**, **float**, **char**, and **string** types, as well as **Student** and **Employee** types is the 1st 4 are **scalar types** which can hold 1 and only 1 value under them whereas the next 2 are **complex types** which can hold more than 1 value under them.

How to consume a type?

Ans: types can't be consumed directly because they do not have any memory allocation.

int = 100; **//Invalid**

So, to consume a type first we need to create a copy of that type:

int i = 100; **//Valid**

Note: In the above case **"i"** is a copy of pre-defined type **int** for which memory gets allocated and the above rule of types can't be consumed directly, applies both to **pre-defined** and **user-defined** types also.

int i; **//i is a copy of pre-defined type int**

```
string s;           //s is a copy of pre-defined type string
Student ss;         //ss is a copy of user-defined type Student
Employee emp;       //emp is a copy of user-defined type Employee
```

Note: Generally, copies of **scalar types** like **int**, **float**, **char**, **bool**, **string**, etc. are known as **variables**, whereas copies of **complex types** which we have defined like **Student** and **Employee** are known as **objects** or **instances**.

Conclusion: After defining a **class** or **structure** if we want to consume them, first we need to create a **copy** of them and then only the **memory** which is required for execution gets allocated and by using that copy (**Object** or **Instance**) only we can call members that are defined under them.

CPP Program

```
class Example
{
    -Collection of Members (Variables & Functions)
};
void main() <= Entry Point
{
    -Create the object of class
    -Call members of class by using the object created
}
```

Note: CPP is the first **Object Oriented Programming Language** which came into existence, but still, it suffers from a **criticism** that it is **not fully** Object-Oriented Language; because in CPP Language we can't write main function inside of the class and according to the standards of **Object-Oriented Programming** each and every **Member** of the **Program** should be inside of the **Class**.

The reason why we write **main function** outside of **class** is, if it is defined inside of the **class** then it becomes a **member** of that **class** and **members** of a **class** can be called only by using **object** of that **class**, but unfortunately we create **object** of **class** inside **main** function only, so until and unless **object** of class is created **main** function can't be called and at the same time until and unless **main** function starts its execution, **object** creation will not take place and this is called as **"Circular Dependency"** and to avoid this problem, in **CPP Language** we write **main** function outside of the **class**.

Object Oriented Programming in Java: Java language came into existence in the year **1995** and here also a class is a collection of members like **variables** and **methods**. While designing the language, designers have taken it as a challenge that their language should not suffer from the criticism that it is not fully Object Oriented, so they want **"main"** method of the class to be present inside of the class only and still execute without the need of class object and to do that they have divided members of a class into 2 categories, like:

- Non-static Members
- Static Members

Every member of a class is by default a non-static member only and what we have learnt till now in **C** or **C++** Languages is also about non-static members only, whereas if we prefix any of those members with **static** keyword, we call them as **Static Members**.

```

class Test
{
    int x = 100;           //Non-Static Member
    static int y = 200;    //Static Member
}

```

Note: Static members of the class doesn't require object of that class for both **initialization** and **execution** also, whereas non-static members require it, so in Java Language "**main method**" is defined inside of the class only but declared as **static**, so even if it is inside of the class also it can start the execution without the need of class object.

Java Program

```

class Example
{
    -Collection of Members (Static & Non-Static)
    public static void main(string[] args)
    {
        -Create the object of class
        -Call non-static members of class by using the object created
        -Call static members of class by prefixing the class name
    }
}

```

Object Oriented Programming in C#: C# Language came into existence after **Java** and was influenced by **Java**, so in **C#** Language also the programming style will be same as **Java** i.e., defining **Main** method inside the class by declaring it as "**static**".

C# Program

```

class Example
{
    -Collection of Members (Static & Non-Static)
    static void Main()
    {
        -Create the instance of class
        -Call non-static members of class by using the instance created
        -Call static members of class by prefixing the class name
    }
}

```

Note: In **Java** or **.NET Languages** if at all the class contains only **Main** method in it, we **don't** require creating **object** or **instance** of that class to run the **class**.

Visual Studio Installation:

Step 1: Visit the site: <https://visualstudio.microsoft.com/downloads>.

Step 2: Download and install the latest version of **Visual Studio Community Edition** i.e., VS 2022 (Version 17) latest. When you click on the download button it will download the installer for downloading and we find it in the bottom of LHS in the Browser or you can also find it in downloads folder, click on it to launch the installer.

Step 3: Once the installer is loaded and opened choose the below options in it:

1. Workloads:

- ☐ ASP.NET and web development
 - ☐ Azure development
 - ☐ .NET desktop development
 - ☐ Data storage and processing
 - ☐ Visual Studio extension development
2. **Individual Components:** Select all the Checkbox's under ".NET" option except "Out of support" Checkbox's and "ML.NET Model Builder" CheckBox and Android, Mac or iOS and Linux options. Now scroll down and go to "Code Tools" section and under that select the CheckBox "LINQ to SQL Tools".
 3. **Language Packs:** Don't change anything here (default is English).
 4. **Installation Folders:** Don't change anything here also.

Step 4: Click on Download and Install button to complete the installation.

Writing programs by using C# Language: C# language has lot of standards to be followed while writing code, as following:

1. It's a case sensitive language so we need to follow the below rules and conventions:
 - I. All keywords in the language must be in lower case (rule).
 - II. While consuming the libraries, names will be in Pascal Case (rule). E.g.: WriteLine, ReadLine
 - III. While defining our own classes and members to name them we can follow any casing pattern, but Pascal case is suggested (convention).
2. A C# program should be saved with ".cs" extension.
3. We can use any name as a file name under which we write the program, but class name is suggested to be used as file name also.
4. To write programs in C# we use an IDE (Integrated Development Environment) known as Visual Studio but we can also write them by using any text editor like Notepad also.

Syntax to define a class:

```
[<modifiers>] class <Name>
{
    -Define Members here           [] => Optional
}
```

- **Modifiers** are some special keywords that can be used on a class like public, internal, static, abstract, partial, sealed, etc.
- **class** is a keyword to tell that we are defining a class just like we used, struct keyword to define a structure in C Language.
- **<Name>** refers to name of the class for identification.
- **Members** refer to contents of the class like fields, methods, etc.

Syntax to define Main Method in the class:

```
static void Main( [string[] args] )
{
    -Stmt's
}
```


- **static** is a keyword we use to declare a member as **static member** and if a member is declared as **static**, **instance** of the class is not required to **call** or **execute** it. In **C#** - **Main** method should be declared **static** to start the execution from there.
- **void** is a keyword to specify that the method is **non-value** returning.
- **Main** is name of the method, which can't be changed and more over it should be in **Pascal Case** only.
- If required (**optional**) we can pass **parameters** to **Main** method but it should be of type **string array** only.
- Statements refer to the **logic** we want to implement.

Writing the first program in C# using Notepad:

Step 1: Open **Notepad** and write the below code in it:

```
class First
{
    static void Main()
    {
        System.Console.Clear();
        System.Console.WriteLine("My first C# program using Notepad.");
    }
}
```

Step 2: Saving the program.

Create a **folder** on any drive of your computer with the name **"CSharp"** and save the above file into that folder naming it as **"First.cs"**.

Step 3: Compilation of the program.

We need to compile our **C#** program by using **C# Compiler** at **"Developer Command Prompt"** provided along with the **Visual Studio** software, and to do that go to **Windows Search** and search for **"Developer Command Prompt for VS"**, click on it to open. Once it is open it will be pointing to the location where **Visual Studio** software is installed, so change to the location where you have created the folder and compile the program as below:

Syntax: **csc <File Name>**

E.g.: **<drive>:\CSharp> csc First.cs**

Once the program is compiled successfully it generates an output file with the name **First.exe** that contains **"CIL (Common Intermediate Language) or MSIL (Microsoft Intermediate Language) Code"** in it which we need to execute.

Step 4: Execution of the program.

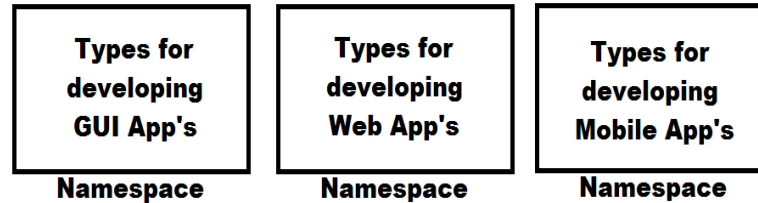
Now at the same Command Prompt we can run our First.exe file as below:

E.g.: **<drive>:\CSharp> First**

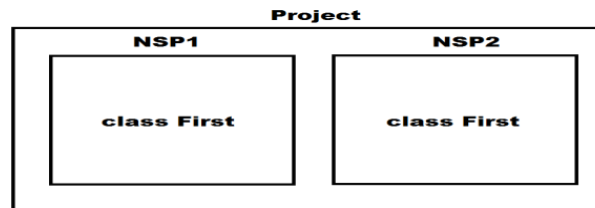
System.Console.WriteLine & System.Console.Clear: **Console** is a pre-defined class under the **libraries** of our language which provides with a set of **static** members using which we can perform **IO** operations on the standard **IO** devices. **WriteLine** is a method in the class **Console** to display output on the monitor, and apart from **WriteLine** method there are many other methods present in the class **Console** like: **Write**, **Read**, **ReadLine**, **ReadKey**, **Clear**, etc. and all these methods are also **static**, so we can call them directly by prefixing the class name.

System is a **namespace**, and a **namespace** is a logical container for types like: **Class**, **Structure**, **Interface**, **Enum** and **Delegate**, and we use these namespaces in a language for 2 reasons:

1. **Grouping** related types i.e., types that are designed for developing similar kind of App's are grouped together under a namespace for easy access and identification as following:



2. To overcome the **naming - collision** i.e., if a project contains multiple types with the same name, we can overcome conflict between names by putting them under separate namespaces, as following:



Note: Every **pre-defined type** in our **Libraries** is defined under some **namespace** and we can also define types under **namespaces**, and we will learn this process while working with **Visual Studio**.

If a type is defined under any namespace, then, whenever and wherever we want to consume the type, we need to prefix namespace name to type name, and this is the reason why in our previous program we have referred to “**Console**” class as “**System.Console**”. To overcome the problem of prefixing namespace, name every time before the type, we are provided with an option of “**importing a namespace**” which is done by “**using directive**” as following:

Syntax: **using <namespace>;**
using System;
using Microsoft.VisualBasic;

Note: We can import any no. of namespaces as above but each import should be a separate statement.

What is a directive?

Ans: directive in our language is an **instruction** that is given to the **compiler** which it must follow, by **importing** the **namespace** we are telling the **C# compiler** that types consumed in the program are from the **imported** namespace.

To test the process of importing a namespace write the below code in Notepad and execute:

```
using System;  
class Second  
{  
    static void Main()  
    {  
        Console.Clear();  
        Console.WriteLine("Importing a namespace.");  
    }  
}
```

}

Note: If there are **multiple** namespaces containing a **type** with same name then it's not possible to consume those types by **importing** the **namespace**, and in such cases it's mandatory to refer to each type by **prefixing** the **namespace** name to them as following:

E.g.: **NSP1.First** **NSP2.First**

using static directive: This is a new feature introduced in "**C# 6.0**" which allows us to **import** a **type** and then consume all the **static members** of that type **without** a type name prefix.

Syntax: **using static <namespace.type>;**
using static System.Console;

To test the process of importing a class write below code in Notepad and execute:

```
using static System.Console;
class Third
{
    static void Main()
    {
        Clear();
        WriteLine("Importing a type.");
    }
}
```

Data Types in C#

<u>C# Types</u>	<u>CIL Types</u>	<u>Size/Capacity</u>	<u>Default Value</u>
<u>Integer Types</u>			
byte	System.Byte	1 byte (0 - 255)	0
short	System.Int16	2 bytes (-2 ^ 15 to 2 ^ 15 - 1)	0
int	System.Int32	4 bytes (-2 ^ 31 to 2 ^ 31 - 1)	0
long	System.Int64	8 bytes (-2 ^ 63 to 2 ^ 63 - 1)	0
sbyte	System.SByte	1 byte (-128 to 127)	0
ushort	System.UInt16	2 bytes (0 to 2 ^ 16 - 1)	0
uint	System.UInt32	4 bytes (0 to 2 ^ 32 - 1)	0
ulong	System.UInt64	8 bytes (0 to 2 ^ 64 - 1)	0
<u>Decimal Types</u>			
float	System.Single	4 bytes	0
double	System.Double	8 bytes	0
decimal	System.Decimal	16 bytes	0
<u>Boolean Type</u>			
bool	System.Boolean	1 byte	False
<u>DateTime Type</u>			
DateTime	System.DateTime	8 bytes	01/01/0001 00:00:00
<u>Unique Identifier Type</u>			
Guid	System.Guid	32 bytes	00000000-0000-0000-0000-000000000000

Character Types			
char	System.Char	2 bytes	\0
string	System.String		Null
Base Type			
object	System.Object		Null

- All the above types are known as primitive/pre-defined types i.e., they are defined under the libraries of our language which can be consumed from anywhere.
- All C# Types after compilation of source code gets converted into CIL Types and in CIL Format these types are either classes or structures defined under the "System" namespace. String and Object types are classes, whereas rest of the other 15 types, are structures.
- short, int, long and sbyte types can store signed integer (Positive or Negative) values whereas ushort, uint, ulong and byte types can store un-signed integer (Pure Positive) values only.
- Guid is a type used for storing Unique Identifier values that are loaded from SQL Server Database, which is a 32-byte alpha-numeric string holding a Global Unique Identifier value and it will be in the following format: 00000000-0000-0000-0000-000000000000.
- The size of char type has been increased to 2 bytes for giving support to Unicode characters i.e., characters of languages other than English.
- We are aware that every English language character has a numeric value representation known as ASCII; characters of languages other than English also have that numeric value representation and we call it as Unicode.

char ch = 'A'; => ASCII (65) => Binary (1000001)

char ch = 'अ'; => Unicode (2309) => Binary (100100000101)

- Just like ASCII values converts into binary for storing, by a computer; Unicode values also converts into binary, but the difference is ASCII requires 1 byte of memory for storing its value whereas Unicode requires 2 bytes of memory for storing its value.
- String is a variable length type i.e.; it doesn't have any fixed size and its size varies based on the value that is assigned to it.
- Object is a parent of all the types, so capable of storing any type of value in it and more over it is also a variable length type.

Syntax to declare fields and variables in a class:

[<modifiers>] [const] [readonly] <type> <name> [=default value] [,...n]

```
class Test
{
    int x;                //Field (Global Scope)
    static void Main()
    {
        int y = 100;      //Variable (Local Scope)
    }
}
```

- “<type>” refers to the data type of field or variable we want to declare, and it can be any of the 17 types we discussed above.

- “<name>” refers to the name of the field or variable and it should be unique within the location.
E.g.: int i; float f; bool b; char c; string s; object o; DateTime dt; Guid id;
- Fields and variables can be initialized with any value at the time of their declaration and if they are not initialized then every **field** has a default value which is “0” for all numeric types, “false” for bool type, “\0” for char type, “00000000-0000-0000-0000-000000000000” for Guid type, “01/01/0001 00:00:00” for DateTime type and “null” for string and object types.

Note: Variables doesn't have any default value so it is must to initialize them while declaration or before consumption.

E.g.: int x = 100;

- Modifiers are generally used to define the scope of a field i.e., from where it can be accessed, and the default scope for every member of a class in our language is **private** which can either be changed to **public** or **internal** or **protected**.
- “const” is a keyword to declare a constant and those constants values can't be modified once after their declaration:

const float pi = 3.14f; //Declaration and Initialization

- “readonly” is a keyword to declare a field as readonly and these readonly field values also can't be modified, but after initialization:

readonly float pi; //Declaration
pi = 3.14f; //Initialization

Note: decimal values are by default treated as double by the compiler, so if we want to use them as float the value should be suffixed with character “f” and “m” to use the value as decimal.

float pi = 3.14f; double pi = 3.14; decimal pi = 3.14m;

using System;
class TypesDemo

```
{
    static int x;                   //Field
    static void Main()
    {
        Console.Clear();
        Console.WriteLine("Field x value is: " + x + " and it's type is: " + x.GetType());

        int y = 10;                 //Variable
        Console.WriteLine("Variable y value is: " + y + " and it's type is: " + y.GetType());
        float f = 3.14f;           //Variable
        Console.WriteLine("Variable f value is: " + f + " and it's type is: " + f.GetType());
        double d = 3.14;           //Variable
        Console.WriteLine("Variable d value is: " + d + " and it's type is: " + d.GetType());
        decimal de = 3.14m;       //Variable
        Console.WriteLine("Variable de value is: " + de + " and it's type is: " + de.GetType());
        bool b = true;             //Variable
        Console.WriteLine("Variable b value is: " + b + " and it's type is: " + b.GetType());
        Char ch = 'A';             //Variable
        Console.WriteLine("Variable ch value is: " + ch + " and it's type is: " + ch.GetType());
    }
}
```

}

Note: `GetType` is a pre-defined method which returns the **type (CIL Format)** of a **variable** or **field** or **instance** on which it is called.

Data Types are divided into 2 categories:

1. Value Types
2. Reference Types

Value Types:

- All fixed length types come under the category of value types. E.g.: **integer types**, **decimal types**, **bool type**, **char type**, **DateTime type** and **Guid type**.
- Value types will store their values on **"Stack"** and **Stack** is a **Data Structure** that works on a principal **"First in Last out (FILO)"** or **"Last in First out (LIFO)"**.
- Each program when it starts the execution, a **Stack** will be created and given to that program for storing its values and in the end of program's execution **Stack** is destroyed.
- Every program will be having its own **Stack** for storing values that are associated with the program and no 2 programs can share the same **Stack**.
- **Stack** is under the control of **Operating System** and memory allocation is performed only in **fixed length** i.e., once allocated that is **final** which can't either be **increased** or **decreased** also.

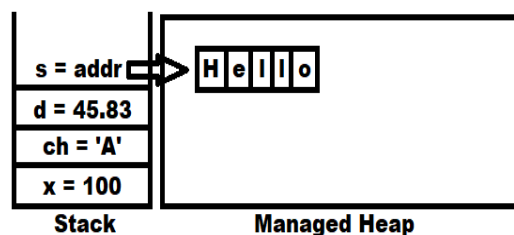
Reference Types:

- All variable length types come under the category of **reference types** and these types will store their values on **"Heap"** memory and their **address** or **reference** is stored on **"Stack"**. E.g.: **String** and **Object** types.
- **Heap** memory doesn't have any limitations like **Stack**, and it provides a beautiful feature called **Dynamic Memory Management** and because of that, all programs in execution can share the same **Heap**.
- In older programming languages like **C** and **C++**, **Heap** memory is under developer's control, whereas in modern programming languages like **Java** and **.NET**, **Heap** memory is under control of a special component known as **"Garbage Collector"**, so we call **Heap** memory in these languages as **"Managed Heap"**.

Suppose if we declare fields in a program as following:

```
int x = 100;   char ch = 'A';   double d = 45.83;   string s = "Hello";
```

Then memory is allocated for them as following:



Nullable Value Types: These are introduced in **C# 2.0** for storing **null** values under **value types** because; by default **value types** can't store **null** values under them whereas **reference types** can store **null** values under them.

```
string str = null;           //Valid  
object obj = null;          //Valid
```

```
int i = null;           //Invalid
decimal d = null;      //Invalid
```

To overcome the above problem **nullable value types** came into picture and if we want a **value type** as **nullable** we need to **suffix** the type with “?” and declare it as following:

```
int? i = null;         //Valid
decimal? d = null;     //Valid
```

Implicitly typed variables: This is a new feature introduced in **C# 3.0**, which allows declaring variables by using “var” keyword, so that the **type** of that **variable** is identified based on the value that is **assigned** to it, for example:

```
var i = 100;           //i is of type int
var f = 3.14f;         //f is of type float
var b = true;          //b is of type bool
var s = "Hello";       //s is of type string
```

Note: While using implicitly typed variables we have 3 restrictions:

1. We can't declare these variables with-out initialization. E.g.: `var x;` //Invalid
2. We can use “var” only on variables but not on fields.
3. Changing the type after declaration is not possible. E.g.: `var i = 100;` //Valid
`i = 34.56;` //Invalid

Dynamic Type: This is a new type introduced in **C# 4.0**, which is very similar to implicitly typed variables we discussed above, but here in place of “var” keyword we use “dynamic”.

Differences between “var” and “dynamic”

<u>Var</u>	<u>Dynamic</u>
Type identification is performed at compilation time.	Type identification is performed at runtime.
Once the type is identified can't be changed to a new type again. <pre>var v = 100; //v is of type int v = 34.56; //Invalid</pre>	We can change the type of dynamic with a new value in every statement. <pre>dynamic d = 100; //d is of type int d = 34.56; //d is of type double (Valid)</pre>
Can't be declared with-out initialization. <pre>var v; //Invalid</pre>	Declaration time initialization is only optional. <pre>dynamic d; //Valid d = 100; //d is of type int d = false; //d is of type bool d = "Hello"; //d is of type string d = 34.56; //d is of type double</pre>
Can be used for declaring variables only.	Can be used for declaring variables and fields also.

```
using System;
class VarDynamic
{
    static void Main()
    {
        var i = 100;
```

```

Console.WriteLine(i.GetType());
var c = 'A';
Console.WriteLine(c.GetType());
var f = 45.67f;
Console.WriteLine(f.GetType());
var b = true;
Console.WriteLine(b.GetType());
var s = "Hello";
Console.WriteLine(s.GetType());
Console.WriteLine("-----");

```

```

dynamic d;
d = 100;
Console.WriteLine(d.GetType());
d = 'Z';
Console.WriteLine(d.GetType());
d = 34.56;
Console.WriteLine(d.GetType());
d = false;
Console.WriteLine(d.GetType());
d = "Hello";
Console.WriteLine(d.GetType());
}
}

```

Boxing and Un-Boxing:

Boxing is a process of converting **values types** into **reference types**:

```

int i = 100;
object obj = i; //Boxing

```

Unboxing is a process of converting a **reference type** which is created from a **value type** back into **value type**, but **un-boxing** requires an **explicit conversion**:

```

int j = Convert.ToInt32(obj); //Un-Boxing

```

Value Type	=> Reference Type		//Boxing
Value Type	=> Reference Type	=> Value Type	//UnBoxing
Reference Type	=> Value Type		//Invalid

Note: “Convert” is a predefined class in “System” namespace and “ToInt32” is a static method under that class, and this class also provides other methods for conversion like “ToDouble”, “ToSingle”, “ToDecimal”, “ToBoolean”, etc, to convert into different types.

Taking input from end user's, into a program:

```

using System;
class AddNums
{
    static void Main()

```



```

{
    Console.Clear();

    Console.Write("Enter 1st number: ");
    string s1 = Console.ReadLine();
    double d1 = Convert.ToDouble(s1);

    Console.Write("Enter 2nd number: ");
    string s2 = Console.ReadLine();
    double d2 = double.Parse(s2);

    double d3 = d1 + d2;

    Console.WriteLine("Sum of " + d1 + " & " + d2 + " is: " + d3);
    Console.WriteLine("Sum of {0} & {1} is: {2}", d1, d2, d3);
    Console.WriteLine($"Sum of {d1} & {d2} is: {d3}");
}
}

```

ReadLine method of the Console class is used for reading the input from end users into our programs and this method will perform 3 actions when used in the program, those are:

1. Waits at the command prompt for the user to enter a value.
2. Once the user finishes entering his value, immediately the value will be read into the program.
3. Returns the value as string by performing boxing because return type of the method is string.

public static string ReadLine()

Note: after reading the value as string in our program we need to convert it back into its original type by performing **explicit un-boxing** which can be done in either of the ways:

<pre> string s1 = Console.ReadLine(); double d1 = Convert.ToDouble(s1); </pre>	<pre> string s2 = Console.ReadLine(); double d2 = double.Parse(s2); </pre>
or	or
<pre> double d1 = Convert.ToDouble(Console.ReadLine()); </pre>	<pre> double d2 = double.Parse(Console.ReadLine()); </pre>

Parse(String): this method is used to convert the **string** representation of a **value** to its equivalent value type on which the method is called.

<pre> string s1 = "100" ; string s2 = "34.56"; string s3 = "true"; </pre>	<pre> int i = int.Parse(s1); double d = double.Parse(s2); bool b = bool.Parse(s3); </pre>
---	---

String Interpolation: String interpolation provides a more **readable** and **convenient** syntax to create formatted strings than a string composite formatting feature. An interpolated string is a string literal that might contain interpolation expressions. When an interpolated string is resolved to a result string, items with interpolation expressions are replaced by the string representations of the expression results. This feature is available starting with **C# 6.0**.

Operators in C#: An **operator** is a special symbol that tells the compiler to perform a specific mathematical or logical operation when used between a set of **operands**. C# has a rich set of built-in operators as below:

Arithmetic Operators	=>	+, -, *, /, %
Assignment Operators	=>	=, +=, -=, *=, /=, %=
Relational Operators	=>	==, !=, <, <=, >, >=
Logical Operators	=>	&&, , !
Unary Operators	=>	++, --
Miscellaneous Operators	=>	sizeof(), typeof(), is, as, ?: (Terinary), ?? (Coalesce)

```
using System;
class OperatorsDemo
{
    static void Main()
    {
        Console.WriteLine(sizeof(double));
        Console.WriteLine(typeof(float));

        double d = 34.56;
        object obj1 = d;
        if(obj1 is double)           //"is" => is a type comparision operator
            Console.WriteLine("d is of type System.Double");
        string str1 = "Hello World";
        object obj2 = str1;
        string str2 = (string)obj2;
        string str3 = obj2 as string;    //"as" => is a type conversion operator
        string str4 = obj2.ToString();
        int i = 100;
        Console.WriteLine(i == 100 ? "Hello India" : "Hello World");

        string Country1 = null, Country2 = null;
        Console.WriteLine(Country1 ?? Country2);
        Country2 = "India";
        Console.WriteLine(Country1 ?? Country2);
        Country1 = "America";
        Console.WriteLine(Country1 ?? Country2);
    }
}
```

Conditional Statements in C#: it's a block of code that executes based on a **condition** and they are divided into 2 categories.

1. **Conditional Branching**
2. **Conditional Looping**

Conditional Branching: these statements allow us to branch the code depending on whether certain conditions are met or not. C# has 2 constructs for branching code, the **"if"** statement which allow us to test whether a specific

condition is met or not, and the “switch” statement which allows us to compare an expression with a number of different values.

Syntax of “if” Condition:

```
if (<condition>)
    {} <statement(s)>; {}
else if (<condition>)
    {} <statement(s)>; {}
[<multiple else if's>]
else
    {} <statement(s)>; {}
```

Note: Curly braces are **optional** if the conditional block contains **single statement** in it or else they are **mandatory**.

```
using System;
class IfDemo
{
    static void Main()
    {
        Console.Write("Enter 1st number: ");
        double d1 = double.Parse(Console.ReadLine());
        Console.Write("Enter 2nd number: ");
        double d2 = double.Parse(Console.ReadLine());

        if(d1 > d2)
            Console.WriteLine("1st number is greater than 2nd number.");
        else if(d1 < d2)
            Console.WriteLine("2nd number is greater than 1st number.");
        else
            Console.WriteLine("Both the given numbers are equal.");
    }
}
```

Syntax of “switch case” Condition:

```
switch (<expression>)
{
    case <value>:
        <stmts>;
        break;
    [<multiple case blocks>]
    default:
        <stmts>;
        break;
}
```

Note: In **C** and **CPP** languages using a break statement after each “case block” is only optional whereas it is mandatory in case of **C#** language, which should be used after “default block” also.

```

using System;
class SwitchDemo
{
    static void Main()
    {
        Console.Write("Enter Student Id. (1-3): ");
        int Id = int.Parse(Console.ReadLine());
        switch(Id)
        {
            case 1:
                Console.WriteLine("Student 1");
                break;
            case 2:
                Console.WriteLine("Student 2");
                break;
            case 3:
                Console.WriteLine("Student 3");
                break;
            default:
                Console.WriteLine("No student exists with the given Id.");
                break;
        }
    }
}

```

Conditional Looping: C# provides 4 different loops that allow us to execute a block of code **repeatedly** until a certain **condition** is met and those are:

1. **for loop** 2. **while loop** 3. **do..while loop** 4. **foreach loop**

Every loop requires 3 things in common:

1. **Initialization:** This set's the **starting** point for a loop.
2. **Condition:** This decides when the loop must **end**.
3. **Iteration:** This takes the loop to the **next level** either in **forward** or **backward** direction.

Syntax of "for loop":

```

for (initializer;condition;iteration)
{
    -<statement's>;
}

```

Example:

```

for(int i = 1;i <= 100;i++)
{
    Console.WriteLine(i);
}

```

Syntax of "while loop":

```

while (<condition>)

```

```
{  
    -<statement's>;  
}
```

Example:

```
int i = 1;  
while(i <= 100)  
{  
    Console.WriteLine(i);  
    i++;  
}
```

Syntax of "do..while loop":

```
do  
{  
    -<statement's>;  
}  
while (<condition>;
```

Example:

```
int i = 1;  
do  
{  
    Console.WriteLine(i);  
    i++;  
}  
while (i <= 100);
```

Note: the minimum no. of **execution's** in case of a "for loop" and "while loop" are "0" because in both these cases the loop starts its execution only when the given **condition** is **satisfied** whereas the minimum no. of **execution's** in case of a "do...while loop" is "1" because in this case after **executing** the loop for **first** time, then it will check for the condition to continue the loop's execution.

Syntax of "foreach loop":

```
foreach(type var_name in array_name|collection_name)  
{  
    -<statements>;  
}
```

Note: **foreach loop** is specially designed for accessing values from an **array** or **collection**.

Jump Statements: these are statements which will transfer the control from 1 line of execution to another line. C# has no. of statements that allows jumping to another line in a program, those are:

1. **goto**
2. **break**
3. **continue**
4. **return**

goto: it allows us to jump directly to another specified line in the program, indicated by a **label** which is an identifier followed by a **colon**.



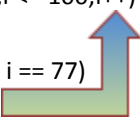
```
goto xxx;
Console.WriteLine("Hello World.");
xxx:
Console.WriteLine("Goto Called.");
```

break: it is used to **exit** from a **case** in a **switch** statement and used to **exit** from any **conditional loop** statement which will switch the control to the statement immediately after end of the loop.



```
for (int i = 1; i <= 100; i++)
{
    Console.WriteLine(i);
    if (i == 50)
        break;
}
Console.WriteLine("End of the loop.");
```

continue: it is used only in a loop which will jump the control to iteration part of the loop without executing any other statement that is present next to it.

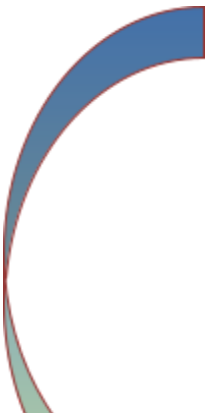


```
for (int i = 1; i <= 100; i++)
{
    if (i == 7 || i == 77)
        continue;
    Console.WriteLine(i);
}
```

return: this is used to terminate the execution of a method in which it is used and jumps out of that method, while jumping out it can also carry a value out of that method which was only optional.

```
using System;
class Table
{
    static void Main()
    {
        Console.Clear();
        Console.WriteLine("Enter an un-signed integer value: ");
        bool Status = uint.TryParse(Console.ReadLine(), out uint x);

        if (Status == false)
        {
            Console.WriteLine("Please enter un-signed integer's only.");
            return;
        }
        if (x == 0 || x == 1)
        {
            Console.WriteLine("Please enter a number greater than 1.");
        }
    }
}
```





```

return;
}

Console.WriteLine();
for (int i=1;i<=10;i++)
{
    Console.WriteLine($"{x} * {i} = {x*i}");
}
} //End of the method
}

```

Arrays

It is a set of similar type values that are stored in a sequential order either in the form of a **row** or **rows & columns**. In C# language also we access the values of an array by using the **index** only which will start from “0” and ends at the “**no. of items - 1**”. In C# arrays can be declared either as **fixed length** or **dynamic**, where a fixed length array can store a pre-defined no. of items whereas the size of a dynamic array increases as we add new items to it.

1-Dimensional Array's: these arrays will store data in the form of a row and are declared as following:

Syntax: <type>[] <array_name> = new <type>[length | size]

Example:

```
int[] arr = new int[5];           //Declaration and Initialization with default values
```

Or

```
int[] arr;                       //Declaration
arr = new int[5];                 //Initialization with default values
```

Or

```
int[] arr = { <list of values> }; //Declaration and Initialization with given set of values
```

```

using System;
class SDArray1
{
    static void Main()
    {
        Console.Clear();
        int x = 0;
        int[] arr = new int[6];

        //Accessing values of a SD Array by using for loop
        for(int i=0;i<6;i++)
            Console.Write(arr[i] + " ");
        Console.WriteLine();

        //Assigning values to a SD Array by using for loop
        for(int i=0;i<6;i++)
        {

```

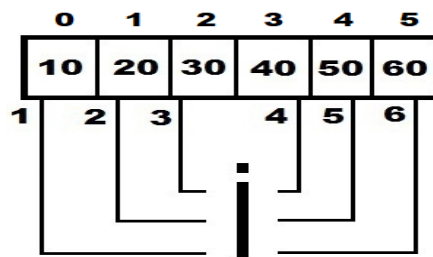
```

x += 10;
arr[i] = x;
}

//Accessing values of a SD Array by using foreach loop
foreach(int i in arr)
    Console.Write(i + " ");
Console.WriteLine();
}
}

```

foreach loop: this loop is specially designed for accessing values from an array or a collection. When we use foreach loop for accessing values, the loop starts providing access to values of the array or collection by assigning the values to loop variable in a sequential order as following:



Differences between for loop and foreach loop in accessing values of an array or collection:

1. In case of a “for loop”, the loop variable refers to **index** of the array whereas in case of a “foreach loop”, the loop variable refers to **values** of the array.
2. By using a “for loop” we can either access (**get**) or assign (**set**) values to an array whereas by using a “foreach loop” we can only access (**get**) the values from an array.
3. In case of a “for loop”, the data type of loop variable is always **int** only irrespective of the type of values in the array, whereas in case of a “foreach loop”, the data type of loop variable will be same as the **type** of values in the array.

```

int[] iarr = { 10, 20, 30, 40, 50 };
double[] darr = { 12.34, 34.56, 56.78, 78.90, 90.12 };
string[] sarr = { "Red", "Blue", "Green", "Yellow", "Magenta" };

```

for (int i=0;i<iarr.Length;i++)	foreach(int i in iarr)
for (int i=0;i<darr.Length;i++)	foreach(double d in darr)
for (int i=0;i<sarr.Length;i++)	foreach(string s in sarr)

Array Class: this is a pre-defined class under the “System” namespace which provides with a set of **members** in it to perform **actions** on an **array**, those are:

Sort(Array arr)	=> void	//Method
Reverse(Array arr)	=> void	//Method
Copy(Array source, Array target, int n)	=> void	//Method
GetLength(int dimension)	=> int	//Method
Length	=> int	//Property (Field)


```

using System;
class SDArray2
{
    static void Main()
    {
        Console.Clear();
        int[] arr = { 54, 79, 59, 8, 42, 22, 93, 3, 73, 38, 67, 48, 18, 61, 32, 86, 15, 27, 81, 96 };

        for(int i=0;i<arr.Length;i++)
            Console.Write(arr[i] + " ");
        Console.WriteLine();

        Array.Sort(arr);
        foreach(int i in arr)
            Console.Write(i + " ");
        Console.WriteLine();

        Array.Reverse(arr);
        foreach(int i in arr)
            Console.Write(i + " ");
        Console.WriteLine();

        int[] brr = new int[10];

        Array.Copy(arr, brr, 7);
        foreach(int i in brr)
            Console.Write(i + " ");
        Console.WriteLine();
    }
}

```

2-Dimensional Array's: these arrays will store data in the form of **rows & columns**, and are declared as following:

Syntax: <type>[,] <array_name> = new <type>[rows, columns]

Example:

int[,] arr = new int[4,5];	//Declaration and Initialization with default values
or	
int[,] arr;	//Declaration
arr = new int[4,5];	//Initialization with default values
or	
int[,] arr = { <list of values> };	//Declaration and Initialization with given set of values

```

using System;
class TDArray
{
    static void Main()
    {
        int x = 0; int[,] arr = new int[4, 5];
    }
}

```

```
//Accessing values of TD Array by using foreach loop
foreach(int i in arr)
    Console.Write(i + " ");
Console.WriteLine();

//Assigning values to TD Array by using nested for loop
for(int i=0;i<arr.GetLength(0);i++)
{
    for(int j=0;j<arr.GetLength(1);j++) {
        x += 5; arr[i,j] = x;
    }
}

//Accessing values of TD Array by using nested for loop
for(int i=0;i<arr.GetLength(0);i++)
{
    for(int j=0;j<arr.GetLength(1);j++)
        Console.Write(arr[i,j] + " ");
    Console.WriteLine();
}
}
```

Assigning values to 2-D Array at the time of it's declaration:

```
int[,] arr = {
    { 11, 12, 13, 14, 15 },
    { 21, 22, 23, 24, 25 },
    { 31, 32, 33, 34, 35 },
    { 41, 42, 43, 44, 45 }
};
```

Jagged Arrays: these are also 2-Dimensional arrays only which will store the data in the form of rows and columns but the difference is in-case of a 2-Dimensional array all the rows will be having equal no. of columns whereas in case of a jagged array the column size varies from row to row. Jagged arrays are also known as “array of arrays” because here each row is considered as a single dimensional array and multiple single dimensional arrays with different sizes are combined together to form a new array.

Syntax: <type>[][] <array_name> = new <type>[rows][]

Example:

```
int[][] arr = new int[4][];           //Declaration
or
int[][] arr = { <list of values> };    //Declaration & Initialization with given set of values
```

Note: in case of a jagged array, we can't initialize the array with default values at the time of its declaration i.e. first we need to specify the no. of rows and then pointing to each row we need to specify the no. of columns to that row, as following:

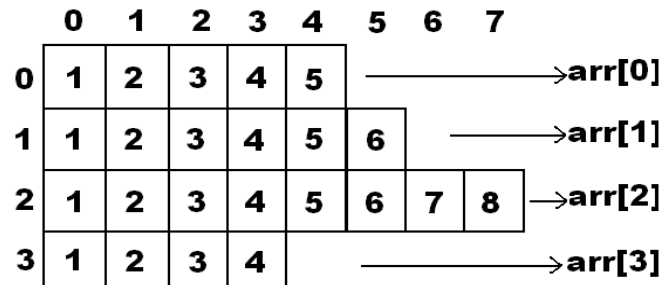
```
int[][] arr = new int[4][];           //Declaration
```

```

arr[0] = new int[5];           //Initialization of 1st row
arr[1] = new int[6];           //Initialization of 2nd row
arr[2] = new int[8];           //Initialization of 3rd row
arr[3] = new int[4];           //Initialization of 4th row

```

Internally the memory is allocated for the array as following:



```

using System;
class JArrayDemo
{
    static void Main() {
        Console.Clear();
        int[][] arr = new int[4][];
        arr[0] = new int[5];
        arr[1] = new int[6];
        arr[2] = new int[8];
        arr[3] = new int[4];
    }
}

```

//Accessing values of Jagged Array by using nested foreach loop

```
foreach(int[] iarr in arr)
```

```
{
    foreach(int x in iarr)
        Console.Write(x + " ");
    Console.WriteLine();
}
```

```
Console.WriteLine("-----");
```

//Accessing values of Jagged Array by using for loop in foreach loop

```
foreach(int[] iarr in arr)
```

```
{
    for(int i=0;i<iarr.Length;i++)
        Console.Write(iarr[i] + " ");
    Console.WriteLine();
}
```

```
Console.WriteLine("-----");
```

//Assigning values to Jagged Array by using for loop in foreach loop

```
foreach(int[] iarr in arr)
```

```
{
```

```

for(int i=0;i<iarr.Length;i++)
{
    iarr[i] = i + 1;
}
}

```

//Accessing values of Jagged Array by using nested for loop

```

for(int i=0;i<arr.GetLength(0);i++)
{
    for(int j=0;j<arr[i].Length;j++)
        Console.Write(arr[i][j] + " ");
    Console.WriteLine();
}
Console.WriteLine("-----");

```

//Assigning values to Jagged Array by using nested for loop

```

for(int i=0;i<arr.GetLength(0);i++)
{
    for(int j=0;j<arr[i].Length;j++)
    {
        arr[i][j] = i + 1;
    }
}

```

//Accessing values of Jagged Array by using foreach loop in for loop

```

for(int i=0;i<arr.GetLength(0);i++)
{
    foreach(int x in arr[i])
        Console.Write(x + " ");
    Console.WriteLine();
}
}

```

Assigning values to Jagged Array at the time of its declaration:

```

int[][] arr =
{
    new int[5] { 11, 12, 13, 14, 15 },
    new int[6] { 21, 22, 23, 24, 25, 26 },
    new int[8] { 31, 32, 33, 34, 35, 36, 37, 38 },
    new int[4] { 41, 42, 43, 44 }
};

```

Implicitly typed arrays: Just like we can declare **variables** by using “**var**” keyword we can also declare **arrays** by using the same “**var**” keyword and here also the **type identification** is performed based on the **values** that are assigned to the **array**.

```

var iarr = new[] { 10, 20, 30, 40, 50 };           //Implicitly type integer array
var sarr = new[] { "Red", "Blue", "Green", "Yellow", "Magenta" }; //Implicitly typed string array
var darr = new[] { 12.34, 34.56, 56.78, 78.96, 90.12 }; //Implicitly typed double array

var jarr = new[]
{
    new[] { 11, 12, 13, 14, 15 },
    new[] { 21, 22, 23, 24, 25, 26 },
    new[] { 31, 32, 33, 34, 35, 36, 37, 38 },
    new[] { 41, 42, 43, 44 },
    new[] { 51, 52, 53, 54, 55, 56, 57 }
};
//Implicitly typed jagged integer array

```

Command Line Arguments: Arguments which are passed by the user or programmer to the **Main** method are known as **Command-Line Arguments**. Main method is the entry point for the execution of a program and this **Main** method can accept an **array of strings**.

```

using System;
class Params
{
    static void Main(string[] args)
    {
        foreach(string str in args) {
            Console.WriteLine(str);
        }
    }
}

```

After compilation of the program execute the program at Command Prompt as following:

```
<drive>:\CSharp> Params 100 Hello 34.56 A true ↵
```

Note: We can pass **any no. of values** as well as **any type of values** as **command line arguments** to the program, but each value should be separated with a **space** and all those values we passed will be captured in the **string array (args)** of **Main** method. In the above case (**100, Hello, 34.56, A, true**) are 5 values we have supplied to the **Main** method of **Params** class as command line arguments.

Adding a given set of numbers that are passed as Command Line Arguments:

```

using System;
class AddParams
{
    static void Main(string[] args)
    {
        double Sum = 0;
        foreach(string str in args)
        {

```

```

    Sum = Sum + double.Parse(str);
}
Console.WriteLine("Sum of given {0} no's is: {1}", args.Length, Sum);

Or

Console.WriteLine($"Sum of given {args.Length} no's is: {Sum}");
}
}

```

After compilation of the program execute the program at Command Prompt as following:

```

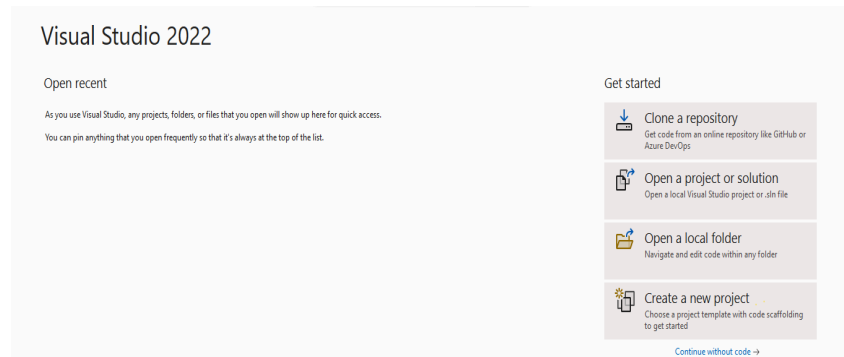
<drive>:\CSharp> AddParams ↵
<drive>:\CSharp> AddParams 100 ↵
<drive>:\CSharp> AddParams 150 75 ↵
<drive>:\CSharp> AddParams 10 20 30 ↵
<drive>:\CSharp> AddParams 34.56 28.93 98.45 63.28 ↵
<drive>:\CSharp> AddParams 938.387 534 348.378 836 174.392 ↵
<drive>:\CSharp> AddParams 18 48.37 75 56.43 97 85.19 ↵

```

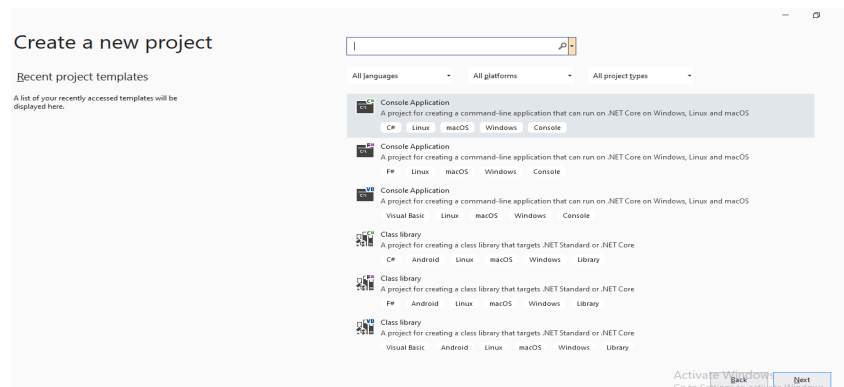
Working with Visual Studio

Visual Studio is an IDE (Integrated Development Environment) used for developing .NET Applications by using any .NET Language like C#, Visual Basic, F# etc., as well as we can develop any kind of applications like Console, Windows, and Web etc.

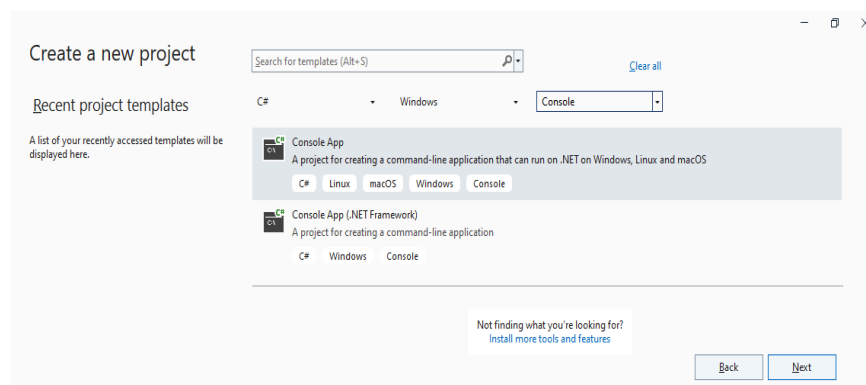
To open Visual Studio, go to Windows Search and search for Visual Studio 2022 and click on it to open, which will launch as following:



Applications that are developed under Visual Studio are known as Projects, where each Project is a collection of items like Class, Interface, Structure, Enum, Delegate, Html Files, XML Files, and Text Files etc. To create a Project, click on “Create a new project” option in the above Page which opens a new window as following:



In the above window under “All languages” DropDownList select “C#”, under “All platforms” DropDownList select “Windows” and under “All project_types” DropDownList select “Console” which will display the options as following:



Now select “Console Application” in the above window and click “Next” button which opens a new window as following:

In that above window under “Project Name” TextBox enter the name of project as “FirstProject”, under Location TextBox enter or select our personal folder location i.e., “<drive>:\CSharp” and click on “Next” button:

This will open a new window asking to select the Target Framework choose .NET 5.0 (Out of support), make sure all Checkbox’s on this Window are un-checked and then click on “Create” button:

This action will create a new project and by default the project comes with a class named as Program under the file Program.cs which will look as below:

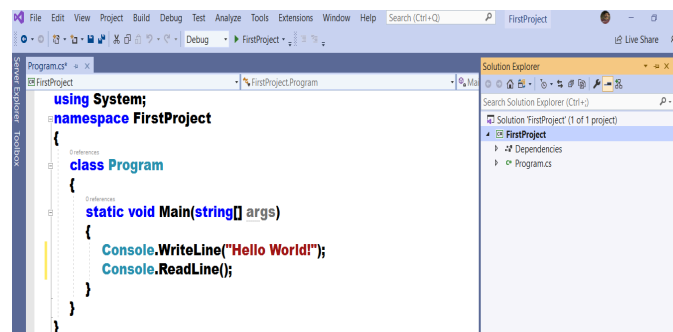
```
using System;
namespace FirstProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");
        }
    }
}
```


To run the class either hit **Ctrl + F5** or go to **“Debug”** menu and select the option **“Start Without Debugging”** which will **save**, **compile**, and **executes** the program by displaying the output **“Hello World!”** on the console window because we have opened a **“Console App.”** Project. To close the console window that is opened, it will display a message **“Press any key to continue . . .”**, so once we hit any key it will close the window and takes us back to the **Visual Studio**.

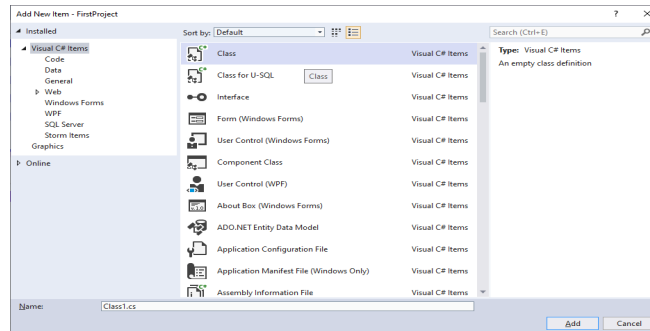
We can also run the class by hitting **F5** or clicking on the **▶ FirstProject** button in the **“Tool Bar”** or go to **“Debug”** menu and select the option **“Start Debugging”**, and in this also case also it will **save**, **compile** and **executes** the program but we can't view the output because the **Console** window gets closed immediately and in this case to view the output we need to hold console window and to do that use **“Console.ReadLine();”** method after **“Console.WriteLine(“Hello World!”);”** in **Main** method of the program which should now look as below:

```
using System;
namespace FirstProject {
    internal class Program {
        static void Main(string[] args) {
            Console.WriteLine("Hello World!");
            Console.ReadLine();
        }
    }
}
```

Adding new items in the project: under **Visual Studio** we find a window in **RHS** known as **Solution Explorer** used for organizing the **Project**, which allows us to **view**, **add** and **delete** items under the **projects**, if it is not visible in the **RHS** then go to **“View”** menu and select **“Solution Explorer”** which will launch it on **RHS** that looks as below:



To add new classes under **project**, right click on the **project** in **Solution Explorer** and select **Add =>** choose **“New Item”** option, which opens the **“Add New Item”** window as below:



In the above window select “Class” template, specify a name to it in the **TextBox** at bottom of the **Window** or leave the existing name and click on **Add** button, which adds the class under our project with the name “Class1.cs” (as i did not change the name). The new class we added, also comes under the same **Namespace**, i.e., “FirstProject” and we find the below code in it:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace FirstProject
{
    internal class Class1
    {
    }
}
```

By default, the class will have 5 import statements, importing a set of **namespaces** and all these statements are not important for us now and what we need now is only **System Namespace**, so delete the others.

Now under the class define a Main method which should now look as below:

```
using System;
namespace FirstProject
{
    internal class Class1
    {
        static void Main()
        {
            Console.WriteLine("Second class under the project.");
            Console.ReadLine();
        }
    }
}
```

Now when we **run** the project, we get an **error** stating that there are **multiple entry points** in the project because the 2 classes that are defined in the project contains a **Main** method and every **Main** method is an entry point, so to resolve the problem we need to set a property known as “Startup Object”.

To set the “Startup Object” property, open **Solution Explorer** => either double click on the project or right click on the project & select the option “Edit Project File”, which opens an “XML File” with the name “FirstProject.csproj” added to the document window in Visual Studio.

Under the project file, by default we find the below code:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net5.0</TargetFramework>
  </PropertyGroup>
</Project>
```

To set the “Startup Object” property, add “<StartupObject></StartupObject>” element within the <PropertyGroup> element and the code in “FirstProject.csproj” file should be as below now:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net5.0</TargetFramework>
    <StartupObject>FirstProject.Class1</StartupObject>
  </PropertyGroup>
</Project>
```

Note: follow the same process to run the new classes we add in the project.

Creating a new project using .NET 6.0 or above versions: Same as the above create another project, name it as “SecondProject” and while choosing Framework Version, select .NET 6.0 (Long Term Support) and make sure all Checkbox’s on the window are un-checked and click on Create button which will now add Program.cs file under the Project.

Note: Starting with .NET 6 (C# 10.0) and above, the project template for new C# Console App’s generates the below code in Program.cs file:

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

This is a new feature in “C# 10.0” i.e., we don’t require to define a class and Main method explicitly which means the code we write in the file will directly execute as if the code we write in a Main method. In the above code first line is a comment and second line is a WriteLine statement to print output on the monitor. Whereas if we create the project by choosing the Framework as .NET 5.0 (Out of support) then it is C# 9.0 and up to this version defining class & Main method is mandatory to run the code, so we find code in “Program.cs” file as below:

```
using System;
namespace FirstProject
{
    internal class Program
    {
```

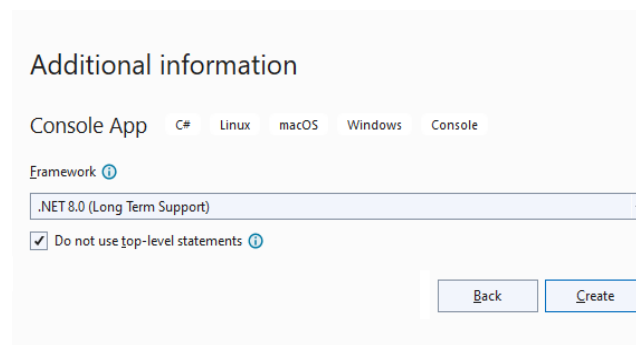
```

static void Main(string[] args)
{
    Console.WriteLine("Hello World!");
}
}
}

```

The above 2 forms of code represent the same class, program. Both are valid with **C# 10.0**. When you use the newer version, you only need to write the body of the **Main** method. The compiler generates a **Program** class with an entry point method (i.e., **Main**) and places all your **top level statements** in that method. You don't need to include the other program elements because the compiler **generates** them for you. This feature is added in **.NET 6.0** and will be same in **.NET 7.0** and **.NET 8.0** also.

If you don't want to use the **top-level statements** and generate a **class** explicitly, create another project with the name **"ThirdProject"** and while choosing the **Framework Version** choose either **.NET 6.0** or **.Net 7.0** or **.NET 8.0** and check the Checkbox **"Do not use top-level statements"** while creating the project as below:



The above action will define code in Program.cs file as below:

```

namespace ThirdProject
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}

```

If you notice the above code we don't find **"using System"** statement on the top because from **".NET 6.0** or **C# 10.0"** there is a concept of **"Global Imports"** i.e., we don't require writing **import statements** in all the files as we have done in case of **Notepad** with the help of **"using directive"** i.e., we can now write all the import statements that are required in our project with-in a single file prefixing the keyword **"global"**, so that they are applied to all the classes in our project.

Syntax: **global using** <Namespace_Name>;

Example: **global using** System;

Note: By default, some namespaces are already **imported** for us to consume in our project created under **VS 2022** when choosed **“.NET 6.0 or above”** within the file **“GlobalUsings.g.cs”** and the content of the file is as below:

```
// <auto-generated/>
global using global::System;
global using global::System.Collections.Generic;
global using global::System.IO;
global using global::System.Linq;
global using global::System.Net.Http;
global using global::System.Threading;
global using global::System.Threading.Tasks;
```

Note: Starting from **.NET 6.0** i.e., **C# 10.0** the project file contents are modified i.e., they have been added with new elements in the file that looks as below:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

- ② **OutputType** element is used to specify the generated output file after compilation of the project will be having an **“.exe”** extension.
- ② **TargetFramework** element is used to specify the **.NET Runtime Version** we are using to build the project and currently we are using the latest version of **“.NET”** i.e., **“.NET 8.0”** and in this version of Runtime, **C#** version is **“12.0”**, same as this if we choose **“.NET 7.0”** then C# version is **11.0**, for **“.NET 6.0”** C# version is **10.0**, for **“.NET 5.0”** C# version is **“9.0”** and if we choose **“.NET Core 3.1”** then the version of **C#** is **“8.0”**.
- ② **ImplicitUsings** element is used to **enable** the feature **global imports** which is new from **“C# 10.0”** and if set as **disable** the feature will not work and we need to explicitly import the namespaces on top of each class with the help of **“using”** directive.
- ② **Nullable** element is used to enable a new feature **“Non-Nullable Reference Types”** which was introduced in **“C# 8.0”**. By default, **Reference Types** are **Nullable**, but we can make them **non-Nullable** by enabling the **Nullable** feature in the project file, so when we assign a **Null** value to them, we get a **warning**, but if we really want to assign a **null** value to them, we need to declare them by suffixing with a **“?”**, for example: **string?** and **object?**. If we want to disable the feature of reference types not accepting null values by default, change the value **“enable”** as **“disable”** under the **Nullable** element of the project file.

Note: now also to run the new classes added in the project, we need to set the **StartupObject Element** in the project file as below:

```

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <StartupObject>ThirdProject.Class1</StartupObject>
  </PropertyGroup>
</Project>

```

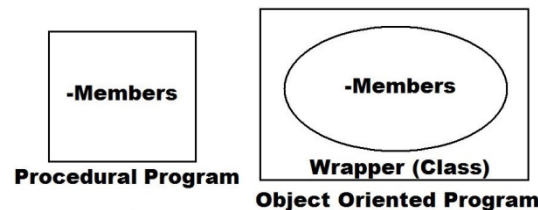
Object Oriented Programming

This is an approach that we use in the industry for developing application, introduced in late 70's or early 80's replacing traditional **Procedural Programming Approach** because **Procedural Programming Approach** doesn't provide **Security** and **Re-usability**, whereas these 2 are the main strength of **Object-Oriented Programming Approach**.

Any language to be called as **Object Oriented** needs to satisfy 4 important principals that are prescribed under the standards of **Object-Oriented Programming**, and they are:

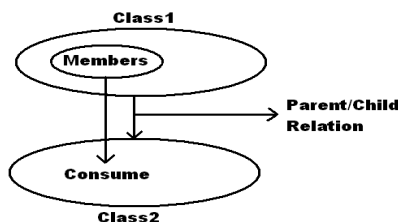
1. **Encapsulation** => hiding the data
2. **Abstraction** => hiding the complexity
3. **Inheritance** => re-usability
4. **Polymorphism** => behaving in different ways based on input received

Encapsulation: this is all about hiding of data or members of a program by wrapping them under a container known as a Class, which provides security for all its contents.



Abstraction: this is all about hiding the complexity of code and then providing with a set of interfaces to consume those functionalities, for example functions/methods in a program are good example for this, because here we are only aware of how to call them, but we are never aware of the underlying logic behind that implementation.

Inheritance: this provides **re-usability** i.e., members that are defined in 1 class can be consumed from other classes by establishing **parent/child** relation between the classes.



Polymorphism: behaving in different ways based on the input received is known as polymorphism i.e., whenever the input changes then the output or behavior also changes accordingly.

Class: It's a **user-defined type** which is in-turn a collection of **members** like:

- **Fields**
- **Methods**
- **Constructors**
- **Finalizers**
- **Properties**
- **Indexers**
- **Events**
- **De-constructors (Introduced in C# 7.0)**

Method: It is a named block of code which performs an **action** whenever it is called and after completion of that action it may or may not return any result of that action, and they are divided into 2 categories:

1. **Value returning method (Function)**
2. **Non-value returning method (Sub-Routine)**

Syntax to define a method:

```
[<modifiers>] void | type <Name> ( [<Parameter List>] )  
{  
- Stmt's or Logic  
}  
[] => Optional  
<> => Any
```

Modifiers are some special keywords which can be used on a method if required (**optional**) like **public**, **internal**, **protected**, **static**, **virtual**, **abstract**, **override**, **sealed**, **partial**, **private** etc.

void | type is to tell whether our method is value returning or non-value returning i.e., "**void**" implies that the method is non-value returning, whereas if we want our method to return any value then we need to specify the **type** of value it has to return by using the "**type**".

Example for non-value returning methods:

```
public static void Clear()  
public static void WriteLine(<type> var)
```

Example for value returning method:

```
public static string ReadLine()
```

Note: the return type of a method need not be any **pre-defined type** like **int**, **float**, **char**, **bool**, **DateTime**, **Guid**, **string**, **object**, etc., but it can also be any **user-defined type** also.

<Name> refers to "**ID**" of the method for identification.

[<Parameter List>]: if required we can pass parameters to our methods for execution, and parameters of a method will make an action **dynamic**, for example:

```
GetLength(0)    => Returns rows  
GetLength(1)    => Returns columns
```

Syntax to pass parameters to a method:

[ref | out] [params] <type> <var> [=default value] [,...n]

Where should we define methods?

Ans: As per the rule of **Encapsulation** methods should be defined inside of a **class**.

How to execute a method that is defined under a class?

Ans: The methods that are defined in a class must be **explicitly** called for execution, except **Main** method because **Main** is implicitly called.

How to call a method that is defined in a class?

Ans: Methods are of 2 types:

1. Non-Static

2. Static

Note: By default, every method of a class is **non-static** only, and if we want to make it as **static**, we need to prefix the **"static"** modifier before the method as we are doing in case of **Main** method.

To call a method that is defined under any class we require to create **instance** of that class provided the methods are **non-static**, whereas if the methods are **static**, we can call them directly by **prefixing** class name, for example **WriteLine** and **ReadLine** are **static** methods in class **Console** which we are calling in our code as **Console.WriteLine** and **Console.ReadLine**.

How to create instance of a class?

Ans: We create the instance of class as following:

Syntax: <class_name> <instance_name> = new <class_name> ([<List of values>])

Example:

Program p = new Program(); //Declaration and Initialization

or

Program p; //Declaration

p = new Program(); //Initialization

Note: with-out using **"new"** keyword we can't create the **instance** of a class in **Java** and **.NET** Languages.

Where should we create the instance of a class?

Ans: instance of a class can be created either with-in the **same class** or in **other classes** also.

If instance is created in the **same class**, it should be created under any **static block**; generally, we create instances in **Main** method because of 2 reasons:

1. **Entry Point of the program.**
2. **It is a static block.**

If **instance** is created in **another class**, then it can be created in any block of that new class i.e., either **static** or **non-static** also.

To try all the above, create a new **Console App.** project in **Visual Studio** naming it as **"OOPSPProject"**, delete all the code that is present in the default file **"Program.cs"** and write the below code over there:


```

namespace OOPSProject
{
    internal class Program
    {
        //Non-value returning method without parameters
        public void Test1() //Static in behavior
        {
            int x = 5;
            for (int i = 1; i <= 10; i++)
            {
                Console.WriteLine($"{x} * {i} = {x * i}");
            }
        }
        //Non-value returning method with parameters
        public void Test2(int x, int ub) //Dynamic in behavior
        {
            for (int i = 1; i <= ub; i++)
            {
                Console.WriteLine($"{x} * {i} = {x * i}");
            }
        }
        //Value returning method without parameters
        public string Test3() //Static in behavior
        {
            string str = "hello world";
            str = str.ToUpper();
            return str;
        }
        //Value returning method with parameters
        public string Test4(string str) //Dynamic in behavior
        {
            str = str.ToUpper();
            return str;
        }
        static void Main()
        {
            //Creating instance of the class.
            Program p = new Program();

            //Calling non-value returning methods.
            p.Test1();
            Console.WriteLine();
            p.Test2(8, 15);
            Console.WriteLine();

            //Calling value returning methods

```

```

string s1 = p.Test3();
Console.WriteLine(s1);
string s2 = p.Test4("hello india");
Console.WriteLine(s2);
Console.ReadLine();
}
}
}

```

Consuming a class from other classes: It is possible to **consume** a **class** and its **members** from other classes in 2 different ways:

1. **Inheritance**
2. **Creating an instance**

To test the second, add a new class in the project naming it as “TestProgram.cs” and write the below code in it:

```

internal class TestProgram
{
    public void CallMethods()
    {
        Program p = new Program();
        p.Test1();
        Console.WriteLine();
        p.Test2(9, 12);
        Console.WriteLine();
        Console.WriteLine(p.Test3());
        Console.WriteLine(p.Test4("hello america"));
    }
    static void Main()
    {
        new TestProgram().CallMethods();    //Un-named instance
        Console.ReadLine();
    }
}

```

Note: Un-named **instances** are created and used when we want to call any single **member** of a **class** or when we want to use that **instance** only for 1 time.

Code files in a project: When we want to add a new class under any project, we first open the “Add New Item” window and in that we choose “Class Item Template”, which when added will add a file with a **class template** in it, same as that we also find “Code File Item Template” which when added will add a **blank file** and we need to write everything manually in it, just like we write code using **Notepad**.

Defining multiple classes in a file: It’s possible to define “n” no. of classes under a single “.cs” file, but “Main” method can be defined under 1 **class** only. Even if it is not **mandatory** it is **advised** to use the “Class Name” under which we defined “Main” method as the “File Name”.

User-defined return types to a Methods: The return type of a method need not be any pre-defined type but can also be any user-defined type also i.e., a type which is defined representing some complex data.

To test all the above, add a "Code File" under the project naming it as "UserDefinedTypes.cs" and write the below code in it:

```
namespace OOPSPProject
{
    class Emp
    {
        public int? Id;
        public string? Name, Job;
        public double? Salary;
        public bool? Status;
    }
    class UserDefinedTypes
    {
        public Emp GetEmpDetails(int Id)
        {
            Emp emp = new Emp();
            emp.Id = Id;
            emp.Name = "Raju";
            emp.Job = "Manager";
            emp.Salary = 50000.00;
            emp.Status = true;
            return emp;
        }
        static void Main()
        {
            UserDefinedTypes udt = new UserDefinedTypes();
            Emp obj = udt.GetEmpDetails(1001);
            Console.WriteLine(obj.Id + " " + obj.Name + " " + obj.Job + " " + obj.Salary + " " + obj.Status);
            Console.ReadLine();
        }
    }
}
```

In the above case "Emp" is a new type (User-Defined and Complex) and that type is used as a return type for our method "GetEmpDetails".

Parameters of a Method: we define parameters to methods for making actions dynamic i.e., as discussed earlier every method is an action and to make those actions dynamic, we define parameters to methods.

Syntax for defining parameters to a method:

[ref | out] [params] <type> <parameter_name> [= default value] [, ..n]

Parameters of a method are classified as:

1. Input Parameters

2. Output Parameters

3. InOut Parameters

- **Input** parameters will bring values into the method for execution.
- **Output** parameters will carry results out of the method after execution.
- **InOut** Parameters are a combination of above 2 i.e., these parameters will 1st bring a value into the method for execution and after execution, the same parameter will carry results out of the method.

By default, every parameter is an **input** parameter whereas if we want to declare any parameter as **output** we need to prefix "**out**" keyword and to declare a parameter as **InOut** we need to prefix "**ref**" keyword before the parameter, as following:

```
public void Test(int a, out int b, ref int c)
```

To test **Output Parameters**, add a new class in the **Project** naming it as "**OutputParameters.cs**" and write the below code in it:

```
internal class OutPutParameters
{
    public void Math1(int a, int b, out int c, out int d)
    {
        c = a + b;
        d = a * b;
    }
    //Introduced in C# 7.0 i.e., Tuples
    public (int, int) Math2(int a, int b)
    {
        int c = a + b;
        int d = a * b;
        return (c, d);
    }
    static void Main()
    {
        OutPutParameters p = new OutPutParameters();

        int Sum1, Product1;
        p.Math1(100, 25, out Sum1, out Product1);
        Console.WriteLine("Sum of the given number's is: " + Sum1);
        Console.WriteLine("Product of the given number's is: " + Product1 + "\n");

        p.Math1(100, 25, out int Sum2, out int Product2); //C# 7.0 Feature
        Console.WriteLine("Sum of the given number's is: " + Sum2);
        Console.WriteLine("Product of the given number's is: " + Product2 + "\n");

        (int Sum3, int Product3) = p.Math2(100, 25);
        Console.WriteLine("Sum of the given number's is: " + Sum3);
        Console.WriteLine("Product of the given number's is: " + Product3 + "\n");

        var (Sum4, Product4) = p.Math2(100, 25);
        Console.WriteLine("Sum of the given number's is: " + Sum4);
        Console.WriteLine("Product of the given number's is: " + Product4 + "\n");
        Console.ReadLine();
    }
}
```

```
}  
}
```

Tuple: A tuple is a data structure in C#, often used when we want to return more than one value from a method. A tuple can be used to return a set of values as a result from a method and this feature was introduced in C# 7.0.

To test InOut parameters add a new class in the Project naming it as "InOutParameters.cs" and write the below code in it:

```
internal class InOutParameters  
{  
    public void Factorial(ref uint a)  
    {  
        if (a == 0 || a == 1)  
        {  
            a = 1;  
        }  
        else  
        {  
            uint result = 1;  
            for(uint i=2;i<=a;i++)  
            {  
                result = result * i;  
            }  
            a = result;  
        }  
    }  
    static void Main()  
    {  
        InOutParameters obj = new InOutParameters();  
        uint f = 5;  
        Console.WriteLine("Value of f before execution of the method: " + f);  
        obj.Factorial(ref f);  
        Console.WriteLine("Value of f after execution of the method: " + f);  
        Console.ReadLine();  
    }  
}
```

Params Keyword: By prefixing this keyword before an array parameter of any method we get a chance to call that method without explicitly creating an array and pass to the method, but we can directly pass a set of values in a "Comma-Seperated" list.

```
public void AddParams(params double[] args)
```

For example WriteLine method of Console class is defined as below:

```
public static void WriteLine(string format, params object[] args)
```

So we are able to call that method in our earlier program as following:

```
Console.WriteLine("{0} * {1} = {2}", x, i, x * i);
Console.WriteLine("{0}, {1}, {2}, ..., {n}", val0, val1, val2, ..., valn);
```

Note: while using the “params” keyword we have 2 restrictions:

1. We can use it only on 1 parameter of the method.
2. It can be used only on the last parameter of that method.

Default values to parameters: While defining methods we can assign default values to parameters of that method, so that those parameters will become “optional” and while calling that method it is not mandatory to pass values to those parameters. If the method is called without passing a value to those parameters then default value of that parameter will be used, for example:

```
public void AddNums(int x, int y = 50, int z = 25)
```

Note: In the above case **x** is a mandatory parameter whereas **y** and **z** are optional parameters and while defining methods with mandatory and optional parameters, mandatory parameters should be in the 1st place of parameter list, followed by optional parameters in the last.

To test “params” keyword and “default valued parameters” add a new class in the project naming it as “MethodParameters.cs” and write the below code in it:

```
internal class MethodParameters
{
    public void AddParams(params double[] args)
    {
        double Sum = 0;
        foreach (double arg in args)
        {
            Sum = Sum + arg;
        }
        Console.WriteLine($"Sum of {args.Length} no's in the array is: {Sum}");
    }
    public void AddNums(int x, int y = 50, int z = 25)
    {
        Console.WriteLine($"Sum of given 3 no's is: {x + y + z}");
    }
    static void Main()
    {
        MethodParameters obj = new MethodParameters();

        obj.AddParams(56.87);
        obj.AddParams(78, 12.35);
        obj.AddParams(12.34, 56.32, 87.21);
        obj.AddParams(10, 20, 30, 40, 50);
        Console.WriteLine();

        obj.AddNums(100);
        obj.AddNums(100, 100);
    }
}
```

```

obj.AddNums(100, z:100);
obj.AddNums(100, 100, 100);
Console.ReadLine();
}
}

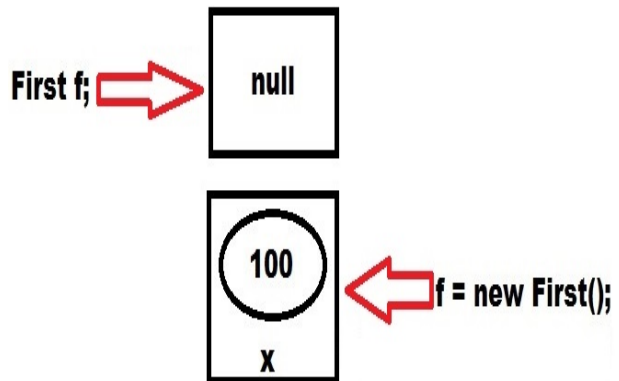
```

Understanding the difference between variable, instance, and reference of a class: to understand about a **variable**, **instance** and **reference** of a class, first add a new class in our **Project** naming it as **“First.cs”** and write the below code in it:

```

internal class First
{
    public int x = 100;
    static void Main()
    {
        First f; //f is a variable of class
        f = new First(); //f is a instance of class
        Console.WriteLine(f.x);
        Console.ReadLine();
    }
}

```



Every **member** of a class, if it is **non-static** can be accessed from the **Main Method** only by using **instance** of that class. So, in the above case to print the value of **“x”**, we created **instance** of class **First** under **Main** method.

A **variable** of class is a **copy** of class which is **not initialized** so it doesn't have any **memory allocation** and can't be used for **calling** or **accessing** the **members**.

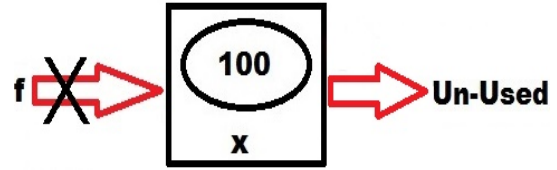
An **instance** of class is a **copy** of class which is initialized by using **“new”** keyword and for an **instance** **memory is allocated**, so by using this **instance** we can **access** or **call** members of that class.

De-referencing an Instance: it's possible to **de-reference** the **instance** of any class by assigning **“null”** to it and once **“null”** is assigned to **instance** we can't use that instance for calling members of class and if we try to do so, we get a **runtime error**. To test this, **re-write** the code under **“Main Method”** of class **“First”** as below:

```

First f = new First();
Console.WriteLine(f.x); //Valid
f = null;
Console.WriteLine(f.x); //Invalid (Causes runtime error)
Console.ReadLine();

```



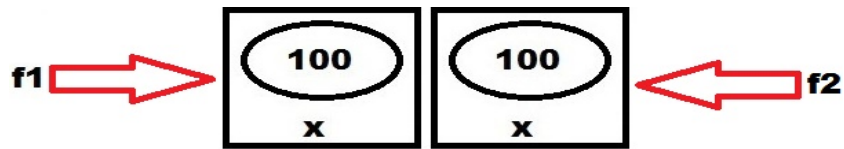
Note: once **null** is assigned to an **instance**, internally the **memory** which is **allocated** for that **instance** is **not de-allocated** immediately, but only gets **marked** as **un-used** and all those **un-used** objects memory will be **de-allocated** by “Garbage Collector” whenever it comes into **action**.

Creating multiple instances to a class: it is possible to create **multiple instances** to a class and each **instance** we create for the **class** will be having a **separate memory allocation** for its **members** as following:

```

First f1 = new First();
First f2 = new First();

```

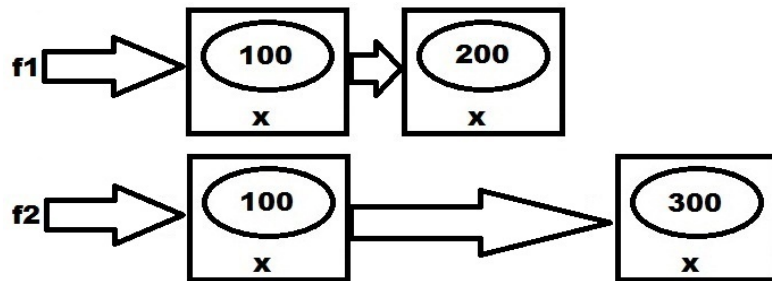


Instances are **unique** i.e., any **modifications** that we perform on the **members** of 1 **instance** will not **reflect** to the **members** of other **instances** of the class, and to test this **re-write** the code under **Main Method** of class **First** as below:

```

First f1 = new First();
First f2 = new First();
Console.WriteLine(f1.x + " " + f2.x);
f1.x = 200;
Console.WriteLine(f1.x + " " + f2.x);
f2.x = 300;
Console.WriteLine(f1.x + " " + f2.x);
Console.ReadLine();

```

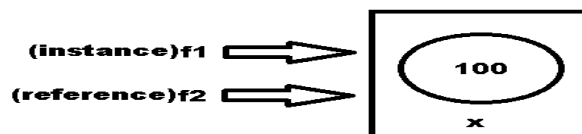


Reference of a class: we can initialize the **variable** of a class by using any **existing instance** of that class and we call it as a **reference** of the class. **References** of class will not have any **memory allocation** like **instances**, i.e., they will be **consuming** the **memory** of **instance** using which they are initialized, so a reference is just a **pointer** to an **instance**, as following:

```

First f1 = new First();
First f2 = f1; //f2 is a reference of class First

```

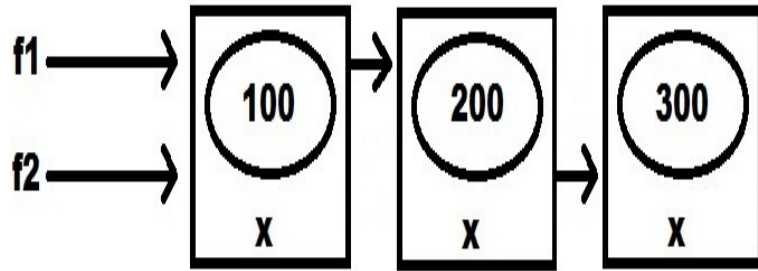


Because an **instance** and **reference** are accessing the **same memory**, changes that are performed on the **members** by using the **instance** will reflect when those **members** are **accessed** by using **reference** and **vice versa**. To test this, **re-write** code under **Main method** of class **First** as below:


```

First f1 = new First();
First f2 = f1;
Console.WriteLine(f1.x + " " + f2.x);
f1.x = 200;
Console.WriteLine(f1.x + " " + f2.x);
f2.x = 300;
Console.WriteLine(f1.x + " " + f2.x);
Console.ReadLine();

```

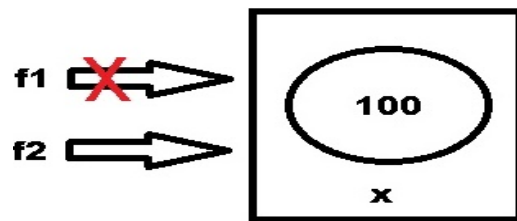


Note: when an **instance** and **references** are accessing the **same memory** and if **null** is assigned to any 1 of them, then the 1 to whom **null** is assigned can't access the **memory** anymore, but still the **others** can access it as is for calling the **members**. To test this, **re-write** code under **Main method** of class **First** as below:

```

First f1 = new First();
First f2 = f1;
f1 = null;
Console.WriteLine(f2.x); //Valid
Console.WriteLine(f1.x); //Invalid (Causes runtime error)
Console.ReadLine();

```



Variable of Class: this is a **copy** of **class** which is **not initialized**, so by using this we can't **call** any **members** of that **class**.

Instance of Class: this is a **copy** of **class** which is **initialized** by using the **new** keyword and by using this we can **call** **members** of that **class**.

Reference of Class: this is a **copy** of **class** which is **initialized** by using any **existing instance** of that class and this works **same** as an **instance**. By using the **reference** also, we can **call members** of that **class**.

What happens internally when we create the instance of a class?

Ans: When we **create** the **instance** of any class internally following **actions** will take place:

1. Reads the classes to identify their members.
2. Invokes the constructors of all those classes.
3. Allocates the memory that is required for execution.

Constructor

This is a **special method** present under a class **responsible** for **initializing** the **data members (fields)** of that class. This method is invoked **automatically** when we **create** the **instance** of class. The **name** of **constructor** method is **same name** of the class and more over it's a **non-value** returning method. Every class requires a **constructor** in it, if we want to **create** the **instance** of that class or else, we can't **create** the **instance** of that class.

Note: While defining a class it's the **responsibility** of **developers** to define a **constructor explicitly** under their class, and if they **fail** to do so, **on-behalf** of the **developer** an **implicit constructor** gets defined in those classes; so, till now we are creating **instances** of the classes we defined, by using those **implicit constructors** only.

For example, if we define a class as below:

```
class Test
{
    int i = 10; string s; bool b;           //Fields
}
```

After compilation of the above class, it will be as below with an implicit constructor:

```
class Test
{
    int i = 10; string s; bool b;           //Fields
    public Test()                          //Implicit Constructor
    {
        i = 10; s = null; b = false;
    }
}
```

- Implicit constructors are **public**.
- While declaring a **field** if we assign any value to it, then **constructor** will initialize the field with that value only or else it will initialize the field with default value of that type.
- We can also define our own constructors in classes and if we do that **implicit constructor** will not be defined.

Syntax to define a Constructor Explicitly:

```
[<modifiers>] <Class_Name>(<Parameter List> )
{
    -Statements to execute
}
```

To test defining a **constructor explicitly**, add a new class in the project naming it as **"ConDemo.cs"** and write the below code in it:

```
internal class ConDemo
{
    public ConDemo() //Explicit Constructor
    {
        Console.WriteLine("Constructor is called.");
    }
    public void Demo() //Method
    {
        Console.WriteLine("Method is called.");
    }
    static void Main()
    {
        ConDemo cd1 = new ConDemo();
        ConDemo cd2 = new ConDemo();
        ConDemo cd3 = cd2;
        cd1.Demo();
        cd2.Demo();
        cd3.Demo();
    }
}
```

```

    Console.ReadLine();
}
}

```

Constructor of a class must be **explicitly called** for execution and we do that while **creating** the **instance** of class as following:

Syntax: <Class_Name> <Instance_Name> = new <Constructor_Name> ([<List of Values>])

Example: ConDemo obj = new **ConDemo** ();

Calling the constructor

If **constructor** is **called** then only **memory allocation** is performed, so **instances** of a class will have memory allocation because they **call** the **constructor** explicitly whereas **reference** of class will not have memory allocation because they do not **call** the **constructor**.

Constructors are defined implicitly or explicitly?

Ans: Either or.

Constructors must be called explicitly or called implicitly?

Ans: Must be called explicitly.

Constructors are of 2 types:

1. Default or Parameter-less
2. Parameterized

Constructors can also be **parameterized** i.e., just like a **method** can be defined with **parameters**; **constructor** can also be defined with **parameters**. If **constructor** is defined with **parameters**, we call it as “**Parameterized Constructor**” where as a **constructor** without any **parameters** is called as “**Default/Parameter-less Constructor**”.

Default constructors can be defined either **explicitly** or will be defined **implicitly** provided there is no **explicit** constructor defined under that class, whereas **implicit** constructors will never be **parameterized** i.e., if a constructor is **parameterized** then it is very **true**, that it is an **explicit** constructor.

Note: if **Constructors** of a class are **parameterized** then **values** to those **parameters** should be sent while creating **instance** of that class.

To test **Parameterized Constructors**, add a new class in our project naming it as “**ParamConDemo.cs**” and write the below code in it:

```

internal class ParamConDemo
{
    public ParamConDemo(int i)
    {
        Console.WriteLine($"Parameterized constructor is called: {i}");
    }
    static void Main()

```

```

{
    ParamConDemo cd1 = new ParamConDemo(100);
    ParamConDemo cd2 = new ParamConDemo(200);
    ParamConDemo cd3 = new ParamConDemo(300);
    Console.ReadLine();
}
}

```

Why to define a constructor explicitly in our class when there are implicit constructors?

Ans: We define constructors explicitly in our class for various reasons like:

1. Implicit constructors are parameter-less which will initialize fields of a class either with a **default value** of that type or a **fixed given value**, even if we create multiple instances of class, whereas if constructors are defined explicitly (**parameterized**), then we get a chance of passing new values to the **fields** every time the **instance** of class is created.

To test this, add a new class under our project naming it as "Second.cs" and write the below code in it:

```

internal class Second
{
    public int x;           //Field
    public Second(int x)    //Variable
    {
        this.x = x;
    }
}

```

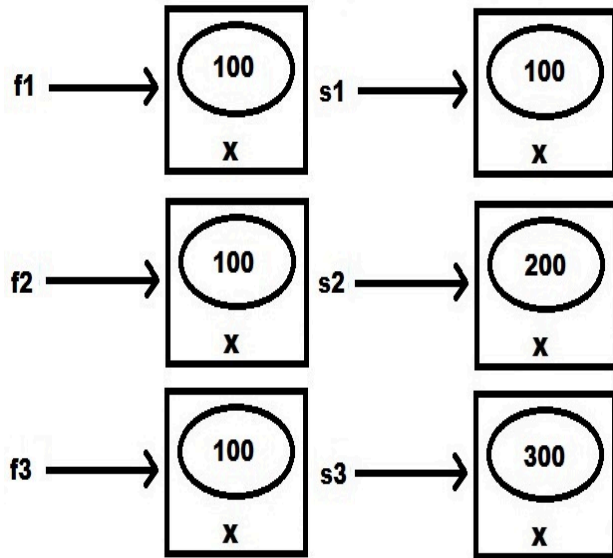
Note: "this" is a keyword which refers to the **class** and by using this we can access **non-static members** of a class from other **non-static blocks** when there is a **naming conflict**.

Earlier we have defined a class "First" with a public field "x" in it, and we have initialized it with a static value "100", and in the above class also we have a public field "x" which was initialized thru a **Constructor**, so in the 1st case even if we create multiple instances of class **First**; under every instance the value of "x" will be "100" only whereas in case of class **Second** for each instance of class we create we can pass a new value for initialization because initialization is performed thru the **constructor**.

To test that add a new class in our project naming it as "TestClasses.cs", and write the below code in it:

```
internal class TestClasses
```

```
{  
    static void Main()  
    {  
        First f1 = new First();  
        First f2 = new First();  
        First f3 = new First();  
        Console.WriteLine(f1.x + " " + f2.x + " " + f2.x);  
  
        Second s1 = new Second(100);  
        Second s2 = new Second(200);  
        Second s3 = new Second(300);  
        Console.WriteLine(s1.x + " " + s2.x + " " + s2.x);  
        Console.ReadLine();  
    }  
}
```



2. Every class **requires** some values for **execution** and the values that are **required** for a class to **execute** should be **passed** to the class with the help of a **constructor**.
3. Just like **parameters** of a method will make a **method dynamic**, same as that **parameters** of **constructor** will make the whole class **dynamic**.

Static Modifier

It is a **keyword** using which we can declare a **class** and its **members** as **static** i.e., if static keyword is pre-fixed before a class or its members then they will become **static** or else by default every class and its member are **non-static** only.

Members of a class are divided into 2 categories, like:

1. Non-Static or Instance Members
2. Static Members

Members that require instance of a class for initialization and execution are known as **non-static** or **instance members**, whereas members that doesn't require instance of the class for **initialization** and **execution** are known as **static** members.

Non-Static Fields Vs Static Fields:

- ◆ If a field is explicitly declared by using static modifier it is a **static field**, whereas rest of every other **field** is **non-static** only.

```

class Test
{
    int x = 100;           //Non-Static
    static int y = 200;    //Static
    static void Main()
    {
        int z = 300;      //Static
    }
}

```

Note: variables declared under **static blocks** are also **static**.

- ❖ **Static** fields of a class are initialized **immediately** once the **execution** of that class starts whereas **non-static** fields are initialized only after **creating** the **instance** of that class as well as each and every time a **new instance** is created.
- ❖ In the **life cycle** of a class a **static** field gets initialized **1 & only 1** time whereas a **non-static** field gets initialized for **"0"** times if **no instances** are created & **"n"** times if **"n" instances** are created.
- ❖ The initialization of **non-static** fields is associated with a **constructor** call, so the best place to **initialize non-static** fields is a **constructor**.

Note: **static** fields can also be **initialized** thru **constructor** but still we never do that because, it's a **single copy** thru out the **life cycle** of a class and every new **instance** will override the **old values**.

Constant Fields: If a field is explicitly declared by using **"const"** keyword we call it as a **constant** field and those **constant** field can't be modified once after their declaration, so it is must to initialize them at the time of declaration only because they do not have a default value.

E.g.: const float pi = 3.14f;

The behavior of a **constant** field will be very similar to the behavior of a **static** field i.e., initialized immediately once the execution of class starts maintaining a single copy thru-out the life cycle of a class and the only difference between **static** and **constant** fields is static fields can be **modified** but not constant fields.

ReadOnly Fields: If a field is explicitly declared by using **"readonly"** keyword we call it as a **readonly** field and like **constant** fields, **readonly** fields also can't be modified, but after their initialization i.e., it's not mandatory to initialize **readonly** fields at the time of declaration because they can also be **initialized** after their declaration i.e., under a **constructor**.

E.g.: readonly bool flag; //Declaration

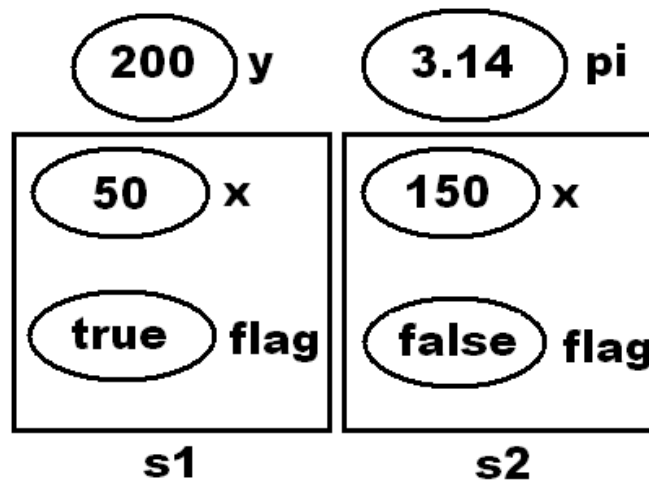
- ❖ The behavior of **readonly** fields will be like the behavior of **non-static** fields i.e., they are initialized only after creating the instance of class and maintains a separate copy for each instance that is created.
- ❖ The only difference between **non-static** and **readonly** fields is **non-static** fields can be modified but not **readonly** fields.
- ❖ The difference between **constant** and **readonly** fields is **constant** is a single fixed value for the whole class whereas **readonly** is a fixed value specific to each instance of the class.

To test all the above add a new class in our project naming it as "Fields.cs" and write the below code in it:

```

internal class Fields
{
    int x;
    static int y = 200;
    const float pi = 3.14f;
    readonly bool flag;
    public Fields(int x, bool flag)
    {
        this.x = x;
        this.flag = flag;
    }
    static void Main()
    {
        Console.WriteLine("Static field y is: " + y);
        Console.WriteLine("Constant field pi is: " + pi);
        y = 500; //Can be modified
        //pi = 5.67f; //Can't be modified & error if un-commented
        Console.WriteLine("Modified static field y is: " + y);
        Console.WriteLine("-----");
        //Creating instances of the class
        Fields s1 = new Fields(50, true);
        Fields s2 = new Fields(150, false);
        Console.WriteLine("Non-Static Fields: " + (s1.x + " " + s2.x));
        Console.WriteLine("ReadOnly Fields: " + (s1.flag + " " + s2.flag));
        s1.x = 100; //Can be modified
        s2.x = 300; //Can be modified
        //s1.flag = false; //Can't be modified & Error if un-commented
        //s2.flag = true; //Can't be modified & Error if un-commented
        Console.WriteLine("Modified Non-Static Fields: " + (s1.x + " " + s2.x));
        Console.ReadLine();
    }
}

```



Note: While accessing **fields** of a class from other classes use class name for accessing **static** and **constant** fields whereas use **instance** of class for accessing **non-static** and **readonly** fields.

- **Static field** initializes immediately once the execution of class starts maintaining a **single copy** thru out the life cycle of class and its value is **modifiable**.
- **Constant field** also initializes immediately once the execution of class starts maintaining a **single copy** thru out the life cycle of class and its value is **non-modifiable**.
- **Non-static field** initializes only after creating the instance of class, as well as for **each instance** of the class that is created, maintaining a **separate copy** for each instance and its value is **modifiable**.
- **Readonly field** also initializes only after creating the instance of class, as well as for **each instance** of the class that is created, maintaining a **separate copy** for each instance and its value is **non-modifiable**.

Non-Static Methods Vs Static Methods:

If a method is explicitly declared by using **static** keyword, then it is a **static** method whereas rest of every other method is **non-static** only.

While defining methods, if a method is **non-static** and if we want to consume any **static** members of class in it, we can consume them directly whereas if the method is **static**, we can consume **non-static** members of class in that method only by using class **instance**.

Rules for consuming members within a class:

Static Member => Static Block	//Direct Access
Static Member => Non-Static Block	//Direct Access
Non-Static Member => Non-Static Block	//Direct Access
Non-Static Member => Static Block	//Can be accessed only by using the class instance

Rules for consuming members out of the class:

Static Members	//Using class name
Non-Static Members	//Using class instance

To test all the above add a new **"Code File"** in the project naming it as **"TestMethods.cs"** and write the below code in it:

```
namespace OOPSPProject
{
    internal class Methods
    {
        int x = 200;
        static int y = 100;
        public void Add()
        {
            Console.WriteLine(x + y);
        }
        public static void Sub()
```



```

{
    Methods m = new Methods();
    Console.WriteLine(m.x - y);
}
}
internal class TestMethods
{
    static void Main()
    {
        Methods obj = new Methods();
        obj.Add(); //Add is non-static so calling it with instance
        Methods.Sub(); //Sub is static so calling it with class name
        Console.ReadLine();
    }
}
}

```

Non-Static Constructor Vs Static Constructor:

- A Constructor if explicitly declared by using static modifier is a static Constructor whereas rest of the other are non-static only and till now every Constructor, we defined is non-static only.
- Static Constructors are implicitly called whereas non-static Constructors must be explicitly called.
- As we are aware that Constructors are responsible for initializing fields in a class; Non-Static Constructor will initialize Non-Static and Readonly Fields, whereas Static Constructor will initialize Static and Constant fields.
- Static Constructor executes immediately once the execution of class starts and more over it is the first block of code to execute in a class, whereas Non-Static Constructor gets executed only after creating the instance of class as well as each and every time a new instance is created i.e., Static Constructor executes 1 and only 1 time in the life cycle of a class whereas Non-Static Constructor get executed for "0" times if no instances are created and "n" times if "n" instances are created.
- Static Constructor can't be parameterized because they are implicitly called and more over it's the first block of code to execute in a class, so we don't have any chance of sending values to its parameter's whereas parameterized Non-Static Constructors can be defined.

Note: We have already learnt earlier that, every class will contain an **implicit** constructor if not defined **explicitly** and those **implicit** constructors are defined based on the following criteria:

1. Non-static constructor will be defined in every class except in a static class.
2. Static constructor will be defined only if the class contains any static fields.

```

class Test                //Case 1
{
}

```

***After compilation there will be a non-static constructor in class.**

```

class Test                //Case 2

```

```
{
    int i = 10;
}
```

*After compilation there will be a non-static constructor in class.

```
class Test                //Case 3
{
    static int i = 100;
}
```

*After compilation there will be both static and non-static constructors also.

```
static class Test         //Case 4
{
}
```

*After compilation there will not be any constructor in class.

```
static class Test         //Case 5
{
    static int i = 100;
}
```

*After compilation there will be a static constructor in class.

To test all the above add a new class in the project naming it as "Constructors.cs" and write the below code in it:

```
internal class Constructors
{
    static Constructors()
    {
        Console.WriteLine("Static constructor is called.");
    }
    Constructors()
    {
        Console.WriteLine("Non-static constructor is called.");
    }
    static void Main()
    {
        Console.WriteLine("Main method is called.");
        Constructors c1 = new Constructors();
        Constructors c2 = new Constructors();
        Constructors c3 = new Constructors();
        Console.ReadLine();
    }
}
```

Static Class: These are introduced in **C# 2.0**. If a class is **explicitly** declared by using **static** modifier, we call it as a **static class** and this class can contain only **static members** in it. We can't create the **instance** of **static class** and more over it is not required also.

```
static class Class1
{
    //Define only static members here.
}
```

Note: **Console** is a static class in our **Libraries** so every member of **Console** class is a **static** member only and to check that, right click on **Console** class in **Visual Studio** and choose the option “Go to definition” which will open “Metadata” or “Source Code” of that class.

Entity

Any **living** or **non-living** object that is associated with a set of **attributes** is known as an **entity** and application development is all about **dealing** and **managing** these **entities** only. To develop an application, we follow the below process:

Step 1: Identify each **entity** that is associated with the **application**.

- **School Application:** Student, Teacher, Book
- **Retail Business Application:** Customer, Employee, Product, Supplier

Step 2: Identify each **attribute** of that **entity**.

- **Student:** Id, Name, Address, Phone, Class, Section, Fees, Marks, Grade
- **Teacher:** Id, Name, Address, Phone, Qualification, Subject, Salary, Designation
- **Customer:** Id, Name, Address, Phone, Balance, Account Type, EmailId, PanCard, Aadhar
- **Employee:** Id, Name, Address, Phone, Job, Salary, Department, EmailId, PanCard

Step 3: Design a **Database** based on the following **guidelines**:

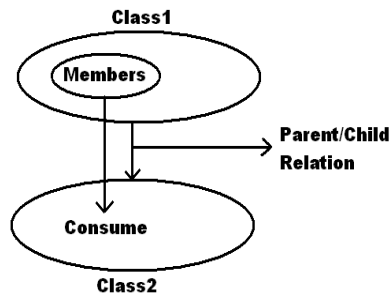
- Create a table representing each entity.
- Every column of the table should represent each attribute of the entity.
- Each record under table should be a unique representation for the entity.

Step 4: Design an application by using any **Programming Language** of your choice which should act as an **UI (User Interface)** between the **End User** and **Database** in managing the data present under **Database**, by adopting following **guidelines**:

- Define a class where each class should represent an entity.
- Define properties where each property should be a representation for each attribute.
- Each instance of the class we create will be a unique representation for each entity.

Inheritance

It is a process of consuming members that are defined in one class from other classes by establishing **parent/child** relationship between the classes, so that **child class** can consume members of its **parent class** as if they are **owner** of those members.



Note: Child class even if it can **consume** members of its parent class as an **owner**, still it can't access **private** members of their **parent** like **Constructors** and **Finalizers**

Syntax:

```
[<modifiers>] class <CC Name> : <PC Name>
```

Example:

```
class Class1
{
    -Define Members
}
class Class2 : Class1
{
    -Consume members of parent i.e., Class1 from here
}
```

To test inheritance, add a new class under the project naming it as "Class1.cs" and write the below code in it:

```
internal class Class1
{
    public Class1()
    {
        Console.WriteLine("Class1 constructor is called.");
    }
    public void Test1()
    {
        Console.WriteLine("Method 1");
    }
    public void Test2()
    {
        Console.WriteLine("Method 2");
    }
}
```

Now add another class in the project naming it as "Class2.cs" and write the below code in it:

```

internal class Class2 : Class1
{
    public Class2()
    {
        Console.WriteLine("Class2 constructor is called.");
    }
    public void Test3()
    {
        Console.WriteLine("Method 3");
    }
    public void Test4()
    {
        Console.WriteLine("Method 4");
    }
    static void Main()
    {
        Class2 c = new Class2();
        c.Test1(); c.Test2();    //Calling members of parent class
        c.Test3(); c.Test4();    //Calling members of current class
        Console.ReadLine();
    }
}

```

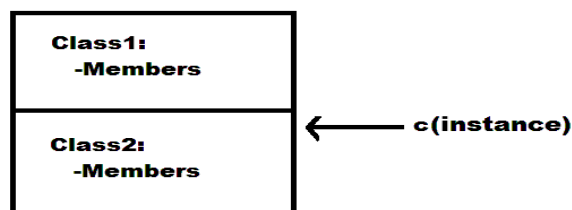
Rules and regulations that has to be followed while working with Inheritance:

Rule 1: In inheritance parent class **Constructor** must be **accessible** to child class or else **inheritance** will **not** be possible. The reason why parent's **Constructor** should be **accessible** to child is, because whenever **child** class instance is created, control first jumps to child class **Constructor** and child class **Constructor** will in turn call its parent class **Constructor** for execution and to test this, place a break point at child class's **Main** method and debug the code by hitting **F11**.

The reason why child **Constructor** calls its parent class **Constructor** is, because if child class wants to consume members of its parent class, they must be **initialized** first and then only child classes can consume them and we are already aware that members of a class are initialized by its own **Constructor**.

Note: Constructors are **never** inherited i.e., Constructors are **specific** to any class which can initialize members of that particular class only but not of parent or child classes.

When we **create** the **instance** of any class, it will first read all its parent classes to gather the information of members that are present under those classes, so in our previous case when the instance of **Class2** is created it gathers information of **Class1** also as following:



Rule 2: In inheritance child class can **access** members of their parent class whereas parent classes **can never access** members of their child class which are **purely defined** under the child class. To test this, re-write the code under **Main** method of child class i.e., **Class2** as following:

```
Class1 p = new Class1();
p.Test1(); p.Test2();           //Valid
//p.Test3(); p.Test4();        //Invalid and in-accessible
Console.ReadLine();
```

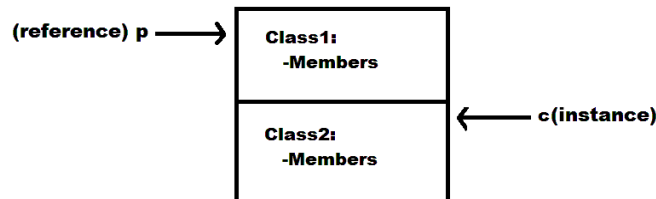
Rule 3: Earlier we have learnt that variable of a class can be initialized by using instance of same class to make it as a reference, for example:

```
Class2 c1 = new Class2();
Class2 c2 = c1;
```

Same as the above we can also initialize variables of parent class by using its child classes instance as following:

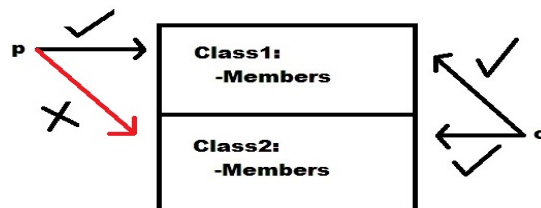
```
Class2 c = new Class2();
Class1 p = c;
```

In this case both parent class reference and child class instance will be accessing the same memory, but owner of that memory is child class instance.



In the above case even if parent class reference is initialized by the child class instance and consuming the memory of child class instance, now also it is not possible to access the member's which are **purely defined** under the child class and to test that rewrite the code under Main method of child class i.e., **Class2** as following:

```
Class2 c = new Class2();
Class1 p = c;
p.Test1(); p.Test2();           //Valid
//p.Test3(); p.Test4();        //Invalid and in-accessible now also
Console.ReadLine();
```



Note: We can never initialize child class variables by using parent class instance either **implicitly** or **explicitly** also.

```
Class1 p = new Class1(); //Creating parent class instance
Class2 c = p;           //Invalid (Implicit conversion and compile time error)
Class2 c = (Class2)p;   //Invalid (Explicit conversion and runtime error)
Class2 c = p as Class2; //Invalid (Explicit conversion and runtime error)
```

We can **initialize** child class variables by using a parent class **reference** which is **initialized** by using the same child class **instance** by performing an **explicit** conversion.

Creating parent's reference by using child class instance:

```
Class2 c = new Class2();  
Class1 p = c;
```

Initializing child's variable by using the above parent's reference:

```
Class2 obj = (Class2)p; //Valid (Explicit)
```

Or

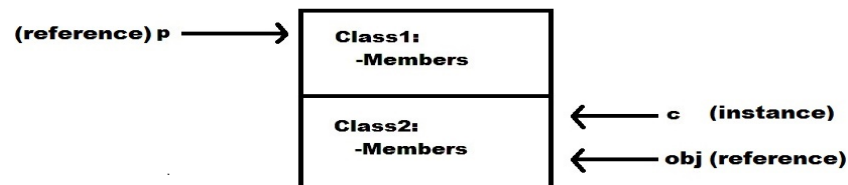
```
Class2 obj = p as Class2; //Valid (Explicit)
```

```
Child Instance    => Parent Reference    //Valid
```

```
Child Instance    => Parent Reference    => Child Reference    //Valid
```

```
Parent Instance   => Child Reference      //Invalid
```

Note: in the above case the new reference **"obj"** also start's accessing the same memory allocated for the instance **"c"** and with the new reference we call the members of both **"Class1"** and **"Class2"** also.

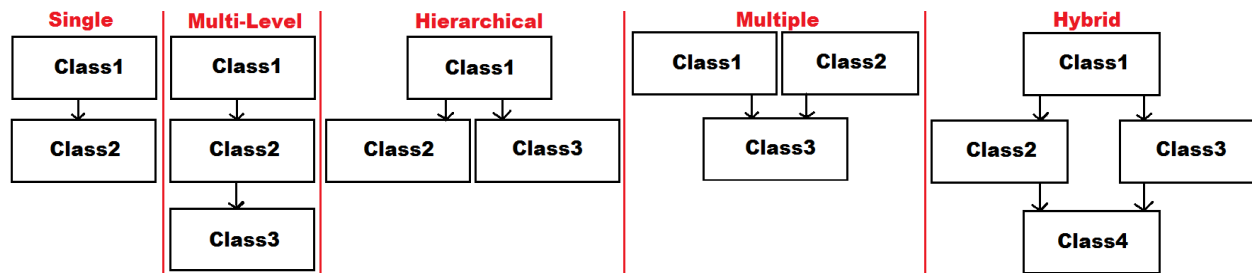


Rule 4: Every class that is **pre-defined** or **user-defined** has a **default** parent class i.e., **Object** class of **System** namespace. **Object** is the **ultimate parent** of all classes in **.NET** class hierarchy providing **low level services** to child classes. So, every class by default contains 4 methods that are inherited from the **"Object"** Class and those are **"Equals"**, **"GetHashCode"**, **"GetType"** and **"ToString"**, and these 4 methods can be called or consumed from any class. To test this, re-write code under **Main** method of child class (**Class2**) as following:

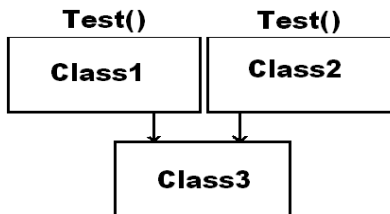
```
Object obj = new Object();  
Console.WriteLine(obj.GetType() + "\n");  
Class1 p = new Class1();  
Console.WriteLine(p.GetType() + "\n");  
Class2 c = new Class2();  
Console.WriteLine(c.GetType());  
Console.ReadLine();
```

Types of Inheritance: This talks about no. of child classes a parent has or the no. of parent classes a child has. According to the standards of **Object-Oriented Programming** we have **5** types of inheritances, and they are:

- i. Single
- ii. Multi-Level
- iii. Hierarchical
- iv. Multiple
- v. Hybrid



Rule 5: Both Java and .NET Language's doesn't provide the support for Multiple and Hybrid inheritances thru classes, and what they support is Single, Multi-Level and Hierarchical inheritances only because Multiple Inheritance suffers from ambiguity problem, for example:



Note: C++ Language supports all 5 types of Inheritances because it is the 1st Object Oriented Programming Language that came into existence and at the time of its introduction, this problem was not anticipated.

Rule 6: In the first rule of inheritance, we have discussed that whenever the instance of child class is created it will implicitly call its parent class constructor for execution, but this implicit calling will take place only if parent classes Constructor is "default or parameter less", whereas if at all the parent classes Constructor is parameterized then child class Constructor can't implicitly call parent class Constructor for execution because it requires parameter values. To resolve the above problem developer needs to explicitly call parent classes Constructor from child class Constructor by using "base" keyword and pass all the required parameter values.

To test the above, re-write constructor of parent class i.e., Class1 as following:

```

public Class1(int i)
{
    Console.WriteLine("Class1 constructor is called: " + i);
}
  
```

Now when we run child class i.e., Class2, we get an error stating that there is no value sent to formal parameter "i" of Class1 (Parent Class) and to resolve this problem re-write constructor of Class2 as following:

```

public Class2(int x) : base(x)
{
    Console.WriteLine("Class2 constructor is called.");
}
  
```

In the above case child classes constructor is also parameterized so while creating the instance of child class we need to explicitly pass all the required values to its constructor and those values are first loaded into the constructor and from there those values are passed to parent classes constructor thru the "base" keyword, and to test this go to Main method of Class2, and re-write the code in it as below and debug:

```

Class2 c = new Class2(50);
  
```


How do we use inheritance in application development?

Ans: Inheritance is a process which comes into picture from the **initial stages** of an application development. As discussed earlier, if we want to develop an **application**, we need to follow the below process:

Step 1: Identification of the **Entities**.

E.g.: School Application: **Student**, **Teaching Staff**, **Non-Teaching Staff**

Step 2: Identification of **Attributes** for each **Entity**.

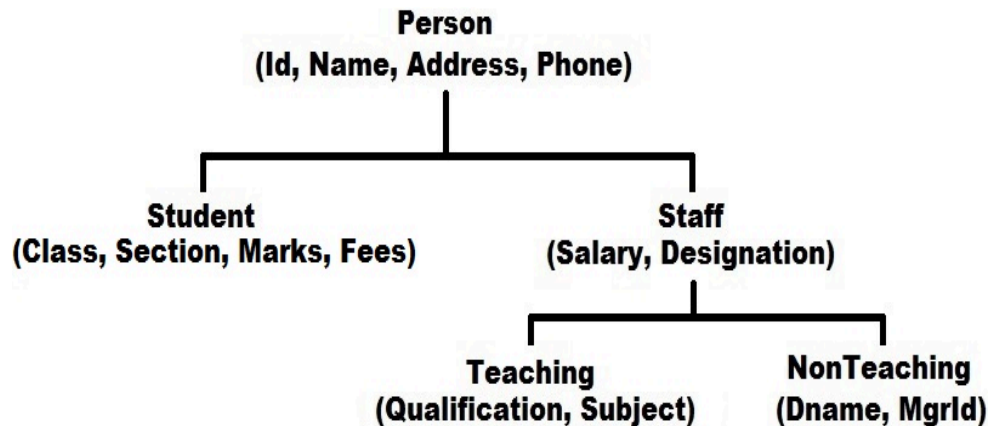
Student	Teaching Staff	Non-Teaching Staff
Id	Id	Id
Name	Name	Name
Phone	Phone	Phone
Address	Address	Address
Class	Designation	Designation
Section	Salary	Salary
Marks	Qualification	Dname
Fees	Subject	MgrId

Step 3: Designing the **Database**.

Step 4: Developing an **application** that works like an **UI**.

While developing the **application**, to bring **re-usability** into the applications we use **inheritance** and to do that follow the below guidelines:

1. Identify all the **common attributes** between **entities** and put them in a **hierarchical order** as below:



2. Now define **classes** based on the above **hierarchy**:

```
public class Person
{
    public int Id;
    public string Name, Phone, Address;
}
```

```
public class Student : Person
{
    int Class;
    char Section;
    float Marks, Fees;
}
public class Staff : Person
{
    public double Salary;
    public string Designation;
}
public class Teaching : Staff
{
    string Subject, Qualification;
}
public class NonTeaching : Staff
{
    int MgrId;
    string Dname;
}
```

Polymorphism

Behaving in different ways depending upon the input received is known as **Polymorphism** i.e., whenever **input** changes then automatically the **output** or **behaviour** also changes accordingly. This can be implemented in our language in 3 different ways:

1. Overloading
2. Overriding
3. Hiding/Shadowing

Overloading: This is again of different types like **Method Overloading**, **Operator Overloading**, **Constructor Overloading**, **Indexer Overloading** and **De-constructor Overloading**.

Method Overloading: It is an approach of defining multiple methods in a class with the **same name** by changing their **parameters**. Changing **parameters** means we can change any of the **following**:

1. Change the no. of parameters passed to method.
2. Change the type of parameters passed to method.
3. Change the order of parameters passed to method.

- public void Show()
- public void Show(int i)
- public void Show(string s)
- public void Show(int i, string s)
- public void Show(string s, int i)

Note: in overloading a **return type** change without parameter change is not taken into **consideration**, for example:

public string Show() => Invalid

To test **method overloading**, add a new class in the project naming it as **"OverloadMethods.cs"** and write the following code in it:

```
internal class OverloadMethods
{
    public void Show()
    {
        Console.WriteLine(1);
    }
    public void Show(int i)
    {
        Console.WriteLine(2);
    }
    public void Show(string s)
    {
        Console.WriteLine(3);
    }
    public void Show(int i, string s)
    {
        Console.WriteLine(4);
    }
}
```

```

public void Show(string s, int i)
{
    Console.WriteLine(5);
}
static void Main()
{
    OverloadMethods obj = new OverloadMethods();
    obj.Show();
    obj.Show(10);
    obj.Show("Hello");
    obj.Show(10, "Hello");
    obj.Show("Hello", 10);
    Console.ReadLine();
}
}

```

What is Method Overloading?

Ans: It's an approach of defining a **method** with **multiple behaviors** and those **behaviors** will vary based on the **number**, **type** and **order** of **parameters**. For example, **IndexOf** is an overloaded **method** under **String** class which returns the **index** position of a **character** or **string** based on the **input** values of that method, for example:

```

string str = "Hello World";
str.IndexOf('o'); => 4 => Returns the first occurrence of a character
str.IndexOf('o', 5); => 7 => Returns the next occurrence of a character

```

Note: **Write** and **WriteLine** methods of **Console** class are also overloaded for printing any type of value that is passed as input to the method, as following:

- WriteLine()
 - WriteLine(int value)
 - WriteLine(bool value)
 - WriteLine(double value)
 - WriteLine(string value)
 - WriteLine(string format, params object[] values)
 - +12 more overloads
-

Inheritance based overloading: It's an approach of **overloading** parent classes' **methods** under the child class, and to do this child class doesn't require **taking any permission** from the parent class, for example:

```

Class1
public void Test()

Class2 : Class1
public void Test(int i)

```

Method Overriding: it's an approach of **re-implementing** parent classes' methods under child class exactly with the same **name** and **signature (parameters)**.

Difference between Method Overloading and Method Overriding:

Method Overloading	Method Overriding
It's all about defining multiple methods with the same name by changing their parameters.	It's all about defining multiple methods with the same name and same parameters.
This can be performed with-in a class or between parent-child classes also.	This can be performed only between parent-child classes but can't be performed with-in a class.
To overload parent's method under child, child doesn't require any permission from parent.	To override parent's method under child, parent should first grant the permission to child.
This is all about defining multiple behaviours to a method.	This is all about changing existing behaviour of a parent's method under child.

How to override a parent classes method under child class?

Ans: To override any parent classes' method under child class, first that method should be declared "overridable" by using "virtual" modifier in parent class as following:

```
Class1 =>
        public virtual void Show()           //Overridable
```

Every **virtual** method of parent class can be **overridden** by child class, if required by using "override" modifier as following:

```
Class2 : Class1 =>
        public override void Show()           //Overriding
```

Note: overriding **virtual** methods of parent class under child class is **not mandatory** for child class.

In **overriding**, parent class defines a method in it as **virtual** and gives it to the child class for **consumption**, so that it's giving a permission to the child class either to consume the method "as is" or **override** the method as per its requirement, if at all the original behavior of that method is not **satisfactory** to the child class.

To test **inheritance-based method overloading** and **method overriding**, add a new class in the project naming it as "LoadParent.cs" and write the following code in it:

```
internal class LoadParent
{
    public void Test()
    {
        Console.WriteLine("Parent Class Test Method Is Called.");
    }
    public virtual void Show() //Overridable
    {
        Console.WriteLine("Parent Class Show Method Is Called.");
    }
    public void Display()
    {
        Console.WriteLine("Parent Class Display Method Is Called.");
    }
}
```

Now add another class in the project naming it as "LoadChild.cs" and write the following code in it:

```
internal class LoadChild : LoadParent
{
    //Overloading parent's Test method in child
    public void Test(int i)
    {
        Console.WriteLine("Child Class Test Method Is Called.");
    }
    static void Main()
    {
        LoadChild c = new LoadChild();
        c.Test();           //Executes parent class Test method
        c.Test(10);         //Executes child class Test method
        c.Show();           //Executes parent class Show method
        c.Display();        //Executes parent class Display method
        Console.ReadLine();
    }
}
```

Inheritance-Based Overloading: In the above classes **Test** method of parent class has been **overloaded** in child class and then by using child class instance we are able to call both parent and child classes methods also, from the child class.

Method Overriding: In the above classes **Show** method of parent class is declared **virtual** which gives a chance for child classes to **override** that method but the child class did not **override** the method, so a call to that method by using child classes instance will invoke the parent classes **Show** method only and this proves us **overriding** is **optional** and to confirm that run the child class **LoadChild** and watch the output of **Show** method.

In this case if child class overrides the parent classes **virtual** method, then a call to that method by using child class **instance** will execute or invoke its own method but not of the parent classes, and to test that add a new method in class **LoadChild** as following:

```
//Overriding parent's Show method in child class
public override void Show()
{
    Console.WriteLine("Child Class Show Method Is Called.");
}
```

Now if we run the child class i.e., **LoadChild** and watch the output of **Show** method we will notice child classes **Show** method getting executed in place of parent classes **Show** method and this is what we call as changing the behavior.

Can we override any parent classes' methods under child classes without declaring them as virtual?

Ans: No.

Can we re-implement any parent classes' methods under the child classes without declaring them as virtual?

Ans: Yes.

We can re-implement a parent class method under the child class by using 2 different approaches:

- Overriding
- Hiding/Shadowing

Method Hiding/Shadowing: This is also an approach of re-implementing parent classes methods under child class exactly with the same name and parameters just like overriding but the difference between the 2 is; in overriding child class can re-implement only virtual methods of parent class where as in-case of hiding/shadowing child class can re-implement any method of the parent class i.e., even if the method is not declared as virtual also re-implementation can be performed.

Class1 =>

public void Display()

Class2 : Class1 =>

public [new] void Display() //Hiding/Shadowing

In the above case using "new" keyword while re-implementing the method in child class is only optional and if we don't use it, compiler gives a warning message at the time of compilation, saying that there is already a method with the same name in parent class and your new method in child class will hide that old method, so by using "new" keyword we are informing the compiler that we are intentionally defining a new method with the same name and parameters under our child class.

Before testing hiding/shadowing first run the child class i.e., LoadChild and watch the output of Display method and here we notice that parent classes Display method getting executed, now add a new method in the child class LoadChild as following:

```
//Hiding/Shadowing parent class Display method in child class
public new void Display()
{
    Console.WriteLine("Child Class Display Method Is Called.");
}
```

Now run the child class LoadChild again and watch the difference in output i.e., in this case child classes Display method is called in place of parent class Display method.

In the above 2 classes we have performed the following:

LoadParent

```
public void Test()
public virtual void Show()
public void Display()
```

LoadChild : LoadParent

```
public void Test(int i)      => Overloading
public override void Show()      => Overriding
public new void Display()      => Hiding/Shadowing
```

In case of **Overriding** and **Hiding**, after **re-implementing** the parent classes methods under child class, instance of child class starts calling its own methods but not of parent class, whereas if required there is still a chance of calling those parent class methods from child class in 2 different ways:

1. By creating the parent classes **instance** under the child class, we can call parent class methods from child class and to test that re-write code under Main method of child class i.e., LoadChild as following:

```
LoadParent p = new LoadParent();
p.Show();           //Executes parent class Show method
p.Display();        //Executes parent class Display method
LoadChild c = new LoadChild();
c.Show();           //Executes child class Show method
c.Display();        //Executes child class Display method
Console.ReadLine();
```

2. By using base keyword also, we can call parent class methods from child class, but keywords like **"this"** and **"base"** can't be used in **static blocks**.

To test this first add 2 new methods under the child class i.e., LoadChild as following:

```
public void PShow()
{
    base.Show();
}

public void PDisplay()
{
    base.Display();
}
```

In the above case the 2 new methods we defined in child class, acts as an **interface** in calling parent classes methods from the child class, so now by using child class instance only we can call both parent and child classes methods also.

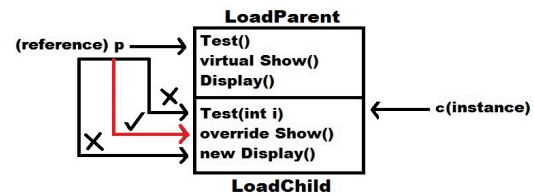
To test this, re-write code under Main method of child class i.e., LoadChild as following:

```
LoadChild c = new LoadChild();
c.PShow();           //Executes parent class Show method
c.PDisplay();        //Executes parent class Display method
c.Show();            //Executes child class Show method
c.Display();         //Executes child class Display method
Console.ReadLine();
```

Note: Earlier in the 3rd rule of **inheritance** we have learnt that parent class **reference** even if created by using child class **instance** can't access any members of the child class which are **purely defined** under child class but we have an exemption for that rule, that is, parent's reference can call or access **overridden members** of the child class because **overridden members** are not considered as **pure child class members** because they have been **re-implemented** with **permission** from the parent class only.

To test that re-write code under Main method of child class i.e., LoadChild as following:

```
LoadChild c = new LoadChild();
LoadParent p = c;
p.Show(); //Executes child class Show method
p.Display(); //Executes parent class Display method only
Console.ReadLine();
```



In the above case Display is considered as pure child class member only because it's re-implemented by child class without taking any permission from parent, so parent will never recognize it.

Polymorphism is divided into 2 types:

1. **Static or Compile-time Polymorphism (Early Binding)**
2. **Dynamic or Run-time Polymorphism (Late Binding)**

In **static** or **compile-time polymorphism**, the decision which **polymorphic** method must be executed for a method call is performed at **compile time**. Method **overloading** is an example for this and here compiler identifies which **overloaded** method it must execute for a particular method call at the time of program **compilation** by checking the type and number of parameters that are passed to the method and if no method matches the method call it will give an **error**.

In **dynamic** or **run-time polymorphism**, the decision which **polymorphic** method must be executed for a method call is made at **runtime** rather than **compile time**. Run-time **polymorphism** is achieved by method **overriding** because method **overriding** allows us to have methods in the **parent** and **child** classes with the same name and same parameters also. By runtime **polymorphism**, we can point to any child class by using the **reference** of the parent class, which is initialized by child class instance, so the **determination** of the method to be executed is based on the **instance** being referred to by **reference**.

<u>Static Polymorphism</u>	<u>Dynamic Polymorphism</u>
1. Occurs at compile-time.	1. Occurs at runtime.
2. Achieved through static binding.	2. Achieved through dynamic binding.
3. Method overloading should exist.	3. Method overriding should exist.
4. Inheritance is not involved.	4. Inheritance is involved.
5. Happens in the same class.	5. Happens between parent-child classes.
6. Reference creation thru instance is not required.	6. Requires parent class reference creation thru child class instance.

Operator Overloading

It's an approach of defining multiple behaviors to an operator, which varies based on the operands between which we use the operator. For example: "+" is an **addition operator** when used between **numeric operands** and it is a **concatenation operator** when used between **string operands**.

Number + Number => Addition
String + String => Concatenation

The behaviour for an **operator** is **pre-defined** i.e., developers or designers of the **language** have already implemented logic that must be executed when an **operator** is used between 2 **operands** under the **libraries** of the **language** with the help of a special method known as **"Operator Method"**.

Syntax of an Operator Method:

```
[<modifiers>] static <type> operator <opt>(<operand types>)  
{  
    -Logic  
}
```

- Operator methods must be static only.
- **<type>** refers to the return type of method i.e., when the operator is used between 2 types what should be the result type.
- **operator** is name of the method, which should be in lower case and can't be changed.
- **<opt>** refers to the operator for which we want to write behaviour like "+" or "-" or "==" , etc.
- **<operand types>** refers to type of operands between which we want to use the operator.

Under libraries, operator methods have been defined as following:

```
public static int operator +(int a, int b)  
public static int operator -(int a, int b)  
public static string operator +(string a, string b)  
public static string operator +(string a, int b)  
public static bool operator >(int a, int b)  
public static float operator +(int a, float b)  
public static decimal operator +(double a, decimal b)  
public static bool operator ==(string a, string b)  
public static bool operator !=(string a, string b)
```

Note: same as the **above** we can also define **operator methods** for using an **operator** between new types of **operands**.

To test **"Operator Overloading"**, **"Method Overriding"** and **"Hiding/Shadowing"** add a new class naming it as **"Matrix.cs"** under the project and write the below code:

```
internal class Matrix  
{  
    //Declaring attributes for a 2 * 2 Matrix  
    int a, b, c, d;
```

```

//Initializing attributes of the Matrix in constructor
public Matrix(int a, int b, int c, int d)
{
    this.a = a; this.b = b; this.c = c; this.d = d;
}
//Overriding the ToString() method inherited from Object class to return values of the Matrix in 2 * 2 format
public override string ToString()
{
    return a + " " + b + "\n" + c + " " + d + "\n";
}
//Implementing the + operator so that it can be used between 2 Matrix operands
public static Matrix operator +(Matrix obj1, Matrix obj2)
{
    Matrix obj = new Matrix(obj1.a + obj2.a, obj1.b + obj2.b, obj1.c + obj2.c, obj1.d + obj2.d);
    return obj;
}
//Implementing the - operator so that it can be used between 2 Matrix operands
public static Matrix operator -(Matrix obj1, Matrix obj2)
{
    Matrix obj = new Matrix(obj1.a - obj2.a, obj1.b - obj2.b, obj1.c - obj2.c, obj1.d - obj2.d);
    return obj;
}
//Re-Implementing the == operator using Hiding/Shadowing so that it can be used between 2 Matrix's to perform
    values equal comparison because original implementation is reference equal comparison
public static bool operator ==(Matrix obj1, Matrix obj2)
{
    if (obj1.a == obj2.a && obj1.b == obj2.b && obj1.c == obj2.c && obj1.d == obj2.d)
        return true;
    else
        return false;
}
//Re-Implementing the != operator using Hiding/Shadowing so that it can be used between 2 Matrix's to perform
    values not equal comparison because original implementation is reference not equal comparison
public static bool operator !=(Matrix obj1, Matrix obj2)
{
    if (obj1.a != obj2.a || obj1.b != obj2.b || obj1.c != obj2.c || obj1.d != obj2.d)
        return true;
    else
        return false;
}
}

```

`ToString` is a method defined in the parent class `"Object"` and by default that method returns `"Name"` of the type to which an `instance` belongs when we call it on any type's instance. `ToString` method is declared as `virtual` under the class `"Object"` so any child class can `override` it as per their `requirements` as we performed it in our `"Matrix"` class to change the `behaviour` of that method, so the new method will return `values` that are associated with `"Matrix"` but not the `type name`.

The “==” and “!=” operators are also implemented in the parent class “Object”, but their original behaviour is to perform a reference equal and reference not-equal comparison between type instances but not values equal and values non-equal comparison. We can also change the behaviour of those operator methods by using the concept of hiding (but not overriding because they are not declared as virtual) as we have done in our Matrix class, so that the 2 operators will now perform values equal and values not-equal comparison in place of reference equal and reference not-equal comparison.

To consume all the above add a new class TestMatrix.cs and write the below code:

```
internal class TestMatrix
{
    static void Main()
    {
        //Creating 4 instances of Matrix class with different values
        Matrix m1 = new Matrix(20, 19, 18, 17);
        Matrix m2 = new Matrix(15, 14, 13, 12);
        Matrix m3 = new Matrix(10, 9, 8, 7);
        Matrix m4 = new Matrix(5, 4, 3, 2);
        //Performing Matrix Arithmetic
        Matrix m5 = m1 + m2 + m3 + m4;
        Matrix m6 = m1 - m2 - m3 - m4;
        //Printing values of each Matrix:
        Console.WriteLine(m1);
        Console.WriteLine(m2);
        Console.WriteLine(m3);
        Console.WriteLine(m4);
        Console.WriteLine(m5);
        Console.WriteLine(m6);

        //Performing Matrix equal comparision
        if (m1 == m2)
            Console.WriteLine("Yes, m1 is equal to m2.");
        else
            Console.WriteLine("No, m1 is not equal to m2.");
        //Performing Matrix not equal comparision
        if (m1 != m2)
            Console.WriteLine("Yes, m1 is not equal to m2.");
        else
            Console.WriteLine("No, m1 is equal to m2.");
        Console.ReadLine();
    }
}
```

In the above case when we call WriteLine method by passing Matrix class instance as a parameter to it, will internally invoke the overloaded WriteLine Method which takes “Object” as a parameter and that method will internally call ToString method on that instance, and because we have overwritten the ToString method in our Matrix class, a call to it in WriteLine method will invoke Matrix classes ToString method which returns the values that are associated with Matrix instance and prints them in a 2 * 2 Matrix format (Dynamic Polymorphism).

Constructor Overloading

Just like **methods** in a class can be **overloaded**, **constructors** in a class also can be **overloaded** and it is called as **"Constructor Overloading"**. It's an approach of defining **multiple constructors** under a class and if **constructors** of a class are **overloaded** then instance of that class can be created by using any available **constructor** i.e., it is not **mandatory** to call any **particular** constructor for **instance** creation. To test this, add a new code file under the project naming it as **"TestOverloadCons.cs"** and write the below code in it:

```
namespace OOPSProject
{
    internal class OverloadCons
    {
        int i; bool b;
        public OverloadCons()
        {
            //Initializes i & b with default values
        }
        public OverloadCons(int i)
        {
            //Initializes b with default value and i with given value
            this.i = i;
        }
        public OverloadCons(bool b)
        {
            //Initializes i with default value and b with given value
            this.b = b;
        }
        public OverloadCons(int i, bool b)
        {
            //Initializes both i & b with given values
            this.i = i;
            this.b = b;
        }
        public void Display()
        {
            Console.WriteLine($"Value of i is: {i} and value of b is: {b}");
        }
    }
    internal class TestOverloadCons
    {
        static void Main()
        {
            OverloadCons c1 = new OverloadCons();
            c1.Display();

            OverloadCons c2 = new OverloadCons(10);
            c2.Display();
        }
    }
}
```

```

        OverloadCons c3 = new OverloadCons(true);
        c3.Display();

        OverloadCons c4 = new OverloadCons(10, true);
        c4.Display();
        Console.ReadLine();
    }
}
}

```

By overloading constructors in a class, we get a chance to initialize fields of that class in 3 different ways:

1. With a **default** or **parameter-less** constructor defined in class we can **initialize** all **fields** of that class with **default values**.
2. With a **parameterized** constructor defined in class we can **initialize** all **fields** of that class with **given values**.
3. With a **parameterized** constructor defined in class we can **initialize** some fields of that class with **default values** and some **fields** with **given values**.

Note: If a class contains **multiple attributes** in it and if we want to **initialize** them in a “mix & match” combination then we **overload** constructors, and the no. of **constructors** to be defined will be 2 power “n” where “n” is the no. of attributes. In our above class we have 2 attributes, so we have defined 4 constructors.

Copy Constructor

It is a **constructor** using which we can create a new **instance** of the class with the help of an **existing instance** of the **same class**, which **copies** the **attribute** values from the **existing instance** into the **new instance** and the main purpose of this constructor is to **initialize** a **new instance** with the values from an **existing instance**. The “Formal Parameter Type” of a copy constructor will be the same “Class Type” in which it is defined.

To test Copy Constructors, add a new class under the project naming it as “CopyConDemo.cs” and write the following code:

```

internal class CopyConDemo
{
    int Id;
    string Name;
    double Balance;
    public CopyConDemo(int Id)
    {
        this.Id = Id;
        Name = "Vijay";
        Balance = 5000.00;
    }
    public CopyConDemo(CopyConDemo cd)
    {
        this.Id = cd.Id;
        this.Name = cd.Name;
        this.Balance = cd.Balance;
    }
}

```

```

public void Display()
{
    Console.WriteLine($"Id: {Id}; Name: {Name}; Balance: {Balance}");
}
static void Main()
{
    CopyConDemo cd1 = new CopyConDemo(1005);
    cd1.Display();
    CopyConDemo cd2 = new CopyConDemo(cd1);
    cd2.Display();
    Console.WriteLine();
    cd1.Balance = 10000;
    cd1.Display();
    cd2.Display();
    Console.WriteLine();
    cd2.Balance = 20000;
    cd1.Display();
    cd2.Display();
    Console.ReadLine();
}
}

```

In the above case “cd2” is a new **instance** of the class which is created by **copying** the values from “cd1” and here any **changes** that are **performed** on members of “cd1” will not **reflect** to members of “cd2” and **vice versa** because they have their own **individual memory** which is **not accessible** to **others**.

Types of Constructors: constructors are divided into **5 Categories** like:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor
4. Static Constructor
5. Private Constructor

Default Constructor: a constructor defined without any parameters is known as a default constructor, which will initialize fields of a class with default values. If a class is not defined with any explicit constructor, then the class will contain an implicit default constructor.

Parameterized Constructor: if a constructor is defined with at least 1 parameter then we call it as parameterized constructor and these constructors must be explicitly defined but never implicitly defined. Parameterized constructors are used for initializing fields of a class with given set of values which we can pass while creating the instance of that class.

Copy Constructor: it's a constructor which takes the same type as its “**Parameter**” and initializes the fields of class by copying values from an existing instance of the same class. A Copy constructors will not create a reference to the class i.e., it will create a new instance for the class by allocating memory for all the members of that class and very importantly any changes made on the source will not reflect to the new instance and vice-versa.

Static Constructor: if a constructor is defined explicitly by using static modifier, we call it as a static constructor and this constructor is the **first block** of code which executes under the class, and they are responsible for initializing static fields and more over these constructors **can't be overloaded** because they **can't be parameterized**. This constructor is called implicitly before the first instance is created or any static members are referenced.

Private Constructor: If a constructor is explicitly declared by used **private modifier**, we call it as a private constructor. If a class contains only private constructors and no public constructors, other classes cannot create instances of that class as well as inheritance is also not possible.

Sealed Class: if a class is explicitly declared by using **sealed** modifier, we call it as a **sealed class** and these classes can't be inherited by other classes, for example:

```
sealed class Class1
{
    -Members
}
```

In the above case Class1 is a sealed class so it can't be inherited by any other class, for example:

```
class Class2 : Class1    =>    In-valid
```

Note: even if a sealed class can't be **inherited** it is still possible to consume the members of a **sealed** class by creating its **instance**, for example **String** is a sealed class in our **libraries**, so we **can't inherit** from **String** class but we can still **consume** it in all our classes by creating the **instance** of **String** class.

Sealed Method: If a parent class method can't be **overridden** under a child class, then we call that method as **sealed method**. By default, every method of a class is **sealed**, because we can never **override** any method of parent class under the child class unless the method is declared as **virtual**. If a method is declared as **virtual** under a class, then any child class of it in a **linear hierarchy (Multi-Level Inheritance)** can override that method, for example:

```
Class1
public virtual void Show()

Class2 : Class1
public override void Show()    //Valid

Class3 : Class2
public override void Show()    //Valid
```

Note: in the above case even if **Class2** is not **overriding** the method also **Class3** can **override** the method.

When a **child** class is **overriding** parent classes' **virtual** methods, it can **seal** those methods by using **sealed modifier** on them, so that **further overriding** of those methods can't be performed by its child classes, for example:

```
Class1
public virtual void Show()

Class2 : Class1
public sealed override void Show()    //Valid

Class3 : Class2
public override void Show()    //In-valid
```


Note: in the above case **Class2** has **sealed** the method while **overriding**, so **Class3** can't **override** the method.

Abstract Class and Abstract Method

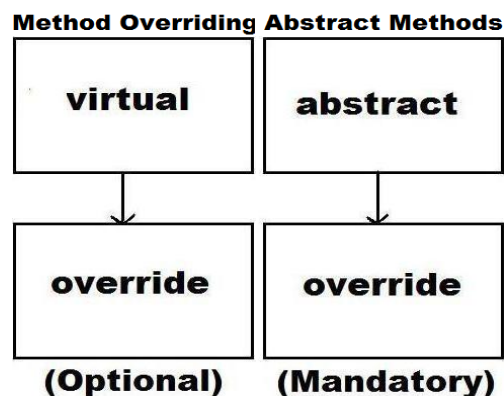
Abstract Method: a method without any body is known as **abstract method** i.e., an **abstract method** contains only **declaration** without any **implementation**. To declare a method as **abstract** it is must to use "**abstract**" modifier on that method **explicitly**.

Abstract Class: a class under which we declare **abstract members** is known as **abstract class** and must also be declared by using "**abstract**" modifier.

```
abstract class Math
{
    public abstract void Add(int x, int y);
}
```

Note: each and every **abstract member** of an **abstract class** must be implemented by the **child** class of the **abstract** class without **fail** (**mandatory**).

The concept of **abstract method's** is near similar to **method overriding** i.e., in case of **overriding**, if at all a parent class contains any methods declared as **virtual** then child classes **can re-implement** those methods by using **override** modifier whereas in case of **abstract methods** if at all a parent class contains any methods declared as **abstract** then every child class **must implement** all those methods by using the same **override** modifier only.



An abstract class can contain both **abstract** and **non-abstract (concrete)** members also, and if at all any child class of the **abstract class** wants to consume any **non-abstract members** of its parent, **must first** implement all the **abstract members** of its parent.

Abstract Class:

- Non-Abstract/Concrete Members
- Abstract Members

Child Class of Abstract Class:

- Implement each and every abstract member of parent class
- Now only we can consume concrete members of parent class

Note: we can't create the instance of an abstract class, so abstract classes are never useful to themselves, i.e., an abstract class is always a parent providing services to child classes.

To test an abstract class and abstract methods add a new class under the project naming it as "AbsParent.cs" and write the following code in it:

```
internal abstract class AbsParent
{
    public void Add(int a, int b)
    {
        Console.WriteLine(a + b);
    }
    public void Sub(int a, int b)
    {
        Console.WriteLine(a - b);
    }
    public abstract void Mul(int a, int b);
    public abstract void Div(int a, int b);
}
```

Now add another class "AbsChild.cs" to implement the above abstract classes - abstract methods and write the following code in it:

```
internal class AbsChild : AbsParent
{
    public override void Mul(int a, int b) //Overriden Member
    {
        Console.WriteLine(a * b);
    }
    public override void Div(int a, int b) //Overriden Member
    {
        Console.WriteLine(a / b);
    }
    public void Mod(int a, int b) //Pure Child Class Member
    {
        Console.WriteLine(a % b);
    }
    static void Main()
    {
        AbsChild c = new AbsChild();
        c.Add(100, 50); c.Add(75, 17);
        c.Mul(12, 13); c.Div(870, 15); c.Mod(121, 5);
        Console.ReadLine();
    }
}
```

Note: even if the instance of an abstract class can't be created it is still possible to create the reference of an abstract class by using its child classes instance, and with that reference we can call each and every concrete

method of **abstract class** as well as its **abstract methods** which are implemented by **child class** but not any **pure child class** methods, and to test this re-write code under **Main** method of the class "**AbsChild**" as following:

```
AbsChild c = new AbsChild();
AbsParent p = c;
p.Add(100, 50); p.Sub(75, 17);    //Methods defined in Parent class
p.Mul(12, 13); p.Div(870, 15);    //Methods implemented (override) by child class
//p.Mod(121, 5);                  //Invalid and In-accessible (Pure child class members are not accessible)
Console.ReadLine();
```

What is the need of Abstract Classes and Abstract Methods in Application development?

Ans: The concept of **Abstract Classes** and **Abstract Methods** is an extension to **inheritance** i.e., in **inheritance** we have already learnt that, we can **eliminate redundancy** between entities by identifying all the common attributes between the entities we wanted to implement, by putting them under a parent class.

For example, if we are designing a **Mathematical Application** then we follow the below process of implementation:

Step1: Identifying the **Entities** of an Mathematical Application.

- Cone
- Circle
- Triangle
- Rectangle

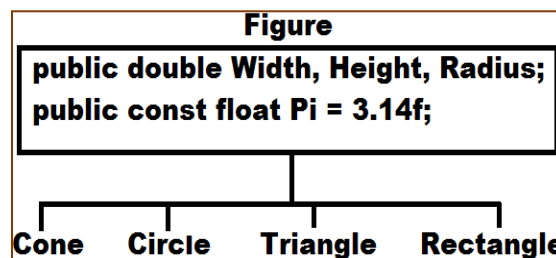
Step 2: Identifying the **Attributes** of each Entity.

- Cone: Height, Radius, Pi
- Circle: Radius, Pi
- Triangle: Base (Width), Height
- Rectangle: Length (Height), Breadth (Width)

Step 3: Designing the **Database** by following the rules we learnt in entity implementations.

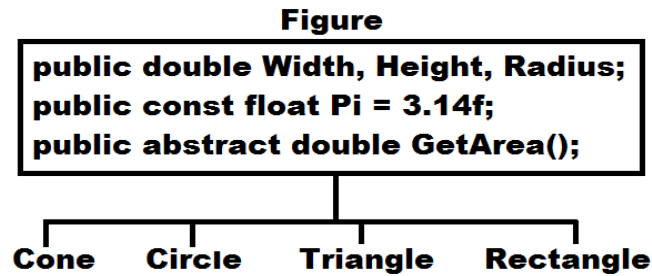
Step 4: Develop an **application** and define classes representing each and every entity.

Note: while defining classes representing entities, as learnt in **inheritance** first we need to define a parent class with all the common attributes as following:



In the above case, "**Figure**" is a Parent class containing all the common attributes between the 4 entities. Now we want a method that returns **Area** of each figure, and even if the method is common for all the classes, still we can't define it in the parent class **Figure**, because the **formula** to calculate area varies from **figure** to **figure**. So,

to resolve the problem, without **defining** the method in parent class we need to **declare** it in the parent class **Figure** as abstract and restrict each child class to implement logic for that method as per their requirement as following:



In the above case because **GetArea()** method is declared as **abstract** in the parent class, it is **mandatory** for all the child classes to implement that method under them, but logic can be varying from each other whereas signature of the method can't change. Now all the child classes must do the following:

1. Define a constructor to initialize the attributes that are required for that entity.
2. Implement **GetArea()** method and write logic for calculating the Area of that corresponding figure.

To test the above add a "Code File" under project naming it as "TestFigures.cs" and write the following code:

```
namespace OOPSProject
{
    public abstract class Figure
    {
        public const float Pi = 3.14f;
        public double Width, Height, Radius;
        public abstract double GetArea();
    }
    public class Cone : Figure
    {
        public Cone(double Height, double Radius)
        {
            this.Height = Height;
            base.Radius = Radius; //Here this and base are same
        }
        public override double GetArea()
        {
            return Pi * Radius * (Radius + Math.Sqrt((Height * Height) + (Radius * Radius)));
        }
    }
    public class Circle : Figure
    {
        public Circle(double Radius)
        {
            this.Radius = Radius;
        }
        public override double GetArea()
```

```

    {
        return Pi * Radius * Radius;
    }
}

public class Triangle : Figure
{
    public Triangle(double Base, double Height)
    {
        this.Width = Base;
        this.Height = Height;
    }
    public override double GetArea()
    {
        return 0.5 * Width * Height;
    }
}

public class Rectangle : Figure
{
    public Rectangle(double Length, double Breadth)
    {
        this.Width = Length;
        this.Height = Breadth;
    }
    public override double GetArea()
    {
        return Width * Height;
    }
}

internal class TestFigures
{
    static void Main()
    {
        Cone cone = new Cone(18.92, 34.12);
        Console.WriteLine($"Area of Cone is: {cone.GetArea()}\n");

        Circle circ = new Circle(45.36);
        Console.WriteLine($"Area of Circle is: {circ.GetArea()}\n");

        Triangle trin = new Triangle(34.98, 27.87);
        Console.WriteLine($"Area of Triangle is: {trin.GetArea()}\n");

        Rectangle rect = new Rectangle(45.29, 76.12);
        Console.WriteLine($"Area of Rectangle is: {rect.GetArea()}\n");

        Console.ReadLine();
    }
}

```

```
}  
}
```

Interface

Interface is also a **type** like a **class** but can contain only “Abstract Members” in it and all those **abstract members** should be **implemented** by a **child class** of the **interface**.

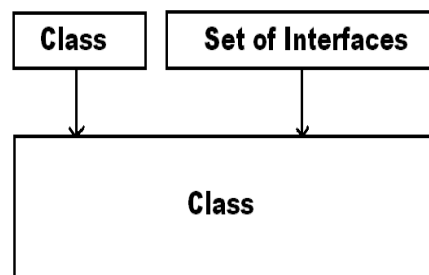
Just like a **class** can have another **class** as its **parent**, it can also have an **interface** as its **parent** but the main difference is if a **class** is a **parent**, we call it as **inheriting** whereas if an **interface** is a **parent**, we call it as **implementing**.

Inheritance is divided into 2 categories:

1. Implementation Inheritance
2. Interface Inheritance

If a class is inheriting from another **class**, we call it as **Implementation Inheritance** whereas if a class is implementing an **interface**, we call it as **Interface Inheritance**. **Implementation Inheritance** provides **re-usability** because by **inheriting** from a class we can **consume** members of **parent class** in **child class** whereas **Interface Inheritance** doesn't provide any **re-usability** because in this case we need to **implement abstract members** of a **parent** in **child class** without **fail**, but not **consume**.

Note: we have already discussed in the **5th rule** of **inheritance** that **Java** and **.NET Languages** doesn't support **multiple inheritance** thru **class**, because of **ambiguity problem** i.e., a class can have 1 and only 1 **immediate parent** class to it; but both in **Java** and **.NET languages** multiple inheritance is supported thru the **interfaces** i.e., a class can have more than 1 **interface** as its **immediate parent**.



Syntax to define a interface:

```
[<modifiers>] interface <Name>  
{  
  -Abstract member declarations.  
}
```

- We can't declare any fields under an interface.
- Default scope for members of an interface is public whereas it is private in case of a class.
- Every member of an interface is by default abstract, so we again don't require using abstract modifier on it.
- Just like a class can inherit from another class, an interface can also inherit from another interface, but not from a class.

Adding an Interface under Project: Just like we have “Class Item Template” in “Add New Item” window to define a class we are also provided with an “Interface Item Template” to define an Interface. To test working with interfaces, add 2 interfaces under the project naming them as **IMath1.cs**, **IMath2.cs** and write the following code:

```
internal interface IMath1
{
    void Add(int x, int y);
    void Sub(int x, int y);
}

internal interface IMath2
{
    void Mul(int x, int y);
    void Div(int x, int y);
}
```

To implement methods of both the above interfaces add a new class under the project naming it as “**ClsMath.cs**” and write the following code:

```
internal class ClsMath : Program, IMath1, IMath2
{
    public void Add(int a, int b)
    {
        Console.WriteLine(a + b);
    }
    public void Sub(int a, int b)
    {
        Console.WriteLine(a - b);
    }
    public void Mul(int a, int b)
    {
        Console.WriteLine(a * b);
    }
    public void Div(int a, int b)
    {
        Console.WriteLine(a / b);
    }
    static void Main()
    {
        ClsMath obj = new ClsMath();
        obj.Add(100, 34); obj.Sub(576, 287); obj.Mul(12, 38); obj.Div(456, 2);
        Console.ReadLine();
    }
}
```

Points to Ponder:

1. The implementation class can inherit from another class and implement “n” no. of interfaces, but class name must be first in the list followed by interface names.

E.g.: `internal class ClsMath : Program, IMath1, IMath2`

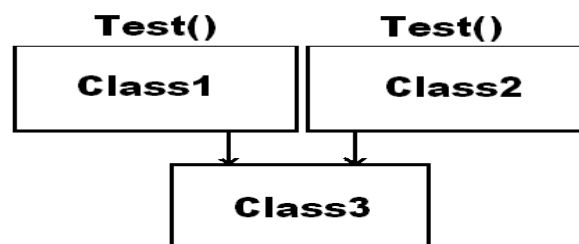
2. While declaring abstract members in an interface we don’t require using “abstract” modifier on them and in the same way while implementing those abstract members we don’t require to use “override” modifier also.

Just like we can't **create instance** of an **abstract class**, we can't **create instance** of an **interface** also; but here also we can create a **reference** of **interface** by using its **child class instance** and with that **reference** we can call all the members of **parent interface** which are implemented in child class and to test this **re-write** code under **Main** method of class **"ClsMath"** as following:

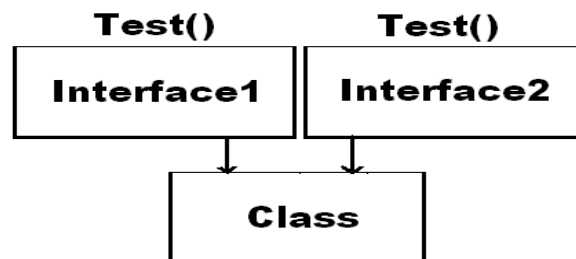
```
ClsMath obj = new ClsMath();  
IMath1 i1 = obj; IMath2 i2 = obj;  
i1.Add(150, 25); i1.Sub(97, 47);  
i2.Mul(12, 17); i2.Div(870, 15);  
Console.ReadLine();
```

Multiple Inheritance with Interfaces:

Earlier in the **5th** rule of **inheritance** we have discussed that **Java** and **.NET** Languages doesn't support **multiple inheritances** thru **classes** because of **ambiguity** problem.



Whereas in **Java** and **.NET** Languages, **multiple inheritance** is supported thru **interfaces** i.e., a class can have any no. of interfaces as its immediate parent, but still we don't come across any **ambiguity** problems because child **class** of an **interface** is not **consuming** parent's members but **implements** them.



If we come across any situation as **above**, we can **implement** the **interface** methods under **class** by using 2 different approaches:

1. Implement the method of both **interfaces** only for 1 time under the **class** and both **interfaces** will assume the implemented **method** is of its only and in this case, we can call the method directly by using class **instance**.
2. We can also implement the method of both **interfaces** separately for each **interface** under the **class** by pre-fixing **interface** name before **method** name and we call this as **explicit implementation**, but in this case, we need to call the **method** by using **reference** of **interface** that is created with the help of a child class **instance**.

To test the above add 2 new **interfaces** under the **project** naming them as **Interface1.cs**, **Interface2.cs** and write the following code:

```
internal interface Interface1
```

```
internal interface Interface1
```



```

{
    void Test();
    void Show();
}

```

```

{
    void Test();
    void Show();
}

```

Now add a new class under the project naming it as **"ImplClass.cs"** for implementing both the above **interfaces** and write the following code:

```

internal class ImplClass : Interface1, Interface2
{
    //Implementing Test method using 1st approach
    public void Test()
    {
        Console.WriteLine("Method declared under 2 interfaces.");
    }
    //Implementing Show method using 2nd approach
    void Interface1.Show()
    {
        Console.WriteLine("Method declared under Interface1.");
    }
    //Implementing Show method using 2nd approach
    void Interface2.Show()
    {
        Console.WriteLine("Method declared under Interface2.");
    }
    static void Main()
    {
        ImplClass c = new ImplClass();
        c.Test();

        Interface1 i1 = c;
        Interface2 i2 = c;

        i1.Show();
        i2.Show();
        Console.ReadLine();
    }
}

```

Structure

Structure is also a **type** like a **class** and **interface** which can contain only non-abstract members in it. A **structure** can contain all the **members** what a class can contain like **constructor**, **static constructor**, **constants**, **fields**, **methods**, **properties**, **indexers**, **operators**, and **events**.

Class:

Contains both **non-abstract/concrete** and **abstract** members

Interface:

Contains only **abstract** members

Structure:

Contains only **non-abstract/concrete** members

Differences between Class and Structure

<u>Class</u>	<u>Structure</u>
This is a reference type.	This is a value type.
Memory is allocated for its instances on Managed Heap , so we get the advantage of Automatic Memory Management thru Garbage Collector .	Memory is allocated for its instances on Stack , so Automatic Memory Management is not available but faster in access.
Recommended for representing entities with larger volumes of data.	Recommended for representing entities with smaller volumes of data.
All pre-defined reference types in our libraries like string (System.String) and object (System.Object) are defined as classes .	All pre-defined value types in our libraries like int (System.Int32), float (System.Single), bool (System.Boolean), char (System.Char) and Guid (System.Guid) are defined as structures.
"new" keyword is mandatory for creating the instance and in this process, we need to call any constructor that is available in the class.	"new" keyword is optional for creating the instance and if "new" is not used it will call default constructor which is defined implicitly, whereas it is still possible to use "new" and call other constructors also.
Contains an implicit default constructor if no constructor is defined explicitly .	Contains a default constructor every time (mandatory) which can be implicitly or explicitly (from C# 10) defined.
We can declare fields and those fields can be initialized at the time of declaration.	We can declare fields , but those fields can't be initialized at the time of declaration.
Fields can also be initialized thru a constructor as well as referring thru instance also we can initialize them.	Fields can only be initialized thru a constructor as well as referring thru instance also we can initialize them.
Constructor is mandatory for creating the instance which can either be default or parameterized also.	Default constructor is mandatory for creating the instance without using new keyword and apart from that we can also define parameterized constructors.
Developers can define any constructor like default or parameterized also, or else implicit default constructor gets defined.	Developers can define parameterized constructors only up to C# 9.0 whereas from C# 10.0 developers can also define default constructors. Default constructor is mandatory if at all we want to create instance without using "new" keyword.
If defined with " 0 " constructors, after compilation there will be " 1 " constructor and if defined with " n " constructors, after compilation there will be " n " constructors only.	If defined with " 0 " constructors, after compilation here also there will be " 1 " constructor whereas if defined with " n " parameterized constructors, after compilation there will be " n + 1 " constructors along with default .
Supports both, implementation as well as interface inheritances also i.e., a class can inherit from another class as well as implement an interface also.	Supports only interface inheritance but not implementation inheritance i.e., a structure can implement an interface but can't inherit from another structure .

Syntax to define a structure:

```
[<modifiers>] struct <Name>
{
    -Define only non-abstract Members
}
```

Adding a Structure under Project: we are not provided with any **structure item template** in the add new item window, like we have **class** and **interface** item templates, so we need to use **code file item template** to define a **structure** under the project.

Add a Code File under project, naming it as “MyStruct.cs” and write the below in it:

```
namespace OOPSPProject
{
    internal struct MyStruct
    {
        int x;
        public MyStruct(int x)
        {
            this.x = x;
        }
        public void Display()
        {
            Console.WriteLine("Method defined under a structure: " + x);
        }
        static void Main()
        {
            MyStruct m1 = new MyStruct();
            m1.Display();
            MyStruct m2;
            m2.x = 10; m2.Display();
            MyStruct m3 = new MyStruct(20);
            m3.Display();
            Console.ReadLine();
        }
    }
}
```

Consuming a Structure: we can consume a **structure** and its **members** from another **structure** or **class** also; but only by creating its **instance** because structure doesn't support **inheritance**.

To test this, add a new **class** under the project naming it as **“TestStruct.cs”**, change the **class** keyword to **struct** and write the below code:

```
internal struct TestStruct
{
    static void Main()
    {
        MyStruct obj1 = new MyStruct(); obj1.Display();
    }
}
```

```

MyStruct obj2 = new MyStruct(30); obj2.Display();
Console.ReadLine();
}
}

```

Working with Multiple Projects and Solution

While developing an application sometimes code will be written under more than 1 project also, where collection of all those projects is known as a **Solution**. Whenever we create a new project by default Visual Studio will create one **Solution** and under it the project gets added, where a **Solution** is a collection of **Projects** and **Project** is a collection of **Items** or **Files** and each **Item** or **File** is a collection of **Types** (**Class**, **Structure**, **Interface**, **Enum** and **Delegate**), and each **Type** is a collection of **Members** (**Fields**, **Methods**, **Constructors**, **Finalizers**, **Properties**, **Indexers**, **Events** and **Deconstructor**)

A **Solution** also requires a **Name**, which can be specified by us while creating a new **Project** or else it will take **Name** of the first **Project** that is created under **Solution**, if not specified. In our case **Solution Name** is **"OOPSProject"** because our **Project Name** is **"OOPSProject"**. A **Solution** can have **Projects** of different **.NET Languages** as well as can be of different **Project Templates** also like **Windows App's**, **Console App's**, **Class Library** etc. but a project cannot contain items of different **.NET Languages** i.e., they must be specific to 1 **Language** only.

To add a new **Project** under our **"OOPSProject"** solution, right click on **Solution Node** in **Solution Explorer** and select add **"New Project"** which opens the new **Project Window**, under it select **Language** as **Visual C#**, **Template** as **Console Application**, name the **Project** as **"SecondProject"** and click **Ok** which adds the new **Project** under the **"OOPSProject"** solution.

By default, the new **Project** also comes with a class **"Program"** but under **"SecondProject Namespace"**, now write the below code in the file by deleting the existing code:

```

internal class Program
{
    static void Main()
    {
        Console.WriteLine("Second project under the solution.");
        Console.ReadLine();
    }
}

```

To run the above class, first we need to set a property i.e., **"Startup Project"**, because there are multiple **Projects** under the **Solution** and **Visual Studio** by default runs first **Project** of the **Solution** only i.e., **"OOPSProject"** under the solution. To set the **"Startup Project"** property and run classes under **"SecondProject"** open **Solution Explorer**, right click on **"SecondProject"**, select **"Set as Startup Project"**, and then run the **Project**.

Note: if the new **Project** is added with new **Classes** we need to again set **"Startup Object"** property under **Second Project's** project file, because each project has its own property **Window**.

Saving Solution and Projects: The application what we have created right now is saved physically on hard disk in the same hierarchy as seen under **Solution Explorer** i.e., first a folder is created representing the **Solution** and under

that a **separate folder** is created representing each **Project** and under that **Items** or **Files** corresponding to that **Project** gets saved and the path of the **Project** will be as following:

```
<drive>:\<our_personal_folder>\OOPSPProject\OOPSPProject => Project1
<drive>:\<our_personal_folder>\OOPSPProject\SecondProject => Project2
```

Note: A **Solution** will be having a file called **Solution file**, which gets saved with **“.sln”** extension and a **Project** also has a file called **Project file**, where a **C# Project** file gets saved with **“.csproj”** extension which can contain **“C#”** items only.

Compilation of Projects: whenever a **Project** is compiled it generates an output file known as **“Assembly”** that contains **“CIL Code”** of all the **“Types”** that are defined in the **Project**.

What is an Assembly?

- It's an output file that is generated after compilation of a project which contains **CIL Code** in it.
- Assembly file contains the **CIL Code** of each type that is defined under the project.
- An **Assembly** is a **unit of deployment**, because when we need to install an application on client machines what we install is these **Assemblies** only and all the **.NET Libraries** are installed on our machines in the form of **Assemblies** when we install **Visual Studio**.
- The name of an **assembly** file is the same name of the **project** and can't be **changed**.
- In **.NET Framework** the assembly files of a project will be present under the project folder's **“bin\debug”** folder. In **.NET Core**, assembly file of a project will be present under **“bin\debug\netcoreapp<Version>”** folder and here version represents the **Core Runtime** version. From **.NET 5**, assembly file of a project will be present under **“bin\debug\net<Version>”** folder and here also version represents the **Runtime** version.
- In **.NET Framework** the **extension** of an **assembly** file can either be a **“.exe”** or **“.dll”** which is based on the type of project we open, for example if the project is an **“Application Project”** then it will generate **“.exe”** assembly whereas if it is a **“Library Project”** then it will generate **“.dll”** assembly. From **.NET Core** every project will generate **“.dll”** assembly and apart from that **“Application Project's”** will generate an additional **“.exe”** assembly also i.e., **“Library Projects”** will be generating **“.dll”** only now also where as **“Application Project's”** will generate both **“.exe”** and **“.dll”** also.

.NET Framework:

- Application Projects => Generates only **“.exe”**.
- Library Projects => Generates only **“.dll”**.

.NET Core & above:

- Application Projects => Generates both **“.exe”** and **“.dll”** also.
- Library Projects => Generates only **“.dll”**.

Note: Generally, **“.dll”** assemblies can't run but a **“.dll”** assembly that are generated by **Application Projects** can run or execute on **Linux** and **MAC** Machines also by using the tool: **“.NET Core CLI (Command Line Interface)”** as following:

```
dotnet <Assembly_Name>.dll
```

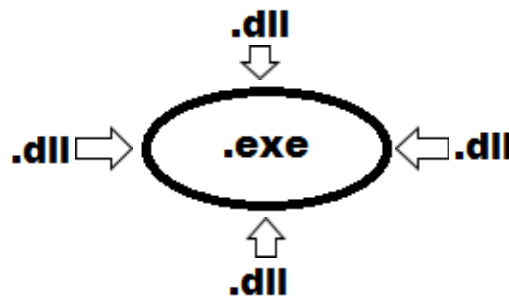
What is an **“.exe”** assembly?

Ans: In Windows OS “.exe” assemblies are known as **in-process** components i.e., these assemblies are physically loaded into the **memory** for execution and run-on **Windows Machines**.

What is a “.dll” assembly?

Ans: In Windows O.S. “.dll” assembly are known as **out-process** components i.e., these assemblies sit out of the memory providing support to the 1 who is running in the memory. In .NET Framework, “.dll” assemblies can never run on their own i.e., they can only be consumed from other projects, whereas from .NET Core and above the “.dll” assemblies that are generated by “Application Projects” can run on **Windows, Linux, and Mac** Machine with the help of **.NET Core CLI**.

Note: An assembly is a unit of deployment, because when we need to install or deploy an application on client machines what we install is these assemblies only and every application is a blend of “.dll’s” and “.exe” assemblies combined to give better efficiency.



The assembly files of our 2 projects i.e., OOPSPProject and SecondProject will be at the following location:

<drive>:\<folder>\OOPSPProject\OOPSPProject\bin\Debug\net8.0\OOPSPProject.dll & .exe => Assembly1

<drive>:\<folder>\OOPSPProject\SecondProject\bin\Debug\net8.0\SecondProject.dll & .exe => Assembly2

ildasm: Intermediate Language Dis-Assembler. We use it to dis-assemble an Assembly file and view the contents of it. To check it out, open **Visual Studio Developer Command Prompt**, go to the location where the assembly files of the project are present and use it as following:

ildasm <name of the .dll assembly file>

Note: in **.NET Framework** we can dis-assemble both “.exe” and “.dll” assemblies also whereas from **.NET Core** we can dis-assemble only “.dll” assemblies.

E.g.: Open **Visual Studio Developer Command Prompt**, go to the below location and try the following:

<drive>:\<our_folder>\OOPSPProject\OOPSPProject\bin\Debug\net8.0> ildasm OOPSPProject.dll

<drive>:\<our_folder>\OOPSPProject\SecondProject\bin\Debug\net8.0> ildasm SecondProject.dll

Q. Can we consume types defined in a project from other types of same project?

Ans: Yes, we can consume them directly because all those types were under the same project and will be considered as a family.

Q. Can we consume the types of 1 project from other projects?

Ans: Yes, we can consume, but not directly, as they are under different projects. To consume them first we need to add reference to the **Project** or **assembly** in which the types are defined, to the project who wants to consume.

Q. How to add the reference of a project or assembly to a project?

Ans: To add reference of a **project** or **assembly** to a project, open solution explorer, right click on the project to whom reference must be added, select **"Add => Project Reference"** option, which opens a window **"Reference Manager"** and in that window if the project which we want to consume is in the same solution then we see the **Project** right over there in the middle panel with a **Checkbox** beside, select the **Checkbox** and click on **"Ok"** button whereas if the project we want to consume is under another **Solution** then, select **"Browse"** option in **LHS**, then click on **"Browse"** button below, select the **assembly** we want to consume from its physical location and click ok. Now we can consume types of that **assembly** by prefixing with their **namespace** or **importing** the **namespace**.

Note: In .NET Framework we can add reference to **".exe"** or **".dll"** assemblies and consume them, whereas from **.NET Core** onwards we can't add reference to **".exe"** assemblies i.e., we can add reference to **".dll"** assemblies only.

To test this, go to **"OOPSPProject"** Solution, right click on the **"SecondProject"** we have newly added, select **add reference** and **add reference** of **"OOPSPProject"** - **Project (recommended)** or **Assembly**. Now add a new class under the **"SecondProject"** naming it as **"Class1.cs"** and write the below code in it:

```
using OOPSPProject;
internal class Class1
{
    static void Main()
    {
        Cone cone = new Cone(18.92, 34.12);
        Console.WriteLine($"Area of Cone is: {cone.GetArea()}\n");

        Circle circ = new Circle(45.36);
        Console.WriteLine($"Area of Circle is: {circ.GetArea()}\n");

        Triangle trin = new Triangle(34.98, 27.87);
        Console.WriteLine($"Area of Triangle is: {trin.GetArea()}\n");

        Rectangle rect = new Rectangle(45.29, 76.12);
        Console.WriteLine($"Area of Rectangle is: {rect.GetArea()}\n");
        Console.ReadLine();
    }
}
```

Assemblies and Namespaces: An assembly is an output file which gets generated after compilation of a project and it is **physical**. The name of an assembly file will be same as project name and can't be changed at all.

Project: Source Code

Assembly: Compiled Code (IL Code)

A namespace is a **logic container** of types which are used for grouping types. By default, every project has a namespace, and its name is same as the project name, but we can change namespace names as per our requirements and more over a project can contain multiple namespaces in it also.

For Example: **DBOperations** (**Library Project**) when compiled generates an **Assembly** with the names as **DBOperations.dll**, under it, namespaces can be as following:

```

namespace SQL
{
    Class1
    Class2
}
namespace Oracle
{
    Class3
    Class4
}
namespace MySQL
{
    Class5
    Class6
}

```

Whenever we want to consume a type which is defined under 1 project from other projects, we need to follow the below steps:

Step 1: add a reference to the **project** or **assembly** under which the types we want to consume are defined.

Step 2: import the appropriate **namespace** under which the types are defined.

Step 3: now either **create** the **instance** of the type or **inherit** the type and consume them.

Access Specifier's

These are a special kind of modifiers using which we can define the scope of a type and its members i.e., who can access them and who cannot. C# supports 5 access specifiers in it, those are:

1. Private 2. Internal 3. Protected 4. Protected Internal 5. Public 6. Private Protected (C# 7.3)

Note: members that are defined in a type with any scope or specifier are always accessible within the type, restrictions come into picture only when we try to access them outside of the type.

Private: members declared as **private** under a **class** or **structure** can be accessed only within the **type** in which they are defined and more over their **default scope** is **private** only. **Interfaces** can't contain any **private** members and their **default scope** is **public**. **Types** can't be declared as **private**, so this applies only to **members**.

Protected: members declared as **protected** under a class can be accessed only from their **child** class i.e., **non-child** classes can't consume them. **Types** can't be declared as **protected**, so this applies only to **members**.

Internal: members and types that are declared as **internal** can be consumed only within the **project**, both from a **child** or **non-child**. The **default scope** for a type in **C#** is **internal** only.

Protected Internal: members declared as **protected internal** will have **dual scope** i.e., within the project they behave as **internal** providing access to whole project and outside the project they will change to **protected** and provide access to their child classes. **Types** can't be declared as **protected internal**, so this applies only to **members**.

Public: a **type** or **member** of a type if declared as **public** is **global** in scope which can be accessed from anywhere.

Private Protected (Introduced in C# 7.3 Version): **members** declared as **private protected** in a class are accessible only from the **child** classes that are defined in the same project. Types can't be declared as **private protected**, so this applies only to **members**.

To test access specifiers, create a new **C# Project** of type **"Console App."**, name the project as **"AccessDemo1"** and re-name the solution as **"MySolution"**, click **Next** and choose the **Framework ".NET 8.0"** and Check the **CheckBox "Do not use top-level statements."**, so that it generates **Program class** and **Main method** also.

By default, the project comes with a class **Program** and its default scope is **internal**, so change it as **public** so that it can be **accessed** from other **projects** also and write the below code in the class:

//Consuming members of a class from the same class

```
public class Program
{
    private void Test1_Private()
    {
        Console.WriteLine("Private Method");
    }
    internal void Test2_Internal()
    {
        Console.WriteLine("Internal Method");
    }
    protected void Test3_Protected()
    {
        Console.WriteLine("Protected Method");
    }
    protected internal void Test4_ProtectedInternal()
    {
        Console.WriteLine("Protected Internal Method");
    }
    public void Test5_Public()
    {
        Console.WriteLine("Public Method");
    }
    private protected void Test6_PrivateProtected()
    {
        Console.WriteLine("Private Protected Method");
    }
    static void Main(string[] args)
    {
        Program p = new Program();
        p.Test1_Private();
        p.Test2_Internal();
        p.Test3_Protected();
    }
}
```

```

        p.Test4_ProtecedInternal();
        p.Test5_Public();
        p.Test6_PrivateProtected();
        Console.ReadLine();
    }
}

```

Now add a new class “Two.cs” under the project and write the following:

//Consuming members of a class from child class of same project

```

internal class Two : Program
{
    static void Main()
    {
        Two t = new Two();
        t.Test2_Internal();
        t.Test3_Protected();
        t.Test4_ProtecedInternal();
        t.Test5_Public();
        t.Test6_PrivateProtected();
        Console.ReadLine();
    }
}

```

Now add another new class “Three.cs” in the project and write the following:

//Consuming members of a class from non-child class of same project

```

internal class Three
{
    static void Main()
    {
        Program p = new Program();
        p.Test2_Internal();
        p.Test4_ProtecedInternal();
        p.Test5_Public();
        Console.ReadLine();
    }
}

```

Now Add a new “Console App” project under “MySolution”, name it as “AccessDemo2”, rename the default file “Program.cs” as “Four.cs” so that class name also changes to Four, add a reference to “AccessDemo1” project to the new project and write the below code in the class Four:

//Consuming members of a class from child class of another project

```

internal class Four : AccessDemo1.Program
{
    static void Main(string[] args)
    {
        Four f = new Four();
    }
}

```

```

f.Test3_Protected();
f.Test4_ProtectedInternal();
f.Test5_Public();
Console.ReadLine();
}
}

```

Now add a new class under AccessDemo2 project, naming it as Five.cs and write the following:

//Consuming members of a class from non-child class of another project

```

internal class Five
{
    static void Main()
    {
        Program p = new Program();
        p.Test5_Public();
        Console.ReadLine();
    }
}

```

<u>Cases</u>	Private	Internal	Protected	Private Protected	Protected Internal	Public
Case 1: Same Class - Same Project	Yes	Yes	Yes	Yes	Yes	Yes
Case 2: Child Class - Same Project	No	Yes	Yes	Yes	Yes	Yes
Case 3: Non-Child Class - Same Project	No	Yes	No	No	Yes	Yes
Case 4: Child Class - Another Project	No	No	Yes	No	Yes	Yes
Case 5: Non-Child Class - Another Project	No	No	No	No	No	Yes

Language Independency or Language Interoperability

As discussed earlier the code written in **1 .NET Language** can be consumed from any other **.NET Languages** and we call this as **Language Interoperability**. Differences between **VB.NET** and **C#** languages:

- VB is not a case-sensitive language like C#.
- VB does not have any curly braces; the end of a block is represented with a matching End Stmt.
- VB does not have any semi colon terminators so each statement must be in a new line.
- The extension of source files will be “.vb” in VB.NET and “.cs” in C#.

To test this, add a new **“Console App”** project under **“MySolution”** choosing the language as **Visual Basic** and name the project as **“AccessDemo3”**. By default, the project comes with a file **“Module1.vb”**, so open solution explorer, delete that file under project and add a new class naming it as **“TestCS.vb”**. Now add the reference of **“AccessDemo1”** project to the new project, and write the below code under the class **TestCS**:

```

Imports AccessDemo1
Public Class TestCS : Inherits Program
    Shared Sub Main()
        'Creating instance of the class
        Dim obj As New TestCS()
        obj.Test3_Protected()
        obj.Test4_ProtectedInternal()
    End Sub
End Class

```

```

        obj.Test5_Public()
        Console.ReadLine()
    End Sub
End Class

```

Note: to run the class set **Startup Project** as current project i.e., “AccessDemo3” and execute.

Consuming VB.NET Code in CSharp: Now to test consuming VB.NET code in C#, add a new project under “MySolution”, choosing the language as **Visual Basic**, project type as “Class Library” and name the project as “AccessDemo4”. A **Class Library** is a collection of types that can be **consumed** but not **executed**. After **compilation** the extension of project’s assembly will be “.dll”.

In VB.Net language methods are divided into 2 categories like:

1. Functions (Value returning methods)
2. Sub-Routines (Non-value returning methods)

By default, the project comes with a class Class1 within the file Class1.vb, write the below code in it:

```

Public Class Class1
    Public Function SayHello(Name As String) As String
        Return "Hello " & Name
    End Function
    Public Sub AddNums(x As Integer, y As Integer)
        Console.WriteLine($"Sum of given 2 no's is: {x + y}")
    End Sub
    Public Sub Math(a As Integer, b As Integer, ByRef c As Integer, ByRef d As Integer)
        c = a + b
        d = a * b
    End Sub
End Class

```

Now to compile the project open solution explorer, right click on “AccessDemo4” project and select “Build” option which **compiles** and **generates** an assembly “AccessDemo4.dll”.

Now add a new class under “AccessDemo2” project with the name “TestVB.cs”, add reference of “AccessDemo4” project to “AccessDemo2” project and write the below code under the new class **TestVB**:

```

using AccessDemo4;
internal class TestVB
{
    static void Main()
    {
        Class1 obj = new Class1();
        obj.AddNums(100, 50);
        string str = obj.SayHello("Raju");
        Console.WriteLine(str);
        int Sum = 0, Product = 0;
        obj.Math(100, 25, ref Sum, ref Product);
        Console.WriteLine("Sum of the given 2 no's is: " + Sum);
    }
}

```

```

    Console.WriteLine("Product of the given 2 no's is: " + Product);
    Console.ReadLine();
}
}

```

Note: to run the class set both **Startup Project** and **Startup Object** properties also.

Q. How to restrict a class not to be accessible for any other class to consume?

Ans: This can be done by declaring all the class constructors as private.

Q. How to restrict a class not to be inherited for any other class?

Ans: This can be done by declaring class as sealed.

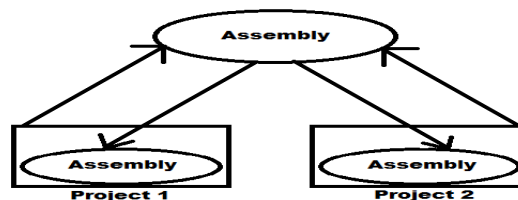
Q. How to restrict a class not to be accessible for any other class to consume by creating its instance?

Ans: This can be done by declaring all the class constructors as protected.

Assemblies are of 2 types:

1. Private Assembly
2. Shared Assembly

Private Assembly: By default, every assembly is private, if reference of these assemblies was added to any project; a copy of assembly is created and given to that project, so that each project maintains a private copy of assembly.



Creating an assembly to test it is by default private: create a new project of type **Class Library** and name it as **"PAssembly"**, which will by default come with a class **Class1** under the file **Class1.cs**. Now write the following code under the class:

```

public string SayHello()
{
    return "Hello from private assembly.";
}

```

Now compile the project by opening the Solution Explorer, right click on the project and select **"Build"** which will compile and generate an assembly with the name as **PAssembly.dll**.

Note: we can find path of the assembly in output window present at bottom of the Visual Studio.

Consuming the assembly, we have created in multiple projects: We can consume an **assembly** under any no. of projects, but if the **assembly** is **private**, it will create multiple copies of the **assembly** i.e., in how many projects the reference is added that many no. of copies are also created for the **assembly**.

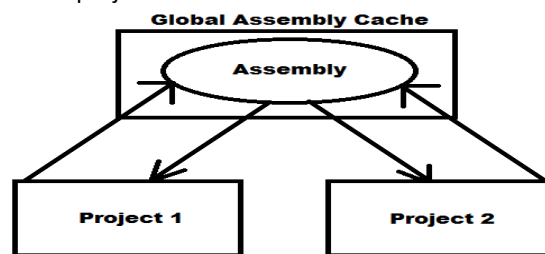
To test this, create 2 new projects of type **Console App.s**, naming it as **“TestPAssembly1”** and **“TestPAssembly2”**. Add the reference of **“PAssembly.dll”** we have created above to both the projects, from its **physical location** and write the below code under **Main** method of the default class **Program**:

```
PAssembly.Class1 obj = new PAssembly.Class1();  
Console.WriteLine(obj.SayHello());  
Console.ReadLine();
```

Run both the projects to test them, and then go and verify under **bin/debug/net8.0** folder of both the projects where we can find a copy of **“PAssembly.dll”** because **PAssembly.dll** is a private assembly.

Note: the advantage of a private assembly is faster execution as it was in the local folder of consumer project, whereas the draw back was multiple copies gets created when multiple projects add the reference to consume it.

Shared Assemblies: If we intend to use an assembly among several applications, private assemblies are not feasible, in such cases we can install the Assembly into a centralized location known as the “Global Assembly Cache”. Each computer where the “.NET Runtime” is installed has this machine-wide code cache. The Global Assembly Cache stores assemblies specifically designated to be shared by several applications on the computer. All .NET Libraries are assemblies and are shared “.dll” assemblies only, so we can find them under GAC. If an assembly is shared multiple copies of the assembly will not be created even if being consumed by multiple projects i.e., only a single copy under GAC serves all the projects.



Note: administrators often protect the Windows directory using an access control list (ACL) to control write and execute access. Because the global assembly cache is installed in the Windows directory, it inherits that directory's ACL. It is recommended that only users with Administrator privileges be allowed to add or delete files from the global assembly cache.

Location of GAC Folder: <OS Drive>:\Windows\Microsoft.NET\assembly\GAC_MSIL

How to make an assembly as Shared?

Ans: to make an assembly as Shared we need to install the assembly into GAC.

How to install an assembly into GAC?

Ans: To manage assemblies in GAC like install, un-install and view we are provided with a tool known as Gacutil.exe (Global Assembly Cache Tool). This tool is automatically installed with VS. To run the tool, use the Visual Studio Command Prompt. These utilities enable you to run the tool easily, without navigating to the installation folder. To use Global Assembly Cache on your computer: On the taskbar, click Start, click All Programs, click Visual Studio, click Visual Studio Tools, and then click Visual Studio Command Prompt and type the following:

```
gacutil -i | -u | -l [<assembly name>]  
or
```

gacutil /i | /u | /l [<assembly name>]

What assemblies can be installed into the GAC?

Ans: We can install only Strong Named Assemblies into the GAC.

What is a Strong Named Assembly?

Ans: assemblies deployed in the global assembly cache must have a strong name. When an assembly is added to the global assembly cache, integrity checks are performed on all files that make up the assembly.

Strong Name: A strong name consists of the assembly's identity - its simple text like: name, version number, and public key.

1. **Name:** it was the name of an assembly used for identification. Every assembly by default has name.
2. **Version:** software's maintain versions for discriminating changes that has been made from time to time. As an assembly is also a software component it will maintain versions, whenever the assembly is created it has a default version for it i.e., 1.0.0.0, which can be changed when required.
3. **Public Key:** as GAC contains multiple assemblies in it, to identify each assembly it will maintain a key value for the assembly known as public key, which should be generated by us and associate with the assembly to make it Strong Named.

You can ensure that a name is globally unique by signing an assembly with a strong name. Strong names satisfy the following requirements:

- Strong names guarantee name uniqueness by relying on unique key pairs. No one can generate the same assembly's name that you can. Strong names protect the version lineage of an assembly.
- A strong name can ensure that no one can produce a subsequent version of your assembly. Users can be sure that a version of the assembly they are loading comes from the same publisher that created the version the application was built with.
- Strong names provide a strong integrity check. Passing the .NET Framework security checks guarantees that the contents of the assembly have not been changed since it was built.

Generating a Public Key: To sign an assembly with a strong name, you must have a public key pair. This public cryptographic key pair is used during compilation to create a strong-named assembly. You can create a key pair using the Strong Name tool (Sn.exe) from visual studio command prompt as following:

Syntax: sn -k <file name>

E.g.: sn -k Key.snk

Note: the above statement generates a key-value and writes it into the file "**Key.snk**". Key/Value pair files usually have the extension of "**.snk**" (strong name key).

Creating a Shared Assembly:

Step 1: generate a public key. Open VS command prompt, go into your personal folder and generate a public key as following:

<drive>:\<CSharp> sn -k Key.snk

Step 2: create a new project and add the key file to it before compilation so that the assembly which is generated will be Strong Named. To do this open a new project of type **Class Library**, name it as "**SAssembly**" and write the below code under the class Class1:

```
public string SayHello1()
{
    return "Hello from shared assembly => 1.0.0.0";
}
```

To associate key file we have generated, with the project, open project properties window and to do that open **Solution Explorer**, right click on the **Project** and choose the option **"Properties"** and in the window opened select **Build** tab on **LHS** and under that click on **"Strong naming"** item, which displays a **Checkbox** as **"Sign the output assembly to give it a strong name."** in **RHS**, select it, which display a **File Upload Control** with **Browse** button to select the file from its physical location, so click on **Browse** and select the **"Key.snk"** file from its physical location which adds the file to the project and we can find that information in **".csproj"** file under the XML key **"<SignAssembly>True</SignAssembly>"** and **"<AssemblyOriginatorKeyFile>"**. Now compile the project using **"Build"** option that will generate **"SAssembly.dll"** which is **Strong Named**.

Step 3: installing assembly into GAC by using the **"Global Assembly Cache Tool"**. To install the assembly into GAC open Visual Studio - Developer Command Prompt in Administrator Mode, go to the location where **"SAssembly.dll"** is present and write the below statement:

```
<drive>:\<folder\SAssembly\SAssembly\bin\Debug\net8.0> gacutil -i SAssembly.dll
```

Step 4: testing the Shared Assembly. Create a new project of type Console App., name it as **"TestSAssembly1"**. Add reference to **"SAssembly.dll"** from its physical location and write the below code under **Main** method of the default class **Program**:

```
SAssembly.Class1 obj = new SAssembly.Class1();
MessageBox.Show(obj.SayHello1());
```

Note: Run the project, test it, and now this project i.e., **"TestSAssembly1"** can run by using the **"SAssembly.dll"** which is present in **"GAC"** i.e., we don't require a copy of the **"SAssembly.dll"** to be present under the local folder.

Versioning Assemblies: Every assembly is associated with a set of **attributes** that describes about general info of an assembly like **Title**, **Company**, **Description**, **Version** etc. These attributes will be under **".csproj"** file of the project. To view the **".csproj"** file right click on **SAssembly Project** in **Solution Explorer** and select **"Edit Project File"** which will open the project file. This is an **XML** file and under this file right now we find code as below:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <SignAssembly>true</SignAssembly>
    <AssemblyOriginatorKeyFile>Key.snk</AssemblyOriginatorKeyFile>
  </PropertyGroup>
</Project>
```

We can also specify various other details regarding the **assembly** like **Company**, **Version**, **etc**, **etc**, under this file by using **XML Tags**, for example if we want to specify the name of the **Company** who designed and

developed this assembly we can use "<Company>" tag and we need to use it inside of "<PropertyGroup>" tag as following:

<Company>NIT</Company>

Why do we maintain version numbers to an assembly?

Ans: Version no. is maintained for discriminating the changes that has been made from time to time. Version No is changed for an assembly if there are any modifications or enhancements made in the code. The default version of every assembly is 1.0.0.0 and version no is a combination of 4 values like:

1. Major Version
2. Minor Version
3. Build Number
4. Revision

What are the criteria for changing the version no. of an assembly?

Ans: we change version no. of an assembly basing on the following criteria:

1. Change the Major version value when we add new types under the assembly.
2. Change the Minor version value when we modify any existing types under the assembly.
3. Change the Build number when we add new members under types.
4. Change the Revision value when we modify any existing members under types.

Where do we change the version no. of an assembly?

Ans: we need to change the version no. of an assembly under the project file, and to do that open Solution Explorer, right click on the project, select the option "Edit Project File" which opens the Project File in XML format. Now under <PropertyGroup> tag we need to add the below statements in the last:

<AssemblyVersion>Specify the new version no. here</AssemblyVersion>
<FileVersion>Specify the new version no. here</FileVersion>

Testing the process of changing version no of an assembly: Open the SAssembly project we have developed earlier and add a new method under Class1 as below:

```
public string SayHello2()
{
    return "Hello from shared assembly => 1.0.1.0";
}
```

Open project property file and set Company, Description, Assembly Version & File Version attributes as below:

<Company>NIT</Company>
<Description>This is a shared assembly developed by Naresh I Technologies.</Description>
<AssemblyVersion>1.0.1.0</AssemblyVersion>
<FileVersion>1.0.1.0</FileVersion>

Now Re-build the project and add the new version of "SAssembly.dll" i.e., "1.0.1.0" also into GAC using the Gacutil Tool.

Note: GAC allows placing of multiple versions of an assembly in it and provides different applications using different versions of the assembly to execute correctly using their required version. Now if we open the GAC folder there, we will find 2 versions of "SAssembly" i.e., "1.0.0.0" and "1.0.1.0".

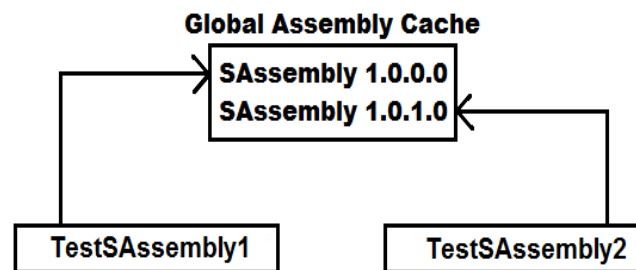
Assemblies and Side-by-Side Execution: Side-by-side execution is the ability to store and execute multiple versions of an application or component on the same computer. Support for side-by-side storage and execution of different versions of the same assembly is an integral part of strong naming and is built into the infrastructure of the .NET

Runtime. Because the strong-named assembly version number is part of its **identity**, the **.NET Runtime** can store multiple versions of the same assembly in the **Global Assembly Cache** and loads those assemblies at run time.

To test side-by-side execution, create a new project of type Console App., name it as **"TestSAssembly2"**. Add reference to **"SAssembly.dll"** from its physical location and write the below code under **Main** method of default class **Program**:

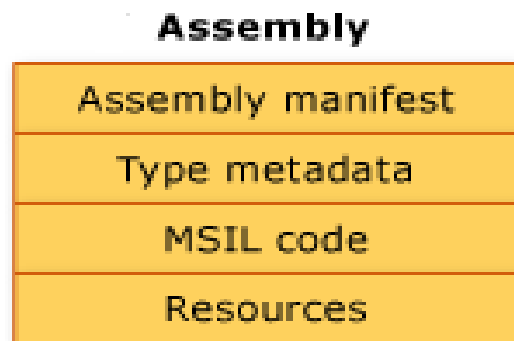
```
SAssembly.Class1 obj = new SAssembly.Class1();  
MessageBox.Show(obj.SayHello2());
```

To check **side-by-side** execution of projects run the **"exe files"** of **"TestSAssembly1"** and **"TestSAssembly2"** projects at the same time, where each project will use its required version of **"SAssembly"** and execute, as below:



In general, an assembly is divided into four sections:

- The assembly manifest, which contains assembly metadata.
- Type metadata.
- Microsoft intermediate language (MSIL) Code or CIL Code that implements the types.
- A set of resources.



Assembly Manifest: contains information about the attributes that are associated with an assembly like **Assembly Name**, **Assembly Version**, **File Version**, **Company**, **Strong Name Information**, **List of files** in the assembly etc.

Type Metadata: describes every type and member defined in your code in a language-neutral manner. Metadata stores the following information:

- Description of the assembly.
 - Identity (name, version, culture, public key).
 - Other assemblies that this assembly depends on.

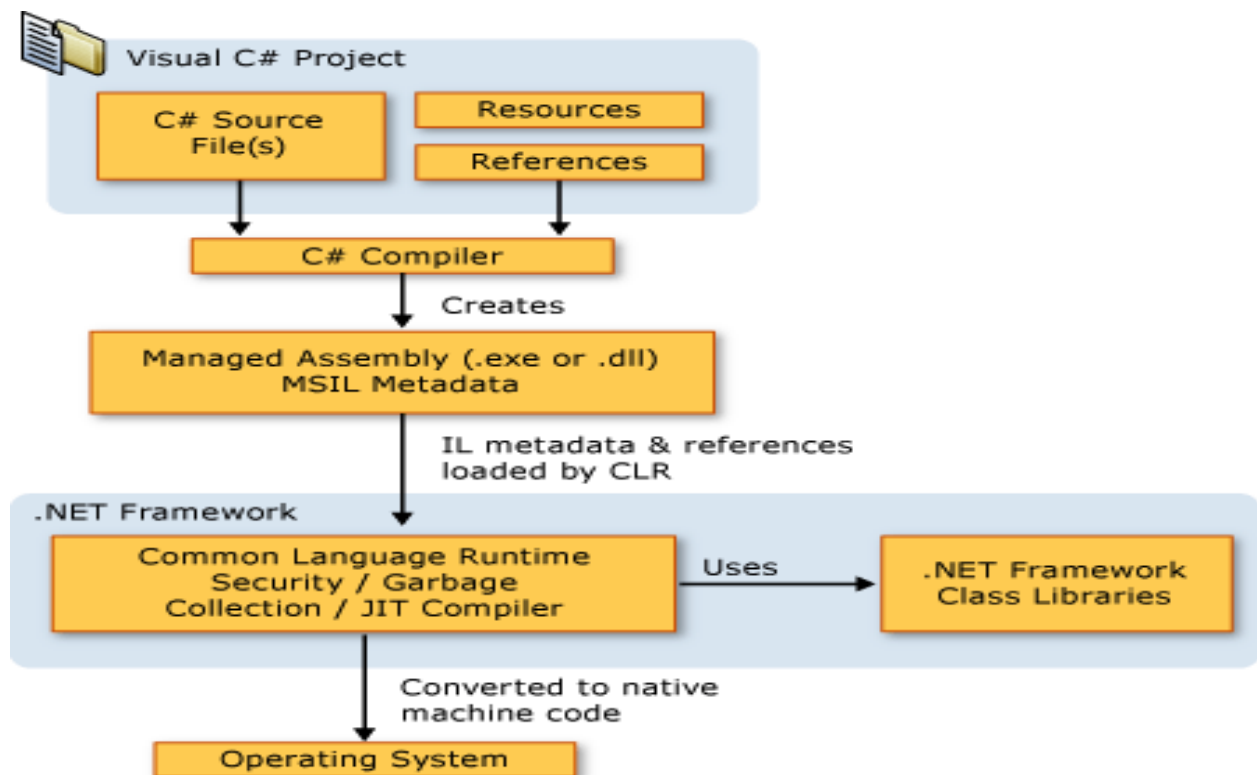
- Security permissions needed to run.
- Description of types.
 - Name, visibility, base class, and interfaces implemented.
 - Members (methods, fields, properties, events, nested types).

Metadata provides the following major benefits:

1. Self-describing files, common language runtime modules and assemblies are self-describing.
2. Language interoperability, metadata provides all the information required about compiled code for you to inherit a class from a file written in a different language.

MSIL Code or CIL Code: during compilation of any .NET programming languages, the source code is translated into CIL code rather than platform or processor-specific code. CIL is a CPU and platform-independent instruction set that can be executed in any environment supporting the Common Language Infrastructure, such as the .NET runtime on Windows, or the cross-platform Mono runtime.

Compilation and Execution Process of a C# Project



Finalizer

Finalizers are used to destruct objects (**instances**) of classes. A **Finalizer** is also a special method just like a **Constructor**, whereas **Constructors** are called when **instance** of a class is **created**, and **Finalizers** are called when **instance** of a class is **destroyed**. Both will have the same name i.e., the name of class in which they are defined, but to **differentiate** between each other we **prefix Finalizer** with a tilde (~) operator. For Example:

```

class Test
{
    Test()
    {
        //Constructor
    }
    ~Test()
    {
        //Finalizer
    }
}

```

Remarks:

- **Finalizers** cannot be defined in **structs** i.e., they are only used with **classes**.
- A **finalizer** does not take **modifiers** or have **parameters**.
- A class can only have one **finalizer** and cannot be **inherited** or **overloaded**.
- **Finalizers** cannot be called. They are invoked **automatically**.
- A **constructor** and **finalizer** will have the same name i.e., the class name and to differentiate between each other we prefix **finalizers** with “~” operator.

The **programmer** has no control over when the **finalizer** is called because this is determined by the **garbage collector**; **garbage collector** calls the **finalizer** in any of the following cases:

1. Called in the **end** of a program's execution and **destroys** all **instances** that are **associated** with the **program**.
2. In the **middle** of a **program's** execution also the **garbage collector** checks for **instances** that are no longer being used by the **application**. If it considers an **instance** is eligible for **destruction**, it calls the **finalizer** and **reclaims** the **memory** used to store the **instance**.
3. It is possible to **force** - **garbage collector** by calling **GC.Collect()** method to check for un-used **instances** and **reclaim** the **memory** used to **store** those **instances**.

Note: we can **force** the **garbage collector** to do clean up by calling the **GC.Collect** method, but in most cases, this should be **avoided** because it may create **performance issues**, i.e., when the **garbage collector** comes into **action** for **reclaiming memory** of **un-used instances**, it will **suspend** the execution of programs.

To test this, open a new **Console App. Project** in **.NET Framework** naming it as “**FinalizersProject**”, rename the default class “**Program.cs**” as “**DestDemo1.cs**” using **Solution Explorer** and write the below code over there:

```

internal class DestDemo1
{
    public DestDemo1()
    {
        Console.WriteLine("Instance1 is created.");
    }
    ~DestDemo1()

```

```

{
    Console.WriteLine("Instance1 is destroyed.");
}
static void Main(string[] args)
{
    DestDemo1 d1 = new DestDemo1();
    DestDemo1 d2 = new DestDemo1();
    DestDemo1 d3 = new DestDemo1();
    //d1 = null; d3 = null; GC.Collect(); //(Write all the 3 statements in the same line with comments)
    Console.ReadLine();
}
}

```

Execute the above program by using **Ctrl + F5** and watch the output of program, first it will call **Constructor** for 3 times because 3 **instances** are created and then waits at **ReadLine** statement to execute; now hit **enter key** to finish the execution of **ReadLine**, immediately **finalizer** gets called for 3 times because it is the end of programs **execution**, so all 3 instances associated with the program are **destroyed**. This proves that **finalizer** is called in the end of a **program's execution**.

Now **un-comment** the **commented** code in **Main** method of above **program** and **re-execute** the program again by using **Ctrl + F5** to watch the difference in output, in this case 2 **instances** are **destroyed** before execution of **ReadLine** because we have marked them as **un-used** by assigning **"null"** and called **Garbage Collector** explicitly, and the third instance is destroyed in end of **program's execution**.

Finalizers and Inheritance: As we are aware that whenever a child class **instance** is created, child class **constructor** will call its parents class **constructor** implicitly, same as this when a child class **instance** is destroyed it will also call its parent classes **finalizer**, but the difference is **constructors** are called in **"Top to Bottom"** hierarchy and **finalizers** are called in **"Bottom to Top"** hierarchy. To test this, add a new class **"DestDemo2.cs"** and write the below code:

```

internal class DestDemo2 : DestDemo1
{
    public DestDemo2()
    {
        Console.WriteLine("Instance2 is created.");
    }
    ~DestDemo2()
    {
        Console.WriteLine("Instance2 is destroyed.");
    }
    static void Main()
    {
        DestDemo2 obj = new DestDemo2();
        Console.ReadLine();
    }
}

```

Conclusion about Finalizers: In general, C# does not require as much memory management; it is needed when you develop with a Language that does not target a runtime with garbage collection, for example C++ Language. This is because the .NET Garbage Collector which implicitly manages the allocation and release of memory for your instances. However, when your application encapsulates un-managed resources such as Files, Databases, and Network Connections, you should use finalizers to free those resources.

Properties

A property is a member that provides a flexible mechanism to read, write, or compute the value of a private field. Properties can be used as if they are public fields, but they are special methods called accessors.

Suppose a class is associated with any value and if we want to expose that value outside of the class, access to that value can be given in 4 different ways:

- I. By storing the value under a public field, access can be given to that value outside of the class, for Example:

```
public class Circle
{
    public double Radius = 12.34;
}
```

Now by creating the instance of the above class we can get or set a value to the field as following:

```
class TestCircle
{
    static void Main()
    {
        Circle c = new Circle();
        double Radius = c.Radius;           //Getting the old value of Radius
        c.Radius = 56.78;                   //Setting a new value for Radius
    }
}
```

Note: in this approach it will provide Read/Write access to the value i.e., anyone can get the old value of the field as well as anyone can set with a new value for the field, so we don't have any control on the value.

- II. By storing the value under a private field also we can provide access to the value outside of the class by defining a property on that field. The advantage in this approach is it can provide access to the value in 3 different ways:

1. Only get access (Read Only Property)
2. Only set access (Write Only Property)
3. Both get and set access (Read/Write Property)

Syntax to define a property:

```
[<modifiers>] <type> Name
{
    [ get { stmt's } ]           //Get Accessor
```

```
[ set { stmt's } ] //Set Accessor  
}
```

- A **property** is **one** or **two code blocks**, representing a **get** access or **and/or** a **set** accessor.
- The code block for the **get accessor** is executed when the property is **read** and the body of the **get** accessor resembles that of a **method**. It must **return** a **value** of the **property type**. The **get accessor** resembles a **value returning** method without any **parameters**.
- The code block for the **set accessor** is executed when the property is **assigned** with a **new value**. The **set accessor** resembles a non-value returning method with parameter, i.e., it uses an **implicit parameter** called **"value"**, whose **type** is the same as **property type**.
- A property without a **set accessor** is considered as **read-only**. A property without a **get accessor** is considered as **write-only**. A property that has **both accessors** is considered as **read-write**.

Remarks:

- Properties can be marked as **public**, **private**, **protected**, **private protected**, **internal**, or **protected internal**. These access modifiers define how users of the class can access the property. The get and set accessors for the same property may have different access modifiers. For example, the get may be public to allow read-only access from outside the class, and the set may be private or protected.
 - A property may be declared as a **static** property by using the **static** keyword. This makes the property available to callers at any time, even if no **instance** of the class exists.
 - A property may be marked as a **virtual** property by using the **virtual** keyword, which enables derived classes to **override** the property behavior by using the **override** keyword. A property **overriding** a **virtual** property can also be **sealed**, specifying that for derived classes it is no longer **virtual**.
 - A property can be declared as **abstract** by using the **abstract** keyword, which means that there is no implementation in the class, and **derived classes** must write their own **implementation**.
-

To test properties first add a new code file Cities.cs and write the following code:

```
namespace OOPSProject  
{  
    public enum Cities  
    {  
        Bengaluru, Chennai, Delhi, Hyderabad, Kolkata, Mumbai  
    }  
}
```

Now add a new class Customer.cs and write the following code:

```
public class Customer  
{  
    int _Custid;  
    bool _Status;  
    string _Name, _State;  
    double _Balance;  
    Cities _City;  
    public Customer(int Custid)  
    {  
        _Custid = Custid;  
    }  
}
```

```
_Status = false;
_Name = "John";
_Balance = 5000.00;
_City = 0;
_State = "Karnataka";
Country = "India";
}
```

//Read Only Property

```
public int Custid
{
    get { return _Custid; }
}
```

//Read-Write Property

```
public bool Status
{
    get { return _Status; }
    set { _Status = value; }
}
```

//Read-Write Property (With a condition in Set Accessor)

```
public string Name
{
    get { return _Name; }
    set
    {
        if (_Status)
        {
            _Name = value;
        }
    }
}
```

//Read-Write Property (With a condition in Get & Set Accessor)

```
public double Balance
{
    get
    {
        if (_Status)
        {
            return _Balance;
        }
        else
        {
            return 0;
        }
    }
    set
```



```

{
    if (_Status)
    {
        if (value >= 500)
        {
            _Balance = value;
        }
    }
}
}

//Read-Write Property (Enumerated Property)
public Cities City
{
    get { return _City; }
    set
    {
        if(_Status)
        {
            _City = value;
        }
    }
}

//Read-Write Property (With a different scope to each property accessor (C# 2.0))
public string State
{
    get { return _State; }
    protected set
    {
        if(_Status)
        {
            _State = value;
        }
    }
}

//Read-Write Property (Automatic or Auto-Implemented property (C# 3.0))
public string Country
{
    get;
    private set;
}

//Read-Only Property (Auto property initializer (C# 6.0))
public string Continent { get; } = "Asia";
}

```

Note: The contextual keyword **value** is used in the **set accessor** in ordinary property declarations. It is like an **input** parameter of **method**. The word **value** references the **value** that client code is attempting to assign to the property.

Enumerated Property: It is a property that provides with a set of **constants** to choose from, for example **BackgroundColor** property of the **Console** class that provides with a list of **constant** colors to choose from, under an **Enum ConsoleColor**. E.g.: `Console.BackgroundColor = ConsoleColor.Blue`; An **Enum** is a distinct type that consists of a set of **named constants** called the **enumerator** list. Usually, it is best to define an **Enum** directly within a **namespace** so that all classes in the **namespace** can access it with equal convenience. However, an **Enum** can also be **nested** within a **class** or **structure**.

Syntax to define an Enum:

```
[<modifiers>] enum <Name>
{
    <List of named constant values>
}
public enum Days
{
    Monday, Tuesday, Wednesday, Thursday, Friday
}
```

Note: By default, the first value is represented with an index 0, and the value of each successive enumerator is increased by 1. For example, in the above enumeration, Monday is 0, Tuesday is 1, Wednesday is 2, and so forth.

To define an Enumerated Property, adopt the following process:

Step 1: define an **Enum** with the list of constants we want to provide for the property to choose.

E.g.: `public enum Days { Monday, Tuesday, Wednesday, Thursday, Friday };`

Step 2: declare a **field** of type **Enum** on which we want to define a property.

E.g.: `Days _Day = 0;` or `Days _Day = Days.Monday;` or `Days _Day = (Days)0;`

Step 3: now define a **property** on the **Enum** field for providing access to its values.

```
public Days Day
{
    get { return _Day; }
    set { _Day = value; }
}
```

Auto-Implemented properties: In C# 3.0 and later, auto-implemented properties make property-declaration more concise when **no additional logic is required in the property accessors**. E.g.: **Country** property in our customer class, but up to **CSharp 5.0** it is important to remember that auto-implemented properties must contain both **get** and **set blocks** either with the same access modifier or different also whereas from **CSharp 6.0** it's not mandatory because of a new feature called **"Auto Property Initializer"**, which allows to initialize a property at declaration time.

In our Customer class the Country property we have defined can be implemented as below also:

E.g.: `public string Country { get; } = "India";`

To consume the properties, we have defined above add a new class TestCustomer.cs and write the following:

```

internal class TestCustomer
{
    static void Main()
    {
        Customer obj = new Customer(1001);
        Console.WriteLine("Custid: " + obj.Custid + "\n");
        //obj.Custid = 1005; //Invalid, because the property is defined read only

        if (obj.Status)
            Console.WriteLine("Customer Status: Active");
        else
            Console.WriteLine("Customer Status: In-Active");
        Console.WriteLine("Customer Name: " + obj.Name);
        obj.Name += " Smith"; //Update fails because status is in-active
        Console.WriteLine("Name when update failed: " + obj.Name);
        Console.WriteLine("Balance when status is in-active: " + obj.Balance + "\n");

        obj.Status = true; //Activating the status
        if (obj.Status)
            Console.WriteLine("Customer Status: Active");
        else
            Console.WriteLine("Customer Status: In-Active");
        Console.WriteLine("Customer Name: " + obj.Name);
        obj.Name += " Smith"; //Update succeeds because status is in-active
        Console.WriteLine("Name when update succeeded: " + obj.Name);
        Console.WriteLine("Balance when status is active: " + obj.Balance + "\n");

        obj.Balance -= 4600; //Transaction fails
        Console.WriteLine("Balance when transaction failed: " + obj.Balance);
        obj.Balance -= 4500; //Transaction succeeds
        Console.WriteLine("Balance when transaction succeeded: " + obj.Balance + "\n");

        Console.WriteLine($"Customer City: {obj.City}");
        obj.City = Cities.Hyderabad;
        Console.WriteLine($"Modified City: {obj.City}");

        Console.WriteLine("Customer State: " + obj.State);
        //obj.State = "Telangana"; //Invalid because set accessor is accessible only to child classes

        Console.WriteLine("Customer Country: " + obj.Country);
        Console.WriteLine("Customer Continent: " + obj.Continent);
        Console.ReadLine();
    }
}

```

Object Initializers (Introduced in C# 3.0)

Object initializers let you assign values to any accessible **properties** of an **instance** at creation time without having to **explicitly invoke** a **parameterized constructor**. You can use **object initializers** to initialize type objects in a **declarative manner** without explicitly invoking a **constructor** for the **type**. Object Initializers will use the **default constructor** for initializing **fields** thru **properties**.

To test these, add a new Code File naming it as "TestStudent.cs" and write the following code in it:

```
namespace OOPSPProject
{
    public class Student
    {
        int? _Id, _Class;
        string? _Name;
        float? _Marks, _Fees;
        public int? Id
        {
            get { return _Id; }
            set { _Id = value; }
        }
        public int? Class
        {
            get { return _Class; }
            set { _Class = value; }
        }
        public string? Name
        {
            get { return _Name; }
            set { _Name = value; }
        }
        public float? Marks
        {
            get { return _Marks; }
            set { _Marks = value; }
        }
        public float? Fees
        {
            get { return _Fees; }
            set { _Fees = value; }
        }
        public override string ToString()
        {
            return "Id: " + _Id + "\nName: " + _Name + "\nClass: " + _Class + "\nMarks: " + _Marks + "\nFees: " + _Fees;
        }
    }

    internal class TestStudent {
```

```

static void Main() {
    Student s1 = new Student { Id = 101, Name = "Raju", Class = 10, Marks = 575.00f, Fees = 5000.00f };
    Student s2 = new Student { Id = 102, Name = "Vijay", Class = 10 };
    Student s3 = new Student { Id = 103, Marks = 560.00f, Fees = 5000.00f };
    Student s4 = new Student { Id = 104, Class = 10, Fees = 5000.00f };
    Student s5 = new Student { Id = 105, Name = "Raju", Marks = 575.00f };
    Student s6 = new Student { Id = 106, Class = 10, Marks = 575.00f };

    Console.WriteLine(s1); Console.WriteLine(s2); Console.WriteLine(s3);
    Console.WriteLine(s4); Console.WriteLine(s5); Console.WriteLine(s6);
    Console.ReadLine();
}
}
}

```

Indexers

Indexers allow **instances** of a **class** or **struct** to be **indexed** just like **arrays**. **Indexers** resemble **properties** except that their accessors take **parameters**. Indexers are syntactic conveniences that enable you to create a class or struct that client applications can access just as an **array**. Defining an **indexer** allows you to create classes & structures that act like “**virtual arrays**”, so instances of that class or structure can be accessed using the **[]** array access operator. Defining an **indexer** in **C#** is like defining operator “**[]**” in **C++** but is considerably simpler and more flexible. For classes or structure that encapsulate array or collection - like functionality, defining an indexer allows the users of that class or structure to use the array syntax to access the class or structure. An indexer doesn't have a specific name like a **property** it is defined by using “**this**” keyword.

Syntax to define Indexer:

```

[<modifiers>] <type> this[<Parameter List>]
{
    [ get { -Stmts } ] //Get Accessor
    [ set { -Stmts } ] //Set Accessor
}

```

Indexers Overview:

- “**this**” keyword is used to define the indexers.
- The **out** and **ref** keyword are not allowed on parameters.
- A get accessor returns a value. A set accessor assigns a value.
- The **value** keyword is only used to define the value being assigned by the set indexer.
- Indexers do not have to be **indexed** by **integer** value; it is up to you how to define the **look-up** mechanism.
- Indexers can be **overloaded**.
- Indexers can't be defined as **static**.
- Indexers can have more than one formal parameter, for example, accessing a **two-dimensional** array.

To test indexers, add a Code File under the project naming it as "TestEmployee.cs" & write the below code in it:

```
namespace OOPSProject
{
    public class Employee
    {
        int? _Id;
        string? _Name, _Job;
        double? _Salary;
        bool? _Status;

        public Employee(int Id)
        {
            _Id = Id;
            _Name = "Nicholas";
            _Job = "Manager";
            _Salary = 50000.00;
            _Status = true;
        }
        public object? this[int Index]
        {
            get
            {
                if (Index == 1)
                    return _Id;
                else if (Index == 2)
                    return _Name;
                else if (Index == 3)
                    return _Job;
                else if (Index == 4)
                    return _Salary;
                else if (Index == 5)
                    return _Status;
                else
                    return null;
            }
            set
            {
                if (Index == 2)
                    _Name = (string?)value;
                else if (Index == 3)
                    _Job = (string?)value;
                else if (Index == 4)
                    _Salary = (double?)value;
                else if (Index == 5)
                    _Status = (bool?)value;
            }
        }
    }
}
```

```

    }
}
public object? this[string Key]
{
    get
    {
        if (Key.ToUpper() == "ID")
            return _Id;
        else if (Key.ToUpper() == "NAME")
            return _Name;
        else if (Key.ToUpper() == "JOB")
            return _Job;
        else if (Key.ToUpper() == "SALARY")
            return _Salary;
        else if (Key.ToUpper() == "STATUS")
            return _Status;
        else
            return null;
    }
    set
    {
        if (Key.ToLower() == "name")
            _Name = (string?)value;
        else if (Key.ToLower() == "job")
            _Job = (string?)value;
        else if (Key.ToLower() == "salary")
            _Salary = (double?)value;
        else if (Key.ToLower() == "status")
            _Status = (bool?)value;
    }
}
}

```

```

internal class TestEmployee

```

```

{
    static void Main()
    {
        Employee Emp = new Employee(1005);
        Console.WriteLine("Employee ID: " + Emp[1]);
        Console.WriteLine("Employee Name: " + Emp[2]);
        Console.WriteLine("Employee Job: " + Emp[3]);
        Console.WriteLine("Employee Salary: " + Emp[4]);
        Console.WriteLine("Employee Status: " + Emp[5]);
        Console.WriteLine();
    }
}

```

Emp["Id"] = 1010; //Can't be assigned with a new value, because we have not defined setter for ID

```

Emp[3] = "Sr. Manager";
Emp["Salary"] = 75000.00;

Console.WriteLine("Employee ID: " + Emp["Id"]);
Console.WriteLine("Employee Name: " + Emp["name"]);
Console.WriteLine("Employee Job: " + Emp["JOB"]);
Console.WriteLine("Employee Salary: " + Emp["SaLaRy"]);
Console.WriteLine("Employee Status: " + Emp["Status"]);
Console.ReadLine();
}
}
}

```

Deconstructor

These are newly introduced in **C# 7.0** which can also be used to provide access to the values or expose the values associated with a class to the outside environment, apart from **public fields**, **properties**, and **indexers**. **Deconstructor** is a special method with the name **"Deconstruct"** that is defined under the class to expose (**Read Only**) the attributes of a class and this will be defined with a code that is **reverse** to a **constructor**.

To understand **Deconstructor**, add a code file in our project naming it as **"TestTeacher.cs"** and write the below code in it:

```

namespace OOPSProject
{
    public class Teacher
    {
        int? Id;
        string? Name, Subject, Designation;
        double? Salary;
        public Teacher(int? Id, string? Name, string? Subject, string? Designation, double? Salary)
        {
            this.Id = Id;
            this.Name = Name;
            this.Subject = Subject;
            this.Designation = Designation;
            this.Salary = Salary;
        }
        public void Deconstruct(out int? Id, out string? Name, out string? Subject, out string? Designation, out double? Salary)
        {
            Id = this.Id;
            Name = this.Name;
            Subject = this.Subject;
            Designation = this.Designation;
            Salary = this.Salary;
        }
    }
}

```



```

class TestTeacher
{
    static void Main()
    {
        Teacher obj = new Teacher(1005, "Suresh", "English", "Lecturer", 25000.00);
        (int? Id1, string? Name1, string? Subject1, string? Designation1, double? Salary1) = obj;
        Console.WriteLine("Teacher Id: " + Id1);
        Console.WriteLine("Teacher Name: " + Name1);
        Console.WriteLine("Teacher Subject: " + Subject1);
        Console.WriteLine("Teacher Designation: " + Designation1);
        Console.WriteLine("Teacher Salary: " + Salary1 + "\n");
        Console.ReadLine();
    }
}

```

In the above case “**Deconstruct**” (name cannot be changed) is a special method which will expose the attributes of **Teacher** class. We can capture the values exposed by “**Deconstructors**” by using **Tuples**, through the instance of class we have created.

We can even capture the values as below:

E.g.: (var Id2, var Name2, var Subject2, var Designation2, var Salary2) = obj;

The above statement can be implemented as following also:

E.g.: var (Id2, Name2, Subject2, Designation2, Salary2) = obj;

Now you print the above values as below and to test that, add the below code in the **Main** method of “**TestTeacher**” class just above the **ReadLine** method.

```

var (Id2, Name2, Subject2, Designation2, Salary2) = obj;
Console.WriteLine("Teacher Id: " + Id2);
Console.WriteLine("Teacher Name: " + Name2);
Console.WriteLine("Teacher Subject: " + Subject2);
Console.WriteLine("Teacher Designation: " + Designation2);
Console.WriteLine("Teacher Salary: " + Salary2 + "\n");

```

We can also **overload** Deconstructors to access specific values from the list of attributes and to test that add the following **Deconstructor** in the **Teacher** class.

```

public void Deconstruct(out int? Id, out string? Name, out string? Subject)
{
    Id = this.Id;
    Name = this.Name;
    Subject = this.Subject;
}

```

Now we can capture only those 3 values and to test that add the below code in the **Main** method of “**TestTeacher**” class just above the **ReadLine** method.

```
var (Id3, Name3, Subject3) = obj;
Console.WriteLine("Teacher Id: " + Id3);
Console.WriteLine("Teacher Name: " + Name3);
Console.WriteLine("Teacher Subject: " + Subject3 + "\n");
```

Without Overloading the **Deconstructors** also we can access required attribute values by just putting “_” at the place whose values we don’t want to access, and to test this **Add** the below code in the **Main** method of **TestTeacher** class just above the **ReadLine** method.

```
var (Id4, _, Subject4, _, Salary4) = obj;
Console.WriteLine("Teacher Id: " + Id4);
Console.WriteLine("Teacher Subject: " + Subject4);
Console.WriteLine("Teacher Salary: " + Salary4 + "\n");
```

```
var (Id5, _, _, Designation5, Salary5) = obj;
Console.WriteLine("Teacher Id: " + Id5);
Console.WriteLine("Teacher Designation: " + Designation5);
Console.WriteLine("Teacher Salary: " + Salary5 + "\n");
```

Exceptions and Exception Handling

In the development of an application, we will be coming across 2 different types of errors, like:

- **Compile time errors.**
- **Runtime errors.**

Errors which occur in a program due to syntactical mistakes at the time of program compilation are known as compile time errors and these are not considered to be dangerous.

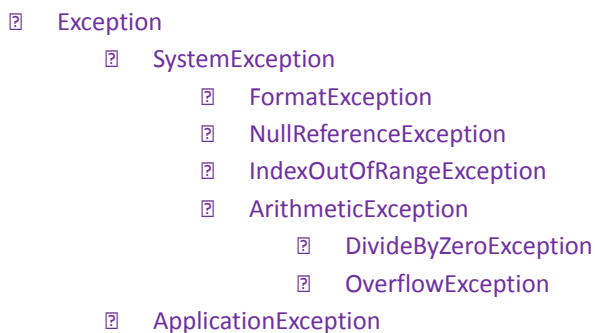
Errors which occur in a program while the execution of a program is taking place are known as runtime errors, which can occur due to various reasons like wrong implementation of logic, wrong input supplied to the program, missing of required resources etc. Runtime errors are dangerous because when they occur under the program, the program terminates abnormally at the same line where the error got occurred without executing the next lines of code. To test this, add a new class naming it as **ExceptionDemo.cs** and write the following code:

```
internal class ExceptionDemo
{
    static void Main()
    {
        Console.Write("Enter 1st number: ");
        int x = int.Parse(Console.ReadLine());
        Console.Write("Enter 2nd number: ");
        int y = int.Parse(Console.ReadLine());
        int z = x / y;
        Console.WriteLine("The result of division is: " + z);
        Console.WriteLine("End of the Program.");
    }
}
```

Execute the above program by using Ctrl + F5, and here there are chances of getting few runtime errors under the program, to check them enter the value for y as '0' or enter character input for x or y values, and in both cases when an error got occurred program gets terminated abnormal on the same line where error got occurred.

Exception: In C#, errors in the program at run time are caused through the program by using a mechanism called Exceptions. Exceptions are classes derived from class Exception of System namespace. Exceptions can be thrown by the .NET Framework CLR (Common Language Runtime) when basic operations fail or by code in a program. Throwing an exception involves creating an instance of an Exception-derived class, and then throwing that instance by using the throw keyword. There are so many Exception classes under the Framework Class Library where each class is defined representing a different type of error that occurs under the program, for example: FormatException, NullReferenceException, IndexOutOfRangeException, ArithmeticException etc.

Exceptions are basically 2 types like SystemExceptions and ApplicationExceptions. System Exceptions are pre-defined exceptions that are fatal errors which occur on some pre-defined error conditions like DivideByZero, FormatException, and NullReferenceException etc. ApplicationExceptions are non-fatal errors i.e. these are error that are caused by the programs explicitly. Whatever the exception it is every class is a sub class of class Exception only and the hierarchy of these exception classes will be as following:



Exception Handling: It is a process of stopping the abnormal termination of a program whenever a runtime error occurs under the program; if exceptions are handled under the program, we will be having the following benefits:

1. As abnormal termination is stopped, statements that are not related with the error can be still executed.
2. We can also take any corrective actions which can resolve the problems that may occur due to the errors.
3. We can display user friendly error messages to end users in place of pre-defined error messages.

How to handle an Exception: to handle an exception we need to enclose the code of the program under some special blocks known as try and catch blocks which should be used as following:

```
try
{
    -Statement's where there is a chance of getting runtime errors.
    -Statement's which should not execute when the error occurs.
}
catch(<Exception Class Name> [<Variable>])
{
```

-Statement's which should execute only when the error occurs.
}
[---<multiple catch blocks if required>---]

To test handling exceptions, add a new class TryCatchDemo.cs and write the following code:

```
internal class TryCatchDemo
{
    static void Main()
    {
        try
        {
            Console.Write("Enter 1st number: ");
            int x = int.Parse(Console.ReadLine());
            Console.Write("Enter 2nd number: ");
            int y = int.Parse(Console.ReadLine());
            int z = x / y;
            Console.WriteLine("The result of division is: " + z);
        }
        catch(DivideByZeroException)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine("Value of divisor can't be zero.");
            Console.ForegroundColor = ConsoleColor.White;
        }
        catch (FormatException)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine("Input values must be integers.");
            Console.ForegroundColor = ConsoleColor.White;
        }
        catch(Exception ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine(ex.Message);
            Console.ForegroundColor = ConsoleColor.White;
        }
        Console.WriteLine("End of the Program.");
    }
}
```

If we enclose the code under, try and catch blocks the execution of program will take place as following:

- If all the statements under try block are successfully executed (i.e., no error in the program), from the last statement of try the control directly jumps to the first statement which is present after all the catch blocks.
- If any statement under try causes an error from that line, without executing any other lines of code in try, control directly jumps to catch blocks searching for a catch block to handle the error:

- If a catch block is available that can handle the exception, then exceptions are caught by that catch block, executes the code inside of that catch block and from there again jumps to the first statement which is present after all the catch blocks.
- If a catch block is not available to handle that exception which got occurred, abnormal termination takes place again on that line.

Note: `Message` is a property under the `Exception` class which gets the error message associated with the exception that got occurred under the program, this property was defined as `virtual` under the class `Exception` and `overridden` under all the child classes of class `Exception` as per their requirement, that is the reason why when we call `ex.Message` under the last catch block, even if “`ex`” is the reference of parent class, it will get the error message that is associated with the child exception class but not of itself because we have already learnt in overriding that “parent’s reference which is created by using child classes instance will call child classes overridden members” i.e., nothing but `dynamic polymorphism`.

Finally Block: this is another block of code that can be paired with try along `with catch` or `without catch` also and the speciality of this block is, code written under this `block` gets executed at `any cost` i.e., when an exception got occurred under the program or an exception did not occur under the program. All statements under try gets executed only when there is no exception under the program and statements under catch block will be executed only when there is exception under the program whereas code under `finally` block gets executed in `both` the cases.

To test finally block add a new class “FinallyDemo.cs” and write the following code:

```
internal class FinallyDemo
```

```
{
    static void Main()
    {
        try
        {
            Console.Write("Enter 1st number: ");
            int x = int.Parse(Console.ReadLine());
            Console.Write("Enter 2nd number: ");
            int y = int.Parse(Console.ReadLine());

            if(y == 1) {
                return;
            }

            int z = x / y;
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine("The result of division is: " + z);
            Console.ForegroundColor = ConsoleColor.White;
        }
        catch (Exception ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine(ex.Message);
            Console.ForegroundColor = ConsoleColor.White;
        }
    }
}
```

```

finally
{
    Console.ForegroundColor = ConsoleColor.Blue;
    Console.WriteLine("Finally block got executed.");
    Console.ForegroundColor = ConsoleColor.White;
}
Console.WriteLine("End of the Program.");
}
}

```

Execute the above program for 2 times, first time by giving input which doesn't cause any error and second time by giving the input which causes an error and check the output where in both the cases **finally block** is executed.

In both the cases not only **finally** block along with its "End of the program." statement also gets executed, now test the program for the third time by giving the divisor value i.e., value to **y** as 1, so that, the if condition in the try block gets satisfied and **return** statement gets executed. As we are aware that **return** is a **jump** statement which jumps out of the method in execution, but in this case, it will jump out only after executing the **finally block** of the method because once the control enters **try** block, we cannot stop the execution of **finally** block.

Note: try, catch, and finally blocks can be used in 3 different combinations like:

- I. **try and catch** in this case exceptions that occur in the program are caught by the catch block so abnormal termination will not take place.
- II. **try, catch and finally:** in this case behavior will be same as above but along with it finally block keeps executing in any situation.
- III. **try and finally:** in this case exceptions that occur in the program are not caught because there is no catch block so abnormal termination will take place but still the code under finally block gets executed.

Note: to test **try & finally**, comment the **catch block** in the previous program and execute the program again, now when there is any **runtime error**, exception occurs and program gets **abnormally terminated** but in this case also we see **finally block** getting **executed**.

Application Exceptions: these are **non-fatal** application errors i.e.; these are errors that are caused by the programs explicitly. Application exceptions are generally raised by **programmers** under their programs basing on their **own error conditions**, for example in a division program we don't want the divisor value to be an **odd number** which is a condition specific to the programmer or developer. If a programmer wants to raise an exception explicitly under his program, he needs to do 2 things under the program.

1. Create the instance of any exception class.
2. Throw that instance by using throw statement. E.g.: **throw <instance of exception class>**

While creating an Exception class instance to throw explicitly we are provided with different options in choosing which exception class instance must be created to throw, like:

1. If any pre-defined **Exception** class is matching with our requirement, then we can create **instance** of that class and **throw**.

2. We can create instance of a pre-defined class i.e., `ApplicationException` by passing the error message that has to be displayed when the error got occurred as a `parameter` to the class `constructor` and then throw that instance.

E.g., `ApplicationException ex = new ApplicationException("<error message>");`
`throw ex;`

or

`throw new ApplicationException("<error message>");`

3. We can also define our own exception class, create instance of that class, and throw it when required. If we want to define a new exception class, we need to follow the below process:
 - I. Define a new class inheriting from any pre-defined Exception class (but `ApplicationException` is preferred choice as we are dealing with application exceptions) so that the new class also is an exception.
 - II. **Override** the **Message** property inherited from parent by providing the required error message.

To test this first add a new class under the project naming it DivideByOddNoException.cs and write the below:

```
public class DivideByOddNoException : ApplicationException
{
    public override string Message
    {
        get
        {
            return "Attempted to divide by odd number.";
        }
    }
}
```

Add a new class ThrowDemo.cs and write the below code:

```
internal class ThrowDemo
{
    static void Main()
    {
        Console.Write("Enter 1st number: ");
        int x = int.Parse(Console.ReadLine());
        Console.Write("Enter 2nd number: ");
        int y = int.Parse(Console.ReadLine());
        if(y % 2 > 0)
        {
            throw new ApplicationException(); //Either use default or parameterized constructor
            //throw new ApplicationException("Divisor can't be an odd number.");
            //throw new DivideByOddNoException();
        }
        int z = x / y;
        Console.WriteLine("The result of division is: " + z);
        Console.WriteLine("End of the Program.");
    }
}
```

```
}
```

Test the above program for the first time by giving the divisor value as “0” and you get the error message “Error in the application.”. Now comment the first throw statement and uncomment the second throw statement and try again with “0” as divisor which displays error message as “Divisor value should not be an odd number.”.

Now comment the second throw statement and uncomment the third throw statement so that when the divisor value is an odd number DivideByOddNoException will raise and displays the error message “Attempted to divide by odd number.”.

Delegates

Delegate is a **type** which holds the method(s) **reference** in an **object**. It is also referred to as a **type safe function pointer**. Delegates are roughly like **function-pointers** in **C++**; however, delegates are **type-safe** and **secure**.

A delegate **instance** can **encapsulate** a **static** or a **non-static** method also and **call** that method for execution. Effective use of a **delegate** improves the **performance** of applications.

Methods can be called in 2 different ways in C#, those are:

1. Using instance of a class if it is non-static and name of the class if it is static.
2. Using a delegate (either static or non-static).

To call a method by using delegate we need to adopt the following process:

1. Define a delegate.
2. Instantiate the delegate.
3. Call the delegate by passing required parameter values.

Syntax to define a Delegate:

```
[<modifiers>] delegate void | <type> Name([<Parameter List>])
```

Note: while defining a **delegate** you should follow the same **signature** of the method i.e., **parameters** of **delegate** should be same as the **parameters** of method and **return types** of **delegates** should be same as the **return types** of method, we want to call by using the **delegate**.

```
public void AddNums(int x, int y)
{
    Console.WriteLine(x + y);
}

public delegate void AddDel(int x, int y);

public static string SayHello(string name)
{
    return "Hello " + name;
}

public delegate string SayDel(string name);
```

Instantiate the delegate: In this process we create the **instance** of the **delegate** and bind the **method** we want to call by using the **delegate** to the **delegate**.


```
AddDel ad = new AddDel(AddNums);      or      AddDel ad = AddNums;
SayDel sd = new SayDel(SayHello);      or      SayDel sd = SayHello;
```

Calling the delegate: Call the **delegate** by passing required parameter values, so that the **method** which is bound with delegate gets executed.

```
ad(10, 20); ad(30, 40) ad(50, 60)
```

```
string s1 = sd("Raju"); string s2 = sd("Suneetha"); string s3 = sd("Srinivas");
```

Where to define a delegate?

Ans: Delegates can be defined either in a **class/structure** or with in a **namespace** just like we define other **types**.

Add a code file under the project naming it as Delegates.cs and write the following code:

```
namespace OOPSProject
{
    public delegate void MathDelegate(int x, int y);
    public delegate string WishDelegate(string str);
    public delegate void CalculatorDelegate(int a, int b, int c);
}
```

Add a class DelDemo1.cs under the project and write the following code:

```
internal class DelDemo1
{
    public void AddNums(int x, int y, int z)
    {
        Console.WriteLine($"Sum of given 3 no's is: {x + y + z}");
    }
    public static string SayHello(string Name)
    {
        return $"Hello {Name}, have a nice day!";
    }
    static void Main()
    {
        DelDemo1 obj = new DelDemo1();

        CalculatorDelegate cd = obj.AddNums;
        cd(10, 20, 30); cd(40, 50, 60); cd(70, 80, 90);

        WishDelegate wd = DelDemo1.SayHello;
        Console.WriteLine(wd("Raju"));
        Console.WriteLine(wd("Vijay"));
        Console.WriteLine(wd("Naresh"));

        Console.ReadLine();
    }
}
```

Multicast Delegate: It is a **delegate** which holds the **reference** of more than **one method**. Multicast delegates must contain only methods that return **void**. If we want to call multiple methods using a single delegate all the methods should have the same **Parameter types**. To test this, add a new class **DelDemo2.cs** under the project and write the following code:

```
internal class DelDemo2
{
    public void Add(int x, int y)
    {
        Console.WriteLine($"Add: {x + y}");
    }
    public void Sub(int x, int y)
    {
        Console.WriteLine($"Sub: {x - y}");
    }
    public void Mul(int x, int y)
    {
        Console.WriteLine($"Mul: {x * y}");
    }
    public void Div(int x, int y)
    {
        Console.WriteLine($"Div: {x / y}");
    }
    static void Main()
    {
        DelDemo2 obj = new DelDemo2();
        MathDelegate md = obj.Add;
        md += obj.Sub; md += obj.Mul; md += obj.Div;

        md(100, 25);
        Console.WriteLine();
        md(760, 20);
        Console.WriteLine();
        md -= obj.Mul;
        md(930, 15);
        Console.ReadLine();
    }
}
```

Anonymous Methods (Introduced in C# 2.0): In versions of **C#** before **2.0**, the only way to **instantiate** a **delegate** was to use **named** methods. **C# 2.0** introduced **anonymous methods** which provide a **technique** to pass a code block as a **delegate parameter**. **Anonymous** methods are basically methods without a **name**. An **anonymous** method is **in-line**, **un-named** method in the code. It is created using the **delegate** keyword and doesn't require **modifiers**, **name**, and **return type**. Hence, we can say an **anonymous** method has only **body** without **name**, **return type** and **optional parameters**. An **anonymous** method behaves like a regular method and allows us to write **in-line** code in place of **explicit** named methods. To test this, add a new class **DelDemo3.cs** and write the below code:

```

internal class DelDemo3
{
    static void Main()
    {
        CalculatorDelegate cd = delegate (int a, int b, int c)
        {
            Console.WriteLine($"Product of given numbers: {a * b * c}");
        };
        cd(10, 20, 30); cd(40, 50, 60); cd(70, 80, 90);

        WishDelegate wd = delegate (string user)
        {
            return $"Hello {user}, welcome to the application.";
        };

        Console.WriteLine(wd("Raju"));
        Console.WriteLine(wd("Pooja"));
        Console.WriteLine(wd("Praveen"));
        Console.ReadLine();
    }
}

```

Lambda Expression (Introduced in CSharp 3.0): While **Anonymous Methods** were a new feature in 2.0; **Lambda Expressions** are simply an improvement to syntax when using **Anonymous** method. Lambda Operator “=>” was introduced so that there is no longer a need to use the **delegate** keyword or provide the **type** of the parameters. The types can usually be **inferred** by **compiler** from usage based on the **delegate**. To test this, add a new class **DelDemo4.cs** and write the below code:

```

internal class DelDemo4
{
    static void Main()
    {
        CalculatorDelegate cd = (a, b, c) =>
        {
            Console.WriteLine($"Product of given numbers: {a * b * c}");
        };

        cd(10, 20, 30);
        cd(40, 50, 60);
        cd(70, 80, 90);

        WishDelegate wd = user =>
        {
            return $"Hello {user}, welcome to the application.";
        };
    }
}

```

```

Console.WriteLine(wd("Raju"));
Console.WriteLine(wd("Pooja"));
Console.WriteLine(wd("Praveen"));
Console.ReadLine();
}
}

```

Expression Bodied Members (Introduced in C# 6.0 & 7.0): Expression body definitions let you provide a member's implementation in a very **concise** and **readable** form. You can use an expression body definition whenever the logic for any supported member consists of a **single** expression.

An expression body definition has the following general syntax:

member => expression;

To test this, add a new class DelDemo5.cs and write the following code:

```

internal class DelDemo5
{
    static void Main()
    {
        CalculatorDelegate cd = (a, b, c) => Console.WriteLine($"Product of given numbers: {a * b * c}");
        cd(10, 20, 30);
        cd(40, 50, 60);
        cd(70, 80, 90);

        WishDelegate wd = user => $"Hello {user}, welcome to the application.";
        Console.WriteLine(wd("Raju"));
        Console.WriteLine(wd("Pooja"));
        Console.WriteLine(wd("Praveen"));
        Console.ReadLine();
    }
}

```

Why would we need to write a method without a name is **convenience** i.e., it's a **shorthand** that allows you to write a method in the same place you are going to use it. Especially useful in places where a method is being used **only once** and the method definition are **short**. It saves you the effort of declaring and writing a separate method to the containing class. Benefits are like **reduced typing**, i.e., no need to specify the name of the method, its return type, and its access modifier as well as when reading the code, you don't need to look **elsewhere** for the method definition. Anonymous methods should be **short**; a **complex** definition makes calling code difficult to read.

Support for **expression body definitions** was introduced for **methods** and **read-only properties** in **C# 6.0** and was expanded in **C# 7.0**. Expression body definitions can be used with type members listed in following table:

<u>Member</u>	<u>Supported as of...</u>
Method	C# 6.0
Read-only property	C# 6.0

Property	C# 7.0
Constructor	C# 7.0
Finalizer	C# 7.0
Indexer	C# 7.0
Deconstructor	C# 7.0

For example, below is a class defined without using expression bodied members:

```
internal class Circle1
{
    double _Radius;
    const float _Pi = 3.14f;
    public Circle1(double Radius)
    {
        _Radius = Radius;
    }
    public void Deconstruct(out double Radius)
    {
        Radius = _Radius;
    }
    ~Circle1()
    {
        Console.WriteLine("Instance is destroyed.");
    }
    public float Pi
    {
        get { return _Pi; }
    }
    public double Radius
    {
        get { return _Radius; }
        set { _Radius = value; }
    }
    public double GetRadius()
    {
        return _Pi * _Radius * _Radius;
    }
    public double GetPerimeter()
    {
        return 2 * _Pi * _Radius;
    }
}
```

Above class can be defined as following by using expression bodied members:

```
internal class Circle2
{
    const float _Pi = 3.14f;
```

```

double _Radius;

public Circle2(double Radius) => _Radius = Radius;           //C# 7.0

public void Deconstruct(out double Radius) => Radius = _Radius; //C# 7.0

~Circle2() => Console.WriteLine("Instance is destroyed.");    //C# 7.0

public float Pi => _Pi;                                       //C# 6.0

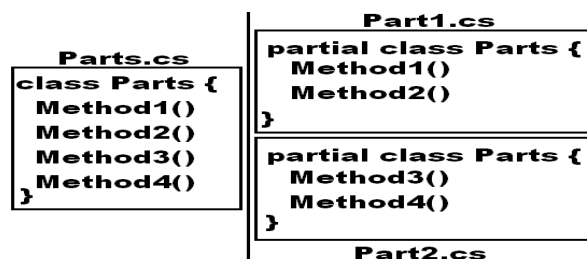
public double Radius                                         //C# 7.0
{
    get => _Radius;
    set => _Radius = value;
}

public double GetRadius() => _Pi * _Radius * _Radius;        //C# 6.0
public double GetPerimeter() => 2 * _Pi * _Radius;           //C# 6.0
}

```

Partial Types (Introduced in C# 2.0): It is possible to **split** the **definition** of a **class** or **struct** or **interface** over two or more **source files**. Each **source file** contains a **section** of the **type definition**, and all **parts** are combined when the application is **compiled**. There are several situations when **splitting** a **class** definition is **desirable** like:

- ② When working on **large projects**, spreading a **type** over **separate files** enable **multiple programmers** to work on it at the **same time**.
- ② **Visual Studio** uses these **partial classes** for **auto generation** of **source code** in the development of **Windows Forms Apps**, **WPF Apps**, **Web Forms Apps**, **Web Services** and so on.



Points to Remember:

- The **partial** keyword indicates that other parts of the class, struct, or interface can be defined in the namespace.
- All the parts must use the **partial** keyword.
- All the parts must be available at compile time to form the final type.
- All the parts must have the same accessibility, such as **public** or **internal**.
- If any part is declared **abstract**, then the whole type is considered **abstract**.
- If any part is declared **sealed**, then the whole type is considered **sealed**.
- If any part declares a base type, then the whole type inherits that class.
- Parts can specify different base interfaces, and the final type should implement all the interfaces listed by all the partial declarations.

- Any class, struct, or interface members declared in a partial definition are available to all the other parts.
- The final type is the combination of all the parts at compile time.
- The partial modifier is not available on delegate or enumeration declarations.

To test partial classes, add 2 new code files under the project Part1.cs and Part2.cs and write the below code:

```
namespace OOPSProject
{
    partial class Parts
    {
        public void Method1()
        {
            Console.WriteLine("Part1 - Method1");
        }
        public void Method2()
        {
            Console.WriteLine("Part1 - Method2");
        }
    }
}
namespace OOPSProject
{
    partial class Parts
    {
        public void Method3()
        {
            Console.WriteLine("Part2 - Method3");
        }
        public void Method4()
        {
            Console.WriteLine("Part2 - Method4");
        }
    }
}
```

Now to test the above partial class, add a new class TestParts.cs under the project and write the below code:

```
internal class TestParts
{
    static void Main()
    {
        Parts p = new Parts();
        p.Method1();
        p.Method2();
        p.Method3();
        p.Method4();
        Console.ReadLine();
    }
}
```

}

Collections

Arrays are simple **data structures** used to store data items of a specific type. Although commonly used, **arrays** have **limited capabilities**. For **example**, you must specify an array's **size** at the time of **declaration** and if at **execution** time, you wish to modify it, you must do so manually by **creating** a new **array** or by using **Array** class's **Resize** method, which creates a new **array** and **copies** the **existing** elements into the **new array**.

Collections are a set of **pre-packaged data structures** that offer greater **capabilities** than traditional **arrays**. They are **reusable**, **reliable**, **powerful**, and **efficient** and have been carefully **designed** and **tested** to ensure **quality** and **performance**. **Collections** are like **arrays** but provide **additional functionalities**, such as **dynamic resizing** - they **automatically increase** their size at **execution** time to **accommodate additional elements**, **inserting** of **new elements** and **removing** of **existing elements**.

Initially in 2002 **.NET** introduced so many collection classes under the namespace **System.Collections** (which is defined in the assembly **System.Collections.NonGeneric.dll**) like **Stack**, **Queue**, **LinkedList**, **SortedList**, **ArrayList**, **Hashtable** etc. and you can work out with these classes in your **application** where you need the **appropriate behavior**.

To work with **Collection** classes, create a new project of type **"Console App."** naming it as **"CollectionsProject"**, now under the first-class **Program** write the following code to use the **Stack** class which works on the principle **First in Last Out (FILO)** or **Last in First Out (LIFO)**:

```
using System.Collections;
internal class Program
{
    static void Main(string[] args)
    {
        Stack s = new Stack();

        s.Push('A'); s.Push(100); s.Push(false); s.Push(34.56); s.Push("Hello");

        foreach(object obj in s) {
            Console.Write(obj + " ");
        }
        Console.WriteLine();

        Console.WriteLine(s.Pop());
        foreach (object obj in s) {
            Console.Write(obj + " ");
        }
        Console.WriteLine();

        Console.WriteLine(s.Peek());
        foreach (object obj in s) {
            Console.Write(obj + " ");
        }
    }
}
```



```

    }
    Console.WriteLine();
    Console.WriteLine($"No. of items in the Stack: {s.Count}");
    s.Clear();
    Console.WriteLine($"No. of items in the Stack: {s.Count}");
    Console.ReadLine();
}
}

```

Using Queue class which works on the principle First in First Out (FIFO): Add a new class in the project naming it as "Class1.cs" and write the below code in it:

```

using System.Collections;
internal class Class1
{
    static void Main()
    {
        Queue q = new Queue();

        q.Enqueue('A'); q.Enqueue(100); q.Enqueue(false); q.Enqueue(34.56); q.Enqueue("Hello");

        foreach(object obj in q) {
            Console.Write(obj + " ");
        }
        Console.WriteLine();

        Console.WriteLine(q.Dequeue());
        foreach (object obj in q) {
            Console.Write(obj + " ");
        }
        Console.ReadLine();
    }
}

```

Auto-Resizing of Collections: The **capacity** of a collection increases dynamically i.e., when we add new **elements** to a Collection the **size** keeps on **incrementing** automatically. Every **collection** class has 3 **constructors** to it and the behavior of **collections** will be as following when the instance is created using different **constructor**:

- i. **Default Constructor:** initializes a new **instance** of the **collection** class that is **empty** and has the **default initial capacity** as **zero** which becomes **4** after adding the **first** element and from then when ever needed the current capacity **doubles**.
 - ii. **Collection(int Capacity):** Initializes a new **instance** of the **collection** class that is **empty** and has the specified **initial capacity**, here also when requirement comes the current capacity **doubles**.
 - iii. **Collection(Collection c):** This is a **Copy Constructor**. Initializes a new **instance** of the **collection** class that contains elements copied from an **old collection** and that has the same **initial capacity** as the number of elements **copied**, here also when requirement comes current capacity **doubles**.
-

ArrayList: this collection class works same as an **array** but provides **auto resizing**, **inserting**, and **deleting** of items. To work with an ArrayList, add a new class in the project naming it as "Class2.cs" and write the below code in it:

```
using System.Collections;
internal class Class2
{
    static void Main()
    {
        ArrayList Coll1 = new ArrayList();
        Console.WriteLine($"Initial capacity: {Coll1.Capacity}");

        Coll1.Add('A');
        Console.WriteLine($"Capacity of the collection after adding 1st item: {Coll1.Capacity}");

        Coll1.Add(100); Coll1.Add(false); Coll1.Add(34.56);
        Console.WriteLine($"Capacity of the collection after adding 4th item: {Coll1.Capacity}");

        Coll1.Add("Hello");
        Console.WriteLine($"Capacity of the collection after adding 5th item: {Coll1.Capacity}");

        for (int i=0;i< Coll1.Count;i++) {
            Console.Write(Coll1[i] + " ");
        }
        Console.WriteLine();

        //Coll1.Remove(false);
        //Coll1.RemoveAt(2);
        Coll1.RemoveRange(2, 1);
        foreach(object obj in Coll1) {
            Console.Write(obj + " ");
        }
        Console.WriteLine();

        Coll1.Insert(2, true);
        foreach (object obj in Coll1) {
            Console.Write(obj + " ");
        }
        Console.WriteLine("\n");

        ArrayList Coll2 = new ArrayList(Coll1);
        foreach (object obj in Coll2) {
            Console.Write(obj + " ");
        }
        Console.WriteLine();
        Console.WriteLine($"Initial capacity of new collection: {Coll2.Capacity}");
        Coll2.Add(false);
        Console.WriteLine($"Capacity of new collection after adding new item: {Coll2.Capacity}");
    }
}
```

```

Coll2.TrimToSize();
Console.WriteLine($"Capacity of new collection after calling TrimToSize: {Coll2.Capacity}");
Console.ReadLine();
}
}

```

Hashtable: it is a **collection** with stores elements in it as “Key/Value Pairs” i.e., **Array** and **ArrayList** also has a **key** to access the **values** under them which is the **index** that starts at “0” to number of **elements - 1**, whereas in case of **Hashtable** these **keys** can also be defined by us and can be of any **data type**. To work with Hashtable add a new class in the project naming it as “**Class3.cs**” and write the below code in it:

```

using System.Collections;
internal class Class3
{
    static void Main()
    {
        Hashtable Emp = new Hashtable();
        Emp.Add("Emp-Id", 1001);
        Emp.Add("Emp-Name", "Scott");
        Emp.Add("Job", "CEO");
        Emp.Add("Mgr-Id", null);
        Emp.Add("Salary", 50000.00);
        Emp.Add("Commission", 0.00f);
        Emp.Add("Dept-Id", 10);
        Emp.Add("Dept-Name", "Administration");
        Emp.Add("Location", "Mumbai");
        Emp.Add("Status", true);
        Emp.Add("PAN", "AKYPM 1234K");
        Emp.Add("Aadhar No.", "1234 5678 9012");
        Emp.Add("Mobile", "98392 14256");
        Emp.Add("Home Phone", "2718 6547");
        Emp.Add("Email", "Scott@gmail.com");

        foreach(object key in Emp.Keys)
        {
            Console.WriteLine($"{key}: {Emp[key]}");
        }
        Console.ReadLine();
    }
}

```

Generics: Generics are added in **C# 2.0** introducing to the **.NET Framework** the concept of **type parameters**, which make it possible to design classes, and methods that defer the specification of one or more types until the class or method is declared and instantiated by client code. For example, by using a generic type parameter “T” you can write a single class that other client code can use without incurring the cost or risk of runtime casts or boxing operations, in simple words Generics allow you to define a class with placeholders for the type of its fields,

methods, parameters, etc. Generics replace these placeholders with some specific type at consumption time. To understand these, add a class naming it as **“GenericMethods.cs”** and write the below code:

```
internal class GenericMethods
{
    public bool AreEqual<T>(T a, T b)
    {
        if (a.Equals(b))
            return true;
        else
            return false;
    }
    static void Main()
    {
        GenericMethods obj = new GenericMethods();
        Console.WriteLine(obj.AreEqual<int>(100, 200));
        Console.WriteLine(obj.AreEqual<bool>(true, true));
        Console.WriteLine(obj.AreEqual<double>(34.56, 87.12));
        Console.WriteLine(obj.AreEqual<string>("Hello", "Hello"));
        Console.ReadLine();
    }
}
```

Just like we are passing Type parameter to methods it is possible to pass them to a class also, to test this add a code file naming it as **“TestGenericClass.cs”** and write the following:

```
namespace CollectionsProject
{
    class Math<T>
    {
        public T Add(T a, T b)
        {
            dynamic d1 = a;
            dynamic d2 = b;
            return d1 + d2;
        }
        public T Sub(T a, T b)
        {
            dynamic d1 = a;
            dynamic d2 = b;
            return d1 - d2;
        }
        public T Mul(T a, T b)
        {
            dynamic d1 = a;
            dynamic d2 = b;
            return d1 * d2;
        }
    }
}
```

```

    }

    public T Div(T a, T b)
    {
        dynamic d1 = a;
        dynamic d2 = b;
        return d1 / d2;
    }
}

internal class TestGenericClass
{
    static void Main()
    {
        Math<int> mi = new Math<int>();
        Console.WriteLine(mi.Add(100, 200));
        Console.WriteLine(mi.Sub(234, 123));
        Console.WriteLine(mi.Mul(12, 46));
        Console.WriteLine(mi.Div(900, 45));
        Console.WriteLine();

        Math<double> md = new Math<double>();
        Console.WriteLine(md.Add(145.35, 12.5));
        Console.WriteLine(md.Sub(45.6, 23.3));
        Console.WriteLine(md.Mul(15.67, 3.4));
        Console.WriteLine(md.Div(168.2, 14.5));
        Console.ReadLine();
    }
}
}

```

Generic Collections: these are also introduced in **C# 2.0** which are extension to collections we have been discussing above, in case of collection (non-generic) classes the elements being added in them are of type **object**, so we can store any type of values in them which requires **boxing** and **un-boxing**, whereas in case of **generic collections** we can store specified type of values which provides **type safety**. Microsoft has re-implemented all the existing collection classes under a new namespace **System.Collections.Generic** but the main difference is while creating instance of generic collection classes we need to explicitly specify the type of values we want to store under them. In this namespace we have been provided with many classes like classes in **System.Collections** namespace as following:

Stack<T>, Queue<T>, LinkedList<T>, SortedList<T>, List<T>, Dictionary<TKey, TValue>

Note: **<T>** refers to the **type** of values we want to store under them. For example:

```

Stack<int> si = new Stack<int>();           //Stores integer values only
Stack<float> sf = new Stack<float>();       //Stores float values only
Stack<string> ss = new Stack<string>();     //Stores string values only
Stack<object> so = new Stack<object>();     //Stores any type of value same as non-generic collections

```

List: this class is same as **ArrayList** we have discussed under collections above.

To work with this List, add a new class in the project naming it as "Class4.cs" and write the below code in it:

```
internal class Class4
{
    static void Main()
    {
        List<int> Coll = new List<int>();
        Coll.Add(10); Coll.Add(20); Coll.Add(30); Coll.Add(40); Coll.Add(50);

        for(int i=0;i<Coll.Count;i++) {
            Console.Write(Coll[i] + " ");
        }
        Console.WriteLine();

        Coll.Insert(3, 35);
        foreach(int i in Coll) {
            Console.Write(i + " ");
        }
        Console.WriteLine();

        Coll.Remove(30); or Coll.RemoveAt(2); or Coll.RemoveRange(2, 1);
        foreach (int i in Coll) {
            Console.Write(i + " ");
        }
        Console.WriteLine();

        Console.ReadLine();
    }
}
```

Dictionary: this class is same as **Hashtable** we have discussed under collections but here while creating the object we need to specify the **type** for **keys** as well as for **values** also, as following:

Dictionary<TKey, TValue>

To work with Hashtable add a new class in the project naming it as "Class5.cs" and write the below code in it:

```
internal class Class5
{
    static void Main()
    {
        Dictionary<string, object?> Emp = new Dictionary<string, object?>();
        Emp.Add("Emp-Id", 1001);
        Emp.Add("Emp-Name", "Scott");
        Emp.Add("Job", "CEO");
        Emp.Add("Mgr-Id", null);
    }
}
```

```

Emp.Add("Salary", 50000.00);
Emp.Add("Commission", 0.00f);
Emp.Add("Dept-Id", 10);
Emp.Add("Dept-Name", "Administration");
Emp.Add("Location", "Mumbai");
Emp.Add("Status", true);
Emp.Add("PAN", "AKYPM 1234K");
Emp.Add("Adhar No.", "1234 5678 9012");
Emp.Add("Mobile", "98392 14256");
Emp.Add("Home Phone", "2718 6547");
Emp.Add("Email", "Scott@gmail.com");

foreach(string key in Emp.Keys) {
    Console.WriteLine($"{key}: {Emp[key]}");
}
Console.ReadLine();
}
}

```

Collection Initializers: this is a new feature added in **C# 3.0** which allows to initialize a collection directly at the time of declaration like an array, as following:

```

List<int> Coll1 = new List<int>() { 10, 20, 30, 40, 50 };
List<string> Coll2 = new List<string>() { "Red", "Blue", "Green", "White", "Yellow" };

```

Add a new class in the project naming it as Class6.cs and write the below code in it:

```

internal class Class6
{
    static void Main()
    {
        //Copying values > 40 from 1 list to another list and arranging them in descending order
        List<int> coll1 = new List<int>() { 13,56,29,98,24,54,79,39,8,42,22,93,6,73,35,67,48,18,61,32,86,15,21,81,2 };
        List<int> coll2 = new List<int>();

        foreach(int i in coll1)
        {
            if(i > 40)
            {
                coll2.Add(i);
            }
        }
        coll2.Sort();
        coll2.Reverse();
        Console.WriteLine(String.Join(" ", coll2));
        Console.ReadLine();
    }
}

```

The above program if used an array, code will be as following (Add a new class Class7.cs and write the below):

```
internal class Class7
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        //Copying values > 40 from 1 array to another array and arranging them in descending order
```

```
        int[] arr = { 13, 56, 29, 98, 24, 54, 79, 39, 8, 42, 22, 93, 6, 73, 35, 67, 48, 18, 61, 32, 86, 15, 21, 81, 2 };
```

```
        int Count = 0, Index = 0;
```

```
        foreach(int i in arr)
```

```
        {
```

```
            if(i > 40)
```

```
            {
```

```
                Count += 1;
```

```
            }
```

```
        }
```

```
        int[] brr = new int[Count];
```

```
        foreach(int i in arr)
```

```
        {
```

```
            if(i > 40)
```

```
            {
```

```
                brr[Index] = i;
```

```
                Index += 1;
```

```
            }
```

```
        }
```

```
        Array.Sort(brr);
```

```
        Array.Reverse(brr);
```

```
        Console.WriteLine(String.Join(" ", brr));
```

```
        Console.ReadLine();
```

```
    }
```

```
}
```

In the above 2 programs we are **filtering** the values of a **List** and **Array** which are greater than **40** and then **arranging** them in **descending** order; to do this we have written a **substantial** amount of code which is the traditional process of performing **filters** on **Arrays** and **Collections**.

In **C# 3.0** Microsoft has introduced a new language known as “**LINQ**” much like **SQL** (which we use universally with **Relational Databases** to perform queries). **LINQ** allows you to write query expressions (similar to **SQL Queries**) that can retrieve information from a wide variety of **Data Sources** like **Objects**, **Databases** and **XML**.

Introduction to LINQ: LINQ stands for **Language Integrated Query**. LINQ is a data querying methodology which provides querying capabilities to **.NET** languages with syntax like an **SQL Query**.

LINQ has a great power of **querying** on any **source** of data, where the **Data Source** could be collections of objects (**arrays & collections**), **Database** or **XML Source** and it is divided into 3 parts:

LINQ to Objects:

- Used to perform queries against the in-memory data like an **Array or Collection**.

LINQ to XML (Xling):

- Used to perform queries against an **XML Source**.

LINQ to Databases: under this we again have 2 options like,

- **LINQ to SQL** is used to perform queries against a **Relation Database**, but only **Microsoft SQL Server**.
- **LINQ to Entities** is used to perform queries against any **Relation Database** like **SQL Server, Oracle**, etc.

Advantages of LINQ:

- **LINQ** offers an object-based, language-integrated way to **Query** over data, no matter where that data came from. So, through **LINQ** we can query **Database, XML** as well as **Collections & Arrays**.
- **Compile-time** syntax checking.
- It allows you to **Query** - **Collections, Arrays**, and **classes** etc. in the **native language** of your application like **VB** or **C#** or **F#** with out using **SQL Syntax's**.

LINQ to Objects

This is designed to write **queries** against the **in-memory data** like an **array** or **collection** and **filter** or **sort** the information present under them. **Syntax** of the query we want to use on **objects** will be as following:

from <alias> in <array name | collection name> [<clauses>] select <alias> | new {<Column List>}

- A LINQ-Query **starts** with **from** and **ends** with **select**.
- In **clauses** we need to use the **alias name** just like we use **column names** in **SQL** in case of **scalar types**.
- Clauses in **LINQ** are **where, group by** and **order by**.
- To use **LINQ** in your application first we need to import **"System.Linq"** namespace.

We can write our previous 2 programs where we have **filtered** the data of a **List** or **Array** and arranged in **sorted order** by using **LINQ** and to test that add a new class with the name **"Class8.cs"** and write the below code:

```
internal class Class8
{
    static void Main()
    {
        List<int> coll1 = new List<int>() { 13,56,29,98,24,54,79,39,8,42,22,93,6,73,35,67,48,18,61,32,86,15,21,81,2 };
        var coll2 = from i in coll1 where i > 40 orderby i descending select i;
```

```

Console.WriteLine(String.Join(" ", coll2));

int[] arr = { 13, 56, 29, 98, 24, 54, 79, 39, 8, 42, 22, 93, 6, 73, 35, 67, 48, 18, 61, 32, 86, 15, 21, 81, 2 };
var brr = from i in arr where i > 40 orderby i descending select i;
Console.WriteLine(String.Join(" ", brr));
Console.ReadLine();
}
}

```

Note: the values that are returned by a LINQ query can be captured by using **implicitly typed local variables**, so in above code “coll2” & “brr” are implicitly declared **collection/array** that stores the values retrieved by the **Query**.

In **traditional** process of **filtering** data of an **array** or **collection** we have **repetition** statements that filter arrays focusing on the **process** of getting the results i.e., **iterating** through the elements and checking whether they satisfy the desired **criteria**, whereas **LINQ** specifies, not the steps necessary to get the results, but rather the conditions that selected elements must satisfy and this is known as **declarative programming** - as opposed to **imperative programming** (which we’ve been using so far) in which we specify the **actual steps** to perform a **task**. **Procedural** and **Object-Oriented** Languages are a **subset** of **imperative**.

The **queries** we have used above specifies that the result should consist of all the **int’s** in the **List** or **Array** that are greater than **40**, but it does not specify how to **obtain** the **result**, **C#** compiler **generates** all the necessary code **automatically**, which is one of the great **strengths** of **LINQ**.

LINQ Providers: The syntax of **LINQ** is built into the **language**, and **LINQ** can be used in many different contexts because of the **libraries** known as **providers**. A **LINQ provider** is a set of classes that implement **LINQ** operations and enable programs to interact with **Data Sources** to perform tasks such as **sorting**, **grouping**, and **filtering** elements. **System.Linq** is the **LINQ Provider** or **Library** that we need for writing **Queries** in our code.

To test writing queries on a Collection, add a new Class naming it as Class9.cs and write the below code in it:

```

internal class Class9
{
    static void Main()
    {
        string[] colors = { "Red", "Blue", "Green", "Black", "White", "Brown", "Orange", "Purple", "Yellow", "Aqua" };

        //Gets the list of all colors as is
        var coll1 = from s in colors select s;
        Console.WriteLine(String.Join(" ", coll1) + "\n");

        //Gets the list of all colors in ascending order
        var coll2 = from s in colors orderby s select s;
        Console.WriteLine(String.Join(" ", coll2) + "\n");

        //Gets the list of all colors in descending order
        var coll3 = from s in colors orderby s descending select s;
        Console.WriteLine(String.Join(" ", coll3) + "\n");
    }
}

```

//Gets the list of colors whose length is 5 characters

```
var coll4 = from s in colors where s.Length == 5 select s;  
Console.WriteLine(String.Join(" ", coll4) + "\n");
```

//Getting the list of colors whose name starts with character "B":

```
var coll5 = from s in colors where s[0] == 'B' select s;  
Console.WriteLine(String.Join(" ", coll5));  
var coll6 = from s in colors where s.IndexOf("B") == 0 select s;  
Console.WriteLine(String.Join(" ", coll6));  
var coll7 = from s in colors where s.StartsWith("B") select s;  
Console.WriteLine(String.Join(" ", coll7));  
var coll8 = from s in colors where s.Substring(0, 1) == "B" select s;  
Console.WriteLine(String.Join(" ", coll8) + "\n");
```

//Getting the list of colors whose name ends with character "e":

```
var coll9 = from s in colors where s[s.Length - 1] == 'e' select s;  
Console.WriteLine(String.Join(" ", coll9));  
var coll10 = from s in colors where s.IndexOf("e") == s.Length - 1 select s;  
Console.WriteLine(String.Join(" ", coll10));  
var coll11 = from s in colors where s.EndsWith("e") select s;  
Console.WriteLine(String.Join(" ", coll11));  
var coll12 = from s in colors where s.Substring(s.Length - 1) == "e" select s;  
Console.WriteLine(String.Join(" ", coll12) + "\n");
```

//Getting the list of colors whose name contains character "a" at 3rd place:

```
var coll13 = from s in colors where s[2] == 'a' select s;  
Console.WriteLine(String.Join(" ", coll13));  
var coll14 = from s in colors where s.IndexOf("a") == 2 select s;  
Console.WriteLine(String.Join(" ", coll14));  
var coll15 = from s in colors where s.Substring(2, 1) == "a" select s;  
Console.WriteLine(String.Join(" ", coll15) + "\n");
```

//Getting the list of colors whose name contains character "O or o" in it:

```
var coll16 = from s in colors where s.Contains('O') || s.Contains('o') select s;  
Console.WriteLine(String.Join(" ", coll16));  
var coll17 = from s in colors where s.IndexOf('O') >= 0 || s.IndexOf('o') >= 0 select s;  
Console.WriteLine(String.Join(" ", coll17));  
var coll18 = from s in colors where s.ToUpper().Contains('O') select s;  
Console.WriteLine(String.Join(" ", coll18));  
var coll19 = from s in colors where s.ToLower().IndexOf('o') >= 0 select s;  
Console.WriteLine(String.Join(" ", coll19) + "\n");
```

//Getting the list of colors whose name doesn't contains character "O or o" in it:

```
var coll20 = from s in colors where s.Contains('O') == false && s.Contains('o') == false select s;  
Console.WriteLine(String.Join(" ", coll20));  
var coll21 = from s in colors where s.IndexOf('O') == -1 && s.IndexOf('o') == -1 select s;
```

```

Console.WriteLine(String.Join(" ", coll21));
var coll22 = from s in colors where s.ToUpper().Contains('O') == false select s;
Console.WriteLine(String.Join(" ", coll22));
var coll23 = from s in colors where s.ToLower().IndexOf('o') == -1 select s;
Console.WriteLine(String.Join(" ", coll23) + "\n");
Console.ReadLine();
}
}

```

Note: The type of values being stored in a generic collection can be of user-defined type values also like a class type or structure type that is defined to represent an entity as following:

```
List<Customer> Customers = new List<Customer>();
```

In the above code assume **Customer** is a user-defined class type that represents an entity **Customer**, so we can store objects of **Customer** type under the List where each object can internally represent different attributes of Customer like **Id**, **Name**, **City**, **Balance**, **Status** etc.

To test this above add a class in the project with the name Customer.cs and write the below code in it:

```

public class Customer
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public string? City { get; set; }
    public double Balance { get; set; }
    public bool Status { get; set; }
    public override string ToString() => $"Id: {Id}; Name: {Name}; City: {City}; Balance: {Balance}; Status: {Status}";
}

```

Add another class in the project with the name Class10.cs and write the below code in it:

```

internal class Class10
{
    static void Main()
    {
        //Creating instance of Customer class using Object Initializers.
        Customer c1 = new Customer { Id = 101, Name = "Scott", City = "Delhi", Balance = 15000.00, Status = true };
        Customer c2 = new Customer { Id = 102, Name = "Dave", City = "Mumbai", Balance = 10000.00, Status = true };
        Customer c3 = new Customer { Id = 103, Name = "Sunitha", City = "Chennai", Balance = 15600.00, Status = false };
        Customer c4 = new Customer { Id = 104, Name = "David", City = "Delhi", Balance = 22000.00, Status = true };
        Customer c5 = new Customer { Id = 105, Name = "John", City = "Kolkata", Balance = 34000.00, Status = true };
        Customer c6 = new Customer { Id = 106, Name = "Jane", City = "Hyderabad", Balance = 19000.00, Status = true };
        Customer c7 = new Customer { Id = 107, Name = "Kavitha", City = "Mumbai", Balance = 16500.00, Status = true };
        Customer c8 = new Customer { Id = 108, Name = "Steve", City = "Bengaluru", Balance = 34600.00, Status = false };
        Customer c9 = new Customer { Id = 109, Name = "Sophia", City = "Chennai", Balance = 6300.00, Status = true };
        Customer c10 = new Customer { Id = 110, Name = "Rehman", City = "Delhi", Balance = 9500.00, Status = true };
        Customer c11 = new Customer { Id = 111, Name = "Raj", City = "Hyderabad", Balance = 9800.00, Status = false };
        Customer c12 = new Customer { Id = 112, Name = "Rupa", City = "Kolkata", Balance = 13200.00, Status = true };
        Customer c13 = new Customer { Id = 113, Name = "Ram", City = "Bengaluru", Balance = 47700.00, Status = true };
    }
}

```

```
Customer c14 = new Customer { Id = 114, Name = "Joe", City = "Hyderabad", Balance = 26900.00, Status = false };
Customer c15 = new Customer { Id = 115, Name = "Peter", City = "Delhi", Balance = 17400.00, Status = true };
```

```
//Created a List of Customers and added all the Customer instances into the List
```

```
List<Customer> Customers = new List<Customer>()
{
    c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11, c12, c13, c14, c15
};
```

```
//Implementing LINQ Queries for fetching the data from the List using LINQ to Objects.
```

```
//Fetching all rows and columns from the List un-conditionally:
```

```
//var Coll = from c in Customers select c;
```

```
//Fetching selected columns and giving alias names to columns:
```

```
//var Coll = from c in Customers select new { c.Id, c.Name, IsActive = c.Status };
```

```
//Order By Clause:
```

```
//var Coll = from c in Customers orderby c.Name select c;
```

```
//var Coll = from c in Customers orderby c.Balance descending select c;
```

```
//Where Clause:
```

```
//var Coll = from c in Customers where c.Status == true select c;
```

```
//var Coll = from c in Customers where c.Status == false select c;
```

```
//var Coll = from c in Customers where c.Balance > 25000 select c;
```

```
//var Coll = from c in Customers where c.City == "Hyderabad" select c;
```

```
//var Coll = from c in Customers where c.City == "Bengaluru" && c.Balance > 40000 select c;
```

```
//var Coll = from c in Customers where c.City == "Chennai" || c.Balance > 30000 select c;
```

```
//Group By Clause:
```

```
//var Coll = from c in Customers group c by c.City into G
                select new { City = G.Key, Customers = G.Count() };
```

```
//var Coll = from c in Customers group c by c.City into G
                select new { City = G.Key, MaxBalance = G.Max(c => c.Balance) };
```

```
//var Coll = from c in Customers group c by c.City into G
                select new { City = G.Key, MinBalance = G.Min(c => c.Balance) };
```

```
//var Coll = from c in Customers group c by c.City into G
                select new { City = G.Key, AvgBalance = G.Average(c => c.Balance) };
```

```
//var Coll = from c in Customers group c by c.City into G
                select new { City = G.Key, TotalBalance = G.Sum(c => c.Balance) };
```

```
//Having (Where) Clause:
```

```
//var Coll = from c in Customers group c by c.City into G
                where G.Count() > 2
                select new {
                    City = G.Key, Customers = G.Count() };
```

```
//var Coll = from c in Customers group c by c.City into G
                where G.Max(c => c.Balance) > 25000
```

```

        select new {
            City = G.Key, MaxBalance = G.Max(c => c.Balance) };
//var Coll = from c in Customers group c by c.City into G
        where G.Min(c => c.Balance) > 10000
        select new {
            City = G.Key, MaxBalance = G.Min(c => c.Balance) };
//var Coll = from c in Customers group c by c.City into G
        where G.Average(c => c.Balance) > 20000
        select new {
            City = G.Key, MaxBalance = G.Average(c => c.Balance) };

var Coll = from c in Customers group c by c.City into G
        where G.Sum(c => c.Balance) < 30000
        select new {
            City = G.Key, MinBalance = G.Sum(c => c.Balance) };

foreach (var customer in Coll)
{
    Console.WriteLine(customer);
}
Console.ReadLine();
}
}

```

Note: We don't have "having" clause in LINQ, so wherever you need the functionality of "having", use "where" overthere, because both are used for filtering only. LINQ has not given us 2 separate clauses i.e., if we use "where" before "group by" it works like "where" clause whereas if we use "where" after "group by" it works like "having".

Task Parallel Library (TPL)

The Task Parallel Library (TPL) is a set of public types "System.Threading" and "System.Threading.Tasks" namespaces. The purpose of TPL is to make developers more productive by simplifying the process of adding parallelism and concurrency to applications. The TPL scales the degree of concurrency dynamically to most efficiently use all the processors that are available. In addition, the TPL handles the partitioning of the work, the scheduling of Threads on the Thread Pool, cancellation support, state management, and other low-level details. By using TPL, you can maximize the performance of your code while focusing on the work that your program is designed to accomplish.

Starting with .NET Framework 4, the TPL is the preferred way to write multithreaded and parallel code. However, not all code is suitable for parallelization. For example, if a loop performs only a small amount of work on each iteration, or it doesn't run for many iterations, then the overhead of parallelization can cause the code to run more slowly. Furthermore, parallelization like any multithreaded code adds complexity to your program execution. Although the TPL simplifies multithreaded scenarios, it is recommended that you have a basic understanding of threading concepts, for example, locks, deadlocks, and race conditions, so that you can use the TPL effectively.

To test the examples given below create a new "Console Application" Project naming it as "TPLProject" and choose the Target Framework as: ".NET 8.0 (Long-term support)", check the Checkbox => "Do not use top-level

statements” and click on the “Create” button. First let’s write a program without using multi-Threading and to do that write the below code in the default class “Program” which is present under “Program.cs” file by deleting the existing code in the class:

```
internal class Program
{
    static void Print1()
    {
        for (int i = 1; i <= 100; i++)
        {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print1 Method: {i}");
        }
    }
    static void Print2()
    {
        for (int i = 1; i <= 100; i++)
        {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print2 Method: {i}");
        }
    }
    static void Print3()
    {
        for (int i = 1; i <= 100; i++)
        {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print3 Method: {i}");
        }
    }
    static void Main(string[] args)
    {
        Print1(); Print2(); Print3();
    }
}
```

Note: in the above code we have defined 3 methods in the class and called them in a single threaded model so each method is executed 1 after the other and all the methods are executed by the Main Thread, and we can see the Id of that Thread which will be printed by the statement “Thread.CurrentThread.ManagedThreadId”.

Now let’s re-write the above program using multi-Threading, and to do that add a new class in the project naming it as “Class1.cs” and write the below code in the class:

```
internal class Class1
{
    static void Print1()
    {
        for (int i = 1; i <= 100; i++) {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print1 Method: {i}");
        }
    }
}
```

```

}
static void Print2()
{
    for (int i = 1; i <= 100; i++) {
        Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print2 Method: {i}");
    }
}
static void Print3()
{
    for (int i = 1; i <= 100; i++) {
        Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print3 Method: {i}");
    }
}
static void Main()
{
    Thread t1 = new Thread(Print1);
    Thread t2 = new Thread(Print2);
    Thread t3 = new Thread(Print3);
    t1.Start(); t2.Start(); t3.Start();
    t1.Join(); t2.Join(); t3.Join();
    Console.WriteLine($"Main thread with Id: {Thread.CurrentThread.ManagedThreadId} is exiting.");
}
}

```

Note: in the above code we have defined 3 methods and called them by using 3 **separate threads** so each thread will execute 1 method **concurrently** and, in the program, we will be having 4 threads along with the **Main thread** and we can see the Id of those **Threads** which will be printed by the statement **"Thread.CurrentThread.ManagedThreadId"**.

Now let's re-write the above program using **Task Parallelism**, and to do that add a new class in the **project** naming it as **"Class2.cs"** and write the below code in the class:

```

internal class Class2
{
    static void Print1()
    {
        for (int i = 1; i <= 100; i++) {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print1 Method: {i}");
        }
    }
    static void Print2()
    {
        for (int i = 1; i <= 100; i++) {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print2 Method: {i}");
        }
    }
    static void Print3()
    {

```



```

for (int i = 1; i <= 100; i++) {
    Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; Print3 Method: {i}");
}
}
static void Main()
{
    Task t1 = new Task(Print1);
    Task t2 = new Task(Print2);
    Task t3 = new Task(Print3);
    t1.Start(); t2.Start(); t3.Start();
    t1.Wait(); t2.Wait(); t3.Wait();
    Console.WriteLine($"Main thread with Id: {Thread.CurrentThread.ManagedThreadId} is exiting.");
}
}

```

In the above case in place of **Threads** we have used **Tasks** and these **Tasks** will internally use **Threads** to execute the code and the **“Wait”** method we called here is same as the **“Join”** method we use in **Threads**. The process of creating **Tasks**, calling **Start** and **Wait** methods can be simplified and implemented i.e., we can implement the code in **Main** method of the above program as following also:

```

Task t1 = Task.Factory.StartNew(Print1);
Task t2 = Task.Factory.StartNew(Print2);
Task t3 = Task.Factory.StartNew(Print3);
Task.WaitAll(t1, t2, t3);
Console.WriteLine($"Main thread with Id: {Thread.CurrentThread.ManagedThreadId} is exiting.");

```

In the above code **Factory** is a **static property** of the **Task** class which will refer to **TaskFactory** class and the **StartNew** method of **TaskFactory** class will create a new **Thread**, starts it, and returns the reference of it.

Note: in the above code also, we have defined 3 **methods** and called them by using 3 **separate tasks**, so each **task** will execute 1 **method** concurrently. In this program also we will be having 4 **threads** along with the **Main thread** and we can see the **Id** of those **Threads** in the output.

Calling value returning methods with-out parameters by using Tasks: in the above programs the methods that we called by using **Tasks** are all **non-value returning** as well as they **do not take any parameters** also. Now let's learn how to call **value returning methods** by using **Task** and to do that add a new class in the **project** naming it as **“Class3.cs”** and write the below code in it:

```

internal class Class3
{
    static int GetLength()
    {
        string str = "";
        for (int i = 1; i <= 100000; i++) {
            str += i;
        }
        return str.Length;
    }
}

```

```

static string ToUpper()
{
    string str = "Hello World";
    return str.ToUpper();
}
static void Main()
{
    Task<int> t1 = new Task<int>(GetLength);
    Task<string> t2 = new Task<string>(ToUpper);
    t1.Start(); t2.Start();

    OR

    Task<int> t1 = Task.Factory.StartNew(GetLength);
    Task<string> t2 = Task.Factory.StartNew(ToUpper);

    int Result1 = t1.Result;
    string Result2 = t2.Result;
    Console.WriteLine($"Value of Result1 is: {Result1}");
    Console.WriteLine($"Value of Result2 is: {Result2}");
}
}

```

Note: in the above program the **GetLength1** and **GetLength2** method of the class are value returning. **GetLength1** method concatenates from 1 to 100000 and then returns the length of that string and **GetLength2** method converts a given string to **Upper Case** and returns the converted string. So, in this case to capture the values we need to use the **Task** class which takes the generic parameter **<T>** and in this case **<T>** is of type **integer** for **GetLength1** and of type **string** for **GetLength2** and after execution of the method we can capture the **result** by calling "**Result**" property of **Task** class which returns the result as **integer** for **GetLength1** and **string** for **GetLength2**.

Calling value returning method with parameters by using Tasks: in the above program the methods that we called by using **Task** are value returning methods and now let's learn how to call value returning methods which takes parameters also by using **Task** and to do that add a new class in the project naming it as "**Class4.cs**" and write the below code in it:

```

internal class Class4
{
    static int GetLength(int ub)
    {
        string str = "";
        for (int i = 1; i <= ub; i++)
            str += i;
        return str.Length;
    }
    static string ToUpper(string str)
    {
        return str.ToUpper();
    }
    static void Main()

```

```

{
    Task<int> t1 = new Task<int>(() => GetLength(50000));
    Task<string> t2 = new Task<string>(() => ToUpper("Hello India"));
    t1.Start(); t2.Start();

    OR

    Task<int> t1 = Task.Factory.StartNew(() => GetLength(50000));
    Task<string> t2 = Task.Factory.StartNew(() => ToUpper("Hello India"));

    int Result1 = t1.Result;
    string Result2 = t2.Result;

    Console.WriteLine($"Value of Result1 is: {Result1}");
    Console.WriteLine($"Value of Result2 is: {Result2}");
}
}

```

Note: in the above program GetLength method of class concatenates 1 to a number that is passed to the method as parameter value and then returns the length of that string, so in this case to pass values to the method we need to take the help of a delegate.

Calling non value returning method with parameters by using Tasks: in the above program the methods that we called by using **Task** are value returning and parameterized methods and now let's learn how to call non-value returning and parameterized methods by using **Task** and to do that add a new class in the **project** naming it as **"Class5.cs"** and write the below code in it:

```

internal class Class5
{
    static void GetLength(int ub)
    {
        string str = "";
        for (int i = 1; i <= ub; i++)
            str += i;
        Console.WriteLine(str.Length);
    }
    static void ToUpper(string str)
    {
        Console.WriteLine(str.ToUpper());
    }
    static void Main()
    {
        Task t1 = new Task(() => GetLength(50000));
        Task t2 = new Task(() => ToUpper("Hello India"));
        t1.Start(); t2.Start();
        t1.Wait(); t2.Wait();

        OR

        Task<int> t1 = Task.Factory.StartNew(() => GetLength(50000));
    }
}

```

```

Task<string> t2 = Task.Factory.StartNew(() => ToUpper("Hello India"));
Task.WaitAll(t1, t2);
}
}

```

Thread Synchronization: synchronization is a technique that allows only one thread to access the resource for the time. No other thread can interrupt until the assigned thread finishes its task. In multithreading program, threads are allowed to access any resource for the required execution time. Threads share resources and executes asynchronously. Accessing shared resources (data) is critical task that sometimes may halt the system. We deal with it by making threads synchronized. It is mainly used in case of transactions like deposit, withdraw etc. To test this, add a new class in the project naming it as “Class6.cs” and write the below code in it:

```

class Class6
{
    public static void Print()
    {
        Console.Write("[CSharp Is ");
        Console.WriteLine("Object Oriented");
    }
    static void Main()
    {
        Thread t1 = new Thread(Print);
        Thread t2 = new Thread(Print);
        Thread t3 = new Thread(Print);
        t1.Start(); t2.Start(); t3.Start();
        t1.Join(); t2.Join(); t3.Join();
    }
}

```

When you execute the above code we get un-expected results most of the time because the 3 Threads we have created are interrupting each other. We can use C# **lock** keyword to execute program synchronously. It is used to get lock for the current thread, execute the task and then release the lock. It ensures that other thread does not interrupt the execution until the execution finish. To resolve the problem re-write the **Print()** method code as below:

```

public static void Print()
{
    lock (typeof(Class7))
    {
        Console.Write("[CSharp Is ");
        Console.WriteLine("Object Oriented");
    }
}

```

If we want to perform synchronization with Tasks then here also the process is same and to test this, add a new class in the project naming it as “Class7.cs” and write the below code in it:

```

class Class7

```

```

{
    public static void Print()
    {
        lock (typeof(Class7))
        {
            Console.Write("[CSharp Is ");
            Console.WriteLine("Object Oriented]");
        }
    }
}

static void Main()
{
    Task t1 = new Task(Print);
    Task t2 = new Task(Print);
    Task t3 = new Task(Print);
    t1.Start(); t2.Start(); t3.Start();
    t1.Wait(); t2.Wait(); t3.Wait();

    Or

    Task t1 = Task.Factory.StartNew(Print);
    Task t2 = Task.Factory.StartNew(Print);
    Task t3 = Task.Factory.StartNew(Print);
    Task.WaitAll(t1, t2, t3);
}
}

```

Data Parallelism: this refers to scenarios in which the same operation is performed concurrently (that is, in parallel) on elements in a source like an array or collection. In data parallel operations, the source is partitioned so that multiple threads can operate on different segments concurrently. The **Task Parallel Library (TPL)** supports data parallelism through “Parallel” class which is present under **System.Threading.Tasks** namespace. This class provides method-based parallel implementations of for and foreach loops. You write the loop logic for a “Parallel.For” or “Parallel.ForEach” loops much as you would write a sequential loop. You do not have to create threads or queue the work items i.e., TPL handles all the low-level work for you.

Sequential Foreach Version:

```

foreach (var item in Source_Collection) {
    Process(item);
}

```

Parallel Equivalent:

```

Parallel.ForEach(Source_Collection, item => Process(item));

```

Sequential For Version:

```

for (initializer;condition;iterator) {
    Process(item);
}

```

Parallel Equivalent:

```

Parallel.For(FromStart, ToEnd, item => Process(item));

```

Let's now write a program to understand the difference between sequential for loop and parallel for loop and to do that add a new class in the project naming it as "Class8.cs" and write the below code in it:

```
using System.Diagnostics;
class Class8
{
    static void Main()
    {
        Stopwatch sw1 = new Stopwatch();
        sw1.Start();
        string str1 = "";
        for (int i = 1; i < 200000; i++) {
            str1 = str1 + i;
        }
        sw1.Stop();
        Console.WriteLine("Time taken to execute the code by using sequential for loop: " + sw1.ElapsedMilliseconds);
        Stopwatch sw2 = new Stopwatch();
        sw2.Start();
        string str2 = "";
        Parallel.For(1, 200000, i => {
            str2 = str2 + i;
        });
        sw2.Stop();
        Console.WriteLine("Time taken to execute the code by using parallel for loop: " + sw2.ElapsedMilliseconds);
    }
}
```

Let's now write another program to understand the difference between sequential foreach loop and parallel foreach loop and to do that add a new class in the project naming it as "Class9.cs" and write the below code in it:

```
using System.Diagnostics;
class Class9
{
    static void Main()
    {
        int[] arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
                    31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50 };
        Stopwatch sw1 = new Stopwatch();
        sw1.Start();
        foreach(int i in arr) {
            Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; i value: {i}");
        }
        sw1.Stop();
        Console.WriteLine();
        Stopwatch sw2 = new Stopwatch();
```

```

sw2.Start();
Parallel.ForEach(arr, i => {
    Console.WriteLine($"Thread Id: {Thread.CurrentThread.ManagedThreadId}; i value: {i}");
});
sw2.Stop();

Console.WriteLine("Time taken to execute code by using sequential foreach loop: " + sw1.ElapsedMilliseconds);
Console.WriteLine("Time taken to execute code by using parallel foreach loop: " + sw2.ElapsedMilliseconds);
}
}

```

Note: If you observe the above 2 programs in the first code parallel for loop executed much faster than a sequential for loop whereas in the second case sequential foreach loop executed faster than parallel foreach loop because when we are doing any bulk task inside the loop then parallel loops are faster whereas if you are just iterating and doing a small task inside a loop then sequential loops are faster.

Chaining Tasks using Continuation Tasks: in **asynchronous programming**, it's common for one asynchronous operation, on completion, to invoke a second operation. Continuations allow descendant operations to consume the results of the first operation. A continuation task (also known just as a **continuation**) is an asynchronous task that's invoked by another task, known as the antecedent, when the antecedent finishes. To test this, add a new class in the project naming it as **"Class10.cs"** and write the below code in it:

```

class Class10
{
    static void Method1(int x, int ub) {
        for (int i = 1; i <= ub; i++)
            Console.WriteLine($"{x} * {i} = {x * i}");
    }
    static void Method2(int x, int ub) {
        for (int i = ub; i > 0; i--)
            Console.WriteLine($"{x} * {i} = {x * i}");
    }
    static void Main() {
        Task t = Task.Factory.StartNew(() => Method1(5, 12)).ContinueWith((antecedent) =>
                                                                    Console.WriteLine()).ContinueWith((antecedent) => Method2(5, 12));

        t.Wait();
        Console.ReadLine();
    }
}

```

Asynchronous programming with async and await: **async** and **await** in C# are the code markers, which marks code positions from where the control should resume after a task completes. When we are dealing with UI, and on a button click we called a long-running method like reading a large file or something else which will take a long time and, in that case, the entire application must wait to complete the task. In other words, if a process is blocked in a synchronous application, the whole application gets blocked and stops responding until the whole task completes. To test this, add a new class in the project naming it as **"Class11.cs"** and write the below code in it:

```

class Class11
{
    static void Test1() {
        Console.WriteLine("Started reading values from DB.....");
        Task.Delay(10000).Wait();
        Console.WriteLine("Completed reading values from DB.....");
    }
    static void Test2() {
        Console.Write("Please enter your name: ");
        string? Name = Console.ReadLine();
        Console.WriteLine($"Name you entered is: {Name}");
    }
    static void Main() {
        Test1(); Test2();
    }
}

```

When you run the code you notice **Test1()** method gets delayed for 10 seconds in its execution and all that time **Test2()** method is waiting because the above code is not using **Threads** or **Tasks** so it is a sequential execution. We can still run them asynchronously by using **async** and **await** keywords and to test this re-write the code of **Test1()** method as below:

```

static async void Test1() {
    Console.WriteLine("Started reading values from DB.....");
    await Task.Delay(10000);
    Console.WriteLine("Completed reading values from DB.....");
}

```

Windows Programming

In development of any **application**, we need a user interface (**UI**) to communicate with **End Users** and **User Interfaces** are of 2 types:

1. CUI (Character User Interface)
2. GUI (Graphical User Interface)

Initially we have only **CUI**, e.g.: **Dos**, **Unix OS** etc., where these applications suffer from few criticisms like:

1. They are not user friendly, because we need to learn the commands first to use them.
2. They do not allow navigating from one place to other.

To solve the above problems, in late **80's** GUI applications are introduced by **Microsoft** with its **Windows Operating System**, which has a beautiful feature known as "**Look and Feel**". To develop GUI's Microsoft has provided a language also in early **90's** only i.e., **Visual Basic**, later when **.NET** was introduced the support for **GUI** has been given in all languages of **.NET**.

Developing Graphical User Interfaces: To develop **GUI's** we need some special components known as **Controls** and those **Controls** are readily available in **.NET Language's** as classes under the namespace "**System.Windows.Forms**". All the **Control** classes that are present under this **namespace** were grouped into different **categories** like:

- Common Controls
- Container Controls
- Menus and Tool Bar Controls
- Data Controls
- Components
- Printing Controls
- Dialog Controls
- Reporting Controls

Working with Controls: Whatever the **Control** we want to work with, every **Control** has 3 things in common like: **Properties**, **Methods** and **Events**.

1. **Properties:** these are **attributes** of a **control** which have their **impact** on **look** of the **control**.
E.g.: **Width**, **Height**, **BackColor**, **ForeColor**, etc.
2. **Methods:** these are **actions** performed by a **control**.
E.g.: **Clear()**, **Focus()**, **Close()**, etc.
3. **Events:** these are **time periods** which specify when an **action** has to be performed.
E.g.: **Click**, **Load**, **KeyPress**, **MouseHover**, etc.

Note: the parent class for all the control classes is the class **"Control"**, which is defined with the **Properties**, **Methods** and **Events** that are common for each control like **Button**, **TextBox**, **Form**, **Panel**, etc.

How to develop a GUI Application?

Ans: To develop a **GUI Application** the **base control** that must be created first is **Form**. We call this as **Windows Form** in **Desktop App's**, **Web Form** in **Web App's**, and **Mobile Form** in **Mobile App's**. To create the **Form**, first define a class inheriting from the pre-defined class **"Form"** present under the namespace **"System.Windows.Forms"**, so that the new class also becomes a **Form**.

E.g.: public class Form1 : Form

Step 2: To run the **Form** class, we have defined call the **static method "Run"** of **"Application"** class by passing the instance of **Form** we have created as a parameter to the **method**.

E.g.: Form1 f = new Form1();

Application.Run(f);

Or

Application.Run(new Form1());

Note: we can develop a **Windows Application** by using a **Notepad** following the above process as well as under **Visual Studio** also using **"Windows Forms App."** project template.

Developing Windows Application using Notepad: Open notepad, write the below code in it, **save**, **compile**, and then **execute**.

```
using System.Windows.Forms;
public class Form1 : Form
{
    static void Main()
```

```

{
    Form1 f = new Form1();
    Application.Run(f);
}
}

```

Developing Windows Applications using Visual Studio: To develop a **Windows Application** under **Visual Studio** open the **New Project Window** and there select **Desktop** under “**All project types**” Dropdown List and then in the below project list select “**Windows Forms App.**” project template and specify a name to the project, for example: “**WindowsProject**”.

By default, the project comes with 2 classes in it:

1. **Form1**
2. **Program**

Form1 is the class which is defined inheriting from predefined class Form:

E.g.: `public partial class Form1 : Form`

Here the class Form1 is partial which means it is defined on multiple files, those are:

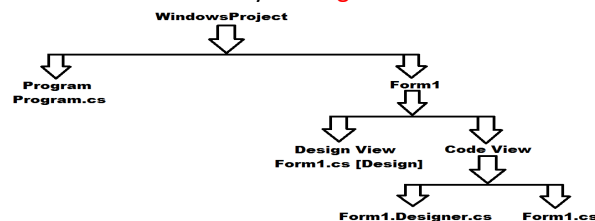
- **Form1.cs**
- **Form1.Designer.cs**

Note: we will not find the **Form1.Designer.cs** file open by default to open it, go to **Solution Explorer** expand the node **Form1.cs** file and under that we find **Form1.Designer.cs** file, double click on it to open.

Windows applications that are developed under Visual Studio have 2 places to work with:

- **Design View**
- **Code View**

Design View is the place where we design the application and this is **accessible** both to **Programmers** and **End User's** also, whereas **Code View** is the place where we write **Code** (i.e., **Designer Code** and **Business Logic**) for the **execution** of application and this is **accessible** only to **Programmers**.



Note: because of the **Design View** what **Visual Studio** provides to us, we call it as **WYSIWYG IDE** (What You See Is What You Get).

The second class in the project is **Program** which is a **static class** and, in this **class**, we find a **Main** method under which object of class **Form1** is created for execution, as following:

```
Application.Run(new Form1());
```

Note: **Program** class is the main **entry point** of the **project** from where the **execution** of our **application** starts.

Adding new Forms in the project: A project can contain any no. of **Forms** in it, to add a new form under our project i.e., “**WindowsProject**”, open **Solution Explorer** => right click on project and select **Add** => “**Windows Form**”, which adds a new Form i.e., **Form2.cs**. To run the new **Form**, go to **Program** class and change the code under **Application.Run** method as **Form2**. For Example:

```
Application.Run(new Form2());
```

Working with Properties of a Control: as we are aware that every control has **properties**, **methods**, and **events**, to access the properties of a control **Visual Studio** provides “**Property Window**” that lists all the **properties** of a control, to open **Property Window** select the **control** and press **F4**. We can change any **property** value in the list of **properties**, under property window like **Width**, **Height**, **BackColor**, **Font**, **ForeColor** etc., for which we can see the **impact** immediately after changing the **property value**. To test this, go to properties of **Form2** we have added right now and change any property values you want.

Whenever we set a **value** to any **property** of a **control** under **property window**, **VS** on behalf of us writes all the necessary code by assigning values to the **properties** we have modified. We can view that code under **InitializeComponent** method of the class which is called in **Constructor** of the class. To view code under **InitializeComponent** method, go to **Code View** and right click on the method called in **constructor** and select “**Go to definition**”, this takes us to **Form2.Designer.cs** file and here also we find the same class **Form2** because it is **partial**.

Working with Events of a Control: An **Event** is a time period which tells when an **action** has to be performed i.e., when exactly we want to execute a **method**. Every **control** will have no. of **events** under it where each **event** occurs on a particular **time period**. We can access the **events** of a control also under **property window** only. To view them in the property window, choose **events Tab** on top of the property window. If we want to write any code that should execute when an **event occurs** double click on the desired **Event** corresponding to a **Control**, which takes you to **Code View** and provides a **Method** for writing the code.

Now in our project add a new Form **Form3.cs**, go to its **Events**, double click on **Load Event** and write the below code under **Form3_Load** method that is generated in **Code View**:

```
MessageBox.Show("Welcome to windows application development.");
```

Again, go to design view, double click on the **Click Event** in property window and write the below code under **Form3_Click** method that is generated in **Code View**:

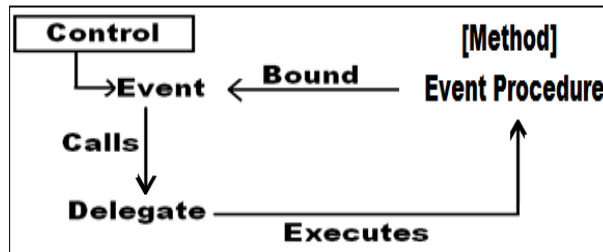
```
MessageBox.Show("You have clicked on the form.");
```

What happens when we double click on an event of a control in the property window?

Ans: When we double click on an **Event** in **Property Window**, internally 2 **actions** get performed:

1. Generates a **method** for **implementing** the **logic** and we call those **methods** as **Event Procedures**, which is a block of code that is bound with an **Event** of **Control** and gets **executed** whenever the **Event occurs**.
2. It will also **generate** a statement in “**InitializeComponent**” method of “**Designer.cs**” file which **binds** the **Controls** - **Event** and **Event Procedure** with each other along with a **Delegate**.

Note: The code written under **Event Procedure** will be executed by the **Event** whenever the **Event** occurs by taking the help of a **Delegate** internally, as following:



In the above case whenever the **Event** occur it will call the **Delegate** which then **executes** the **Event Procedure** that is **bound** with the **Event**. Because **Delegate** is responsible for execution of the **Event**, first the **Control**, **Event**, **Delegate** and **Event Procedure** should be **bound** with each other as following:

Syntax: `<Control Instance>.<Event Name> += new <Delegate Name> (<Event Procedure Name>)`

E.g.:
`this.Load += new EventHandler(Form3_Load);`
`button1.Click += new EventHandler(button1_Click);`
`textBox1.KeyPress += new KeyPressEventHandler(textBox1_KeyPress);`

Events and **Delegates** are pre-defined under the libraries (**Events** are defined in **Control** classes and **Delegates** are defined under some **Namespaces**), what is defined here is only an **Event Procedure**. So, after defining the **Event Procedure** in **Form** class **Visual Studio** links, the **Controls** - **Event**, **Delegate** and **Event Procedure** with each other as we have seen above, and this can be found under the method "**InitializeComponent**" of "**Designer.cs**" file.

Note: 1 **Delegate** can be used by multiple **Events** to execute **Event Procedures**, but all **Events** will not use the same **Delegate**, where different **Events** may use different **Delegates** to execute **Event Procedures**.

Placing controls on a form: By default, we are provided with no. of **Controls** where each **Control** is a **Class**. These **Controls** are available in **Toolbox** window in **LHS** of the **Visual Studio**, which displays all **Controls**, organized under different **Tabs** (groups). To place a **Control** on the **Form** either **double click** on the desired **Control** or select the **Control**, drag it, and place it in the desired location on **Form**.

Note: use **Layout Toolbar** in **Visual Studio** to align **Controls** properly.

How a Form gets created?

Ans: When a **Form** is added to the **Project**, internally following **actions** will takes place:

- i. Defines a **class** inheriting from the pre-defined class "**Form**" so that the new class is also a **Form**.
E.g.: `public partial class Form1 : Form`
- ii. Sets some **initialization properties** like **Name**, **Text** etc., under **InitializeComponent** method.
E.g.: `this.Name = "Form1";`
`this.Text = "Form1";`

How a control get placed on the Form?

Ans: When a **Control** is placed on the **Form**, following **actions** takes place internally:

1. Creates **instance** of appropriate **Control** class.
E.g.: `Button button1 = new Button();`
2. Sets some **initialization properties** that are required like **Name**, **Text**, **Size**, **Location** etc.,

```
E.g.: button1.Location = new Point(12, 12);
      button1.Name = "button1";
      button1.Size = new Size(358, 60);
      button1.TabIndex = 0;
      button1.Text = "button1";
```

- Now the **Control** gets added to **Form** by calling **Controls.Add** method on current **Form**.
E.g.: `this.Controls.Add(button1);`

Note: All the above code will be generated by **Visual Studio** under **InitializeComponent** method of **Designer.cs** file.

The code that is present under a windows application is divided into 2 types:

- Designer Code
- Business Logic

Code which is responsible for **construction** of **Form** is called as **designer code** and code responsible for **execution** of **Form** is called as **business logic**. **Designer code** is generated by **Visual Studio** under **InitializeComponent** method of "**Designer.cs**" file and **business logic** is written by programmers in the form of **Event Procedures** under ".cs" file of a **Form**.

Default Events: as we are aware every control has no. of events to it, but 1 event will be **default** for a Control. To write code under that **default event** of **Control**, directly **double click** on the control which takes to an event procedure associated with that **default event**.

<u>Control</u>	<u>Default Event</u>
Form	Load
Button	Click
TextBox	TextChanged
CheckBox and RadioButton	CheckedChanged
ListView, ListBox, ComboBox and CheckedListBox	SelectedIndexChanged

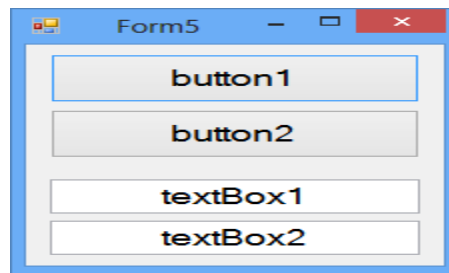
Working with Events and Event Procedures: The concept of **events** and **event procedures** has been derived from the classical **Visual Basic Language**, but there an **event procedure** can be bound with only **single event** of a **single control**, whereas in .NET we can bind an **event procedure** with **multiple events** of a **single control** as well as with **multiple controls** also.

Binding an Event Procedure with multiple Events of a Control: Add a new form to the project **Form4** and double click on it which defines an event procedure **Form4_Load**, now bind the same event procedure with click event of form also, to do this go to events of form, select click event and click on the drop down beside, which displays the list of event procedures available, select "**Form4_Load**" event procedure that is defined previously, which binds the event procedure with click event also, now under the event procedure write the following code and execute:

```
MessageBox.Show("Event Procedure bound with multiple Events of a Control.");
```

Binding an Event Procedure with multiple Controls: Add a new form in the project i.e., **Form5** and design it as below. Now double click on **button1** which generates a click event procedure for **button1**, bind that event procedure with **button2**, **textBox1**, **textBox2** and **Form5** also and write the below code under **event procedure**:

```
MessageBox.Show("Control is clicked by the user.");
```

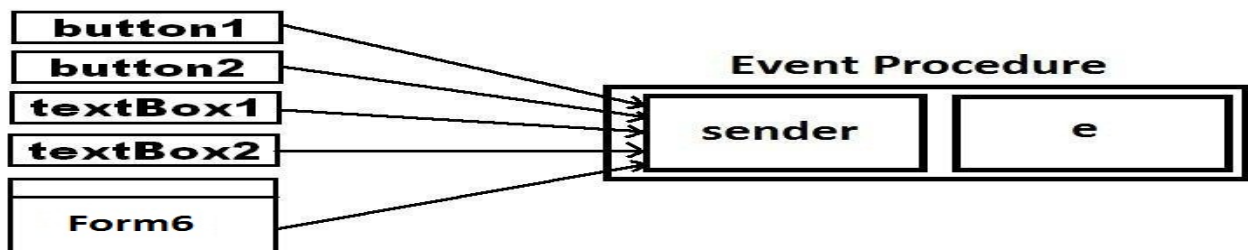


Binding an Event Procedure with multiple Controls & identifying the “Type” of Control which is raising the Event:

Add a new form in the project i.e., **Form6** and design it same as **Form5**. Now double click on **button1** which generates a click event procedure for **button1**, bind that event procedure with **button2**, **textBox1**, **textBox2** and **Form6** also and write the below code under **event procedure**:

```
if (sender is Button) Or if (sender.GetType().Name == "Button")
    MessageBox.Show("Button is clicked by the user.");
else if (sender is TextBox) Or else if (sender.GetType().Name == "TextBox")
    MessageBox.Show("TextBox is clicked by the user.");
else
    MessageBox.Show("Form6 is clicked by the user.");
```

When an **event procedure** is bound with **multiple controls**, any of the **control** can raise the event in **runtime** and **execute** the **event procedure**, but the **instance** of **control** which is raising the **event** will be coming to the **event procedure** and captured under the parameter “**sender**” of **event procedure** as following:



As **sender** is of type **object** it's capable of storing **instance** of any class in it, so after the **instance** of the control class is captured under **sender** by calling **GetType()** method on it we can identify the **type** of control to which that **instance** belongs as we have performed above.

Binding an Event Procedure with multiple Controls and identifying the “exact control” which is raising the Event:

Add a new **Form** in the project i.e., **Form7** and design it same as **Form5**. Now double click on **button1** which generates a click event procedure for **button1**, bind that event procedure with **button2**, **textBox1**, **textBox2** and **Form7** also and write the below code under **event procedure**:

```
if (sender is Button)
{
    Button b = (Button)sender;
    if (b.Name == "button1")
```

```

        MessageBox.Show("Button1 is clicked by the user.");
    else
        MessageBox.Show("Button2 is clicked by the user.");
    }
    else if (sender is TextBox)
    {
        TextBox tb = sender as TextBox;
        if (tb.Name == "textBox1")
            MessageBox.Show("TextBox1 is clicked by the user.");
        else
            MessageBox.Show("TextBox2 is clicked by the user.");
    }
    else
    {
        MessageBox.Show("Form7 is clicked by the user.");
    }
}

```

When an **event procedure** is **bound** with **multiple controls**, and if we want to **identify** the **exact** control which is **raising** the **event**, we need to identify the **"Name (Instance Name)"** of control. But even if **"sender"** represents the **control** which is raising the **event**, using **sender** we cannot find the **control name** because we are already aware that **instance** of a **class** can be stored in its **parent's variable** and make it as a **reference** but with that reference, we cannot access pure child class's members (**Rule No. 3 of Inheritance**). So, if we want to find the name of control that is **raising** the **event**, we need to convert **sender** back into the appropriate control type (**Button** or **TextBox**) from which it is created by performing an **explicit conversion** and then find out the **name** of control instance.

We can convert sender into control type in any of the below 2 ways:

Button b = sender as Button;	or	Button b = (Button)sender;
TextBox tb = sender as TextBox;	or	TextBox tb = (TextBox)sender;

RadioButton and CheckBox Controls: We use these controls when we want the users to select from a list of values provided. RadioButton control is used when we want to allow only a single value to select from the set of values whereas CheckBox control is used when we want to allow multiple selections.

Note: as RadioButton control allows only single selection when we want to use them under multiple options or questions, we need to group related Radio Button's, so that 1 can be selected from each group, to group them we need to place Radio Buttons on separate container controls like Panel, Group Box, Tab Control etc.

Both these 2 controls provide a common boolean property **Checked** which returns true if the control is selected or else returns false, using which we can identify which option has been chosen by the users. Now add a new form in the project and write the below code by designing the form as following:

Code under “Show Selected Radio Button” button’s click event:

```
if (radioButton1.Checked)
    MessageBox.Show("RadioButton1 is selected.");
else if(radioButton2.Checked)
    MessageBox.Show("RadioButton2 is selected.");
else if(radioButton3.Checked)
    MessageBox.Show("RadioButton3 is selected.");
```

Code under “Show Selected Check Box’s” button’s click event:

```
if(checkBox1.Checked)
    MessageBox.Show("CheckBox1 is selected.");
if (checkBox2.Checked)
    MessageBox.Show("CheckBox2 is selected.");
if (checkBox3.Checked)
    MessageBox.Show("CheckBox3 is selected.");
```

CheckedChanged Event: this is the default event of both the above 2 controls (CheckBox and Radio Button) which occur when the controls are selected as well as de-selected also.

To work with CheckedException design a form as below:

- Change the name of every control on the Form for example “Name” TextBox as “txtName”, “Total Fees” TextBox as “txtFees”, “Reset Form” Button as “btnReset” and “Close Form” button as “btnClose”, “Courses” GroupBox as “gbCourses”, “Track” GroupBox as “gbTrack”, “C (750)” CheckBox as “cbC”, “Normal” RadioButton as “rbNormal” and so on.
- Change the Readonly property of “Total Fees” TextBox as true so that it becomes non-editable, enter “0” as default value in Text property and set the TextAlign property as Right.

- Set the **Tag** property for each **CheckBox** and **RadioButton** with their corresponding **fees values** and it should be "0" for **Normal RadioButton**. **Tag** property is used for associating **user-defined data** to any control just like **Text** property, but **Text** value is **visible** to end user and **Tag** value is **not visible** to end user.
- Double click on any 1 **CheckBox** so that **CheckedChanged - Event Procedure** gets generated; bind that **Event Procedure** with all the remaining **CheckBox's**.
- Double click on any 1 **RadioButton** so that **CheckedChanged - Event Procedure** gets generated; bind that **Event Procedure** with all the remaining **RadioButton's**.
- Now go to **Code View** and write the below code.

Class/Global Declarations:

```
int Count = 0;    //Field
```

Code under CheckedChanged Event Procedure of all CheckBox's:

```
rbNormal.Checked = true;
CheckBox cb = sender as CheckBox;
int Amt = int.Parse(txtFees.Text);
if (cb.Checked) {
    Count += 1;
    Amt += Convert.ToInt32(cb.Tag);
}
else {
    Count -= 1;
    Amt -= Convert.ToInt32(cb.Tag);
}
txtFees.Text = Amt.ToString();
```

Code under CheckedChanged Event Procedure of all RadioButton's:

```
RadioButton rb = sender as RadioButton;
int Amt = int.Parse(txtFees.Text);
if (rb.Checked)
    Amt += (Convert.ToInt32(rb.Tag) * Count);
else
    Amt -= (Convert.ToInt32(rb.Tag) * Count);
txtFees.Text = Amt.ToString();
```

Code under Click Event Procedure of Reset Form Button:

```
foreach (Control ctrl in gbCourses.Controls) {
    CheckBox cb = ctrl as CheckBox;
    cb.Checked = false;
}
foreach (Control ctrl in this.Controls)
{
    if (ctrl is TextBox)
    {
        TextBox tb = ctrl as TextBox;
        tb.Clear();
    }
}
```

```

}
}
txtFees.Text = "0";
txtName.Focus();

```

Code under Click Event Procedure of Close Form Button:

```
this.Close();
```

Button, Label and TextBox Controls:

1. **Button:** used for taking acceptance from a user to perform an action.
2. **Label:** used for displaying static text on the UI.
3. **TextBox:** used for taking text input from the user and this control can be used in 3 different ways:
 - I. Single-Line Text (d)
 - II. Multi-Line Text (Text Area)
 - III. Password Field

The default behavior of the control is **Single Line Text**; to make it multiline set the property **Multiline** of the control as **true**. By default, the text area will not have any scroll bars to navigate up and down or left and right, to get them set the **Scrollbars** property either as Vertical or Horizontal or Both, default is none.

Note: by default, the **Word-wrap** property of the control is set as true disabling **horizontal scroll bar** so set it as **false** to get horizontal scroll bar.

To use the control as a **Password Field** either set the **PasswordChar** property of control with a character we want to use as **Password Character** like * or # or \$ or @ etc., or else set the **UseSystemPasswordChar** property as **true** which indicates the text in the control should appear as the default **password character**.

MaskedTextBox: This control looks same as a **TextBox** but can be used for taking the input in **specific formats** from the users. To set a format for input, select the **Mask** property in **property window** and click on the **button** beside it, which opens a new window (**Input Mask**), in it we can either choose a **mask** from list of available **masks** or select **custom** and specify our own mask format using zeros in the **mask textbox** below as following:

Indian Pincode: 000000

Railway PNR: 000-0000000

Credit Card No: 0000-0000-0000-0000

Aadhar No: 0000 0000 0000

Date & Time: 00/00/0000 00:00:00

Note: Even if we set the **Mask** as date, it will not **validate** for a **valid date** and if we want to do that, we need to explicitly write code for the controls **TypeValidationCompleted** event.

To work with all the above controls, add a new form in the project and design it as following:

Setting Tab Order of Form Controls: While working with a windows application we navigate between controls using the Tab key of keyboard. By default, Tab key navigates the focus between controls in the same order how they are placed on form. If we want to set the sequence of tab on our own, it can be done by setting the “Tab Order”. To do this go to View Menu and click on “Tab Order” Menu Item which shows the current tab order, now click on each control in a sequence how we want to move the tab, again go to view menu, and click on “Tab Order” Menu Item.

In the above form check the following business rules (Validations):

1. Check the username, password and confirm password fields to be mandatory.
2. Check password characters to be ranging between 8 and 16 characters.
3. Check confirm password to be matching with password.
4. Disable the password Textbox's once the above 2 rules are satisfied and enable under Clear button.
5. Check DOB to accept a valid date in “dd/MM/yyyy” format and check user has attained 18 years of age at the time of registration and its a mandatory field.
6. Check Mobile TextBox accepts numeric and back spaces only and Mobile No. should start with digits 6 or 7 or 8 or 9 and should be minimum 10 and maximum 10 digits, but not a mandatory field.
7. Check Email TextBox accepts valid Email Id's only, but not a mandatory field.
8. Allow users to close the form if required at any time without any restrictions.

To perform the first 4 validations first set the **MaxLength** property of both **Password Textbox's** to **16** so that they will accept only **16** characters, then define a **Validating Event Procedure** for **User Name - Textbox** and bind that **Event Procedure** with both the **Password Textbox's** also and write the below code under the **Event Procedure**:

```
TextBox tb = sender as TextBox;
//This validation applies to User Name and Password Textbox's
if (tb.Text.Trim().Length == 0)
{
    MessageBox.Show("You can't leave the field empty.", "Warning", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);

    e.Cancel = true;
    return;
}
//This validation applies only to Password Textbox's
if (tb.Name != "txtName") {
    if (tb.Text.Trim().Length < 8) {
        MessageBox.Show("Password should between 8 to 16 chars.", "Warning", MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        e.Cancel = true;
        return;
    }
}

//This validation applies only to Confirm Password Textbox
if (tb.Name == "txtCPwd") {
    if (txtPwd.Text != txtCPwd.Text) {
        DialogResult dr = MessageBox.Show("Confirm Password should match with Password.\n\nDo you remember the password?", "Warning", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
```

```

if (dr == DialogResult.Yes) {
    txtCPwd.Clear();
    txtCPwd.Focus();
}
else {
    txtPwd.Clear();
    txtCPwd.Clear();
    txtPwd.Focus();
}
return;
}
txtPwd.Enabled = txtCPwd.Enabled = false;
}

```

- **Validating** event occurs when the focus is leaving the control and validates the content entered in the control.
- Some **Events** are associated with **properties** with them e.g.: **Validating**, **KeyPress** etc., if we want to consume the properties of an **Event** under its **Event Procedure**, we can make use of the **Parameter** - "e" of the **Event Procedure** which refers to **properties** of current executing **Event**.
- In the above code "**Cancel**" is a property of **Validating** - **Event**, which when set as **true** restricts the focus not to **leave** the control.

For performing 5th **Validation**, generate **TypeValidationCompleted** Event Handler for **Date - MaskedTextBox** and write the below code in it by importing "**System.Globalization**" namespace:

```

if(mtbDOB.Text.Replace("/", "").Trim().Length > 0) {
    bool Status =
        DateTime.TryParseExact(mtbDOB.Text, "dd/MM/yyyy", null, DateTimeStyles.None, out DateTime dt);
    if (Status) {
        if(dt > DateTime.Now.AddYears(-18)) {
            MessageBox.Show("Minimum 18 years of age is required for registration.", "Date Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);

            e.Cancel = true;
        }
    }
    else {
        MessageBox.Show("Date entered must be in a valid date format like dd/MM/yyyy.", "Date Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        e.Cancel = true
    }
}
else {
    MessageBox.Show("Date of birth field is mandatory.", "Warning", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);

    e.Cancel = true;
}
}

```

To perform 6th validation, i.e., if we want the **Phone No - TextBox** to accept only **numeric values** and **back spaces** define a **KeyPress - Event Procedure** for **Phone No - TextBox** and write below code in it:

```
//Un-comment the below statement to find ascii values of characters.
//MessageBox.Show(Convert.ToInt32(e.KeyChar).ToString());
if(char.IsDigit(e.KeyChar) == false && Convert.ToInt32(e.KeyChar) != 8 ) {
    MessageBox.Show("Please enter numeric values only", "Numeric Error", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    e.Handled = true;
}
```

- **KeyPress Event** occurs when we **press** and **release** a **key** while the focus is in the **Control**.
- **KeyChar** property of **KeyPress - Event** gets the **key value** corresponding to the **key pressed**.
- **Handled** property when set as **true** will restrict the **key value** to enter into the **Control**.
- **Char.IsDigit(char)** will return **true** if the given **char** is a **numeric** or else returns **false**.
- **Convert.ToInt32(char)** will return **ascii value** of the given **character**.

If we want **Mobile No - TextBox** to start with digits **6, 7, 8** and **9**, and should have **minimum** and **maximum** of **10** digits, generate a **Validating - Event Procedure** for **Mobile No - TextBox** and write the below code in it by importing **"System.Text.RegularExpressions"** namespace:

```
if (txtMobile.Text.Trim().Length > 0) {
    Regex mobileValidation = new Regex(@"^[6-9]\d{9}$");
    if (!mobileValidation.IsMatch(txtMobile.Text)) {
        MessageBox.Show("Entered number is not a valid mobile number.", "Phone Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        e.Cancel = true;
    }
}
```

To perform 7th validation i.e., if we want **Email - TextBox** to accept a valid **Email Id** generate a **Validating - Event Procedure** to **Email Id - TextBox** and write the below code in it:

```
if (txtEmail.Text.Trim().Length > 0) {
    Regex emailValidation = new Regex(@"^\w+[\w-\.]*\@\w+((-\w+)|(\w*))\.[a-z]{2,3}$");
    if (!emailValidation.IsMatch(txtEmail.Text)) {
        MessageBox.Show("Entered string is not in a valid email format.", "Email Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
        e.Cancel = true;
    }
}
```

To perform 8th validation i.e., closing the **Form** even from **mandatory fields**, go to properties of **Close - Button** and set its **"CausesValidation"** property as **false** so that code under that **Button - Click Event** gets executed before the execution of any other **Controls - Validating Event**, so now write the below code under **Close Button - Click Event Procedure**:

```
if (MessageBox.Show("Are you sure of closing the form?", "Confirmation", MessageBoxButtons.YesNo,
    MessageBoxIcon.Question) == DialogResult.Yes)
```

```

{
    foreach (Control ctrl in this.Controls) {
        if (ctrl is TextBoxBase) {
            ctrl.CausesValidation = false;
        }
    }
    this.Close();
}

```

Note: When we set “CausesValidation” property value as false for a TextBox it will restrict Validating Event of that Control not to occur, so that focus comes out of the Textbox and then Form gets closed.

Code under Save Button Click Event Procedure:

```

MessageBox.Show("Your registration is successfully completed.", "Confirmation", MessageBoxButtons.OK,
    MessageBoxIcon.Information);

```

Code under Clear Button Click Event Procedure:

```

foreach(Control ctrl in this.Controls) {
    if(ctrl is TextBoxBase) {
        TextBoxBase tb = ctrl as TextBoxBase;
        tb.Clear();
    }
}
txtPwd.Enabled = txtCPwd.Enabled = true;
txtName.Focus();

```

RegularExpressions: also known as Regex are some special characters using which we can perform data-validations without writing complex logic.

B => Braces => [] {} ()

C => Carrot => ^

D => Dollar => \$

Braces:

[] => these are used to specify the characters which are allowed.

E.g: [a-m] or [A-K] or [0-6] or [A-Za-z0-9]

{ } => these are used to specify the no. of characters that are allowed.

E.g: {3} or {4, 7} or {4,}

() => these are used to specify a list of options which are accepted.

E.g: (com|net|in|edu)

[A-K] => accepts 1 alphabet between A to K.

[a-m]{5} => accepts 5 lower-case alphabets between a to m.

[a-z]{3}[0-9]{5} => accepts 3 lower-case alphabets in the first followed by 5 numeric.

Note: in all the above 3 cases after the validating expression, it will accept anything we enter over there and to overcome this problem we need to make the expressions, rigid as following:

^[a-z]{3}[0-9]{5}\$ => same as the last expression above but this is rigid expression i.e., accepts only 8 chars.

Special characters in regular expressions: there are some special characters in Regex with pre-defined logic.

\s => accepts whitespace
\S => doesn't accept whitespace
\d => accepts only numeric values.
\D => accepts only non-numeric values.
\w => accepts only alpha-numeric values.
\W => accepts only non-alphanumeric values.

Validation Expressions using Regular Expressions:

\d{6}	=> Indian Postal Code
\d{3}-\d{7}	=> Indian Railway PNR
\d{4} \d{4} \d{4}	=> Aadhar Number
\d{4} \d{4} \d{4} \d{4}	=> Credit Card Number
http(s)?://([\\w-]+\\.)+([\\w-]+(/[\\w- ./?%&=]*)?)	=> Website URL Validation
\\w+([\\w-\\.]*\\@\\w+((-[\\w+]) (\\w*))\\.([a-z]){2,3})	=> Email Validation
[6-9]\\d{9}	=> Mobile No. validation that checks No. starts with 6 or 7 or 8 or 9 and will be maximum 10 and minimum 10 digits.
[0][6-9]\\d{9}	=> Mobile No. validation that checks No. starts with 0 and after that 6 or 7 or 8 or 9 and will be maximum 11 and minimum 11 digits.
^[0][6-9]\\d{9}\$ ^6-9\\d{9}\$	=> Mobile No. validation which checks if the No. starts with 0 then it is 11 digit or else it is 10 digit.
^\\d{6,8}\$ ^6-9\\d{9}\$	=> Validation for either 6-8 digits landline no or 10 digit mobile no that starts with 6 or 7 or 8 or 9.

Note: To use **Regular Expressions** or **Regex** in **Desktop Applications** we are provided with a **pre-defined class** known as **"Regex"** under the namespace **"System.Text.RegularExpressions"** and under this class we find a method with name **"IsMatch"** that can be used for **validating** the data against a given **"Regular Expression"**.

MessageBox: This control is used for displaying messages within a windows application by calling its static method **"Show"**, which returns a value of type **DialogResult** (Enum), using it we can find out which button has been clicked on the **MessageBox** like **Ok** or **Yes** or **No** or **Cancel** etc. Show is an overloaded method that is defined with different overloads as following:

- Show(string msg) => DialogResult
- Show(string msg, string caption) => DialogResult
- Show(string msg, string caption, MessageBoxButtons buttons) => DialogResult
- Show(string msg, string caption, MessageBoxButtons buttons, MessageBoxIcon icon) => DialogResult

MessageBoxButtons is an **Enum** which provides a list of options to choose what **buttons** should be displayed on the **MessageBox** for the user to select.

MessageBoxIcon is an **Enum** which provides a list of options to choose what **icon** should be displayed on the **MessageBox** describing about the message like **Error** or **Warning** or **Question** or **Information** etc., icons.

ComboBox, ListBox, and CheckedListBox: These controls are also used for providing users with a list of values to choose from. **ComboBox** allows only single selection, but it is editable which gives a chance to either **select** from the list of values available or **enter** a new value, it's a combination of 2 controls **DropDownList** + **TextBox**. **ListBox** by default allows **single selection** only but can be changed to **multi-selection** by setting the **SelectionMode** property

either to **MultiSimple** [Mouse Click] or **MultiExtended** [Ctrl + Mouse Click]. **CheckedListBox** is same as **ListBox** but displays a **CheckBox** beside every item for selection and by default it allows **multi-selection**.

Adding values to the controls: we can add values to the 3 controls in different ways like:

1. In the properties of the control, we find a property **Items**, select it, and click on the button beside it which opens a window, enter the values we want to add, but each in a new line.

2. By using **Items.Add** method of the control we can add values, but only one at a time.

<List Control>.Items.Add(object value)

3. By using **Items.AddRange** method of the control an array of values can be added at a time.

<List Control>.Items.AddRange(object[] values)

4. By using **DataSource** property of the control a **DataTable** can be bound to it so that all the records of **Table** get bound to the control, but as it can display only a single column, we need to specify the column to be displayed using the **DisplayMember** property.

<List Control>.DataSource = <Data Table>;

<List Control>.DisplayMember = <Column Name>;

Accessing all values from the controls: for accessing all the values from **List Controls** they provide a property known as **Items** which returns an **object[]** of all items.

<List Control>.Items => Object[]

Accessing selected values from the controls: for accessing the selected values from List Controls we need to make use of the following properties:

ComboBox:

Text => string

SelectedItem => object

SelectedIndex => int

ListBox:

SelectedItem => object

SelectedIndex => int

SelectedItems => object[]

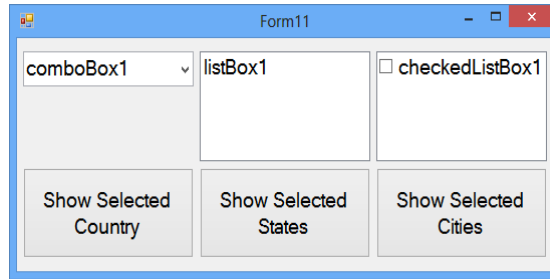
SelectedIndices => int[]

CheckedListBox:

CheckedItems => object[]

CheckedIndices => int[]

Add a new **Form** in the project and design it as below, then go to **Items** property of the **ComboBox** control, click on the **Button** beside it and enter a list of **Countries** in the window opened.



Code under Form Load Event Procedure:

```
listBox1.Items.Add("Kerala");
listBox1.Items.Add("Odisha");
listBox1.Items.Add("Karnataka");
listBox1.Items.Add("Telangana");
listBox1.Items.Add("Tamilnadu");
listBox1.Items.Add("Andhra Pradesh");
string[] Colors = { "Delhi", "Kolkata", "Mumbai", "Chennai", "Bengaluru", "Hyderabad" };
checkedListBox1.Items.AddRange(Colors);
```

Code under ComboBox KeyPress Event Procedure:

```
if (Convert.ToInt32(e.KeyChar) == 13)
{
    if (comboBox1.FindStringExact(comboBox1.Text) == -1)
    {
        comboBox1.Items.Add(comboBox1.Text);
    }
}
```

Adding a new value entered in the **ComboBox** when we hit the enter key and the given value is **not existing** under the list of values.

Code under "Show Selected Country" Button Click Event Procedure:

```
MessageBox.Show("Selected Contry: " + comboBox1.Text);
MessageBox.Show("Selected Contry: " + comboBox1.SelectedItem);
MessageBox.Show("Selected Index: " + comboBox1.SelectedIndex);
```

Code under "Show Selected States" Button Click Event Procedure:

```
foreach(object obj in listBox1.SelectedItems)
{
    MessageBox.Show("Selected State(s): " + obj);
}
```

Code under "Show Selected Cities" Button Click Event Procedure:

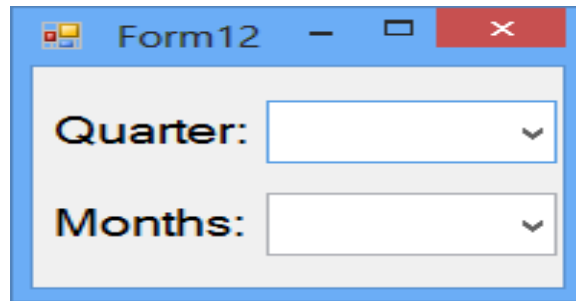
```
string str = "";
foreach(object obj in checkedListBox1.CheckedItems)
{
    str += obj + ", ";
}
str = str.Remove(str.LastIndexOf(", "));
int lastCommaIndex = str.LastIndexOf(", ");
```

```

if (lastCommaIndex != -1)
{
    str = str.Remove(lastCommaIndex, 1);
    str = str.Insert(lastCommaIndex, " and");
}
MessageBox.Show("Selected Cities: " + str);

```

SelectedIndexChanged: it is the default event of all the above 3 controls which gets raised once a value is selected in the list, to test this add a new form in the project, design it as below and add the values “Quarter1, Quarter2, Quarter3 and Quarter4” under the first **ComboBox** by using its items property and write the code.



Code under first ComboBox SelectedIndexChanged Event Procedure:

```

comboBox2.Text = "";
comboBox2.Items.Clear();
switch (comboBox1.SelectedIndex)
{
    case 0:
        comboBox2.Items.AddRange(new string[] { "January", "February", "March" });
        break;
    case 1:
        comboBox2.Items.AddRange(new string[] { "April", "May", "June" });
        break;
    case 2:
        comboBox2.Items.AddRange(new string[] { "July", "August", "September" });
        break;
    case 3:
        comboBox2.Items.AddRange(new string[] { "October", "November", "December" });
        break;
}

```

PictureBox: We use this control for displaying **images** in our application and to load an **Image** into the control we can use any of the following properties:

- ❑ `ImageLocation = <path of the image>`
- ❑ `Image = Image.FromFile(string ImgPath)`
- ❑ `Image = Image.FromStream(Stream stream)`

Use the **BorderStyle** property to control what type of border we want for the **PictureBox**, with any of the following values:

- ? None [d]
- ? FixedSingle
- ? Fixed3D

Use **SizeMode** property of the control to set image placement and control sizing under the **PictureBox** which can be set with any of the following values:

- ? Normal [d]
- ? StretchImage
- ? AutoSize
- ? CenterImage

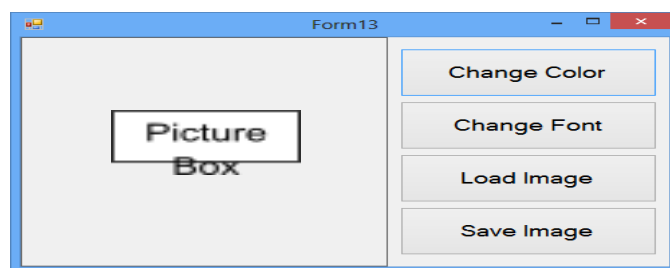
Dialog Controls: These are special controls which provide an interface for displaying a list of values too choose from or for entering of new values, we have 5 dialog controls like **ColorDialog**, **FolderBrowserDialog**, **FontDialog**, **OpenFileDialog** and **SaveFileDialog**.

Dialog controls are not shown directly on the form even after adding them, we can see them at bottom of the studio in design time, to make them visible in runtime we need to explicitly call the method **ShowDialog** on the controls instance, which returns a value of type **DialogResult (Enum)**, using it we can find out which button has been clicked on the **DialogControl** like **Ok** button or **Cancel** button.

Dialog Controls never performs any actions they are only responsible for returning the values to application developers which has been chosen by end users or entered by the end users and then the developers are responsible for capturing those values to perform the necessary actions. To capture the values that are chosen or entered by end users we are provided with following properties:

ColorDialog:	Color
FolderBrowserDialog:	SelectedPath
FontDialog:	Font
OpenFileDialog:	FileName
SaveFileDialog:	FileName

Design a new form as below, add the **ColorDialog**, **FontDialog**, **OpenFileDialog** and **SaveFileDialog** controls to the form and write the below code:



Code under Change Color Button:

```
colorDialog1.Color = button1.BackColor;
DialogResult dr = colorDialog1.ShowDialog();
if (dr == DialogResult.OK)
{
    button1.BackColor = colorDialog1.Color;
}
```

Code under Change Font Button:

```
fontDialog1.Font = button2.Font;
DialogResult dr = fontDialog1.ShowDialog();
if (dr == DialogResult.OK)
{
    button2.Font = fontDialog1.Font;
}
```

Code under Load Image Button:

```
openFileDialog1.FileName = "";
openFileDialog1.Filter = "Jpeg Images|*.jpg|Icon Images|*.ico|Bitmap Images|*.bmp|All Files|*.*";
DialogResult dr = openFileDialog1.ShowDialog();
if (dr == DialogResult.OK)
{
    string filePath = openFileDialog1.FileName;
    pictureBox1.ImageLocation = filePath;
}
```

Code under Save Image Button:

```
saveFileDialog1.FileName = "*.jpg";
saveFileDialog1.Filter = "Jpeg Images|*.jpg|Icon Images|*.ico|Bitmap Images|*.bmp|All Files|*.*";
DialogResult dr = saveFileDialog1.ShowDialog();
if (dr == DialogResult.OK)
{
    string filePath = saveFileDialog1.FileName;
    pictureBox1.Image.Save(filePath);
}
```

Dock Property: It's a property present under all controls except Form, which defines which border of the control is bound to the container. The property can be set with any of the following values:

- ❑ **None:** in this case any of the controls border is not bound to its container.
- ❑ **Left, Right, Top and Bottom:** in this case what option we select from the 4, that border of the control is bound with the same border of the container.
- ❑ **Fill:** in this case all the four borders of the control will be bound with its container, so it occupies all empty space present on the container but leaving the space to existing controls.

Adding Menu's to a Form: To create a **Menu** for our application first we need to place a **MenuStrip** control on **Form** which is present under **Menu's** and **Toolbar's Tab** of **Toolbox**, which comes and sits on top of the **Form** because its **dock property** is set as **Top**.

To add a **Menu** on the **MenuStrip** click on LHS corner of it which shows a textbox asking to **"Type Here"**, enter some Text in it which adds a **Menu**, and repeat the same process for adding of multiple **Menu's**.

To add a **MenuItem** under a **menu**, click on the **Menu** which shows a textbox below asking to “Type Here”, enter some Text in it which adds a **MenuItem**, and repeat the same process for adding of multiple **MenuItem**’s.

Note: both a **Menu** and **MenuItem** when added with internally create an object or instance of the same class i.e., **ToolStripMenuItem**.

If we want Menu’s to be responding for “Alt Keys” of keyboard prefix with “&” before the character that should respond for Alt. E.g.: &File &Edit F&ormat

To define a shortcut for **MenuItem**’s so that they respond to keyboard actions, go to properties of **MenuItem**, select “Shortcut Keys” Property, click on Dropdown beside it, which displays a Window, in that window choose a modifier **Ctrl** or **Alt** or **Shift** and then choose a **Key** from **ComboBox** below.

To group related **MenuItem**’s under a **Menu** we can add separators between **MenuItem**’s, to do it right click on a **MenuItem** and select **Insert => Separator** which adds a **separator** on top of the **MenuItem**.

Note: same as we inserted a separator, we can also insert a **MenuItem** if required, in the middle.

If we want to display any **Image** beside **MenuItem** right click on it and select “Set Image” or “Edit Items” which opens a window, select **Local Resource**, and click on **Import Button** which opens a **DialogBox**, using it select an **Image** from your **Hard disk**.

Sometimes we find check mark beside **MenuItem** to identify a property is on or off, e.g.: **Word Wrap** under **Notepad**. To provide check marks beside a **MenuItem** right click on it and select “Checked”, but to check or uncheck the item in run time we need to write code explicitly under click event of **MenuItem** as following:

```
if (<control>.Checked == true)
{
    <control>.Checked = false;
}
else
{
    <control>.Checked = true;
}
```

ADO.NET

Pretty much every application deal with data in some manner, whether that data comes from memory, databases, XML files, text files, or something else. The location where we store the data can be called as a Data Source or Data Store where a Data Source can be a file, database, address books or indexing server etc.

Programming Languages cannot communicate with Data Sources directly because each Data Source adopts a different Protocol (set of rules) for communication, so to overcome this problem long back Microsoft has introduced intermediate technologies like ODBC and OleDb which works like bridge between the Applications and Data Sources to communicate with each other.

ODBC (Open Database Connectivity) is a standard C programming language middleware API for accessing database management systems (DBMS). ODBC accomplishes DBMS independence by using an ODBC driver as a translation layer between the application and the DBMS. The application uses ODBC functions through an ODBC driver manager with which it is linked, and the driver passes the query to the DBMS. An ODBC driver will be providing a standard set of functions for the application to use and implementing DBMS-specific functionality. An application that can use ODBC is referred to as "ODBC-Compliant". Any ODBC-Compliant application can access any DBMS for which a driver is installed. Drivers exist for all major DBMS's as well as for many other data sources like Microsoft Excel, and even for Text or CSV files. ODBC was originally developed by Microsoft in 1992.

1. It's a collection of drivers, where these drivers sit between the App's and Data Source's to communicate with each other and more over we require a separate driver for every data source.
2. ODBC drivers comes along with your Windows O.S. and we can find them at the following location:
Control Panel => Administrative Tools => ODBC Data Sources
3. To consume these ODBC Drivers first we need to configure them with the data source by creating a "DSN" (Data Source Name).
4. ODBC drivers are open source i.e., there is an availability of these ODBC Drivers for all the leading Operation System's in the market.

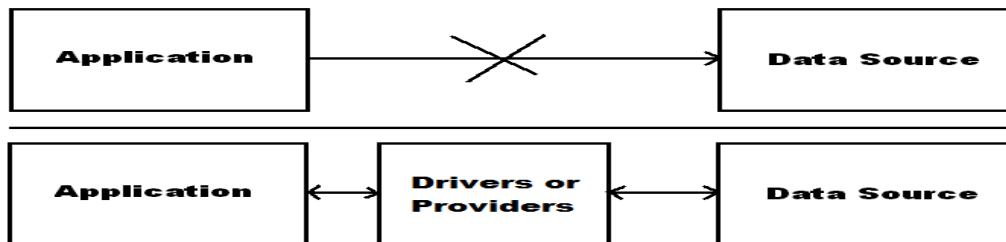
Drawbacks with ODBC Drivers:

1. These drivers must be installed on every machine where the application is executing from and then the application, driver and data source should be manually configured with each other.
2. ODBC Drivers are initially designed for communication with Relational DB only.

OLE DB (Object Linking and Embedding, Database, sometimes written as OLEDB or OLE-DB), an API designed by Microsoft, allows accessing data from a variety of data sources in a uniform manner. The API provides a set of interfaces implemented using the Component Object Model (COM) and SQL. Microsoft originally intended OLE DB as a higher-level replacement for, and successor to, ODBC, extending its feature set to support a wider variety of non-relational databases, such as object databases and spreadsheets that do not necessarily implement SQL. OLE DB is conceptually divided into consumers and providers. The consumers are the applications that need access to the data, and the providers are the software components that implement the interface and thereby provide the data to the consumer. An OLE DB provider is a software component enabling an OLE DB consumer to interact with a data source. OLE DB providers are alike to ODBC drivers. OLE DB providers can be created to access such simple data stores as a text file and spreadsheet, through to such complex databases as Oracle, Microsoft SQL Server, and many others. It can also provide access to hierarchical data stores. These OLE DB Providers are introduced by Microsoft around the year 1996.

1. It's a collection of providers where these providers sit between the Applications and Data Source to communicate with each other, and we require a separate provider for each data source.
2. OleDb Providers are designed for communication with relational & non-relational data source also i.e., it provides support for communication with any Data Source.

3. OleDb Providers sits on server machine so they are already configured with data source and when we connect with any data source they will help in the process of communication.
4. OleDb Providers are developed by using COM and SQL Languages, so they are also un-managed.
5. Microsoft introduced OLEDB as a replacement for ODBC for its Windows Systems.
6. OleDb is a pure Microsoft technology which works only on Windows Platform.

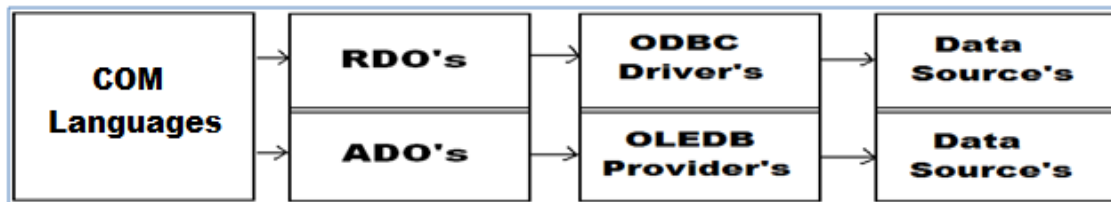


Things to remember while working with Odbc and OleDb:

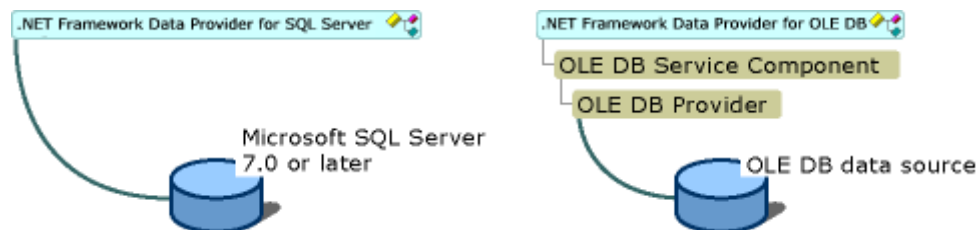
1. ODBC and OleDb are un-managed or platform dependent.
2. ODBC and OleDb are not designed targeting any particular language i.e., they can be consumed by any language like: C, CPP, Visual Basic, Visual CPP, Java, C# etc.

Note: If any language wants to consume **ODBC Drivers** or **OleDb Providers** they must use some built-in libraries of the language in which we are developing the application without writing complex coding.

RDO's and ADO's in COM Language: COM Language used RDO's (**Remote Data Objects**) and ADO's (**ActiveX Data Objects**) for data source communication without having to deal with the comparatively complex ODBC or OLEDB API.



.NET Framework Providers: The .NET Framework Data Provider for SQL Server uses its own protocol to communicate with SQL Server. It is lightweight and performs well because it is optimized to access a SQL Server directly without adding an OLE DB or ODBC layer and it supports SQL Server software version 7.0 or later. The .NET Framework Data Provider for Oracle (Oracle Client) enables data access to Oracle data sources through Oracle client connectivity software. The data provider supports Oracle client software version 8.1.7 or later.



ADO.Net: It is a set of types that expose data access services to the .NET programmer. ADO.NET provides functionality to developers writing managed code like the functionality provided to native COM developers by ADO. ADO.NET provides consistent access to data sources such as Microsoft SQL Server, as well as data sources exposed

through OLE DB and XML. Data-sharing consumer applications can use ADO.NET to connect to these data sources and retrieve, manipulate, and update data. It is an integral part of the .NET Framework, providing access to relational data, XML, and application data. ADO.NET supports a variety of development needs, including the creation of front-end database clients and middle-tier business objects used by applications or Internet browsers.

ADO.Net provides libraries for Data Source communication under the following namespaces:

- System.Data
- System.Data.Odbc
- System.Data.OleDb
- System.Data.SqlClient
- System.Data.OracleClient

Note: System.Data, System.Data.Odbc, System.Data.OleDb and System.Data.SqlClient namespaces are under the assembly System.Data.dll whereas System.Data.OracleClient is under System.Data.OracleClient.dll assembly.

System.Data: types of this namespace are used for holding and managing of data on client machines. This namespace contains following set of classes in it: **DataSet**, **DataTable**, **DataRow**, **DataColumn**, **DataView** and **DataRelation**.

System.Data.Odbc: types of this namespace can communicate with any **Relational Data Source** using **Un-Managed ODBC Drivers**.

System.Data.OleDb: types of this namespace can communicate with any Data Source using **OleDb Providers** (Un-Managed COM Providers).

System.Data.SqlClient: types of this namespace can purely communicate with **SQL Server Database** only using **SqlClient Provider** (Managed .Net Framework Provider).

System.Data.OracleClient: types of this namespace can purely communicate with **Oracle Database** only using **OracleClient Provider** (Managed .Net Framework Provider).

All the above 4 namespaces contain same set of types as following: **Connection**, **Command**, **DataReader**, **DataAdapter**, **Parameter** and **CommandBuilder**, but here each class is referred by prefixing with **Odbc**, **OleDb**, **Sql** and **Oracle** keywords before the class name to **discriminate** between each other as following:

OdbcConnection	OdbcCommand	OdbcDataReader	OdbcDataAdapter	OdbcCommandBuilder	OdbcParameter
OleDbConnection	OleDbCommand	OleDbDataReader	OleDbDataAdapter	OleDbCommandBuilder	OleDbParameter
SqlConnection	SqlCommand	SqlDataReader	SqlDataAdapter	SqlCommandBuilder	SqlParameter
OracleConnection	OracleCommand	OracleDataReader	OracleDataAdapter	OracleCommandBuilder	OracleParameter

Performing Operations on a DataSource: the operations we perform on a **Data Source** will be **Select**, **Insert**, **Update** and **Delete**, and every operation we perform on a **Data Source** involves in **3 steps**, like:

- Establishing a connection with Data Source.

- Sending a request to Data Source by using SQL.
- Capturing the results given by Data Source.

Establishing a Connection with Data Source: It's a process of opening a **channel** for **communication** between **Application** and **Data Source** that is present either on a **local** or **remote** machine to perform **Database** operations and to open the channel for communication we use **Connection** class.

Working with Connection class: To work with any class first we need to know the members of that class like **Constructors**, **Properties**, **Methods**, etc.

Constructors of the Class:

- Connection()
- Connection(string connectionString)

Note: **Connection String** is a collection of attributes that are required for **connecting** with a **Data Source**, those are:

- DSN
- Provider
- Data Source
- User Id and Password
- Integrated Security
- Database or Initial Catalog
- Extended Properties

DSN: this is the only attribute that is required if we want to connect with a data source by using ODBC Drivers and by using this attribute, we need to specify the DSN Name.

Provider: this attribute is required when we want to connect to the data source by using OleDb Providers. So, by using this attribute we need to specify the provider's name based on the data source we want to connect with.

<u>Oracle:</u>	Msdacora or ORAOLEDB.ORACLE
<u>SQL Server:</u>	SqlOledb
<u>MS-Access or MS-Excel:</u>	Microsoft.Jet. Oledb.4.0 (32 Bit OS) Microsoft.Ace.Oledb.12.0 (64 Bit OS)
<u>MS-Indexing Server:</u>	Msidxs

Data Source: this attribute is required to specify the server's name if the Data Source is a Database or else if the Data Source is a File, we need to specify path of the file and this attribute is required in case of any provider communication.

User Id and Password: This attribute is required to specify the credentials for connection with a database and this attribute is required in case of any provider communication.

Integrated Security: this attribute is used while connecting with **SQL Server Database only** to specify that we want to connect with the Server by using Windows Authentication and in this case, we should not use User Id and Password attributes and this attribute is required in case of any provider communication.

Database or Initial Catalog: these attributes are used while connecting with **Sql Server Database only** to specify the name of DB we want to connect with, and this attribute is required in case of any provider communication.

Extended Properties: this attribute is required only while connecting with **MS-Excel** using OleDb Provider.

List of attributes which are required in case of Odbc Drivers, Oledb and Framework Providers

Attribute	ODBC Driver	OLEDB Provider	Framework Provider
DSN	Yes	No	No
Provider	No	Yes	No
Data Source	No	Yes	Yes
User Id and Password	No	Yes	Yes
Integrated Security*	No	Yes	Yes
Database or Initial Catalog*	No	Yes	Yes
Extended Properties**	No	Yes	-

*Only for SQL Server

**Only for Microsoft Excel

Connection String for SQL Server to connect by using different options:

```
OdbcConnection con = new OdbcConnection("Dsn=<Dsn Name>");
```

```
OleDbConnection con = new OleDbConnection("Provider=SqlOleDb;Data Source=<Server Name>;  
Database=<DB Name>;User Id=<User Name>;Password=<Pwd>");
```

```
SqlConnection con = new SqlConnection("Data Source=<Server Name>;Database=<DB Name>;  
User Id=<User Name>;Password=<Pwd>");
```

Note: in case of **Windows Authentication** in place of **User Id** and **Password** attributes we need to use **Integrated Security = SSPI** (Security Support Provider Interface).

Connection String for Oracle to connect by using different options:

```
OdbcConnection con = new OdbcConnection("Dsn=<Dsn Name>");
```

```
OleDbConnection con = new OleDbConnection("Provider=Msdaora (o)r ORAOLEDB.ORACLE;  
Data Source=<Server Name>;User Id=<User Name>;Password=<Pwd>");
```

```
OracleConnection con = new OracleConnection("Data Source=<Server Name>;  
User Id=<User Name>;Password=<Pwd>");
```

Connection String for MS-Excel to connect by using different options:

```
OdbcConnection con = new OdbcConnection("Dsn=<Dsn Name>");
```

```
OleDbConnection con = new OleDbConnection("Provider=Microsoft.Jet.Oledb.4.0;  
Data Source=<Path of Excel Document>;Extended Properties=Excel 8.0");
```

Members of Connection class:

1. **Open():** a method which opens a connection with data source.
2. **Close():** a method which closes the connection that is open.
3. **State:** an enumerated property which is used to get the status of connection.

4. **ConnectionString**: a property which is used to get or set a connection string that is associated with the connection object.

Object of class Connection can be created in any of the following ways:

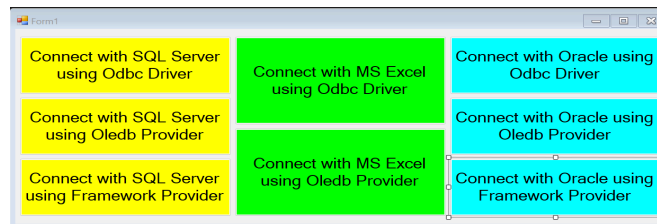
```
Connection con = new Connection();
```

```
con.ConnectionString = "<connection string>";
```

or

```
Connection con = new Connection("<connection string>");
```

Testing the process of establishing a connection: open a new project of type “Windows Forms App.”, name it as “DBExamples” and design the form as following:



Creating DSN for working with ODBC Drivers: To work with ODBC Drivers first we need to configure the drivers installed on our machine with corresponding Databases by creating a DSN (Data Source Name) and to do that go to Control Panel => Administrative Tools or Windows Tools => double click on ODBC Data Sources (64-Bit) to open ODBC Data Source Administrator window, click on Add button, select a driver for SQL Server and click Finish button, which opens a window, in that enter the following details, Name: SqlDSN, Description: Connects with SQL Server Database, Server: <Your Server Name>, click on Next button, choose the Authentication Mode (Windows Or SQL) and provide the credentials in case of SQL Authentication, click on Next button, select the CheckBox “Change the default database to”, and select the Database to which we want to configure with, click on Next button and Click on Finish button which displays a window showing the connection details, click on Ok button which adds the DSN under ODBC Data Source Administrator window.

Again, click on Add button, select a driver for Oracle and click Finish button, which opens a window, in it, enter the following details, Data Source Name: OracleDSN, Description: Connects with Oracle Database, TNS Service Name: <Your Server Name>, User ID: <User Name>/<Password>, click on Ok button which adds the DSN under ODBC Data Source Administrator window.

Again, click on Add button, select a driver for Excel and click Finish button, which opens a window, in it, enter the following details, Data Source Name: ExcelDSN, Description: Connects with Microsoft Excel, click on the “Select Workbook” button and select the Excel file (.xls) from its physical location and click on Ok button which adds the DSN under ODBC Data Source Administrator window.

Note: If you are working with .NET Framework Projects, to consume ADO.NET Types, we need the reference “System.Data.dll” & “System.Data.OracleClient.dll” assemblies to be added to our project, but “System.Data.dll” assembly reference is already added so we need to add reference to “System.Data.OracleClient.dll” assembly only, and to do that open Solution Explorer, right click on References node under project and select “Add Reference” which opens “Reference Manager” dialog box, in that on the LHS under Assemblies option select Framework, now on the RHS select the Checkbox beside “System.Data.OracleClient.dll” assembly, and click Ok. If you are working with .NET Core Projects, we should explicitly install System.Data.Odbc, System.Data.OleDb, System.Data.SqlClient,

and **System.Data.OracleClient** packages, by using **NuGet Package Manager** and to do that go to **Tools Menu**, select the **MenuItem - NuGet Package Manager** and under it choose **Manage NuGet Packages for Solution...** option which opens a new **Window** and in that window go to **Browse** option is **LHS** top and search for all the 4 packages to install, and do that by selecting the **Checkbox** beside the **Project Name** in **RHS** and click on **Install** button.

Now write the below code under Form1.cs:

```
using System.Data.Odbc;
using System.Data.OleDb;
using System.Data.SqlClient;
using System.Data.OracleClient;
```

Code under "Connect with SQL Server using Odbc Driver" Button Click:

```
OdbcConnection con = new OdbcConnection("DSN=SqlDSN");
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with SQL Server using OleDb Provider" Button Click:

```
OleDbConnection con = new OleDbConnection();
//con.ConnectionString = "Provider=SqlOledb;Data Source=Server;Database=Master;User Id=Sa;Password=123";
con.ConnectionString = "Provider=SqlOledb;Data Source=Server;Database=Master;Integrated Security=SSPI";
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with SQL Server using .NET Framework Provider" Button Click:

```
SqlConnection con = new SqlConnection();
//con.ConnectionString = "Data Source=Server;Database=Master;User Id=Sa;Password=123";
con.ConnectionString = "Data Source=Server;Database=Master;Integrated Security=SSPI";
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with MS Excel using Odbc Driver" Button Click:

```
OdbcConnection con = new OdbcConnection("DSN=ExcelDSN");
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with MS Excel using OleDb Provider" Button Click:

```
OleDbConnection con = new OleDbConnection();
con.ConnectionString = "Provider=Microsoft.Ace.Oledb.12.0;Data Source=<Path>;Extended Properties=Excel 8.0";
```

```
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with Oracle using Odbc Driver" Button Click:

```
OdbcConnection con = new OdbcConnection("DSN=OracleDSN");
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with Oracle DB using OleDb Provider" Button Click:

```
OleDbConnection con = new OleDbConnection();
con.ConnectionString = "Provider=OraOledb.Oracle;Data Source=Server;User Id=Scott;Password=tiger";
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Code under "Connect with Oracle using .NET Framework Provider" Button Click:

```
OracleConnection con = new OracleConnection("Data Source=Server;User Id=Scott;Password=tiger");
con.Open();
MessageBox.Show("Connection State: " + con.State);
con.Close();
MessageBox.Show("Connection State: " + con.State);
```

Sending request to Data Source by using SQL to perform CRUD Operations: In this process we send a request to **Data Source** by specifying the type of action (**Select** or **Insert** or **Update** or **Delete**) we want to perform by using an **SQL Statement** like **Select**, **Insert**, **Update**, and **Delete** or by calling a **Stored Procedure** present under the **Data Source**. To send and execute **SQL Statements** or call **Stored Procedure's** in **Data Source** we use **Command** class.

Constructors of the class:

- Command()
- Command(string CommandText, Connection con)

Note: **CommandText** means it can be any **SQL Statement** like **Select** or **Insert** or **Update** or **Delete** or **Stored Procedure** name, whereas **"Connection"** refers to **instance** of **Connection** class we created in **1st** step.

Properties of Command Class:

1. **Connection:** sets or gets the connection object associated with command object.
2. **CommandText:** sets or gets the **SQL** statement or **Stored Procedure** name associated with command object.
3. **CommandType:** sets or gets whether **Command** is configured to execute a **SQL** Statement [default] or call **Stored Procedure**.

The object of class Command can be created in any of the following ways:

```
Command cmd = new Command();  
cmd.Connection = <instance of Connection class>;  
cmd.CommandText = "<SQL Statement or Stored Procedure Name>";
```

Or

```
Command cmd = new Command("<SQL Statement or Stored Procedure Name>", <instance of Connection class>);
```

Methods of Command class:

❏ ExecuteReader	=>	DataReader
❏ ExecuteScalar	=>	object
❏ ExecuteNonQuery	=>	int

Note: after creating instance of **Command** class, we need to call any of these **execute methods** to execute **SQL Statements** and these are the **guidelines** to understand which method to be called when:

- ❏ Use **ExecuteReader** method when we want to execute a **Select Statement** that returns data as **Rows & Columns**. The method returns an object of class **DataReader** which holds data that is retrieved from **Data Source** in the form of **Rows & Columns**.
- ❏ Use **ExecuteScalar** method when we want to execute a **Select Statement** that returns a **single value result**. The method returns result of the **Query** in the form of an **object**.
- ❏ Use **ExecuteNonQuery** method when we want to execute any **SQL Statement** other than **Select**, like **Insert** or **Update** or **Delete** etc. The method returns an **integer** that tells the **no. of rows affected** by the statement.

Note: The above process of calling a **suitable method** to **capture results** is our third step i.e., **capturing the results**.

To try the examples in this document, first create a Database and a set of Tables:

Use Master

Create Database CSDB

Go

Use CSDB

Create Table Dept (Deptno **Int Constraint** Deptno_Pk **Primary Key**, Dname **Varchar**(50), Location **Varchar**(50))

Insert into Dept values(10, 'Marketing', 'Mumbai')

Insert into Dept values(20, 'Sales', 'Chennai')

Insert into Dept values(30, 'Finance', 'Delhi')

Insert into Dept values(40, 'Production', 'Kolkata')

Go

Create table Emp (Empno **Int Constraint** Empno_Pk **Primary Key**, Ename **Varchar**(100), Job **Varchar**(100), Mgr **Int**, HireDate **Date**, Salary **Money**, Comm **Money**, Deptno **Int References** Dept(Deptno))

Insert into Emp Values(1001, 'Scott', 'President', NULL, '01/01/88', 5000, NULL, 10)

Insert into Emp Values(1002, 'Clark', 'Manager', 1001, '01/01/88', 4000, NULL, 10)

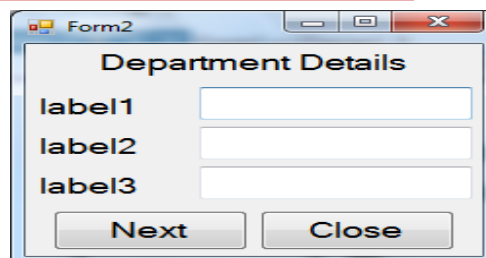
Insert into Emp Values(1003, 'Smith', 'Manager', 1001, '01/01/90', 3500, 500, 20)

```

Insert into Emp Values(1004, 'Vijay', 'Manager', 1001, '01/01/92', 4000, NULL, 30)
Insert into Emp Values(1005, 'Ajay', 'Salesman', 1003, '02/04/89', 3000, 300, 20)
Insert into Emp Values(1006, 'John Smith', 'Salesman', 1003, '02/08/88', 3300, 600, 20)
Insert into Emp Values(1007, 'Venkat', 'Salesman', 1003, '04/15/88', 3300, 0, 20)
Insert into Emp Values(1008, 'Vinod', 'Clerk', 1003, '01/15/88', 2400, NULL, 20)
Insert into Emp Values(1009, 'Suneel', 'Clerk', 1004, '05/12/83', 2000, NULL, 30)
Insert into Emp Values(1010, 'Srinivas', 'Analyst', 1004, '03/01/89', 3400, NULL, 30)
Insert into Emp Values(1011, 'Smyth', 'Analyst', 1004, '03/01/89', 3600, NULL, 30)
Insert into Emp Values(1012, 'Madan', 'Analyst', 1004, '01/09/91', 3100, NULL, 30)
Insert into Emp Values(1013, 'JohnSmith', 'Clerk', 1002, '01/06/88', 1800, NULL, 10)
Insert into Emp Values(1014, 'Raju', 'Clerk', 1005, '06/01/89', 2300, NULL, 20)
Insert into Emp Values(1015, 'Ramesh', 'Clerk', 1011, '08/22/90', 2500, NULL, 30)
Insert into Emp Values(1016, 'Aarush', 'Manager', 1001, '07/15/90', 4200, NULL, 40)
Insert into Emp Values(1017, 'Sridhar', 'Clerk', 1016, '07/20/90', 2500, NULL, 40)
Insert into Emp Values(1018, 'Rahul', 'Supervisor', 1016, '08/01/90', 3500, NULL, 40)
Insert into Emp Values(1019, 'Krishna', 'Fabricator', 1018, '08/12/90', 3100, NULL, 40)
Insert into Emp Values(1020, 'Aaron', 'Fabricator', 1018, '08/21/90', 2900, NULL, 40)
Insert into Emp Values(1021, 'Dave', 'Analyst', 1004, '08/22/90', 3500, NULL, 30)
Insert into Emp Values(1022, 'Kristane', 'Administrator', 1002, '08/22/90', 3000, NULL, 10)
Insert into Emp Values(1023, 'Sophia', 'Administrator', 1003, '08/22/90', 3000, NULL, 20)
Insert into Emp Values(1024, 'Racheal', 'Administrator', 1004, '08/22/90', 3000, NULL, 30)
Insert into Emp Values(1025, 'Elizabeth', 'Administrator', 1016, '08/22/90', 3000, NULL, 40)
Go

```

Add a new Windows Form under the project and design it as following:



```
using System.Data.SqlClient;
```

Declarations:

```

SqlConnection con;
SqlCommand cmd;
SqlDataReader dr;

```

Code under Form Load Event Handler:

```

//Create the object of Connection class providing the required Connection String
con = new SqlConnection("Data Source=Server;Database=CSDB;User Id=Sa;Password=123");
//Create the object of Command class by passing an SQL Statement and Connection instance created above
cmd = new SqlCommand("Select Deptno, Dname, Location From Dept Order By Deptno", con);
//Establish the connection
con.Open();

```

```

//Execute the SQL Statement and capture data in DataReader
dr = cmd.ExecuteReader();
//Accessing and assigning the Column Names to Label Controls
label1.Text = dr.GetName(0) + ": ";
label2.Text = dr.GetName(1) + ": ";
label3.Text = dr.GetName(2) + ": ";
//Calling LoadData method to assign values to TextBox Controls
ShowData();

```

```

private void ShowData() {
    if (dr.Read()) {
        textBox1.Text = dr.GetValue(0).ToString();
        textBox2.Text = dr[1].ToString();
        textBox3.Text = dr["Location"].ToString();
    }
    else {
        MessageBox.Show("You are at the last record of table.", "Information", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
    }
}

```

Code under Next Button Click Event Handler:

```
ShowData();
```

Code under Close Button Click Event Handler:

```

if (con.State != ConnectionState.Closed) {
    con.Close();
}
this.Close();

```

Accessing data from a DataReader: `DataReader` is a class which can hold the data in the form of **rows** and **columns** and to access data from `DataReader` it provides the following members:

1. `GetName(int ColumnIndex)` => **string**

Returns name of the column for given index position.

2. `Read()` => **bool**

Moves record pointer from current location to next row and returns a **boolean** value which tells whether the row to where it moved contains any data or not, which will be **true** if data is **present** or **false** if data is **not present**.

3. `GetValue(int ColumnIndex)` => **object**

4. `Indexer[int ColumnIndex]` => **object**

5. `Indexer[string ColumnName]` => **object**

All the above 3 are used for retrieving column values from the row to which pointer was pointing by specifying the Column Index or Column Name.

6. `FieldCount` => **int**

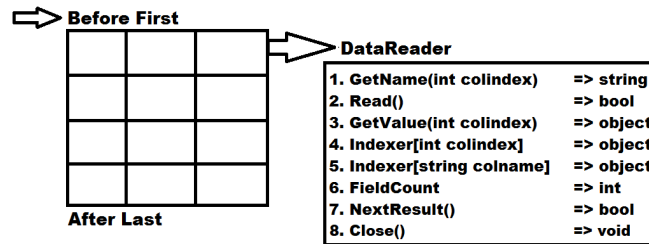
This property returns the **no. of columns** fetched into the `DataReader`

7. `NextResult()` => **bool**

Moves record pointer from **current table** to **next table** and returns a **boolean** value which tells whether the location to which it moved contains a **table** or not, which will be **true** if **present** or **false** if **not present**.

8. **Close()** => **void**

This method closes the DataReader and release the data in it.



Create a new Table in SQL Server to perform CRUD Operations: open **SQL Server Management Studio** and create a Table on **"CSDB" Database** as below:

Create Table Employee(Eno **Int Constraint** Eno_PK **Primary Key**, Ename **Varchar**(50), Job **Varchar**(50), Salary **Money**, Photo **VarBinary**(Max), Status **Bit** Not Null **Default** 1)

Add a new form in our Project i.e., DBExamples, and design it as below:

using System.Data.SqlClient;

Declarations:

SqlConnection con;

SqlCommand cmd;

SqlDataReader dr;

Code under Form Load Event Handler:

con = new SqlConnection("Data Source=Server;Database=CSDB;User Id=Sa;Password=123");

cmd = new SqlCommand();

cmd.Connection = con;

con.Open();

LoadData();

private void LoadData() {

cmd.CommandText = "Select Eno, Ename, Job, Salary, Status From Employee Where Status=1 Order By Eno";

dr = cmd.ExecuteReader();

ShowData();

}

private void ShowData() {

```

if (dr.Read()) {
    textBox1.Text = dr["Eno"].ToString();
    textBox2.Text = dr["Ename"].ToString();
    textBox3.Text = dr["Job"].ToString();
    textBox4.Text = dr["Salary"].ToString();
    checkBox1.Checked = (bool)dr["Status"];
}
else {
    MessageBox.Show("You are at the last record of table.", "Information", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
}
}

```

Code under Next Button Click Event Handler:

```
ShowData();
```

Code under New Button Click Event Handler:

```

foreach (Control ctrl in this.Controls) {
    if (ctrl is TextBox) {
        TextBox tb = ctrl as TextBox;
        tb.Clear();
    }
}
checkBox1.Checked = false;
dr.Close();
cmd.CommandText = "Select IsNull(Max(Eno), 1000) + 1 From Employee";
textBox1.Text = cmd.ExecuteScalar().ToString();
checkBox1.Enabled = btnInsert.Enabled = true;
textBox2.Focus();

```

Code under Insert Button Click Event Handler:

```

cmd.CommandText = $"Insert Into Employee (Eno, Ename, Job, Salary, Status) Values ({textBox1.Text},
    '{textBox2.Text}', '{textBox3.Text}', {textBox4.Text}, {Convert.ToInt32(checkBox1.Checked)})";
if (cmd.ExecuteNonQuery() > 0) {
    MessageBox.Show("Insert operations is successful.", "Success", MessageBoxButtons.OK,
        MessageBoxIcon.Information);

    LoadData();
    checkBox1.Enabled = btnInsert.Enabled = false;
}
else {
    MessageBox.Show("Failed inserting record into the table.", "Failure", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}

```

Code under Update Button Click Event Handler:

```

dr.Close();
cmd.CommandText = $"Update Employee Set Ename='{textBox2.Text}', Job='{textBox3.Text}',
    Salary={textBox4.Text} Where Eno={textBox1.Text}";
if (cmd.ExecuteNonQuery() > 0) {
    MessageBox.Show("Update operations is successful.", "Success", MessageBoxButtons.OK,

```

```

        MessageBoxIcon.Information);
    LoadData();
}
else {
    MessageBox.Show("Failed updating record in the table.", "Failure", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}

```

Code under Delete Button Click Event Handler:

```

if (MessageBox.Show("Are you sure of deleting the current record?", "Confirmation", MessageBoxButtons.YesNo,
    MessageBoxIcon.Question) == DialogResult.Yes)
{
    dr.Close();
    //To delete a record permanently use the below code:
    //cmd.CommandText = $"Delete From Employee Where Eno={textBox1.Text}";

    Or

    //To mark the record as deleted use the below code:
    cmd.CommandText = $"Update Employee Set Status=0 Where Eno={textBox1.Text}";
    if (cmd.ExecuteNonQuery() > 0) {
        MessageBox.Show("Delete operations is successful.", "Success", MessageBoxButtons.OK,
            MessageBoxIcon.Information);

        LoadData();
    }
    else {
        MessageBox.Show("Failed deleting the record from table.", "Failure", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}

```

Code under Close Button Click Event Handler:

```

if (con.State != ConnectionState.Closed) {
    con.Close();
}
this.Close();

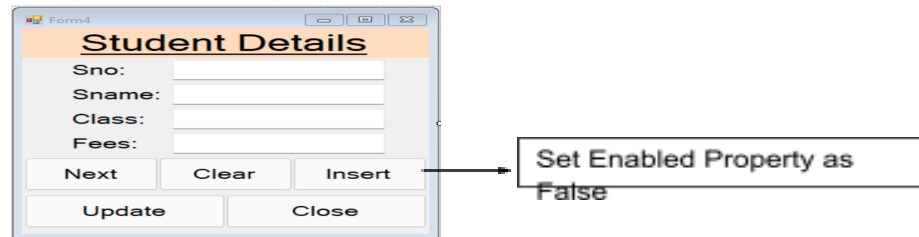
```

Accessing data from Microsoft Excel documents into .NET Application: Microsoft Excel is a file system which stores data in the form of rows and columns same as a Database - Table. An Excel document is referred as Workbook that contains Work Sheets in it, Work Book is considered as Database and Work Sheets are considered as Tables. First row of a Work Sheet will store Column Names.

Creating an Excel Document: Open Microsoft Office Excel and by default the document contains 1 work sheet in it. Now in the first row of the sheet1 enter column names for Student table as Sno, Sname, Class, Fees and from the second row enter few records in it. Now in bottom of the document change the sheet name Sheet1 as Student, click "Save" and under "Save as type" DropDownList choose "Excel 97 – 2003 Workbook", name the document as "School.xls" and save it in your desired location.

Connecting with Excel document from .NET Application: we can connect with an Excel document from .NET application by using Drivers or Providers also. To connect with drivers first we need to configure ODBC driver for

Excel. To configure driver go to **Control Panel => Administrative Tools** or **Windows Tools => Data Sources (ODBC)**, click on it to open **ODBC Data Source Administrator Window**, Click **Add** button, select **Microsoft Excel (*.xls)** driver, click **Finish** button and enter the following details, **Data Source Name: "ExcelDsn"**, **Description: "Connects with Microsoft Excel"**, and click on **Select Workbook** button to choose the **"School.xls"** document from its physical location and click on the **Ok** button which adds the **DSN** under **ODBC Data Source Administrator Window**. Now add a new **Windows Form** in the project and design it as below:



using System.Data.Odbc;

Declarations:

OdbcConnection con;

OdbcCommand cmd;

OdbcDataReader dr;

Code under Form Load Event Handler:

con = new OdbcConnection("DSN=ExcelDSN;ReadOnly=0");

cmd = new OdbcCommand();

cmd.Connection = con;

con.Open();

LoadData();

private void LoadData() {

cmd.CommandText = "Select * From [Student\$]";

dr = cmd.ExecuteReader();

ShowData();

}

private void ShowData() {

if(dr.Read()) {

textBox1.Text = dr["Sid"].ToString();

textBox2.Text = dr["Name"].ToString();

textBox3.Text = dr["Class"].ToString();

textBox4.Text = dr["Fees"].ToString();

}

else {

MessageBox.Show("You are at the last record of table.", "Information", MessageBoxButtons.OK,
MessageBoxIcon.Information);

}

}

Code under Next Button Click Event Handler:

ShowData();

Code under Clear Button Click Event Handler:

textBox1.Text = textBox2.Text = textBox3.Text = textBox4.Text = "";

textBox1.Focus();

```
btnInsert.Enabled = true;
```

Code under Insert Button Click Event Handler:

```
dr.Close();
cmd.CommandText = $"Insert Into [Student$] (Sid, Name, Class, Fees) Values ({textBox1.Text}, '{textBox2.Text}',
                                                                    {textBox3.Text}, {textBox4.Text})";
if (cmd.ExecuteNonQuery() > 0)
{
    MessageBox.Show("Insert operations is successful.", "Success", MessageBoxButtons.OK,
                    MessageBoxIcon.Information);

    LoadData();
    btnInsert.Enabled = false;
}
else {
    MessageBox.Show("Failed inserting record into the table.", "Failure", MessageBoxButtons.OK,
                    MessageBoxIcon.Error);
}
```

Code under Update Button Click Event Handler:

```
dr.Close();
cmd.CommandText = $"Update [Student$] Set Name='{textBox2.Text}', Class={textBox3.Text},
                                                                    Fees={textBox4.Text} Where Sid={textBox1.Text}";
if (cmd.ExecuteNonQuery() > 0)
{
    MessageBox.Show("Update operations is successful.", "Success", MessageBoxButtons.OK,
                    MessageBoxIcon.Information);

    LoadData();
}
else
{
    MessageBox.Show("Failed updating record in the table.", "Failure", MessageBoxButtons.OK,
                    MessageBoxIcon.Error);
}
```

Code under Close Button Click Event Handler:

```
if (con.State != ConnectionState.Closed) {
    con.Close();
}
this.Close();
```

Connecting with Excel using OLEDB Provider: to connect with Excel Documents using OLEDB Provider, Connection String should be as following:

"Provider=Microsoft.Jet.Oledb.4.0;Data Source=<path of the excel file>;Extended Properties=Excel 8.0"

"Provider=Microsoft.Ace.Oledb.12.0;Data Source=<path of the excel file>;Extended Properties=Excel 8.0"

Note: OdbcConnection class opens connection with Excel Document in read-only mode so if we want to perform any manipulations to data in the document, we need to open it in read/write mode by setting the attribute "ReadOnly=0" under the connection string, whereas OleDbConnection will open the document in read/write mode only so no need of using readonly attribute here.

DataReader: it's a class designed for holding the data on client machines in the form of Rows and Columns.

Features of DataReader:

1. Faster access to data from the Data Source because it is "Connection Oriented".
2. Can hold multiple tables in it at a time and to load multiple tables into a DataReader pass multiple Select Statements as "CommandText" to Command separated by a semi-colon.

E.g.: Command cmd = new Command("Select * From Student;Select * From Teacher", con);
DataReader dr = cmd.ExecuteReader();

Note: use NextResult method on DataReader class object to navigate from current table to next table.

dr.NextResult() => bool (return type)

Drawbacks of DataReader:

1. As it is connection oriented requires a continuous connection with Data Source while we are accessing the data, so there are chances of performance degradation if there are more no. of clients accessing data at the same time because that many number of connections should be kept open.
 2. It gives forward only access to the data i.e., allows going either to next record or table but not to previous record or table.
 3. It is a read only object which will not allow any changes to data that is present in it.
-

Dis-Connected Architecture: ADO.NET supports 2 different models for accessing data from Data Sources:

1. Connection Oriented Architecture
2. Disconnected Architecture

In the first case we require a continuous connection with Data Source for accessing data from it and in this case, we use DataReader class for holding data on client machines, where as in the second case we don't require a continuous connection with Data Source for accessing data from it i.e., we require a connection only for loading data from Data Source and in this case, we use DataSet class for holding data on client machines.

DataSet: It's a class present under System.Data namespace designed for holding and managing of the data on client machines apart from DataReader. DataSet class provides the following features:

1. DataSet is also capable of holding multiple tables like a DataReader whereas in case of DataSet those tables can be loaded from different Data Sources.
2. It is designed in disconnected architecture which requires a connection just for loading data but not for holding and accessing data.
3. It provides scrollable navigation to data which allows us to move in any direction i.e., either top to bottom or bottom to top.
4. It is updatable i.e.; changes can be made to data present in DataSet and those changes can be sent back to Database for update.
5. It provides options for searching and sorting of data that is present under it.
6. It provides options for establishing relationships between the tables that are present under it.

Loading Data into DataSet's: The class which is responsible for loading data into DataReader from a Data Source is Command, in the same way DataAdapter class is required for communication between Data Source and DataSet.

DataSource <= Command => DataReader

DataSource <=> DataAdapter <=> DataSet

Note: **DataAdapter** is internally a collection of 4 **Commands** like “**SelectCommand**”, “**InsertCommand**”, “**UpdateCommand**” and “**DeleteCommand**” where each **Command** is an instance of **Command class**, and by using these **Commands**, **DataAdapter** will perform **Select**, **Insert**, **Update** and **Delete** operations on a **Data Source**.

Constructors of DataAdapter class:

```
DataAdapter()  
DataAdapter(Command SelectCmd)  
DataAdapter(string SelectCommandText, Connection con)  
DataAdapter(string SelectCommandText, string ConnectionString)
```

Note: **Select Command Text** means it can be a **Select Stmt** or **Stored Procedure** which contains the **Select Stmt**.

Instance of DataAdapter class can be created in any of the following ways:

```
Connection con = new Connection("<Connection String>");  
Command cmd = new Command("<Select Stmt or SP Name>", con);  
DataAdapter da = new DataAdapter();  
da.SelectCommand = cmd;
```

Or

```
Connection con = new Connection("<Connection String>");  
Command cmd = new Command("<Select Stmt or SP Name>", con);  
DataAdapter da = new DataAdapter(cmd);
```

Or

```
Connection con = new Connection("<Connection String>");  
DataAdapter da = new DataAdapter("<Select Stmt or SPName>", con);
```

Or

```
DataAdapter da = new DataAdapter("<Select Stmt or SPName>", "<Connection String>");
```

Properties of DataAdapter:

1. **SelectCommand**
2. **InsertCommand**
3. **UpdateCommand**
4. **DeleteCommand**

Methods of DataAdapter:

1. **Fill(DataSet ds, string tableName)**
2. **Update(DataSet ds, string tableName)**

Fill method is used for **loading** data from **Data Source** into the **DataSet** and **Update** method is used for **updating** any changes made in the **DataSet** back to **Data Source**:

Fill Method =>

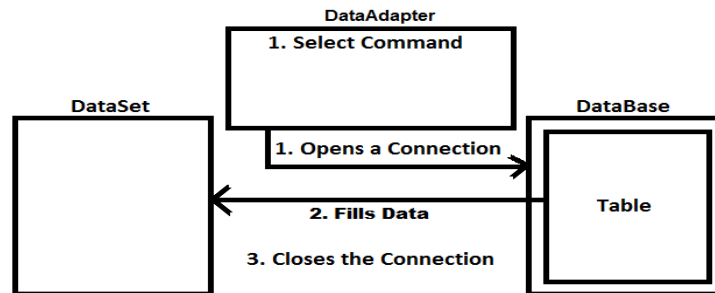
Data Source => DataAdapter => DataSet

Update Method =>

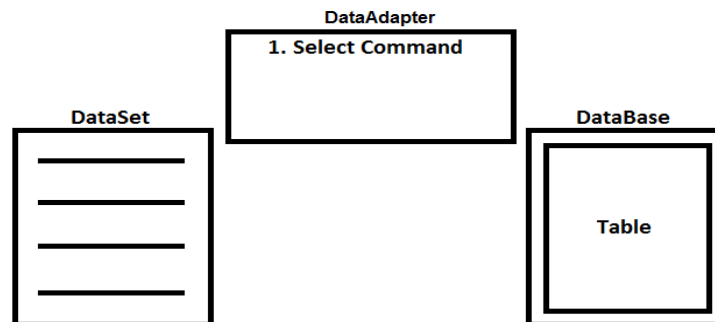
DataSet => DataAdapter => Data Source

When we call Fill method on DataAdapter class, then following actions will take place internally:

- ❑ DataAdapter will open a connection with the Data Source.
- ❑ Executes the Select Command present in it on the Data Source and loads data from table to DataSet.
- ❑ Closes the connection.

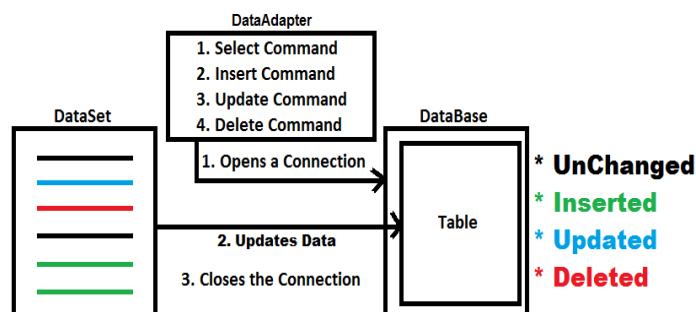


Once the execution of Fill method is completed data gets loaded into the DataSet as below:

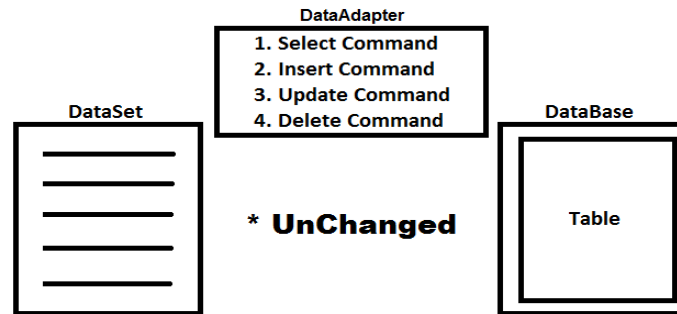


As we are discussing DataSet is **updatable** i.e., we can make **changes** to the **data** that is loaded into it like **adding**, and **modifying** and **deleting** of **records** and after making **changes** to **data** in DataSet if we want to send those changes back to Data Source, we need to call **Update** method on DataAdapter, which performs the following actions:

- ❑ DataAdapter will re-open the connection with Data Source.
- ❑ Changes that are made to data in DataSet will be sent back to corresponding table, where in this process it will make use of Insert, Update and Delete Commands of DataAdapter.
- ❑ Closes the connection.



Once Update method execution is completed data gets re-loaded into DataSet with all unchanged rows:



Accessing data from DataSet: Data Reader's provides pointer-based access to the data, so we can get data only in a sequential order whereas DataSet provides index-based access to the data, so we can get data from any location randomly. DataSet is a collection of tables where each table is represented as a class DataTable and identified by its index position or name. Every DataTable is again collection of Rows and collection of Columns where each row is represented as a class DataRow and identified by its index position and each column is represented as a class DataColumn and identified by its index position or name.

- Accessing a DataTable from DataSet: <dataset>.Tables[index] or <dataset>.Tables[name]
E.g.: ds.Tables[0] or ds.Tables["Employee"]
- Accessing a DataRow from DataTable: <datatable>.Rows[index]
E.g.: ds.Tables[0].Rows[0]
- Accessing a DataColumn from DataTable: <datatable>.Columns[index] or <datatable>.Columns[name]
E.g.: ds.Tables[0].Columns[0] or ds.Tables[0].Columns["Eno"]
- Accessing a Cell from DataTable: <datatable>.Rows[row][col]
E.g.: ds.Tables[0].Rows[0][0] or ds.Tables[0].Rows[0]["Eno"]

To work with DataSet, add a new Form in the project and design it as below:

Form5

Employee Details

Eno:

Ename:

Job:

Salary:

First Prev Next Last

New Insert Update Delete

Save to Database Search Close

Set Read-only Property as True

Set Enabled Property as False

Note: Add reference of "Microsoft.VisualBasic" assembly from Framework Tab of Add Reference window if you are working with .NET Framework Project to use the InputBox control whereas if you are working with .NET Core Project it is by default available.

Now go to Code View and write the below code under the class:

```
using System.Data.SqlClient;
using static Microsoft.VisualBasic.Interaction;
```

Declarations:

```
DataSet ds;  
SqlDataAdapter da;  
int RowIndex = 0;
```

Code under Form Load Event Handler:

```
da = new SqlDataAdapter("Select Eno, Ename, Job, Salary From Employee Order By Eno",  
                        "Data Source=Server;User Id=Sa;Password=123;Database=CSDB");  
da.MissingSchemaAction = MissingSchemaAction.AddWithKey;  
ds = new DataSet();  
da.Fill(ds, "Employee");  
ShowData();
```

private void ShowData()

```
{  
    if (ds.Tables["Employee"].Rows[RowIndex].RowState != DataRowState.Deleted) {  
        textBox1.Text = ds.Tables["Employee"].Rows[RowIndex]["Eno"].ToString();  
        textBox2.Text = ds.Tables["Employee"].Rows[RowIndex]["Ename"].ToString();  
        textBox3.Text = ds.Tables["Employee"].Rows[RowIndex]["Job"].ToString();  
        textBox4.Text = ds.Tables["Employee"].Rows[RowIndex]["Salary"].ToString();  
    }  
    else {  
        MessageBox.Show("Current row is deleted and can't be accessed anymore.", "Information",  
                        MessageBoxButtons.OK, MessageBoxIcon.Information);  
    }  
}
```

Code under First Button Click Event Handler:

```
RowIndex = 0;  
ShowData();
```

Code under Prev Button Click Event Handler:

```
if (RowIndex > 0) {  
    RowIndex -= 1;  
    ShowData();  
}  
else {  
    MessageBox.Show("You are at the first record of table.", "Information", MessageBoxButtons.OK,  
                    MessageBoxIcon.Information);  
}
```

Code under Next Button Click Event Handler:

```
if (RowIndex < ds.Tables[0].Rows.Count - 1) {  
    RowIndex += 1;  
    ShowData();  
}  
else {  
    MessageBox.Show("You are at the last record of table.", "Information", MessageBoxButtons.OK,  
                    MessageBoxIcon.Information);  
}
```

Code under Last Button Click Event Handler:

```
RowIndex = ds.Tables[0].Rows.Count - 1;  
ShowData();
```

Code under New Button Click Event Handler:

```
textBox1.Text = textBox2.Text = textBox3.Text = textBox4.Text = "";  
//Finding the index of last record in DataTable  
int LastRowIndex = ds.Tables["Employee"].Rows.Count - 1;  
  
//Finding the Eno of the last record  
int MaxEno = Convert.ToInt32(ds.Tables["Employee"].Rows[LastRowIndex]["Eno"]);  
  
//Adding 1 to the Eno we found and assigning it to TextBox1  
textBox1.Text = (MaxEno + 1).ToString();  
btnInsert.Enabled = true;  
textBox2.Focus();
```

Steps for adding a DataRow to DataTable of DataSet: To add a DataRow to the DataTable of DataSet adopt the below process:

1. Create a **new row** by calling the **NewRow** method on **DataTable** which creates a new row with the same structure of the **DataTable**.
2. Assign values to the **new row** by treating it as a **single dimensional array**.
3. Call the **Rows.Add** method on **DataTable** and add the row to **DataRowCollection**.

Code under Insert Button Click Event Handler:

```
DataRow dr = ds.Tables["Employee"].NewRow();  
dr["Eno"] = textBox1.Text;  
dr["Ename"] = textBox2.Text;  
dr["Job"] = textBox3.Text;  
dr["Salary"] = textBox4.Text;  
ds.Tables["Employee"].Rows.Add(dr);  
  
//Setting the RowIndex to the index of new row  
RowIndex = ds.Tables["Employee"].Rows.Count - 1;  
btnInsert.Enabled = false;  
MessageBox.Show("DataRow added to DataTable of DataSet.", "Information", MessageBoxButtons.OK,  
                MessageBoxIcon.Information);
```

Updating a DataRow in DataTable of DataSet: To update an existing **DataRow** in **DataTable** of **DataSet**, we need to **re-assign** the modified values back to **DataRow** in **DataTable**, so that the old values gets overridden with new values.

Code under Update Button Click Event Handler:

```
ds.Tables["Employee"].Rows[RowIndex]["Ename"] = textBox2.Text;  
ds.Tables["Employee"].Rows[RowIndex]["Job"] = textBox3.Text;  
ds.Tables["Employee"].Rows[RowIndex]["Salary"] = textBox4.Text;  
MessageBox.Show("DataRow modified in DataTable of DataSet.", "Information", MessageBoxButtons.OK,  
                MessageBoxIcon.Information);
```

Deleting a DataRow from DataTable of DataSet: To delete an existing DataRow in DataTable of DataSet call Delete method pointing to the row that must be deleted on DataTable.

Code under Delete Button Click Event Handler:

```
ds.Tables["Employee"].Rows[RowIndex].Delete();  
btnFirst.PerformClick();    //Raises click event on First button  
MessageBox.Show("DataRow deleted from DataTable of DataSet.", "Information", MessageBoxButtons.OK,  
                MessageBoxIcon.Information);
```

Saving changes made in DataTable of DataSet back to Database: If we want to save changes made in DataTable of DataSet back to Database we need to call Update method on DataAdapter by passing the DataSet which contains modified values as a parameter. If Update method of DataAdapter must work it should contain 3 commands under it i.e., Insert, Update and Delete and these 3 commands must be written by the programmers explicitly or can be generated implicitly with the help of CommandBuilder class. CommandBuilder class constructor if given with DataAdapter that contains a SelectCommand in it will generate the remaining 3 commands based on that Select.

Note: CommandBuilder can generate Update and Delete Commands for a given Select command only when the Database table contains Primary Key Constraint on it and if there is no Primary Key Constraint on the table it will generate only Insert Command.

Code Under Save To Database Button Click Event Handler:

```
SqlCommandBuilder sb = new SqlCommandBuilder(da);  
da.Update(ds, "Employee");  
MessageBox.Show("Data saved to Database Server.", "Success", MessageBoxButtons.OK,  
                MessageBoxIcon.Information);
```

Searching for a DataRow in DataTable of DataSet: To search for a DataRow in DataTable of DataSet call Find method on DataRowCollection which searches for the DataRow on Primary Key Column(s) of table and returns a Row. Use the first method if the primary key constraint is present on a single column or else use the second method if it is a composite primary key.

Find(Object key)	=> DataRow
Find(Object[] keys)	=> DataRow

Note: if the Find method has to work, we need to first load the Primary Key information of Database-Table into DataSet by setting the property value as "AddWithKey" for MissingSchemaAction property of DataAdapter.

Code under Search Button Click Event Handler:

```
string Value = InputBox("Enter Employee No. to Search:");  
if(int.TryParse(Value, out int Eno)) {  
    DataRow dr = ds.Tables[0].Rows.Find(Eno);  
    if (dr != null) {  
        RowIndex = ds.Tables[0].Rows.IndexOf(dr);  
        textBox1.Text = dr["Eno"].ToString();  
        textBox2.Text = dr["Ename"].ToString();  
        textBox3.Text = dr["Job"].ToString();  
        textBox4.Text = dr["Salary"].ToString();  
    }  
    else {
```

```

        MessageBox.Show("There is no Employee existing with given No.", "Warning", MessageBoxButtons.OK,
                        MessageBoxIcon.Warning);
    }
}
else {
    MessageBox.Show("Employee No must be an integer value.", "Error", MessageBoxButtons.OK,
                    MessageBoxIcon.Error);
}

```

Code under Close Button Click Event Handler:

```
this.Close();
```

Configuration Files

While developing applications if there are any values in application which requires changes in future, should not be hard coded i.e., should not be maintained as static values within the application, because if any changes are required for those values in future client will not be able to make changes because they will not have the source code of application for modification. To overcome this problem, we need to identify those values and put them under a special file known as Configuration File, which is an XML file that stores values in it in the form of Key/Value pairs. We store values like Company Name, Address, Phone No, Fax No, Connection Strings etc., in these files. When an application is installed on the client machines along with it the configuration file also will be installed there and because the configuration file is a text file clients can edit those files and make modification to the values under them at any time and those values will be read into the application in runtime.

Note: if you are working with .NET Framework Project's then by default this Configuration File is available under the Project with the name as "App.Config", whereas if you are working with .NET Core Project's we need to add this file explicitly from the Add New Item Window and the item to be added is "Application Configuration File".

Storing values under configuration file: By default, the file comes with a tag <configuration></configuration> and all the values must be present under this tag only in the form of different sections and by default we find Startup section which contains the Runtime and Framework versions to which we are developing the application as below:

```

<configuration>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.8" />
  </startup>
</configuration>

```

Now we can add new sections or blocks like App Settings, Connection Strings, etc. and to test that add App Settings and Connection Strings below Startup and finally the code should look as below:

```

<configuration>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.8" />
  </startup>

  <appSettings>

```

```

<add key="CompanyName" value="Naresh I Technologies"/>
<add key="Address" value="Ameerpet, Hyderabad - 16"/>
<add key="Phone" value="(040) 23746666"/>
<add key="WhatsApp" value="+91 8179191999"/>
<add key="Email" value="info@nareshit.com"/>
<add key="Website" value="www.nareshit.com"/>
</appSettings>

<connectionStrings>
<add name="SqlConStr" connectionString="Data Source=Server;User Id=Sa;Password=123;Database=CSDB"
        providerName="System.Data.SqlClient"/>
<add name="OracleConStr" connectionString="Data Source=Server;User Id=Scott;Password=tiger"
        providerName="System.Data.OracleClient"/>
<add name="ExcelConStr" connectionString="Provider=Microsoft.Ace.Oledb.12.0;Data
        Source=E:\ExcelDocs\School.xls;Extended Properties=Excel 8.0" providerName="System.Data.OleDb"/>
</connectionStrings>
</configuration>

```

Reading configuration file values from applications: to read configuration file values in our applications we are provided with a class `ConfigurationManager` within the namespace `System.Configuration` present under the assembly `System.Configuration.dll` and to consume the class we need to first add reference of the assembly using the "Add Reference" window if you are working with .NET Framework Project's and we find the assemblies under Framework tab whereas if you are working with .NET Core Project's the reference of the assembly is already added. So now import the namespace "`System.Configuration`" and then read the values from config file in to our application as below:

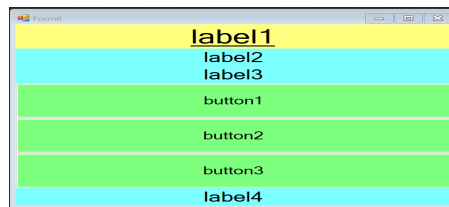
Code for reading App Settings:

```
ConfigurationManager.AppSettings.Get(string key) => string (returns the value for given key)
```

Code for reading Connection Strings:

```
ConfigurationManager.ConnectionStrings[string name].ConnectionString => string (returns the value for given name)
```

To test this, add a new form in the project, design it as below and then write code in code view:



```
using System.Configuration;
```

Code Under Form Load Event Handler:

```
label1.Text = ConfigurationManager.AppSettings.Get("CompanyName");
```

```
label2.Text = ConfigurationManager.AppSettings.Get("Address");
```

```
label3.Text = $"Phone: {ConfigurationManager.AppSettings.Get("Phone")}; What's App:
        {ConfigurationManager.AppSettings.Get("WhatsApp")}";
```

```
label4.Text = $"Email: {ConfigurationManager.AppSettings.Get("Email")}; Website:  
{ConfigurationManager.AppSettings.Get("Website")}";
```

Code under Button1 Click Event Handler:

```
button1.Text = ConfigurationManager.ConnectionStrings["SqlConStr"].ConnectionString;
```

Code under Button2 Click Event Handler:

```
button2.Text = ConfigurationManager.ConnectionStrings["OracleConStr"].ConnectionString;
```

Code under Button3 Click Event Handler:

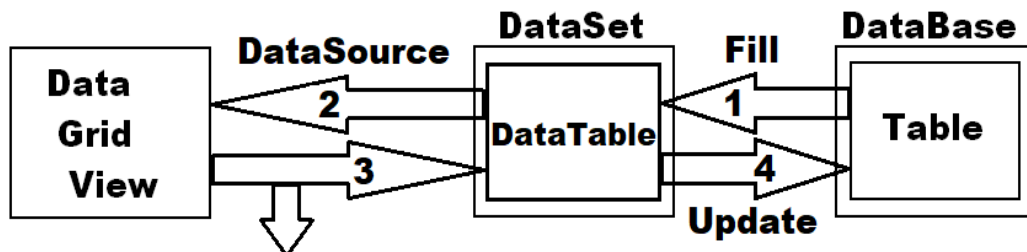
```
button3.Text = ConfigurationManager.ConnectionStrings["ExcelConStr"].ConnectionString;
```

DataGridView

This control is used for displaying the **data** in the form of a table i.e., **rows** and **columns**. To load data in to the control first we need to bind a **DataTable** of **DataSet** to the **DataGridView** control by using its **DataSource** property as following:

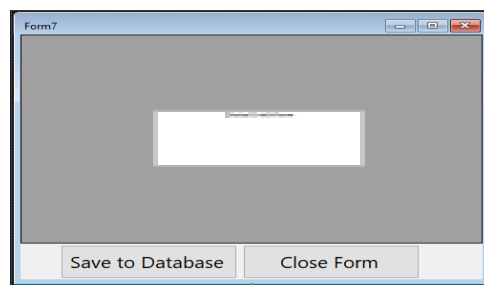
```
dataGridView1.DataSource = <DataTable>
```

DataGridView control has a specialty i.e., changes that are performed to **data** in it gets reflected directly to the **data** of **DataTable** to which it was bound, so that we can **update - DataSet** back to **Database** directly.



Changes made to the data present in **DataGridView** control reflects directly into the **DataTable** that was bound to it with out writing any code.

To test this, add a new form in the project, design it as below and then write code in code view:



```
using System.Configuration;  
using System.Data.SqlClient;
```

Declarations:

```
DataSet ds;  
SqlDataAdapter da;
```

Code under Form Load Event Handler:

```
string ConStr = ConfigurationManager.ConnectionStrings["SqlConStr"].ConnectionString;
da = new SqlDataAdapter("Select Eno, Ename, Job, Salary From Employee Order By Eno", ConStr);
ds = new DataSet();
da.Fill(ds, "Employee");
dataGridView1.DataSource = ds.Tables[0];
```

Code under Save to Database Button click Event Handler:

```
SqlCommandBuilder sb = new SqlCommandBuilder(da);
da.Update(ds, "Employee");
MessageBox.Show("Data saved to Database.", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);
```

Code under Close Form Button click Event Handler:

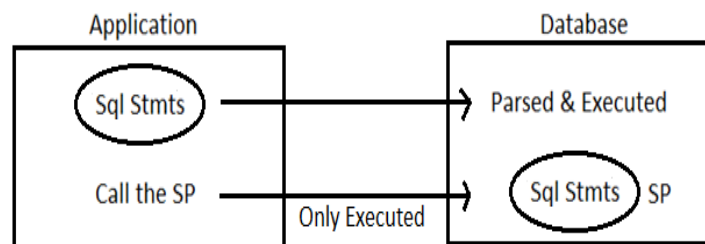
```
this.Close();
```

Major differences between DataSet and DataReader:

- ❑ DataSet is disconnected whereas DataReader is connected while reading data.
- ❑ If we want to cache data and pass to a different tier, DataSet's will be the choice.
- ❑ If we want to move back while reading records, DataReader doesn't support this functionality.
- ❑ DataSet's will internally store the data in XML Format that is the reason why they can hold data loaded from multiple Data Sources also, but not possible in case of DataReader.
- ❑ Using DataReader's increases application performance and reduces system overheads, this is due to one row at a time is stored in memory.
- ❑ DataReader's are well suitable for Web Apps, whereas DataSet's are well suitable for Desktop Apps.

Stored Procedures

Whenever we want to interact with a Database from an application, we use SQL Statements. When we use SQL Statements within the application, we have a problem i.e., when the application runs SQL Statements will be sent to Database for execution and there those SQL Statements will be parsed (compile) and then executed. The process of parsing takes place every time we run the application and because of this performance of our application decreases. To overcome the above problem, write SQL Statements directly under Database only with-in an object known as Stored Procedure and then call them from our application for execution. Because, a Stored Procedure is a pre-compiled block of code which is ready for execution, when called will directly execute the SQL Statements without parsing them every time.



Syntax to define a Stored Procedure:

Create Procedure <Name> [(<Parameter List>)]

As


```
Begin
    <Stmts>;
End;
```

❑ **Stored Procedure** is similar to a **Method** in our language.

```
public void Test()                //Method in C#
Create Procedure Test()           //Stored Procedure in SQL Server
```

❑ If **required** we can also define **parameters** to a **Stored Procedure** but it is only **optional**. If we want to pass parameters to an **SQL Server Stored Procedure**, we need to prefix the character “@” before parameter.

```
public void Test(int x)            //C#
Create Procedure Test(@x int)      //SQL Server
```

❑ A **Stored Procedure** can't return values, so we use **output parameters** to send any **results** out of the **Procedure** and to do that we should **suffix** those parameters with “Out” or “Output” keywords.

```
public void Test(int x, out int y) //C#
Create Procedure Test(@x int, @y int out) //SQL Server
```

Creating a Stored Procedure: We can create a **Stored Procedure** in **SQL Server** either by using **SQL Server Management Studio** or **Visual Studio** also. To create a **Stored Procedure** from **Visual Studio** first we need to **configure** our **Database** under **Server Explorer**, to do this go to **View Menu**, select **Server Explorer** which gets launched on **LHS** of the **Visual Studio**. To **configure** it right click on the node “**Data Connections**”, select “**Add Connection**” which opens a window asking to choose a **Data Source** select “**Microsoft SQL Server (SqlClient)**”, click ok, which opens “**Add Connection**” window and under it provide the following details:

1. **Server Name:** <Enter name of your server>
2. **Authentication:** Windows or SQL Server (provide **Username** and **Password** for **SQL Authentication**)
 3. **Encrypt:** Choose **Optional (False)** under this **DropDown**.
4. Choose “**CADB**” Database under **Select or enter a database name Dropdown**.

Click on the **OK** button which adds the **Database** under **Server Explorer**, expand it, right click on the node **Stored Procedures**, select “**Add New Stored Procedure**” which opens a window and write the below code in it:

```
CREATE PROCEDURE Employee_Select
As
Begin
    Select Eno, Ename, Job, Salary, Photo, Status From Employee Order By Eno;
End;
```

Note: now **right click** on the **document window** and select “**Execute**” which will create the **Stored Procedure** in **Database Server** and we can view that under **Server Explorer** with-in “**Stored Procedures**” node.

Calling a SP from .NET application: To call a **Stored Procedure** from **.NET Application** we use **Command** class and the process of calling will be as following:

1. Create instance of class **Command** by specifying **Stored Procedure Name** as **CommandText**.

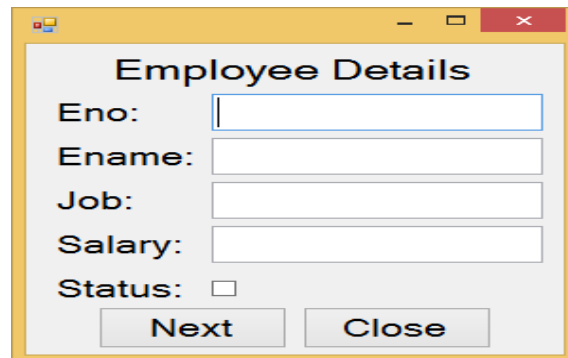
2. Change the **CommandType** property of **Command** as **StoredProcedure** because by default **CommandType** property is set as **Text** which is used for executing **SQL Statements** only and after changing that property value to **StoredProcedure**, **Command** can now call **Stored Procedures**.

cmd.CommandType = CommandType.StoredProcedure;

3. If the **Stored Procedure** has any **Parameters**, we need to add those **Parameters** to the **Command** i.e., if the **parameter** is **input**, we need to add **input parameters** by calling **"AddWithValue"** method and in-case if the **parameter** is **output**, we need to add them by calling **"Add"** method of **Command** class.

4. If the **Stored Procedure** is a **Query Procedure**, then call **ExecuteReader** method of **Command** class which executes the **Stored Procedure** and loads data into **DataReader**, whereas if we want to load data into a **DataSet** then create **DataAdapter** class instance by passing **Command** as a **Constructor Parameter** to it and then call **Fill** method on **DataAdapter** which loads the data into **DataSet**. If the **Stored Procedure** contains any **non-Query Statements**, then call **ExecuteNonQuery** method of **Command** to execute the **Stored Procedure**.

Calling above Stored Procedure using Command class and loading data into DataReader: Add a new **Form** in the project, design it as below and then write code in Code View:



```
using System.Configuration;
using System.Data.SqlClient;
```

Declarations:

```
SqlConnection con;
SqlCommand cmd;
SqlDataReader dr;
```

Code under Form Load Event Handler:

```
string ConStr = ConfigurationManager.ConnectionStrings["SqlConStr"].ConnectionString;
con = new SqlConnection(ConStr);
cmd = new SqlCommand("Employee_Select", con);
cmd.CommandType = CommandType.StoredProcedure;
con.Open();
dr = cmd.ExecuteReader();
ShowData();

private void ShowData() {
    if(dr.Read()) {
        textBox1.Text = dr["Eno"].ToString();
        textBox2.Text = dr["Ename"].ToString();
    }
}
```

```

textBox3.Text = dr["Job"].ToString();
textBox4.Text = dr["Salary"].ToString();
checkBox1.Checked = (bool)dr["Status"];
}
else {
    MessageBox.Show("You are at the last record of table.", "Information", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
}
}

```

Code under Next Button Click Event Handler:

```
ShowData();
```

Code under Close Button Click Event Handler:

```

if (con.State != ConnectionState.Closed) {
    con.Close();
}
this.Close();

```

Calling the above Stored Procedure using DataAdapter class and loading data into DataSet: Add a new Form in the project, place a **DataGridView** control on it setting the **Dock** property as **Fill** and write the below code in it:

```

using System.Configuration;
using System.Data.SqlClient;

```

Declarations:

```

DataSet ds;
SqlDataAdapter da;

```

Code under Form Load Event Handler:

```

string ConStr = ConfigurationManager.ConnectionStrings["SqlConStr"].ConnectionString;
da = new SqlDataAdapter("Employee_Select", ConStr);
da.SelectCommand.CommandType = CommandType.StoredProcedure;
ds = new DataSet();
da.Fill(ds, "Employee");
dataGridView1.DataSource = ds.Tables[0];

```

Performing Select and DML Operations using Stored Procedures: To perform **select**, **insert**, **update** and **delete** operations using **Stored Procedures**, first define the following **Stored Procedures** in our **Database** i.e., "CSDB".

```

ALTER PROCEDURE Employee_Select(@Eno as int = null, @Status as bit = null)
As
Begin
    If @Eno Is Null And @Status Is Null
        Select Eno, Ename, Job, Salary, Photo, Status From Employee Order By Eno;
    Else If @Eno Is Null And @Status Is Not Null
        Select Eno, Ename, Job, Salary, Photo, Status From Employee Where Status=@Status Order By Eno;
    Else If @Eno Is Not Null And @Status Is Null
        Select Eno, Ename, Job, Salary, Photo, Status From Employee Where Eno=@Eno;
    Else If @Eno Is Not Null And @Status Is Not Null

```

```
Select Eno, Ename, Job, Salary, Photo, Status From Employee Where Eno=@Eno And Status=@Status;
End;
```

```
CREATE PROCEDURE Employee_Insert(@Ename Varchar(50), @Job Varchar(50), @Salary Money, @Photo
VarBinary(Max), @Eno Int Out)
As
Begin
    Begin Transaction
    Select @Eno = IsNull(Max(Eno), 1000) + 1 From Employee;
    Insert Into Employee (Eno, Ename, Job, Salary, Photo) Values (@Eno, @Ename, @Job, @Salary, @Photo);
    Commit Transaction;
End;
```

```
CREATE PROCEDURE Employee_Update(@Eno Int, @Ename Varchar(50), @Job Varchar(50), @Salary Money,
@Photo VarBinary(Max))
As
Begin
    Update Employee Set Ename=@Ename, Job=@Job, Salary=@Salary, Photo=@Photo Where Eno=@Eno;
End;
```

```
CREATE PROCEDURE Employee_Delete(@Eno Int)
As
Begin
    Update Employee Set Status=0 Where Eno=@Eno;
End;
```

Parameters of Stored Procedures: Stored Procedures can be defined with **parameters** either to **send** values for execution or **receiving** values after execution. While calling a **Stored Procedure** which has **parameters** from our .NET application, for each **parameter** of the **Stored Procedure** we need to add a matching **parameter** under **Command** i.e., for **input parameter** matching **input parameter** has to be added and for **output parameter** a matching **output parameter** has to be added. Every **parameter** that is added under **Command** has **5 attributes** to it like **Name**, **Value**, **DbType**, **Size** and **Direction** which can be Input (d) or Output.

- **Name** refers to name of the parameter that is defined in Stored Procedure.
- **Value** refers to value being assigned in case of input or value we are expecting in case of output.
- **DbType** refers to data type of the parameter in terms of the Database where the Stored Procedure exists.
- **Size** refers to size of data.
- **Direction** specifies whether the parameter is Input or Output.

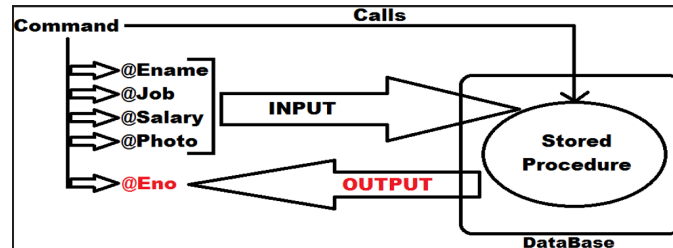
If a **Stored Procedure** is defined with **Input** or **Output** parameters, we need to specify the following **attributes** while adding the parameters:

	Input	Output
Name	Yes	Yes
Value	Yes	No
DbType	Yes [*]	Yes
Size	No	Yes [**]
Direction	No	Yes

*Required only if the value supplied is null.

**Required only in-case of variable length types.

If we want to call the **Select Stored Procedure** i.e., “**Employee_Insert**” which we defined above we need to add both **Input** and **Output** parameters under **Command** as below:



In the above case the 4 Input parameters (@Ename, @Job, @Salary, @Photo) are sent to the **Stored Procedure** for execution and once after the **Stored Procedure** got executed will send the **Output parameter** (@Eno) as a result.

Adding Input Parameter under Command:

```
cmd.Parameters.AddWithValue(string Parameter_Name, object Parameter_Value)
```

Adding Fixed Length Output Parameter under Command:

```
cmd.Parameters.Add(string Parameter_Name, DbType Data_Type).Direction = ParameterDirection.Output;
```

Adding Variable Length Output Parameter under Command:

```
cmd.Parameters.Add(string Parameter_Name, DbType Data_Type, int Size).Direction=ParameterDirection.Output;
```

After executing the SP we can capture Output parameter values as following:

```
Object obj = cmd.Parameters[string pname].Value;
```

Add a new Form in the project, design it as below, and then write below code:

Form14

Employee Details

Eno:

Ename:

Job:

Salary:

Select Clear New

Update Delete Close

Load Image

Note: Set the **ReadOnly** property of **TextBox1** i.e., **Eno TextBox** as **True**. Set the **SizeMode** property of **PictureBox** as **Stretch** and also **BorderStyle** property as **FixedSingle**. Delete the **Text** property value of **ClearImage Button** i.e., it should be **empty string**, set the **BackgroundImage** property with **Cancel** image (present in **Icons** folder which is provided to you) and also set the **BackgroundImageLayout** property as **Center**. Add a **OpenFileDialogControl** on the **Form** and then write the below code in **Code View**:

```
using System.IO;
using System.Configuration;
```

```
using System.Data.SqlClient;
using static Microsoft.VisualBasic.Interaction;
```

Declarations:

```
string imgPath = "";
byte[] imgData = null;
SqlConnection con;
SqlCommand cmd;
SqlDataReader dr;
```

Code under Form Load Event Procedure:

```
string ConStr = ConfigurationManager.ConnectionStrings["SqlConStr"].ConnectionString;
con = new SqlConnection(ConStr);
cmd = new SqlCommand();
cmd.Connection = con;
cmd.CommandType = CommandType.StoredProcedure;
```

Code under Select Button Click Event Handler:

```
string Value = InputBox("Enter Employee No. to Search.");
if (int.TryParse(Value, out int Eno)) {
    try {
        cmd.CommandText = "Employee_Select";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@Eno", Eno);
        cmd.Parameters.AddWithValue("@Status", true);
        con.Open();
        dr = cmd.ExecuteReader();
        if (dr.Read()) {
            textBox1.Text = dr["Eno"].ToString();
            textBox2.Text = dr["Ename"].ToString();
            textBox3.Text = dr["Job"].ToString();
            textBox4.Text = dr["Salary"].ToString();

            if (dr["Photo"] != DBNull.Value) {
                imgData = (byte[])dr["Photo"];
                MemoryStream ms = new MemoryStream(imgData);
                pictureBox1.Image = Image.FromStream(ms);
            }
            else {
                imgData = null;
                imgPath = "";
                pictureBox1.Image = null;
            }
        }
        else {
            MessageBox.Show("No employee exist's with the given number.", "Information", MessageBoxButtons.OK,
                MessageBoxIcon.Information);
        }
    }
}
```

```

    }
    catch(Exception ex) {
        MessageBox.Show(ex.Message, "Error Message", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally {
        con.Close();
    }
    else {
        MessageBox.Show("Employee No. should be integer value.", "Conversion Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}

```

Code under Clear Button Click Event Procedure:

```

imgPath = "";
imgData = null;
pictureBox1.Image = null;
textBox1.Text = textBox2.Text = textBox3.Text = textBox4.Text = "";
textBox2.Focus();

```

Code under Insert Button Click Event Procedure:

```

if (btnNew.Text == "New") {
    btnClear.PerformClick();
    btnNew.Text = "Insert";
}
else {
    try {
        cmd.CommandText = "Employee_Insert";
        cmd.Parameters.Clear();
        cmd.Parameters.AddWithValue("@Ename", textBox2.Text);
        cmd.Parameters.AddWithValue("@Job", textBox3.Text);
        cmd.Parameters.AddWithValue("@Salary", textBox4.Text);
        if (imgPath.Trim().Length > 0) {
            imgData = File.ReadAllBytes(imgPath);
            cmd.Parameters.AddWithValue("@Photo", imgData);
        }
        else {
            cmd.Parameters.AddWithValue("@Photo", DBNull.Value);
            cmd.Parameters["@Photo"].SqlDbType = SqlDbType.VarBinary;
        }
        cmd.Parameters.Add("@Eno", SqlDbType.Int, 4).Direction = ParameterDirection.Output;
        con.Open();
        cmd.ExecuteNonQuery();
        textBox1.Text = cmd.Parameters["@Eno"].Value.ToString();
        imgData = null; imgPath = "";
    }
    catch (Exception ex) {
        MessageBox.Show(ex.Message, "Error Message", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

```

finally {
    con.Close();
    btnNew.Text = "New";
}
}

```

Code under Update Button Click Event Procedure:

```

try {
    cmd.CommandText = "Employee_Update";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@Eno", textBox1.Text);
    cmd.Parameters.AddWithValue("@Ename", textBox2.Text);
    cmd.Parameters.AddWithValue("@Job", textBox3.Text);
    cmd.Parameters.AddWithValue("@Salary", textBox4.Text);
    if(imgData == null && imgPath.Trim().Length == 0) {
        cmd.Parameters.AddWithValue("@Photo", DBNull.Value);
        cmd.Parameters["@Photo"].SqlDbType = SqlDbType.VarBinary;
    }
    else if(imgPath.Trim().Length > 0) {
        imgData = File.ReadAllBytes(imgPath);
        cmd.Parameters.AddWithValue("@Photo", imgData);
    }
    else if(imgPath.Trim().Length == 0 && imgData != null) {
        cmd.Parameters.AddWithValue("@Photo", imgData);
    }
    con.Open();
    cmd.ExecuteNonQuery();
    MessageBox.Show("Record updated in Database-Table.", "Information Message", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
}
catch (Exception ex) {
    MessageBox.Show(ex.Message, "Error Message", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
finally {
    con.Close();
}
}

```

Code under Delete Button Click Event Procedure:

```

try {
    cmd.CommandText = "Employee_Delete";
    cmd.Parameters.Clear();
    cmd.Parameters.AddWithValue("@Eno", textBox1.Text);
    con.Open();
    cmd.ExecuteNonQuery();
    btnClear.PerformClick();
    MessageBox.Show("Record deleted from Database-Table.", "Information Message", MessageBoxButtons.OK,
        MessageBoxIcon.Information);
}
}

```



```

catch (Exception ex) {
    MessageBox.Show(ex.Message, "Error Message", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
finally {
    con.Close();
}

```

Code under Close Button Click Event Procedure:

```

if (con.State != ConnectionState.Closed) {
    con.Close();
}
this.Close();

```

Code under Load Image Button Click Event Procedure:

```

openFileDialog1.Filter = "Jpeg Images | *.jpg | Bitmap Images | *.bmp | All Files | *.*";
DialogResult dr = openFileDialog1.ShowDialog();
if (dr == DialogResult.OK) {
    imgPath = openFileDialog1.FileName;
    pictureBox1.ImageLocation = imgPath;
}

```

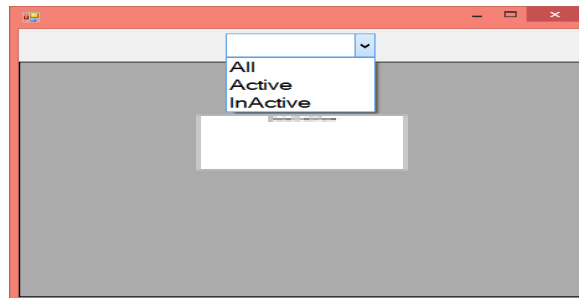
Code under Clear Image Click Event Procedure:

```

imgPath = "";
imgData = null;
pictureBox1.Image = null;

```

Accessing All, Active and In-Active records by using Select Stored Procedure: Add a new **Form** in the **project** for accessing **All**, **Active** and **In-Active** records from the table using **Select Stored Procedure**. To do this place a **ComboBox** control on the **top center**, goto it's **items** property and add the values: **All**, **Active** & **In-Active**. Now place a **DataGridView** Control, goto its properties and Set **AllowUserToAddRows** and **AllowUserToDeleteRows** property as **false**, **ReadOnly** Property as **true** and then set the **Dock** property as **Bottom** and write the below code under **Code View**.



```

using System.Configuration;
using System.Data.SqlClient;

```

Declarations:

```

DataSet ds;
SqlDataAdapter da;

```

Code under Form Load Event Procedure:

```
string ConStr = ConfigurationManager.ConnectionStrings["SqlConStr"].ConnectionString;
da = new SqlDataAdapter("Employee_Select", ConStr);
da.SelectCommand.CommandType = CommandType.StoredProcedure;
comboBox1.SelectedIndex = 0;
```

Code under ComboBox SelectedIndexChanged Event Procedure:

```
da.SelectCommand.Parameters.Clear();
if (comboBox1.SelectedIndex == 1) {
    da.SelectCommand.Parameters.AddWithValue("@Status", true);
}
else if (comboBox1.SelectedIndex == 2) {
    da.SelectCommand.Parameters.AddWithValue("@Status", false);
}
ds = new DataSet();
da.Fill(ds, "Employee");
dataGridView1.AutoGenerateColumns = false;
dataGridView1.DataSource = ds.Tables[0];
GenerateColumns();

private void GenerateColumns() {
    dataGridView1.Columns.Clear();
    DataGridViewTextBoxColumn enoColumn = new DataGridViewTextBoxColumn();
    enoColumn.HeaderText = "Emp-No";
    enoColumn.DataPropertyName = "Eno";
    enoColumn.HeaderCell.Style.Alignment = DataGridViewContentAlignment.MiddleCenter;
    enoColumn.DefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleCenter;
    enoColumn.Resizable = DataGridViewTriState.False;
    enoColumn.Width = 90;

    dataGridView1.Columns.Add(enoColumn);
    DataGridViewTextBoxColumn enameColumn = new DataGridViewTextBoxColumn();
    enameColumn.HeaderText = "Name";
    enameColumn.DataPropertyName = "Ename";
    enameColumn.HeaderCell.Style.Alignment = DataGridViewContentAlignment.MiddleCenter;
    enameColumn.Resizable = DataGridViewTriState.False;
    enameColumn.Width = 110;
    dataGridView1.Columns.Add(enameColumn);

    DataGridViewTextBoxColumn jobColumn = new DataGridViewTextBoxColumn();
    jobColumn.HeaderText = "Job";
    jobColumn.DataPropertyName = "Job";
    jobColumn.HeaderCell.Style.Alignment = DataGridViewContentAlignment.MiddleCenter;
    jobColumn.Resizable = DataGridViewTriState.False;
    jobColumn.Width = 120;
    dataGridView1.Columns.Add(jobColumn);

    DataGridViewTextBoxColumn salaryColumn = new DataGridViewTextBoxColumn();
```

```

salaryColumn.HeaderText = "Salary";
salaryColumn.DataPropertyName = "Salary";
salaryColumn.HeaderCell.Style.Alignment = DataGridViewContentAlignment.MiddleCenter;
salaryColumn.DefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleRight;
salaryColumn.Resizable = DataGridViewTriState.False;
salaryColumn.Width = 130;
dataGridView1.Columns.Add(salaryColumn);

```

```

DataGridViewCheckBoxColumn statusColumn = new DataGridViewCheckBoxColumn();
statusColumn.HeaderText = "Status";
statusColumn.DataPropertyName = "Status";
statusColumn.HeaderCell.Style.Alignment = DataGridViewContentAlignment.MiddleCenter;
statusColumn.DefaultCellStyle.Alignment = DataGridViewContentAlignment.MiddleCenter;
statusColumn.Resizable = DataGridViewTriState.False;
statusColumn.Width = 80;
dataGridView1.Columns.Add(statusColumn);

```

```

DataGridViewImageColumn photoColumn = new DataGridViewImageColumn();
photoColumn.HeaderText = "Emp-Photo";
photoColumn.DataPropertyName = "Photo";
photoColumn.HeaderCell.Style.Alignment = DataGridViewContentAlignment.MiddleCenter;
photoColumn.ImageLayout = DataGridViewImageCellLayout.Stretch;
photoColumn.Resizable = DataGridViewTriState.False;
photoColumn.Width = 250;
dataGridView1.Columns.Add(photoColumn);

```

```

for (int i = 0; i < ds.Tables[0].Rows.Count; i++)
{
    dataGridView1.Rows[i].Height = 250;
}
}

```

Logical Programs

1. Write a program to print the given no is a prime number or not?

```

class PrimeNumberTest
{
    static void Main()
    {
        Console.Write("Enter a number to check it's a prime: ");
        uint Number = uint.Parse(Console.ReadLine());
        if(Number == 0 || Number == 1)
        {
            Console.WriteLine("Please enter a number other than 0 & 1");
            return;
        }
        bool IsPrime = true;
        uint HalfNumber = Number / 2;

```

```

for(uint i = 2; i <= HalfNumber; i++)
{
    if(Number % i == 0)
    {
        IsPrime = false;
        break;
    }
}
if(IsPrime == true)
    Console.WriteLine("Given number is a prime.");
else
    Console.WriteLine("Given number is not a prime.");
Console.ReadLine();
}
}

```

2. Write a program to swap 2 numbers without using 3rd variable?

```

class SwapNumbers1           //Solution 1
{
    static void Main()
    {
        int a = 342, b = 784;
        Console.WriteLine($"Numbers Before Swap: a => {a}; b => {b}");
        a = a * b;    b = a / b;    a = a / b;
        Console.WriteLine($"Numbers After Swap: a => {a}; b => {b}");
        Console.ReadLine();
    }
}

```

```

class SwapNumbers2           //Solution 2
{
    static void Main()
    {
        Console.Write("Enter 1st number: ");
        int a = int.Parse(Console.ReadLine());
        Console.Write("Enter 2nd number: ");
        int b = int.Parse(Console.ReadLine());
        Console.WriteLine($"Numbers Before Swap: a => {a}; b => {b}");
        a = a + b;    b = a - b;    a = a - b;
        Console.WriteLine($"Numbers After Swap: a => {a}; b => {b}");
        Console.ReadLine();
    }
}

```

3. Write a program to print the reverse of a given number?

```

class ReverseNumber
{
    static void Main()
    {

```

```

Console.Write("Enter a number: ");
int Number = int.Parse(Console.ReadLine());
int Reminder, Reverse = 0;
while(Number != 0)
{
    Reminder = Number % 10;
    Reverse = Reverse * 10 + Reminder;
    Number = Number / 10;
}
Console.WriteLine("Reversed Number is: " + Reverse);
Console.ReadLine();
}
}

```

4. Write the program to print the binary value of a given number?

```

class NumberToBinary
{
    static void Main()
    {
        Console.Write("Enter an number to convert into binary: ");
        int Number = int.Parse(Console.ReadLine());
        int[] arr = new int[16];
        int i;
        for(i = 0; Number > 0; i++)
        {
            arr[i] = Number % 2;
            Number = Number / 2;
        }
        Console.Write("Binary value of the given number is: ");
        for(i = i - 1; i >= 0; i--)
        {
            Console.Write(arr[i]);
        }
        Console.ReadLine();
    }
}

```

5. Write a program to check whether a given number is a palindrome?

```

class PalindromeNumber
{
    static void Main()
    {
        Console.Write("Enter a Number: ");
        int Number = int.Parse(Console.ReadLine());
        int OldNumber = Number;
        int Reminder, Reverse = 0;
        while(Number != 0)
        {

```

```

    Reminder = Number % 10;
    Reverse = (Reverse * 10) + Reminder;
    Number = Number / 10;
}
if (OldNumber == Reverse)
    Console.WriteLine("Given number is a palindrome");
else
    Console.WriteLine("Given number is not a palindrome");
Console.ReadLine();
}
}

```

6. Write a program to print the Fibonacci series up to a given upper bound?

class FibonacciSeries

```

{
    static void Main()
    {
        Console.Write("Enter the number of elements for Fibonacci Series: ");
        int Number = int.Parse(Console.ReadLine());
        int Num1 = 0, Num2 = 1, Num3;
        Console.Write(Num1 + " " + Num2 + " ");
        for(int i = 2; i < Number; i++)
        {
            Num3 = Num1 + Num2;
            Console.Write(Num3 + " ");
            Num1 = Num2;
            Num2 = Num3;
        }
        Console.ReadLine();
    }
}

```

7. Write a program to print the factorial of a given number?

class Factorial

```

{
    static void Main()
    {
        Console.Write("Enter a number to find it's factorial: ");
        uint Number = uint.Parse(Console.ReadLine());
        uint Result = 1;
        for(uint i=1; i<=Number; i++)
        {
            Result = Result * i;
        }
        Console.WriteLine("Factorial of given number is: " + Result);
        Console.ReadLine();
    }
}

```

8. Write a program to find whether the give number is an Armstrong number or not?

class ArmstrongNumber

```
{
    static void Main()
    {
        Console.WriteLine("Enter a number to find it is Armstrong: ");
        int Number = int.Parse(Console.ReadLine());
        int Original = Number;
        int Reminder, Sum = 0;
        while(Number > 0)
        {
            Reminder = Number % 10;
            Sum = Sum + (Reminder * Reminder * Reminder);
            Number = Number / 10;
        }
        if (Original == Sum)
            Console.WriteLine($"{Original} is an armstrong number");
        else
            Console.WriteLine($"{Original} is not an armstrong number");
        Console.ReadLine();
    }
}
```

9. Write a program to find the sum of digits of a given number?

class SumOfDigits1

```
{
    static void Main()
    {
        Console.WriteLine("Enter a number to find sum of its digits: ");
        int Number = int.Parse(Console.ReadLine());
        int Reminder, Sum = 0;
        while(Number > 0)
        {
            Reminder = Number % 10;
            Sum = Sum + Reminder;
            Number = Number / 10;
        }
        Console.WriteLine("Sum of the digits of given no is: " + Sum);
        Console.ReadLine();
    }
}
```

10. Write a program to find the sum of digits of a given number until single digit?

class SumOfDigits2

```
{
    static void Main()
    {
        Console.WriteLine("Enter a number to find sum of it's digits: ");
```

```

int Number = int.Parse(Console.ReadLine());
int Reminder, Sum = 0;
do
{
    if(Sum != 0)
    {
        Number = Sum;
        Sum = 0;
    }
    while (Number > 0)
    {
        Reminder = Number % 10;
        Sum = Sum + Reminder;
        Number = Number / 10;
    }
}
while (Sum > 9);
Console.WriteLine("Sum of the digits of given no is: " + Sum);
Console.ReadLine();
}
}

```

11. Write a program to print the given number in words?

```

class NumberToString
{
    static void Main()
    {
        Console.Write("Enter a number: ");
        int Number = int.Parse(Console.ReadLine());
        int Reminder, Reverse = 0;
        while(Number > 0)
        {
            Reminder = Number % 10;
            Reverse = Reverse * 10 + Reminder;
            Number = Number / 10;
        }
        while(Reverse > 0)
        {
            Reminder = Reverse % 10;
            switch (Reminder)
            {
                case 1:
                    Console.Write("one ");
                    break;
                case 2:
                    Console.Write("two ");
                    break;
            }
        }
    }
}

```



```

    case 3:
        Console.Write("three ");
        break;
    case 4:
        Console.Write("four ");
        break;
    case 5:
        Console.Write("five ");
        break;
    case 6:
        Console.Write("six ");
        break;
    case 7:
        Console.Write("seven ");
        break;
    case 8:
        Console.Write("eight ");
        break;
    case 9:
        Console.Write("nine ");
        break;
    case 0:
        Console.Write("zero ");
        break;
    }
    Reverse = Reverse / 10;
}
Console.ReadLine();
}
}

```

12. Write a program to find the given year is a leap year or not?

```

class LeapYear
{
    static void Main()
    {
        Console.Write("Enter the year in 4 digits: ");
        int Year = int.Parse(Console.ReadLine());
        if ((Year % 4 == 0 && Year % 100 != 0) || (Year % 400 == 0))
            Console.WriteLine($"{Year} is a leap year.");
        else
            Console.WriteLine($"{Year} is not a leap year.");
        Console.ReadLine();
    }
}

```

13. Write a program to print the larger number in an array?

```

class LargerNumberInArray

```

```

{
static void Main()
{
    Console.Write("Specify the no of items to compare: ");
    int UB = int.Parse(Console.ReadLine());
    Console.Clear();
    int[] arr = new int[UB];
    for(int i=0;i<UB;i++)
    {
        Console.Write($"Enter Item{i + 1}: ");
        arr[i] = int.Parse(Console.ReadLine());
    }
    int LargeNumber = arr[0];
    for(int i=1;i<UB;i++)
    {
        if(arr[i] > LargeNumber)
        {
            LargeNumber = arr[i];
        }
    }
    Console.WriteLine("Larger number in the array is: " + LargeNumber);
    Console.ReadLine();
}
}

```

14. Write a program to print the given string in reverse?

```

class StringReverse
{
static void Main()
{
    Console.Write("Enter a string: ");
    string input = Console.ReadLine();
    string reverse = "";
    foreach(char ch in input)
        reverse = ch + reverse;
    Console.WriteLine($"Reverse of given string '{input}' is: '{reverse}'");
    Console.ReadLine();
}
}

```

15. Write a program to print the no. of words in each string?

```

class WordCount
{
static void Main()
{
    Console.Write("Enter a string: ");
    string input = Console.ReadLine();
    int Count = 0, CharCount = 0;

```

```

bool Flag = true, EndSpace = false;
bool StartSpace = false;
foreach (char ch in input)
{
    CharCount += 1;
    if (CharCount == 1 && ch == 32)
        StartSpace = true;
    if (ch == 32 && Flag == false)
        continue;
    else {
        Flag = true;
        EndSpace = false;
    }
    if (Count == 0)
        Count = 1;
    if (ch == 32)
    {
        Count += 1;
        Flag = false;
        EndSpace = true;
    }
}
if (StartSpace == true)
    Count -= 1;
if (EndSpace == true)
    Count -= 1;
Console.WriteLine("No of words in the given string are: " + Count);
Console.ReadLine();
}
}

```

16. Write a program to print the length of a given string?

```

class StringLength
{
    static void Main()
    {
        Console.Write("Enter a string: ");
        string input = Console.ReadLine();
        int Length = 0;
        foreach (char ch in input)
            Length += 1;
        Console.WriteLine("Length of given string is: " + Length);
        Console.ReadLine();
    }
}

```

17. Write a program to print the no. of characters in each string?

```

class CharCount

```

```

{
static void Main()
{
    Console.Write("Enter a string: ");
    string input = Console.ReadLine();
    int Length = 0;
    foreach (char ch in input)
    {
        if(ch != 32)
            Length += 1;
    }
    Console.WriteLine("No. of char's in given string are: " + Length);
    Console.ReadLine();
}
}

```

18. Write a program to print the words in reverse order of a given string?

```

class ReverseWords
{
static void Main()
{
    Console.Write("Enter a string: ");
    string input = Console.ReadLine();
    string word = "", reverseWords = "";
    foreach(char ch in input)
    {
        if (ch != 32)
            word = word + ch;
        else
        {
            reverseWords = " " + word + reverseWords;
            word = "";
        }
    }
    if (word != "")
        reverseWords = word + reverseWords;
    Console.WriteLine(reverseWords);
    Console.ReadLine();
}
}

```

19. Write a program to convert the given string into lower case?

```

class StringToLower
{
static void Main()
{
    Console.Write("Enter a string: ");
    string input = Console.ReadLine();

```

```

string output = "";
foreach(char ch in input)
{
    if (ch >= 65 && ch <= 90)
        output += (char)(ch + 32);
    else
        output += ch;
}
Console.WriteLine(output);
Console.ReadLine();
}

```

20. Write a program to convert the given string into upper case?

```

class StringToUpper
{
    static void Main()
    {
        Console.Write("Enter a string: ");
        string input = Console.ReadLine();
        string output = "";
        foreach (char ch in input) {
            if (ch >= 97 && ch <= 122)
                output += (char)(ch - 32);
            else
                output += ch;
        }
        Console.WriteLine(output);
        Console.ReadLine();
    }
}

```

21. Write a program to convert the given string into pascal case?

```

class StringToPascal
{
    static void Main()
    {
        Console.Write("Enter a string: ");
        string input = Console.ReadLine();
        string lower = "";
        foreach (char ch in input)
        {
            if (ch >= 65 && ch <= 90)
                lower += (char)(ch + 32);
            else
                lower += ch;
        }
        string pascal = "";
    }
}

```

```

bool firstChar = true, flag = false;
foreach(char ch in lower)
{
    if (firstChar == true)
    {
        if (ch >= 97 && ch <= 122)
            pascal += (char)(ch - 32);
        firstChar = false;
        continue;
    }
    if (flag == true)
    {
        if (ch >= 97 && ch <= 122)
            pascal += (char)(ch - 32);
        flag = false;
    }
    else
        pascal += ch;
    if (ch == 32)
        flag = true;
}
Console.WriteLine(pascal);
Console.ReadLine();
}
}

```

22. Write a program to find out the unique characters in each string?

```

class UniqueChars
{
    static void Main()
    {
        Console.Write("Enter a string: ");
        string input = Console.ReadLine();
        bool Exists = false;
        int Count1 = 0, Count2 = 0;
        foreach(char ch1 in input)
        {
            Count1 += 1;
            foreach(char ch2 in input)
            {
                Count2 += 1;
                if(Count1 != Count2)
                {
                    if (ch1 != ch2 && ch1 != 32)
                        Exists = false;
                    else
                    {

```

```

        Exists = true;
        break;
    }
}
}
if (Exists == false)
    Console.Write(ch1);
Count2 = 0; Exists = false;
}
Console.ReadLine();
}
}

```

23. Write a program to find out the duplicate characters in each string?

class DuplicateChars

```

{
    static void Main()
    {
        Console.Write("Enter a string: ");
        string input = Console.ReadLine();
        int Length = 0;
        foreach (char ch in input)
            Length += 1;
        char[] arr = new char[Length];
        int Index = 0;
        foreach(char ch in input)
        {
            arr[Index] = ch;
            Index += 1;
        }
        int Count1 = 0, Count2 = 0;
        foreach(char ch1 in arr)
        {
            Count1 += 1;
            foreach(char ch2 in arr)
            {
                Count2 += 1;
                if(Count1 != Count2)
                {
                    if(ch1 == ch2 && ch1 != 32)
                    {
                        Console.WriteLine(ch1);
                        arr[Count1 - 1] = ' ';
                        arr[Count2 - 1] = ' ';
                        break;
                    }
                }
            }
        }
    }
}

```

```

    }
    Count2 = 0;
}
Console.ReadLine();
}
}

```

24. Write a program to print the roman number of a given number?

class NumberToRoman

```

{
    static void Main()
    {
        Console.Write("Enter a string: ");
        int num = int.Parse(Console.ReadLine());
        string roman = ToRoman(num);
        Console.WriteLine(roman);
        Console.ReadLine();
    }
    public static string ToRoman(int num)
    {
        if (num < 0 || num > 3999)
            return "Enter a number between 1 and 3999";
        else if (num >= 1000)
            return "M" + ToRoman(num - 1000);
        else if (num >= 900)
            return "CM" + ToRoman(num - 900);
        else if (num >= 500)
            return "D" + ToRoman(num - 500);
        else if (num >= 400)
            return "CD" + ToRoman(num - 400);
        else if (num >= 100)
            return "C" + ToRoman(num - 100);
        else if (num >= 90)
            return "XC" + ToRoman(num - 90);
        else if (num >= 50)
            return "L" + ToRoman(num - 50);
        else if (num >= 40)
            return "XL" + ToRoman(num - 40);
        else if (num >= 10)
            return "X" + ToRoman(num - 10);
        else if (num >= 9)
            return "IX" + ToRoman(num - 9);
        else if (num >= 5)
            return "V" + ToRoman(num - 5);
        else if (num >= 4)
            return "IV" + ToRoman(num - 4);
        else if (num >= 1)

```

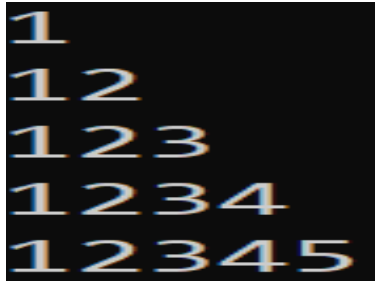


```

        return "I" + ToRoman(num - 1);
    else
        return "";
    }
}

```

25. Write a program to print the below output:



```

1
12
123
1234
12345

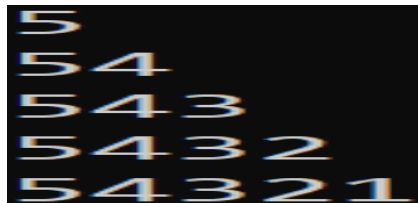
```

```

class Pattern1
{
    static void Main()
    {
        Console.Write("Enter a number: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        for(int i=1;i<=num;i++)
        {
            for(int j=1;j<=i;j++)
                Console.Write(j);
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}

```

26. Write a program to print the below output:



```

5
54
543
5432
54321

```

```

class Pattern2
{
    static void Main()
    {
        Console.Write("Enter a number: ");
        int Number = int.Parse(Console.ReadLine());
        Console.Clear();
        for (int i = Number; i >= 1; i--)
        {

```

```

    for (int j = Number; j >= i; j--)
        Console.Write(j);
    Console.WriteLine();
}
Console.ReadLine();
}

```

27. Write a program to print the below output:

```

1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
16 17 18 19 20 21

```

```

class Pattern3
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int Rows = int.Parse(Console.ReadLine());
        Console.Clear();
        int x = 1;
        for(int i=1;i<=Rows;i++)
        {
            for (int j = 1; j <= i; j++)
                Console.Write($"{x++} ");
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}

```

28. Write a program to print the below output:

```

1
01
101
0101
10101
010101

```

```

class Pattern4
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int Rows = int.Parse(Console.ReadLine());
        Console.Clear();
    }
}

```

```

int x = 0, y = 0;
for (int i = 1; i <= Rows; i++)
{
    if(i % 2 == 0)
    {
        x = 1;
        y = 0;
    }
    else
    {
        x = 0;
        y = 1;
    }
    for (int j = 1; j <= i; j++)
    {
        if (j % 2 == 0)
            Console.Write(x);
        else
            Console.Write(y);
    }
    Console.WriteLine();
}
Console.ReadLine();
}

```

29. Write a program to print the below output:



```

class Pattern5
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        for (int i = 1; i < num; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                Console.Write(j);
            }
            Console.WriteLine();
        }
        for(int i=num;i>=0;i--)
    }
}

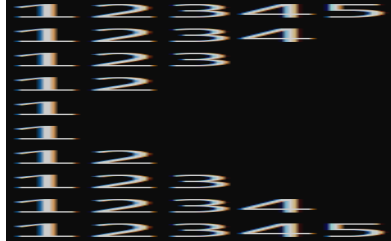
```

```

{
    for (int j = 1; j <= i; j++)
        Console.Write(j);
    Console.WriteLine();
}
Console.ReadLine();
}

```

30. Write a program to print the below output:



class Pattern6

```

{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        for (int i = num; i >= 0; i--)
        {
            for (int j = 1; j <= i; j++)
                Console.Write(j);
            if (i > 0)
                Console.WriteLine();
        }
        for (int i = 1; i <= num; i++)
        {
            for (int j = 1; j <= i; j++)
                Console.Write(j);
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}

```

31. Write a program to print the below output:

```

      1
     121
    12321
   1234321
  123454321

```

```

class Pattern7
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        for(int i=1;i<= num;i++)
        {
            for (int space = 1; space <= (num - i); space++)
                Console.Write(" ");
            for (int j = 1; j <= i; j++)
                Console.Write(j);
            for (int k = (i - 1); k >= 1; k--)
                Console.Write(k);
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}

```

32. Write a program to print the below output:

```

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1

```

```

class Pattern8
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        int result;
        for(int i=0;i<=num;i++)
        {
            result = 1;

```

```

for(int j=i;j <= num - 1;j++)
    Console.Write(" ");
for(int k=0;k<=i;k++)
{
    Console.Write(result + " ");
    result = (result * (i - k) / (k + 1));
}
Console.WriteLine();
}
Console.ReadLine();
}

```

33. Write a program to print the below output:



class Pattern9

```

{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        int count = num - 1;
        for(int i=1;i<num+1;i++)
        {
            for (int j = 1; j <= count; j++)
                Console.Write(" ");
            count--;
            for (int k = 1; k <= 2 * i - 1; k++)
                Console.Write("*");
            Console.WriteLine();
        }
        count = 1;
        for(int i=1;i<=num-1;i++)
        {
            for (int j = 1; j <= count; j++)
                Console.Write(" ");
            count++;
            for (int k = 1; k <= 2 * (num - i) - 1; k++)
                Console.Write("*");
            Console.WriteLine();
        }
    }
}

```

```

    Console.ReadLine();
}
}

```

34. Write a program to print the below output:

```

*  *
* *  * *
* * *  * * *
* * * *  * * * *
* * * * *  * * * * *

```

```

class Pattern10
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
        for(int i=0;i<num;i++)
        {
            for (int j = 0; j <= i; j++)
                Console.Write("*");
            Console.Write(" ");
            for (int j = 0; j <= i; j++)
                Console.Write("*");
            Console.WriteLine();
        }
        Console.ReadLine();
    }
}

```

35. Write a program to print the below output:

```

> * < > * < > * < > * < > * <
> * <                                > * <
> * <                                > * <
> * <                                > * <
> * <                                > * <
> * < > * < > * < > * < > * <

```

```

class Pattern11
{
    static void Main()
    {
        Console.Write("Enter number of rows: ");
        int num = int.Parse(Console.ReadLine());
        Console.Clear();
    }
}

```

```

for(int i=0;i < num;i++)
{
    if(i == 0 || i == num - 1)
    {
        for (int j = 0; j < num; j++)
            Console.Write('*');
        Console.WriteLine();
    }
    else
    {
        for(int j=0;j<num;j++)
        {
            if (j == 0 || j == num - 1)
                Console.Write('*');
            else
                Console.Write(' ');
        }
        Console.WriteLine();
    }
}
Console.ReadLine();
}

```

36. Bubble Sort: how does Bubble Sort work is starting at index zero, we take an item and the item next in the array and compare them. If they are in the right order, then we do nothing, if they are in the wrong order (e.g. the item lower in the array is actually a higher value than the next element), then we swap these items. Then we continue through each item in the array doing the same thing (Swapping with the next element if it's higher).

```

class BubbleSort
{
    static void Main(string[] args)
    {
        int[] arr = { 54, 79, 58, 7, 42, 23, 91, 3, 74, 38, 67, 46, 18, 61, 32, 86, 14, 28 };
        bool itemMoved = false;
        do
        {
            itemMoved = false;
            for (int i = 0; i < arr.Length - 1; i++)
            {
                if (arr[i] > arr[i + 1])
                {
                    int lowerValue = arr[i + 1];
                    arr[i + 1] = arr[i];
                    arr[i] = lowerValue;
                    itemMoved = true;
                }
            }
        }
    }
}

```



```

    }
} while (itemMoved);

foreach (int i in arr)
    Console.Write(i + " ");
Console.ReadLine();
}
}

```

Now since we are only comparing each item with its neighbor, each item may only move a single place when it needs to move several places. So how does Bubble Sort solve this? Well, it just runs the entire process all over again. Notice how we have the variable called “itemMoved”. We simply set this to true if we did swap an item and start the scan all over again. Because we are moving things one at a time, not directly to the right position, and having to multiple passes to get things right, Bubble Sort is seen as extremely inefficient.

37. Selection Sort: It's remarkably a simple algorithm to explain and the way Selection Sort works is an outer loop visits each item in the array to find out whether it is the minimum of all the elements after it. If it is not the minimum, it is going to be swapped with whatever item in the rest of the array is the minimum. For example, if you have an array of 10 elements, this means that “i” goes from 0 to 9. When we are looking at position 0, we check to find the position of the minimum element in positions 1 ... 9. If the minimum is not already at position “i”, we swap the minimum into place. Then we consider “i = 1” and look at positions 2 .. 9. And so on.

class SelectionSort

```

{
    static void Main()
    {
        int[] arr = { 54, 79, 58, 7, 42, 23, 91, 3, 74, 38, 67, 46, 18, 61, 32, 86, 14, 28 };
        for(int i=0;i<arr.Length;i++)
        {
            int min = i;
            for(int j=i+1;j<arr.Length;j++)
            {
                if(arr[min] > arr[j])
                {
                    min = j;
                }
            }
            if(min != i)
            {
                int lowerValue = arr[min];
                arr[min] = arr[i]; arr[i] = lowerValue;
            }
        }
        foreach (int i in arr)
            Console.Write(i + " ");
        Console.ReadLine();
    }
}

```

```
}
```

38. Insertion Sort: In the Insertion Sort algorithm, we build a sorted list from the bottom of the array. We repeatedly insert the next element into the sorted part of the array by sliding it down to its proper position. This will require as many exchanges as Bubble Sort, since only one inversion is removed per exchange.

```
class InsertionSort
```

```
{
    static void Main()
    {
        int[] arr = { 54, 79, 58, 7, 42, 23, 91, 3, 74, 38, 67, 46, 18, 61, 32, 86, 14, 28 };
        for(int i=0;i<arr.Length;i++)
        {
            int item = arr[i];
            int currentIndex = i;
            while(currentIndex > 0 && arr[currentIndex - 1] > item)
            {
                arr[currentIndex] = arr[currentIndex - 1];
                currentIndex--;
            }
            arr[currentIndex] = item;
        }
        foreach (int i in arr)
            Console.Write(i + " ");
        Console.ReadLine();
    }
}
```

39. Shell Sort: Donald Shell published the first version of this sort; hence this is known as Shell sort. This sorting is a generalization of insertion sort that allows the exchange of items that are far apart. It starts by comparing elements that are far apart and gradually reduces the gap between elements being compared. The running time of Shell sort varies depending on the gap sequence it uses to sort the elements.

```
class ShellSort
```

```
{
    static void Main()
    {
        int[] arr = { 54, 79, 58, 7, 42, 23, 91, 3, 74, 38, 67, 46, 18, 61, 32, 86, 14, 28 };
        int n = arr.Length;
        int gap = n / 2;
        int temp;
        while (gap > 0)
        {
            for(int i=0;i + gap < n;i++)
            {
                int j = i + gap;
                temp = arr[j];
                while(j - gap >= 0 && temp < arr[j - gap])
            }
        }
    }
}
```

```

    {
        arr[j] = arr[j - gap];
        j = j - gap;
    }
    arr[j] = temp;
}
gap = gap / 2;
}
foreach (int i in arr)
{
    Console.Write(i + " ");
}
Console.ReadLine();
}
}

```

40. Quick Sort: Like Merge Sort, Quick Sort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array around the picked pivot. There are many different versions of Quick Sort that pick pivot in different ways.

1. Always pick first element as pivot (implemented below).
2. Always pick last element as pivot.
3. Pick a random element as pivot.
4. Pick median as pivot.

The main idea for finding pivot is - the pivot or pivot element is the element of an array, which is selected first to do certain calculations. The key process in Quick Sort is partition. Target of partitions is, given an array and an element x of array as pivot, put x at its correct position in sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x.

```

class QuickSort
{
    static int[] arr;
    public static void Sort(int left, int right)
    {
        int pivot, leftEnd, rightEnd;
        leftEnd = left;
        rightEnd = right;
        pivot = arr[left];
        while (left < right)
        {
            while ((arr[right] >= pivot) && (left < right))
            {
                right--;
            }
            if (left != right)
            {
                arr[left] = arr[right]; left++;
            }
        }
    }
}

```

```

    }
    while ((arr[left] <= pivot) && (left < right))
    {
        left++;
    }
    if (left != right)
    {
        arr[right] = arr[left];
        right--;
    }
}
arr[left] = pivot;
pivot = left;
left = leftEnd;
right = rightEnd;
if(left < pivot)
{
    Sort(left, pivot - 1);
}
if(right > pivot)
{
    Sort(pivot + 1, right);
}
}
static void Main()
{
    arr = new int[] { 54, 79, 58, 7, 42, 23, 91, 3, 74, 38, 67, 46, 18, 61, 32, 86, 14 };
    Sort(0, arr.Length - 1);
    foreach (int i in arr)
    {
        Console.Write(i + " ");
    }
    Console.ReadLine();
}
}

```